QUILA VISUALS

TECHNICAL REPORT

Ads Eventing Platform

Design, implementation, verification and complete source code

Six phases
plus an installable app

18,754 lines
in 214 source files

133 automated tests
116 Java, 7 TypeScript, 10 alert-rule

20,000 events/s
sustained, exactly-once

Prepared by QUILA VISUALS
Issued 16 October 2026
Repository: github.com/michaelonidba-code/primetech

About this document

This report records everything built for the Ads Eventing platform: the architecture and the reasons for it, what each of the six phases delivered, how correctness and performance were verified, what failed and how it was fixed, how to run and operate the system, and, in Appendix B, the complete source code: 214 files, 18,754 lines. Every figure in it was measured on the running system.

Prepared by **QUILA VISUALS** on 16 October 2026. Source: github.com/michaelonidba-code/primetech, pull requests #2 to #8. The PDF has bookmarks for every section and every source file.

Contents

About this document 2

Contents . . . 2

1. Executive summary 4

2. Architecture 5

Data flow . . . 5

Components . . . 5

3. Phase 1: event backbone 6

What was built . . . 6

Measured . . . 6

4. Phase 2: stream processing and reporting 6

What was built . . . 6

Measured . . . 6

5. Phase 3: ad serving with OpenRTB auctions 7

What was built . . . 7

Measured . . . 7

6. Phase 4: budgets and pacing 7

What was built . . . 7

Measured . . . 8

7. Installable app and hosted API 8

What was built . . . 8

Measured . . . 8

8. Phase 5: audit and observability 8

What was built . . . 8

Measured . . . 9

9. Phase 6: load testing and write-up 9

What was built . . . 9

Measured . . . 9

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

10. Testing and verification **10**

Automated tests by module . . . 10

End-to-end reconciliations . . . 10

11. Defects found by measurement **11**

12. Security and privacy **12**

13. Running and operating **13**

Run locally . . . 13

Deploy with HTTPS and sign-in . . . 13

Configuration . . . 13

Kafka topics . . . 14

Service endpoints . . . 14

Alert rules . . . 14

14. Capacity model **16**

15. Pull requests delivered **16**

Appendix A. File inventory **17**

Appendix B. Complete source code **26**

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

1. Executive summary

The project builds, in one repository, a real-time advertising system for a connected-TV ad tier, modelled on the systems that serve and measure ads at streaming scale. A player asks the **ad server** for an ad break; the ad server runs an **OpenRTB 2.6 auction** across demand-side platforms and its own direct campaigns, within budgets set by a **pacing** service. The player reports playback to an **ingest gateway**; events flow through **Kafka** to **Flink**, which removes duplicates, enriches, flags invalid traffic and totals delivery exactly once; **Druid** serves reporting. An **audit** service continuously reconciles independent records of the same money, and **alerts** reach phones through the installable dashboard app.

OUTCOME	RESULT
Event path throughput	**20,000 events/s** sustained on one 4-vCPU machine (1.7 billion a day), exactly-once
Ingest gateway alone	**51,500 events/s**, p99 14–23 ms
Ad serving	**660 auctions/s** across three DSPs, p99 459 ms, nothing shed
Correctness	every reconciliation exact, including **956,446 = 956,446** events through a crash
Budget pacing	**0** line items over budget in measured periods; served-spend counters equal the decision log to the cent
Alerting	fault to phone notification in **49–87 s** in failure drills
Defects found by measurement	**12** bugs no unit test caught, all fixed
Automated tests	**116 Java**, 7 TypeScript, 10 alert-rule cases; CI on every pull request

Technology: Java 21 (virtual threads), Apache Flink 2.2, Redpanda (Kafka API), Apache Druid 37, Redis, Nuxt 4 / Vue 3 / TypeScript, Prometheus, Alertmanager, Grafana, Caddy, Docker Compose, GitHub Actions.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

2. Architecture

Kafka is the system of record between every stage. Druid and Redis are views that can be rebuilt, and the services that keep state (pacing, audit) rebuild it from the topics when they start, so nothing but Kafka needs backing up.

Data flow

- **Serving:** player → ad server (direct line items + OpenRTB 2.6 auction across 3 DSPs, frequency caps, floors) → ad pod back to the player. Every decision is logged to `ad-decisions.v1`.
- **Pacing:** the pacing service reads delivered spend per minute from Flink and writes a throttle and stop threshold per line item to Redis; the ad server reads them every second and writes its served spend back.
- **Measurement:** player → ingest gateway (validate, pseudonymise, acknowledge when durable) → `ad-events.v1` → Flink (de-duplicate, enrich, flag invalid traffic, minute totals, exactly-once) → `ad-events-clean.v1`, `line-item-minutes.v1` → Druid.
- **Assurance:** the audit service reconciles Flink with Druid, Redis with the decision log, and delivery with budgets; Prometheus scrapes every service; Alertmanager routes alerts to the dashboard, which pushes them to phones.

Components

COMPONENT	BUILT WITH	RESPONSIBILITY
Ingest gateway	Java 21, JDK HTTP server, Kafka producer	Validate events one by one, pseudonymise device IDs, acknowledge only when Kafka has the batch
Event bus	Redpanda (Kafka API)	Durable, partitioned, replayable log between every stage
Stream processor	Apache Flink 2.2, RocksDB state	De-duplicate, enrich, flag invalid traffic, minute totals per line item, exactly-once output
Analytics store	Apache Druid 37	Minute rollups with unique-device sketches, SQL
Ad server	Java 21, OpenRTB 2.6	Direct demand and DSP auction, frequency caps, floors, pacing, decision log
Mock DSPs	Java 21	Three bidders with realistic value and latency distributions
Pacing service	Java 21	A closed-loop controller per line item
Audit service	Java 21	Continuous reconciliation of independent records
Dashboard	Nuxt 4, Vue 3, TypeScript, PWA	Live view, sign-in, read-only API, alerts and push notifications
Observability	Prometheus, Alertmanager, Grafana	20 alert rules, an operations dashboard
Edge	Caddy	Automatic HTTPS; the only published service in production

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

3. Phase 1: event backbone

PULL REQUEST #2

What was built

- **Event model and validation** (`ads-common`): `ClientEvent` in, `AdEvent` to Kafka; each event validated on its own so one bad event never fails a batch; rejected events go to `ad-events-dlq.v1` with the reason.
- **Privacy:** device IDs replaced at the gateway by an HMAC-SHA256 pseudonym when the viewer consented, dropped when not. Raw IDs never reach Kafka.
- **Ingest gateway:** JDK HTTP server on virtual threads; Kafka producer with `acks=all` and idempotence, keyed by session; replies 200 only when every event is durable, otherwise 503 with `Retry-After` (at-least-once).
- **Traffic simulator:** realistic ad pods, quartile drop-off, clicks, skewed campaigns; injects duplicates, late and invalid events and keeps exact ground truth.
- **Dashboard:** Nuxt 4 single-page app with live throughput, funnel and rejections.

Measured

MEASURE	RESULT
Sustained ingest	about **51,500 events/s** (4.4 billion a day), p50 ≈ 9 ms, p99 14–23 ms
Invalid events	39,464 injected = 39,464 rejected = 39,464 in the dead-letter topic
Privacy	0 raw device IDs in 3,000 sampled Kafka records
Tuning	capping JVM heaps cut p99 from 55–80 ms to 14–23 ms

4. Phase 2: stream processing and reporting

PULL REQUEST #3

What was built

- **De-duplication** by event ID in RocksDB state with a 30-minute TTL removes the duplicates at-least-once ingest lets through.
- **Enrichment** with campaign metadata through broadcast state; events that arrive before their metadata are buffered and released.
- **Invalid traffic:** a device above 20 impressions in an event-time minute is flagged, excluded from spend, and kept.
- **Minute totals per line item:** event-time windows with 10 minutes of allowed lateness, emitted as revisions; `CoalescingLateTrigger` limits late re-firings (217 revisions of one minute became 6).
- **Exactly-once output:** checkpoints every 10 s, Kafka transactions on both sinks; `RestoreEventTime` restores timestamps lost by broadcast state.
- **Druid 37:** Kafka supervisor reading committed records only, minute rollup, HLL unique devices; dashboard panels for Flink health and campaign delivery.

Measured

MEASURE	RESULT
Clean events vs ground truth	**636,872 = 636,872**, identical per event type; 6,397 duplicates injected, 0 in output

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

MEASURE	RESULT
Line item minutes vs recount	306 / 306 windows match
Crash drill (TaskManager killed)	**956,446 = 956,446**, 0 missing, 0 duplicated; 612 / 612 minutes match
Druid vs clean topic	identical totals, including 8,537,992,000 µ$ of spend

5. Phase 3: ad serving with OpenRTB auctions

PULL REQUEST #4

What was built

- **OpenRTB 2.6 model** for connected-TV ad pods (`podid`, `slotinpod`, `poddur`), with price macros.
- **Mock DSPs:** premium (selective, fast), budget (bids on most, cheap) and slow (high value, long latency tail).
- **Ad server:** pseudonymises the device, fans out to DSPs in parallel within 120 ms while filtering direct line items, applies frequency caps with one Redis `MGET`, runs a per-slot pod auction (floors, one ad per advertiser per break, duration fit, first or second price), sends win notices and logs every decision.
- **Admission control:** bounded in-flight decisions with an immediate 503, so overload degrades instead of hanging.
- The simulator now requests every break from the ad server and reports auction ID and clearing price on its events.

Measured

MEASURE	RESULT
Clean events vs ground truth	**187,027 = 187,027**
Impressions joined to their auction at the clearing price	**44,610 / 44,610**
Druid spend by source vs the auction join	identical: $922.73 direct, $750.67 programmatic
Latency at 227 / 542 / 808 auctions/s	p50/p99 91/129 ms, 104/318 ms, 149/333 ms

6. Phase 4: budgets and pacing

PULL REQUEST #5

What was built

- **Line items** have a tier (guaranteed or standard) and a budget per pacing period; guaranteed line items take a slot ahead of the auction.
- **Pacing service:** one controller per line item, learning its yield and setting `throttle = remaining budget ÷ remaining time ÷ yield`; cuts up to 4× per tick, raises at most 2×, catches up at most at 2× the even rate.
- **Two clocks:** delivered spend (Flink, exact, a minute old) budgets; served spend (ad server, immediate) steers; a stop threshold in served spend stops a line item the moment it is spent.
- **Ad server** reads pacing state from Redis once a second; if it is stale it falls back to a small fixed throttle and a hard stop at budget, so it can under-deliver but never overspend.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

Measured

MEASURE	RESULT
Line items over budget by > 1%	**0** in both measured periods
Median delivered / budget	96.9% and 96.4% (stops early by design)
Served-spend counters vs decision log	identical: $2,406.02 and $2,530.03
Closed-loop simulation	98.9–99.2% of budget delivered, worst deviation 0.8–5.9%

7. Installable app and hosted API

PULL REQUEST #6

What was built

- **Progressive web app:** manifest, icons (including maskable and Apple touch), service worker; installs on Windows, macOS, Android and iOS and opens offline. API responses are never cached.
- **Sign-in:** scrypt password hashes in configuration; HMAC-signed `HttpOnly; Secure; SameSite=Strict` session cookies; changing a password ends its sessions.
- **Read-only API:** every data route is GET and requires a session; login limited to 20 failures per 15 minutes per address and 10 per user name; fails closed with 503 when misconfigured.
- **Hosting:** a production overlay with Caddy for automatic HTTPS and HSTS; only ports 80 and 443 published.

Measured

CHECK	RESULT
Internal ports from outside	all closed
Data routes without / with a session	401 / live data
Form login, cross-origin login, tampered cookie, POST to data	415, 403, 401, 405
21 failed logins with spoofed X-Forwarded-For	21st refused (429): spoofing has no effect

8. Phase 5: audit and observability

PULL REQUEST #7

What was built

- **Audit service:** every minute compares Flink's totals with Druid's per line item and minute, Redis served spend with the decision log, delivered spend with budgets, and delivered with served; results as metrics, records on `audit-results.v1`, and a dashboard panel naming the line items.
- **Prometheus, Alertmanager, Grafana:** 20 alert rules with 10 `promtool` unit tests, inhibition so one cause sends one notification, and a provisioned operations dashboard.
- **Alerts to phones:** Alertmanager calls the dashboard's webhook (bearer token, blocked at Caddy), which sends encrypted Web Push (VAPID) to every subscribed device; push endpoints restricted to the browsers' push services.
- **Found by the audit's first run:** the ad server dropped served-spend increments when Redis was slow (4,744 in three minutes; 21 line items over budget). Fixed by writing absolute per-instance totals in the background (`HSET`), so retries can neither lose nor double-count.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

Measured

DRILL	FAULT → NOTIFICATION ON A DEVICE
Stop the pacing service	**87 s**
Forge $5 of spend in Redis	**64 s**, naming the line item and both amounts
Kill the Flink TaskManager	**49 s**; 2,381 minutes across the failure still identical in Flink and Druid

9. Phase 6: load testing and write-up

PULL REQUEST #8

What was built

- **Load test harness** (`loadtest/run.py`): steps the simulator through rising load and records throughput, latency, backlog, freshness, CPU per service and audit status from Prometheus and the running system; `report.py` renders the results.
- **Found:** time-only topic retention filled the disk at 40,000 events/s; Redpanda crashed and Flink crash-looped with no alert naming the cause. Fixed with size-based retention, a disk alert on free bytes, and new critical alerts for failing checkpoints and crash loops.
- **Profiled:** 14% of Flink's CPU went on compressing small Kafka batches; larger batches cost no visible latency under exactly-once and gave 7% more throughput.
- **Design document** (`docs/DESIGN.md`) and a capacity model for 10 billion events a day.

Measured

OFFERED EVENTS/S	GATEWAY	FLINK	BACKLOG	VISIBLE P50
5,000	4,998/s	4,971/s	none	5.7 s
10,000	9,867/s	9,873/s	none	6.2 s
20,000	20,083/s	20,346/s	none	7.7 s
25,000	24,703/s	20,513/s	grows	13.0 s
30,000	29,601/s	18,218/s	grows	27.1 s

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

10. Testing and verification

Unit tests cover the logic; end-to-end reconciliations cover the system. Continuous integration (GitHub Actions) runs the Maven build and tests, the dashboard type check, unit tests and builds, every Docker image, the production overlay, the Prometheus and Alertmanager configuration and the alert-rule tests.

Automated tests by module

MODULE	TESTS
ad-server	25
ads-common	21
stream-processor	17
audit-service	13
pacing-service	13
event-simulator	10
ingest-gateway	7
mock-dsp	5
openrtb	5
Java total	**116**
dashboard (TypeScript, node:test)	7
alert rules (promtool)	10

End-to-end reconciliations

COMPARED	RESULT
Clean events vs simulator ground truth	636,872 = 636,872
Exactly-once through a TaskManager kill	956,446 = 956,446
Druid vs the clean topic	identical events, types, invalid traffic and spend
Impressions joined to auctions at the clearing price	44,610 / 44,610
Served-spend counters vs the decision log, per period	identical to the cent
Flink minute totals vs Druid across a TaskManager kill	2,381 minutes identical

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

11. Defects found by measurement

None of these failed a unit test. Each was found by reconciling, profiling or deliberately breaking the running system, and each is fixed.

#	DEFECT	FOUND BY	FIX
1	One minute's total revised 217 times by late events	reconciliation	coalescing late trigger (6 revisions)
2	Flink lost event time on records from broadcast state	reconciliation	`RestoreEventTime` operator
3	A replayed run looked like data loss	crash drill	pass the simulator seed through
4	Ad server hung under overload (13,062 stuck connections)	load sweep	admission control, fail-fast Redis pool
5	Campaign catalog never published (82k events unenriched)	reconciliation	fetch metadata first, retry until acknowledged
6	Pacing overspent up to 51% after a cold start	live run	catch-up cap, reset at period rollover
7	A guessed constant under-delivered by 6%	live run	measured serve-to-play delay (15 s)
8	Pacing fallback could overspend by 20%	live measurement	stop at budget in served spend
9	Served-spend increments dropped under load	audit	absolute per-instance totals (HSET)
10	A stale-checkpoint page during recovery	failure drill	2-minute restore grace, rule test
11	A full disk took the system down unannounced	load test	size retention, disk and crash-loop alerts
12	14% of Flink CPU compressing tiny batches	profiling	larger producer batches (+7%)

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

12. Security and privacy

- **Device privacy:** raw device IDs never leave the gateway; consented IDs become keyed HMAC pseudonyms, others are dropped; rotating the key unlinks history.
- **Dashboard sign-in:** scrypt hashes; HMAC-signed session cookies (`HttpOnly`, `Secure`, `SameSite=Strict`, 12 h); a password change ends existing sessions.
- **Brute force:** failed logins limited per client address and per user name; unknown names cost the same time as known ones.
- **Least exposure:** in production only Caddy (80/443) is reachable; Kafka, Flink, Druid, Redis, Prometheus, Alertmanager and Grafana stay on the internal network.
- **Read-only API:** data routes are GET only; the only writes are sign-in, sign-out, notification (un)subscription and the token-protected alert webhook, which Caddy blocks from outside.
- **Server-side request forgery:** push subscription endpoints are restricted to the browsers' push services.
- **Fails closed:** sign-in on but misconfigured → the API answers 503; it never serves data unauthenticated.
- **No data at rest on devices:** the service worker caches only the app shell, never API responses.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

13. Running and operating

Run locally

```
docker compose up --build           # whole stack, about 10 GB of memory
open http://localhost:3000           # dashboard (sign-in off locally)
open http://localhost:3001           # Grafana operations dashboard
mvn verify                           # Java build and 116 tests
```

Deploy with HTTPS and sign-in

```
cp deploy/.env.example deploy/.env    # domain, users, secrets
docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml \
  --env-file deploy/.env up -d --build
```

Configuration

VARIABLE	SERVICE	DEFAULT
DEVICE_ID_SECRET	gateway	**required**, ≥ 16 chars (Compose sets a dev value)
KAFKA_BOOTSTRAP	gateway	localhost:19092
MAX_EVENTS_PER_BATCH, MAX_BODY_BYTES	gateway	1000, 1048576
TARGET_EPS, BATCH_SIZE, CONCURRENCY	simulator	2000, 200, 16
DURATION_SECONDS	simulator	0 (run forever)
DUPLICATE_RATE, LATE_RATE, INVALID_RATE, LATE_DELAY_MS	simulator	0.01, 0.02, 0.005, 30000
BOT_SESSION_SHARE, BOT_DEVICES	simulator	0.01, 5
CAMPAIGNS, SEED	simulator	50, 42 (event ids derive from the seed)
AD_SERVER_URL, AD_REQUEST_CONCURRENCY	simulator	unset (local catalog), 256
KAFKA_BOOTSTRAP	simulator	unset; without an ad server, publishes the campaign catalog
DSP_ENDPOINTS	ad server	name=url,...
TMAX_MS, FLOOR_CPM, AUCTION_TYPE	ad server	120, 5.00, first (or second)
DIRECT_PARTICIPATION, MAX_IN_FLIGHT, DSP_FREQUENCY_CAP_PER_HOUR	ad server	0.015 (fallback without pacing), 256, 4
DASHBOARD_AUTH_DISABLED, DASHBOARD_USERS, DASHBOARD_SESSION_SECRET	dashboard (local Compose)	true, empty, empty. In production set via deploy/.env (see Hosting)
NUXT_SESSION_TTL_HOURS, NUXT_TRUST_PROXY	dashboard	12, false (true in the production overlay)
ALERT_WEBHOOK_TOKEN	Alertmanager, dashboard	a fixed local value; required in production
VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY, VAPID_SUBJECT	dashboard	empty: push notifications off
AUDIT_INTERVAL_SECONDS	audit service	60
TOPIC_PARTITION_RETENTION_BYTES	topic setup	268435456 (256 MiB per partition, alongside 7-day time retention)
SIM_AD_SERVER_URL	simulator	http://ad-server:8090; empty plans ad breaks locally (event-path load tests)
GRAFANA_ADMIN_PASSWORD, GRAFANA_ANONYMOUS	Grafana	admin, true locally; password required and anonymous off in production

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

VARIABLE	SERVICE	DEFAULT
`PACING_PERIOD_MINUTES`, `BUDGET_SCALE`	ad server, pacing service	`30`, `1.0` (budgets sized for 30 min at ~2,000 events/s)
`PACING_TICK_SECONDS`	pacing service	`10`
`REDIS_URL`, `DEVICE_ID_SECRET`	ad server	unset = in-process caps; secret must match the gateway's
`DSP_PROFILE`, `PUBLIC_URL`	mock DSP	`premium`, `budget` or `slow`; URL for win notices
`DEDUP_TTL_MINUTES`	stream-processor	`30`
`OUT_OF_ORDERNESS_SECONDS`, `ALLOWED_LATENESS_MINUTES`, `LATE_FIRING_DELAY_SECONDS`	stream-processor	`5`, `10`, `5`
`IVT_MAX_IMPRESSIONS_PER_DEVICE_MINUTE`	stream-processor	`20`
`METADATA_WAIT_SECONDS`, `CHECKPOINT_INTERVAL_SECONDS`	stream-processor	`60`, `10`
`NUXT_PUBLIC_GATEWAY_URL`	dashboard	`http://localhost:8080` (or `?gateway=` in the URL)
`NUXT_FLINK_URL`, `NUXT_DRUID_URL`, `NUXT_AD_SERVER_URL`	dashboard (server)	`http://localhost:8082`, `http://localhost:8888`, `http://localhost:8090`

Kafka topics

TOPIC	PARTITIONS	RETENTION	KEY	WRITTEN BY
`ad-events.v1`	12	7 days	`sessionId`	gateway
`ad-events-dlq.v1`	3	14 days	`eventId`	gateway
`campaign-metadata.v1`	1	compacted	`campaignId`	ad server (direct catalog at start-up, DSP campaigns on first win)
`ad-events-clean.v1`	12	7 days	`sessionId`	Flink, transactional
`ad-decisions.v1`	12	7 days	`requestId`	ad server
`line-items.v1`	1	compacted	`lineItemId`	ad server (tiers and budgets, read by the pacing service)
`line-item-minutes.v1`	6	compacted + 7 days	`` `lineItemId\ ``	windowStart` Flink, transactional

Service endpoints

SERVICE	ENDPOINTS
Ingest gateway :8080	`POST /v1/events`, `GET /stats`, `/metrics`, `/health`
Ad server :8090	`POST /v1/ad-requests`, `GET /stats`, `/metrics`, `/health`
Pacing service :8095	`GET /stats`, `/metrics`, `/health`
Audit service :8096	`GET /stats`, `/metrics`, `/health`
Dashboard API	`/api/health`, `/api/auth/login`, `/api/auth/logout`, `/api/auth/me`, `/api/gateway`, `/api/pipeline`, `/api/campaigns`, `/api/adserver`, `/api/pacing`, `/api/audit`, `/api/alerts`, `/api/alerts/webhook`, `/api/push/key`, `/api/push/subscribe`, `/api/push/unsubscribe`

Alert rules

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

ALERT	GROUP	SEVERITY	FOR	SUMMARY
ServiceDown	availability	critical	1m	… is down
IngestPublishFailures	ingest	critical	—	Gateway cannot write events to Kafka
IngestRejectionRateHigh	ingest	warning	5m	… of incoming events are rejected
AdDecisionLatencyHigh	ad-serving	warning	5m	Ad decision p99 is …
AdServerShedding	ad-serving	warning	5m	Ad server rejects … of requests
FillRateLow	ad-serving	warning	10m	Fill rate …
FlinkJobNotRunning	stream-processing	critical	1m	The stream processing job is not running
FlinkJobRestarting	stream-processing	warning	—	The Flink job restarted … times in 10 minutes
FlinkJobCrashLooping	stream-processing	critical	—	The Flink job is crash-looping (… restarts in 10 minutes)
FlinkCheckpointsFailing	stream-processing	critical	—	… Flink checkpoints failed in 5 minutes
FlinkCheckpointsStalled	stream-processing	critical	1m	No completed checkpoint for …
FlinkConsumerLagHigh	stream-processing	warning	5m	… events waiting for the stream processor
PacingStateStale	pacing	critical	30s	Pacing state not updated for …
DeliveryDataStale	pacing	warning	2m	Delivered spend is … behind
LineItemOverBudget	pacing	critical	1m	… line item(s) more than 1% over budget
AdServerOnPacingFallback	pacing	warning	5m	The ad server is pacing on its fallback
AuditMismatch	audit	critical	—	Audit check … found a mismatch
AuditNotCompleting	audit	warning	5m	Audit check … has not completed for …
KafkaPartitionsUnavailable	kafka	critical	1m	… Kafka partitions unavailable
KafkaDiskLow	kafka	critical	2m	Kafka (Redpanda) is running out of disk

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

14. Capacity model

Per-core rates measured at saturation and rounded down, scaled to 10 billion events a day with a 3× peak (350,000 events/s, 42,000 auctions/s). An order of magnitude from one small machine, not a quote.

STAGE	PER CORE	CORES AT PEAK	ALSO NEEDS
Ingest gateway	50,000 events/s	7	stateless, behind a load balancer
Kafka	50,000 events/s	7	~12 TB/day before compression; tiered storage; 96+ partitions
Flink	7,500 events/s	47	630 M event IDs of de-duplication state
Druid ingestion	40,000 events/s	9	historicals sized by queries and retention
Ad server	~750 auctions/s	56	regional, close to players
Total		**~125**	**~250 with headroom and zone redundancy**

15. Pull requests delivered

PR	BRANCH	SCOPE
#2	`phase-1-event-backbone`	Event backbone: gateway, simulator, Kafka topics, dashboard
#3	`phase-2-stream-processing`	Flink exactly-once processing and Druid reporting
#4	`phase-3-ad-serving`	Ad server with OpenRTB 2.6 auctions and mock DSPs
#5	`phase-4-pacing`	Budgets, tiers and closed-loop pacing
#6	`pwa-hosted-api`	Installable app, signed-in API, HTTPS deployment
#7	`phase-5-observability`	Audit service, Prometheus, Alertmanager, Grafana, push alerts
#8	`phase-6-load-and-writeup`	Load tests, profiling, capacity model, design document

Each pull request is stacked on the previous one and passed CI (Java build and tests, dashboard checks, Docker images, and from Phase 5 the observability checks). Merge them in order, #2 to #8.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

Appendix A. File inventory

Every file in the repository except images, the npm lock file and Markdown documentation: 214 files, 18,754 lines. The purpose is taken from each file's own documentation comment where it has one.

(repository root) (952 lines)

FILE	LINES	PURPOSE
.dockerignore	7	
.gitignore	14	Java
Dockerfile	57	Multi-stage build producing one image per Java service.
LICENSE	339	
docker-compose.yml	379	The whole stack for local runs: Kafka, Flink, Druid, services, observability, dashboard.
pom.xml	156	Maven parent build: modules and pinned dependency versions.

.github (134 lines)

FILE	LINES	PURPOSE
workflows/ci.yml	71	
workflows/nuxtjs.yml	63	Builds the dashboard in dashboard/ as a static site and deploys it to GitHub Pages.

ad-server (2,232 lines)

FILE	LINES	PURPOSE
pom.xml	59	
main/AdDecisionService.java	250	Answers one ad request: pseudonymise the device id (same keyed hash as the event pipeline, so caps and reporting agree); send one OpenRTB request to every DSP in parallel, with a hard deadline; meanwhile, filter direct-sold line items by targeting; drop candidates over their frequency cap (one Redis read for all of them); run the pod auction, record caps, notify winning DSPs, and log the decision.
main/AdServerApp.java	89	Entry point: wires the ad server (DSP clients, Redis, Kafka, pacing view, HTTP).
main/AdServerConfig.java	74	Ad server settings read from environment variables.
main/AdServerHttp.java	127	HTTP front of the ad server, on virtual threads like the gateway.
main/AdServerMetrics.java	143	Ad server counters, exposed to Prometheus and summarised on /stats for the dashboard.
main/Auction.java	86	Fills an ad pod slot by slot.
main/Candidate.java	31	Something that could fill a slot: a direct-sold creative or a DSP bid.
main/DecisionSink.java	27	Where the ad server records decisions and announces campaigns.
main/DirectDemand.java	56	Direct-sold line items from the campaign catalog, filtered by targeting and by pacing: each line item enters an auction with the probability its pacer sets, and not at all once its budget is spent.
main/DspClient.java	116	Calls one DSP's OpenRTB endpoint.
main/FrequencyCaps.java	21	Per device, per campaign impression counts in fixed clock-hour windows.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

FILE	LINES	PURPOSE
main/InMemoryFrequencyCaps.java	32	Single-instance frequency caps, for tests and for running without Redis.
main/KafkaDecisionSink.java	121	Writes the decision log and campaign metadata to Kafka without waiting for acks: the ad request must not pay for an audit write.
main/PacingView.java	28	The ad server's view of budget pacing: how often each direct line item may compete, and the running total of what it has served.
main/RedisFrequencyCaps.java	83	Frequency caps in Redis: one counter per device, campaign and hour, so every ad server instance shares them.
main/RedisPacingView.java	207	Pacing state from Redis, refreshed in the background once a second so requests never wait on it.
main/ServedLedger.java	95	This ad server instance's served spend per period and line item, and how it combines with the shared totals in Redis.
src/main/resources/simplelogger.properties	7	The Kafka client logs its full config at INFO on startup; keep only warnings.
test/AdDecisionServiceTest.java	265	Real OpenRTB round trips against in-process DSPs: a good one, a slow one and a broken one.
test/AdServerHttpTest.java	94	Test.
test/AuctionTest.java	110	Test.
test/RedisPacingViewTest.java	31	Test.
test/ServedLedgerTest.java	80	Test.

ads-common (1,191 lines)

FILE	LINES	PURPOSE
pom.xml	27	
main/AdEvent.java	33	Canonical ad event written to the Topics#AD_EVENTS topic.
main/CampaignMetadata.java	12	Reference data the stream processor joins onto every event.
main/ClientEvent.java	45	An event as a player sends it to the ingest gateway.
main/DeadLetter.java	16	A rejected event written to Topics#AD_EVENTS_DLQ.
main/DeviceIdHasher.java	36	Pseudonymises device ids with HMAC-SHA256.
main/DeviceType.java	8	Device types accepted on events (CTV, mobile, web, ...).
main/EnrichedAdEvent.java	64	A de-duplicated event with campaign metadata attached and an invalid-traffic verdict.
main/EventNormalizer.java	34	Converts a validated ClientEvent to the canonical AdEvent, applying privacy rules.
main/EventType.java	19	Client-side ad playback events, modelled on the VAST tracking events a CTV player fires.
main/EventValidator.java	79	Stateless, per-event checks done at the edge.
main/IngestRequest.java	7	Body of POST /v1/events.
main/IngestResponse.java	13	Per-batch result.
main/Json.java	18	One shared, thread-safe mapper so every service reads and writes events the same way.
main/LineItemMinute.java	30	Per line item, per event-time minute delivery totals, written to Topics#LINE_ITEM_MINUTES.
main/RejectReason.java	13	Reasons an event is rejected at the gateway (sent to the dead-letter topic).
main/Topics.java	23	Names of every Kafka topic, shared by all services.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

FILE	LINES	PURPOSE
main/adserving/AdDecision.java	37	Audit record of one ad request, written to ad-decisions.v1: what was asked for, who bid, who won each slot and at what price.
main/adserving/AdRequest.java	22	A player asking for the ads of one ad break (pod).
main/adserving/AdResponse.java	27	The ads to play, in order.
main/catalog/AdCatalog.java	135	Synthetic direct-sold campaigns, line items and creatives: the stand-in for a campaign management system.
main/catalog/LineItemConfig.java	22	The commercial terms of a direct-sold line item, as the pacing service needs them.
main/http/JsonHttp.java	64	Small helpers for JSON services on the JDK HTTP server.
main/pacing/PacingPeriod.java	33	Budgets reset every period.
main/pacing/PacingState.java	42	The pacing service's instruction for one line item in the current period, read by the ad server.
main/pacing/ServedSpend.java	47	Served spend per period and line item, as kept in Redis by the ad servers.
test/EnrichedAdEventTest.java	51	Test.
test/EventNormalizerTest.java	53	Test.
test/EventValidatorTest.java	95	Test.
test/JsonTest.java	29	Test.
test/PacingPeriodTest.java	39	Test.
test/TestEvents.java	18	Test.

audit-service (1,024 lines)

FILE	LINES	PURPOSE
pom.xml	51	
main/AuditApp.java	168	Wires the audit to Kafka (decision log, minute totals and budgets in; results out), Druid, Redis and HTTP.
main/AuditLedger.java	96	What the audit has read from Kafka: Flink's minute totals (latest revision of each), direct spend per period and line item from the ad server's decision log, and line item budgets.
main/AuditService.java	196	Runs every reconciliation over data that has settled, and exposes the results as metrics (for alerts), as recent history (for the dashboard) and as records (for the audit trail).
main/CheckResult.java	49	The outcome of one run of one reconciliation.
main/Checks.java	127	The reconciliations, as pure functions over data already read from each source.
main/Sources.java	101	The two sources the audit queries on demand: Druid (SQL over HTTP) and Redis.
src/main/resources/simplelogger.properties	7	The Kafka client logs its full config at INFO on startup; keep only warnings.
test/AuditServiceTest.java	171	Sources that answer from maps, or fail like a down Druid or Redis.
test/ChecksTest.java	58	Test.

dashboard (3,354 lines)

FILE	LINES	PURPOSE
.dockerignore	4	

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

FILE	LINES	PURPOSE
Dockerfile	22	Optional "cacert" build secret: extra CA bundle for builds behind a TLS-inspecting proxy.
app/app.vue	114	
app/assets/main.css	84	
app/components/AdServingPanel.vue	187	
app/components/AlertsPanel.vue	176	
app/components/AuditPanel.vue	188	
app/components/BrandLogo.vue	27	The Quila Visuals logo. The PNG is used as a mask so the logo takes the current text colour and
app/components/CampaignTable.vue	125	
app/components/DashboardView.vue	155	
app/components/FunnelChart.vue	111	Playback funnel: how many impressions went on to each quartile. Bars share one hue because
app/components/LoginForm.vue	106	
app/components/PacingPanel.vue	271	
app/components/PipelinePanel.vue	96	
app/components/StatTile.vue	50	
app/components/ThroughputChart.vue	169	
app/composables/useGatewayStats.ts	73	
app/composables/useInstall.ts	50	
app/composables/useOnline.ts	15	
app/composables/usePolled.ts	19	
app/composables/usePush.ts	94	
app/composables/useSession.ts	56	
app/composables/useVisibleInterval.ts	36	
app/plugins/pwa.client.ts	11	Registers the service worker (public/sw.js) for production builds, under the app's base URL so
app/utils/format.ts	43	
nuxt.config.ts	68	A client-side SPA: the same build is served by Node (Docker, or hosted behind Caddy) or as static
package.json	27	
public/manifest.webmanifest	18	
public/sw.js	119	Service worker: makes the dashboard installable and lets the app shell open offline.
scripts/hash-password.ts	49	Prints a user entry for NUXT_AUTH_USERS. Usage: npm run hash-password -- <user>
server/api/adserver.get.ts	11	Ad server counters (auctions, fill, latency, per-DSP outcomes), proxied so the browser needs no
server/api/alerts.get.ts	35	Active alerts from Alertmanager (silenced and inhibited ones left out), plus recent notifications.
server/api/alerts/webhook.post.ts	53	Alertmanager's webhook: every firing or resolved alert becomes a push notification on each
server/api/audit.get.ts	9	Reconciliation results, proxied from the audit service.
server/api/auth/login.post.ts	60	Exchanges a user name and password for a session cookie (HttpOnly, SameSite=Strict, Secure on HTTPS).

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

FILE	LINES	PURPOSE
server/api/auth/logout.post.ts	4	
server/api/auth/me.get.ts	5	Who is signed in. Reaching this handler means the middleware accepted the session (or auth is off).
server/api/campaigns.get.ts	52	Campaign delivery from Druid. Billable numbers exclude invalid traffic (ivt = 1).
server/api/gateway.get.ts	9	Ingest gateway counters, proxied so the browser only ever talks to this origin.
server/api/health.get.ts	2	Liveness for load balancers and uptime checks. Unauthenticated, so it reveals nothing else.
server/api/pacing.get.ts	10	Budget pacing state per line item, proxied from the pacing service.
server/api/pipeline.get.ts	59	Stream-processor health from the Flink REST API. Runs server-side, so the browser needs no
server/api/push/key.get.ts	5	The VAPID public key the browser needs to subscribe, or enabled: false if push is not set up.
server/api/push/subscribe.post.ts	11	Registers this device for alert notifications. Signed-in users only (the middleware checks).
server/api/push/unsubscribe.post.ts	8	
server/middleware/auth.ts	31	Every data route is read-only and requires a signed session cookie. The only writes are signing in
server/plugins/auth-config.ts	7	Reports the authentication mode once at startup, so a misconfiguration is visible in the logs.
server/utils/auth.ts	129	Password hashing, signed session tokens and login rate limiting for the hosted dashboard API.
server/utils/notifications.ts	20	The last notifications sent, for the app's alerts panel. In memory: the source of truth for
server/utils/push.ts	144	Web Push to installed copies of the app: VAPID keys from config, subscriptions kept in a small
server/utils/session.ts	47	Auth configuration from runtime config, resolved once. Fails closed: if authentication is on but
test/auth.test.ts	71	
tsconfig.json	9	

deploy (109 lines)

FILE	LINES	PURPOSE
.env.example	23	Copy to deploy/.env (git-ignored) and fill in. Values need no quoting.
Caddyfile	17	Automatic HTTPS: Caddy obtains and renews a certificate for DASHBOARD_DOMAIN (its DNS must point
docker-compose.prod.yml	69	Hosted deployment overlay: one public entry point (Caddy, HTTPS) in front of the dashboard,

docs (1,585 lines)

FILE	LINES	PURPOSE
loadtest/events.json	292	load test measurements (data)
loadtest/full.json	176	load test measurements (data)
overview.html	481	one-page illustrated overview of the design (open in a browser)
report/build_report.py	636	

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

event-simulator (1,257 lines)

FILE	LINES	PURPOSE
pom.xml	43	
main/AdServerPodSource.java	96	Asks the ad server for each break, asynchronously, like a player starting an ad pod.
main/CatalogPublisher.java	49	Stands in for the campaign management system until the ad server exists (Phase 3): writes the campaign catalog to the compacted metadata topic, keyed by campaign id.
main/FaultInjector.java	58	Makes the traffic as messy as real device traffic, so the pipeline's handling of it can be tested: retried sends (duplicates), events buffered on a device that come in late, and broken payloads.
main/GatewayClient.java	155	Sends batches the way a well-behaved player SDK would: bounded concurrency, and the whole batch retried with exponential backoff on 5xx, 429 or network errors.
main/GroundTruth.java	50	What the simulator actually produced.
main/PodSource.java	14	Where a viewer's ad break comes from: the local catalog, or the ad server.
main/Session.java	73	One viewer's ad break, planned up front but emitted lazily.
main/SessionPlanner.java	178	Plans realistic ad breaks: a pod of ads played back to back, each firing an impression then quartiles until the viewer drops off, plus an occasional click.
main/SimulatorApp.java	85	Entry point: runs the traffic simulator against the gateway and ad server.
main/SimulatorConfig.java	44	Simulator settings: rate, faults, campaigns, seed, ad server URL.
main/TrafficEngine.java	163	Drives the simulation by time.
src/main/resources/simplelogger.properties	7	The Kafka client logs its full config at INFO on startup; keep only warnings.
test/SessionPlannerTest.java	97	Test.
test/TrafficEngineTest.java	145	Test.

infra (2,319 lines)

FILE	LINES	PURPOSE
alertmanager/alertmanager.yml	37	Alerts go to the dashboard, which shows them and pushes them to every device that installed
create-topics.sh	44	Creates the event topics. Safe to re-run: existing topics are left alone.
druid/ad-events-supervisor.json	54	
druid/druid.env	21	Shared settings for every Druid service (based on Druid's own docker-compose example),
druid/submit-supervisor.sh	15	Registers the Kafka ingestion supervisor once the Druid router answers. Re-submitting the same
grafana/dashboards/ads-eventing.json	1,730	generated Grafana dashboard (33 panels)
grafana/provisioning/dashboards/dashboards.yml	7	
grafana/provisioning/datasources/datasources.yml	15	
prometheus/prometheus.yml	31	
prometheus/rules/ads-eventing.yml	172	Alerting rules. Each alert names what is wrong, what it costs, and where to look first.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

FILE	LINES	PURPOSE
prometheus/tests/ads-eventing-test.yml	193	promtool test rules infra/prometheus/tests/ads-eventing-test.yml (run in CI)

ingest-gateway (814 lines)

FILE	LINES	PURPOSE
pom.xml	47	
main/EventSink.java	18	Where the gateway puts events.
main/GatewayApp.java	43	Entry point: starts the ingest gateway HTTP server and Kafka producer.
main/GatewayConfig.java	33	Gateway settings read from environment variables.
main/GatewayServer.java	159	HTTP front door.
main/IngestMetrics.java	112	Gateway counters.
main/IngestService.java	91	Validates, scrubs and publishes one batch.
main/KafkaEventSink.java	80	Writes accepted events to Kafka (acks=all, idempotent) and rejected ones to the dead-letter topic.
src/main/resources/simplelogger.properties	7	The Kafka client logs its full config at INFO on startup; keep only warnings.
test/FakeSink.java	32	Test.
test/GatewayServerTest.java	111	Test.
test/IngestServiceTest.java	81	Test.

loadtest (723 lines)

FILE	LINES	PURPOSE
__pycache__/report.cpython-313.pyc	148	
__pycache__/run.cpython-313.pyc	318	
report.py	74	
run.py	183	

mock-dsp (461 lines)

FILE	LINES	PURPOSE
pom.xml	43	
main/Bidder.java	99	The bidding logic of a mock DSP, separate from HTTP so it can be tested directly.
main/DspApp.java	33	Entry point: runs one mock DSP with a named bidding preset.
main/DspProfile.java	39	How a mock DSP behaves.
main/DspServer.java	153	HTTP front of a mock DSP: the OpenRTB bid endpoint, win notices and its own counters.
src/main/resources/simplelogger.properties	7	The Kafka client logs its full config at INFO on startup; keep only warnings.
test/BidderTest.java	87	Test.

openrtb (254 lines)

FILE	LINES	PURPOSE
pom.xml	21	

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

FILE	LINES	PURPOSE
main/BidRequest.java	75	OpenRTB 2.6 bid request, limited to the fields a CTV video ad pod needs.
main/BidResponse.java	37	OpenRTB 2.6 bid response.
main/OpenRtb.java	52	Conversions between our model and OpenRTB conventions.
test/OpenRtbTest.java	69	Test.

pacing-service (979 lines)

FILE	LINES	PURPOSE
pom.xml	51	
main/LineItemPacer.java	227	Paces one line item: spends its period budget evenly by setting the probability that it enters an auction (the throttle).
main/PacingApp.java	165	Wires PacingService to Kafka (inputs), Redis (outputs) and HTTP (stats).
main/PacingService.java	152	Holds one pacer per line item, feeds them delivery data, and runs the control tick.
main/PacingStore.java	16	Where pacing reads served spend and publishes its decisions (Redis in production).
main/RedisPacingStore.java	47	Pacing state in one Redis hash, one field per line item, written in a single HSET per tick.
src/main/resources/simplelogger.properties	7	The Kafka client logs its full config at INFO on startup; keep only warnings.
test/LineItemPacerTest.java	194	Closed-loop tests: the pacer drives a simulated market and must hit its budget.
test/PacingServiceTest.java	120	Test.

stream-processor (1,366 lines)

FILE	LINES	PURPOSE
pom.xml	91	
main/AdPipeline.java	82	The processing topology, independent of Kafka so tests can drive it with in-memory sources.
main/CoalescingLateTrigger.java	74	Event-time trigger with batched late firings.
main/Deduplicate.java	51	Drops repeats of an event id.
main/EnrichWithCampaign.java	102	Joins campaign metadata onto events.
main/InvalidTrafficFilter.java	71	General invalid traffic (GIVT) rule: a device that fires more impressions in one event-time minute than a person could watch is a bot or a broken integration.
main/JsonKafka.java	67	JSON (de)serialization for Kafka using the same mapper as the other services.
main/LineItemTotals.java	110	Incremental per-window totals, so a window holds one small accumulator instead of every event.
main/PipelineConfig.java	46	Job settings, read from the environment so the same jar runs locally and in containers.
main/PipelineJob.java	102	Wires AdPipeline to Kafka with exactly-once output.
main/RestoreEventTime.java	20	Sets each record's Flink timestamp back to its event time.
test/AdPipelineTest.java	216	Runs the whole topology on a local Flink cluster with in-memory sources.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

FILE	LINES	PURPOSE
test/CoalescingLateTriggerTest.java	74	Test.
test/DeduplicateTest.java	59	Test.
test/EnrichWithCampaignTest.java	86	Test.
test/Events.java	27	Test fixtures.
test/InvalidTrafficFilterTest.java	88	Test.

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

Appendix B. Complete source code

Every source file in full, grouped by module, with line numbers. Long lines are wrapped and marked with ↪. The generated Grafana dashboard and load-test result files are listed in Appendix A but not reproduced. Use the PDF bookmarks to jump to any file.

(repository root)

.dockerignore

7 lines

```
1  **/target
2  **/node_modules
3  dashboard/.nuxt
4  dashboard/.output
5  .git
6  .idea
7  *.iml
```

.gitignore

14 lines · Java

```
 1  # Java
 2  target/
 3  *.iml
 4  .idea/
 5
 6  # Node / Nuxt
 7  node_modules/
 8  .nuxt/
 9  .output/
10  .data/
11  dist
12
13  .env
14  .DS_Store
```

Dockerfile

57 lines · Multi-stage build producing one image per Java service.

```
 1  # One Maven build shared by every Java service; pick the service with --target.
 2  FROM maven:3.9.11-eclipse-temurin-21 AS build
 3  # Optional, for builds behind a proxy: e.g. -Dhttps.proxyHost=... and a truststore passed as the
 4  # "truststore" build secret (-Djavax.net.ssl.trustStore=/run/secrets/truststore).
 5  ARG MAVEN_OPTS=""
 6  WORKDIR /src
 7  COPY pom.xml ./
 8  COPY ads-common ads-common
 9  COPY ingest-gateway ingest-gateway
10  COPY event-simulator event-simulator
11  COPY stream-processor stream-processor
12  COPY openrtb openrtb
13  COPY mock-dsp mock-dsp
14  COPY ad-server ad-server
15  COPY pacing-service pacing-service
16  COPY audit-service audit-service
17  RUN --mount=type=cache,target=/root/.m2 \
18      --mount=type=secret,id=truststore,required=false \
19      MAVEN_OPTS="$MAVEN_OPTS" mvn -B -q -DskipTests package
20
21  FROM eclipse-temurin:21-jre AS runtime-base
22  RUN useradd --system --uid 10001 app
23  USER app
24  WORKDIR /app
25  ENV JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+UseZGC -XX:+ZGenerational"
26  ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar /app/app.jar"]
27
28  FROM runtime-base AS gateway
29  COPY --from=build /src/ingest-gateway/target/ingest-gateway.jar /app/app.jar
30  EXPOSE 8080
31
32  FROM runtime-base AS simulator
33  COPY --from=build /src/event-simulator/target/event-simulator.jar /app/app.jar
34
35  FROM runtime-base AS ad-server
36  COPY --from=build /src/ad-server/target/ad-server.jar /app/app.jar
37  EXPOSE 8090
38
39  FROM runtime-base AS pacing-service
40  COPY --from=build /src/pacing-service/target/pacing-service.jar /app/app.jar
41  EXPOSE 8095
42
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
FROM runtime-base AS audit-service
COPY --from=build /src/audit-service/target/audit-service.jar /app/app.jar
EXPOSE 8096

FROM runtime-base AS mock-dsp
COPY --from=build /src/mock-dsp/target/mock-dsp.jar /app/app.jar
EXPOSE 9000

# Flink application image: the job jar in usrlib is picked up by "standalone-job" mode.
FROM flink:2.2.1-java21 AS stream-processor
# Mount point for checkpoints; a named volume inherits this ownership (Flink runs as "flink").
USER root
RUN mkdir -p /flink-data && chown flink:flink /flink-data
USER flink
COPY --from=build --chown=flink:flink /src/stream-processor/target/stream-processor.jar /opt/flink/usrlib/stream
  ↪ -processor.jar
```

LICENSE

339 lines

```
                    GNU GENERAL PUBLIC LICENSE
                       Version 2, June 1991

 Copyright (C) 1989, 1991 Free Software Foundation, Inc.,
 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
 Everyone is permitted to copy and distribute verbatim copies
 of this license document, but changing it is not allowed.

                            Preamble

  The licenses for most software are designed to take away your
freedom to share and change it.  By contrast, the GNU General Public
License is intended to guarantee your freedom to share and change free
software--to make sure the software is free for all its users.  This
General Public License applies to most of the Free Software
Foundation's software and to any other program whose authors commit to
using it.  (Some other Free Software Foundation software is covered by
the GNU Lesser General Public License instead.)  You can apply it to
your programs, too.

  When we speak of free software, we are referring to freedom, not
price.  Our General Public Licenses are designed to make sure that you
have the freedom to distribute copies of free software (and charge for
this service if you wish), that you receive source code or can get it
if you want it, that you can change the software or use pieces of it
in new free programs; and that you know you can do these things.

  To protect your rights, we need to make restrictions that forbid
anyone to deny you these rights or to ask you to surrender the rights.
These restrictions translate to certain responsibilities for you if you
distribute copies of the software, or if you modify it.

  For example, if you distribute copies of such a program, whether
gratis or for a fee, you must give the recipients all the rights that
you have.  You must make sure that they, too, receive or can get the
source code.  And you must show them these terms so they know their
rights.

  We protect your rights with two steps: (1) copyright the software, and
(2) offer you this license which gives you legal permission to copy,
distribute and/or modify the software.

  Also, for each author's protection and ours, we want to make certain
that everyone understands that there is no warranty for this free
software.  If the software is modified by someone else and passed on, we
want its recipients to know that what they have is not the original, so
that any problems introduced by others will not reflect on the original
authors' reputations.

  Finally, any free program is threatened constantly by software
patents.  We wish to avoid the danger that redistributors of a free
program will individually obtain patent licenses, in effect making the
program proprietary.  To prevent this, we have made it clear that any
patent must be licensed for everyone's free use or not licensed at all.

  The precise terms and conditions for copying, distribution and
modification follow.

                    GNU GENERAL PUBLIC LICENSE
   TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

  0. This License applies to any program or other work which contains
a notice placed by the copyright holder saying it may be distributed
under the terms of this General Public License.  The "Program", below,
refers to any such program or work, and a "work based on the Program"
means either the Program or any derivative work under copyright law:
that is to say, a work containing the Program or a portion of it,
either verbatim or with modifications and/or translated into another
```

```
language.  (Hereinafter, translation is included without limitation in
the term "modification".)  Each licensee is addressed as "you".

Activities other than copying, distribution and modification are not
covered by this License; they are outside its scope.  The act of
running the Program is not restricted, and the output from the Program
is covered only if its contents constitute a work based on the
Program (independent of having been made by running the Program).
Whether that is true depends on what the Program does.

  1. You may copy and distribute verbatim copies of the Program's
source code as you receive it, in any medium, provided that you
conspicuously and appropriately publish on each copy an appropriate
copyright notice and disclaimer of warranty; keep intact all the
notices that refer to this License and to the absence of any warranty;
and give any other recipients of the Program a copy of this License
along with the Program.

You may charge a fee for the physical act of transferring a copy, and
you may at your option offer warranty protection in exchange for a fee.

  2. You may modify your copy or copies of the Program or any portion
of it, thus forming a work based on the Program, and copy and
distribute such modifications or work under the terms of Section 1
above, provided that you also meet all of these conditions:

    a) You must cause the modified files to carry prominent notices
    stating that you changed the files and the date of any change.

    b) You must cause any work that you distribute or publish, that in
    whole or in part contains or is derived from the Program or any
    part thereof, to be licensed as a whole at no charge to all third
    parties under the terms of this License.

    c) If the modified program normally reads commands interactively
    when run, you must cause it, when started running for such
    interactive use in the most ordinary way, to print or display an
    announcement including an appropriate copyright notice and a
    notice that there is no warranty (or else, saying that you provide
    a warranty) and that users may redistribute the program under
    these conditions, and telling the user how to view a copy of this
    License.  (Exception: if the Program itself is interactive but
    does not normally print such an announcement, your work based on
    the Program is not required to print an announcement.)

These requirements apply to the modified work as a whole.  If
identifiable sections of that work are not derived from the Program,
and can be reasonably considered independent and separate works in
themselves, then this License, and its terms, do not apply to those
sections when you distribute them as separate works.  But when you
distribute the same sections as part of a whole which is a work based
on the Program, the distribution of the whole must be on the terms of
this License, whose permissions for other licensees extend to the
entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest
your rights to work written entirely by you; rather, the intent is to
exercise the right to control the distribution of derivative or
collective works based on the Program.

In addition, mere aggregation of another work not based on the Program
with the Program (or with a work based on the Program) on a volume of
a storage or distribution medium does not bring the other work under
the scope of this License.

  3. You may copy and distribute the Program (or a work based on it,
under Section 2) in object code or executable form under the terms of
Sections 1 and 2 above provided that you also do one of the following:

    a) Accompany it with the complete corresponding machine-readable
    source code, which must be distributed under the terms of Sections
    1 and 2 above on a medium customarily used for software interchange; or,

    b) Accompany it with a written offer, valid for at least three
    years, to give any third party, for a charge no more than your
    cost of physically performing source distribution, a complete
    machine-readable copy of the corresponding source code, to be
    distributed under the terms of Sections 1 and 2 above on a medium
    customarily used for software interchange; or,

    c) Accompany it with the information you received as to the offer
    to distribute corresponding source code.  (This alternative is
    allowed only for noncommercial distribution and only if you
    received the program in object code or executable form with such
    an offer, in accord with Subsection b above.)

The source code for a work means the preferred form of the work for
making modifications to it.  For an executable work, complete source
```

```
code means all the source code for all modules it contains, plus any
associated interface definition files, plus the scripts used to
control compilation and installation of the executable.  However, as a
special exception, the source code distributed need not include
anything that is normally distributed (in either source or binary
form) with the major components (compiler, kernel, and so on) of the
operating system on which the executable runs, unless that component
itself accompanies the executable.

If distribution of executable or object code is made by offering
access to copy from a designated place, then offering equivalent
access to copy the source code from the same place counts as
distribution of the source code, even though third parties are not
compelled to copy the source along with the object code.

  4. You may not copy, modify, sublicense, or distribute the Program
except as expressly provided under this License.  Any attempt
otherwise to copy, modify, sublicense or distribute the Program is
void, and will automatically terminate your rights under this License.
However, parties who have received copies, or rights, from you under
this License will not have their licenses terminated so long as such
parties remain in full compliance.

  5. You are not required to accept this License, since you have not
signed it.  However, nothing else grants you permission to modify or
distribute the Program or its derivative works.  These actions are
prohibited by law if you do not accept this License.  Therefore, by
modifying or distributing the Program (or any work based on the
Program), you indicate your acceptance of this License to do so, and
all its terms and conditions for copying, distributing or modifying
the Program or works based on it.

  6. Each time you redistribute the Program (or any work based on the
Program), the recipient automatically receives a license from the
original licensor to copy, distribute or modify the Program subject to
these terms and conditions.  You may not impose any further
restrictions on the recipients' exercise of the rights granted herein.
You are not responsible for enforcing compliance by third parties to
this License.

  7. If, as a consequence of a court judgment or allegation of patent
infringement or for any other reason (not limited to patent issues),
conditions are imposed on you (whether by court order, agreement or
otherwise) that contradict the conditions of this License, they do not
excuse you from the conditions of this License.  If you cannot
distribute so as to satisfy simultaneously your obligations under this
License and any other pertinent obligations, then as a consequence you
may not distribute the Program at all.  For example, if a patent
license would not permit royalty-free redistribution of the Program by
all those who receive copies directly or indirectly through you, then
the only way you could satisfy both it and this License would be to
refrain entirely from distribution of the Program.

If any portion of this section is held invalid or unenforceable under
any particular circumstance, the balance of the section is intended to
apply and the section as a whole is intended to apply in other
circumstances.

It is not the purpose of this section to induce you to infringe any
patents or other property right claims or to contest validity of any
such claims; this section has the sole purpose of protecting the
integrity of the free software distribution system, which is
implemented by public license practices.  Many people have made
generous contributions to the wide range of software distributed
through that system in reliance on consistent application of that
system; it is up to the author/donor to decide if he or she is willing
to distribute software through any other system and a licensee cannot
impose that choice.

This section is intended to make thoroughly clear what is believed to
be a consequence of the rest of this License.

  8. If the distribution and/or use of the Program is restricted in
certain countries either by patents or by copyrighted interfaces, the
original copyright holder who places the Program under this License
may add an explicit geographical distribution limitation excluding
those countries, so that distribution is permitted only in or among
countries not thus excluded.  In such case, this License incorporates
the limitation as if written in the body of this License.

  9. The Free Software Foundation may publish revised and/or new versions
of the General Public License from time to time.  Such new versions will
be similar in spirit to the present version, but may differ in detail to
address new problems or concerns.

Each version is given a distinguishing version number.  If the Program
specifies a version number of this License which applies to it and "any
later version", you have the option of following the terms and conditions
```

```
either of that version or of any later version published by the Free
Software Foundation.  If the Program does not specify a version number of
this License, you may choose any version ever published by the Free Software
Foundation.

  10. If you wish to incorporate parts of the Program into other free
programs whose distribution conditions are different, write to the author
to ask for permission.  For software which is copyrighted by the Free
Software Foundation, write to the Free Software Foundation; we sometimes
make exceptions for this.  Our decision will be guided by the two goals
of preserving the free status of all derivatives of our free software and
of promoting the sharing and reuse of software generally.

                            NO WARRANTY

  11. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY
FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW.  EXCEPT WHEN
OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES
PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED
OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF
MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.  THE ENTIRE RISK AS
TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU.  SHOULD THE
PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING,
REPAIR OR CORRECTION.

  12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING
WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR
REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES,
INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING
OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED
TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY
YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER
PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE
POSSIBILITY OF SUCH DAMAGES.

                     END OF TERMS AND CONDITIONS

            How to Apply These Terms to Your New Programs

  If you develop a new program, and you want it to be of the greatest
possible use to the public, the best way to achieve this is to make it
free software which everyone can redistribute and change under these terms.

  To do so, attach the following notices to the program.  It is safest
to attach them to the start of each source file to most effectively
convey the exclusion of warranty; and each file should have at least
the "copyright" line and a pointer to where the full notice is found.

    <one line to give the program's name and a brief idea of what it does.>
    Copyright (C) <year>  <name of author>

    This program is free software; you can redistribute it and/or modify
    it under the terms of the GNU General Public License as published by
    the Free Software Foundation; either version 2 of the License, or
    (at your option) any later version.

    This program is distributed in the hope that it will be useful,
    but WITHOUT ANY WARRANTY; without even the implied warranty of
    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
    GNU General Public License for more details.

    You should have received a copy of the GNU General Public License along
    with this program; if not, write to the Free Software Foundation, Inc.,
    51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.

Also add information on how to contact you by electronic and paper mail.

If the program is interactive, make it output a short notice like this
when it starts in an interactive mode:

    Gnomovision version 69, Copyright (C) year name of author
    Gnomovision comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
    This is free software, and you are welcome to redistribute it
    under certain conditions; type `show c' for details.

The hypothetical commands `show w' and `show c' should show the appropriate
parts of the General Public License.  Of course, the commands you use may
be called something other than `show w' and `show c'; they could even be
mouse-clicks or menu items--whatever suits your program.

You should also get your employer (if you work as a programmer) or your
school, if any, to sign a "copyright disclaimer" for the program, if
necessary.  Here is a sample; alter the names:

  Yoyodyne, Inc., hereby disclaims all copyright interest in the program
  `Gnomovision' (which makes passes at compilers) written by James Hacker.

  <signature of Ty Coon>, 1 April 1989
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
  Ty Coon, President of Vice

This General Public License does not permit incorporating your program into
proprietary programs.  If your program is a subroutine library, you may
consider it more useful to permit linking proprietary applications with the
library.  If this is what you want to do, use the GNU Lesser General
Public License instead of this License.
```

docker-compose.yml

379 lines · The whole stack for local runs: Kafka, Flink, Druid, services, observability, dashboard.

```
name: ads-eventing

services:
  redpanda:
    image: redpandadata/redpanda:v24.2.7
    command:
      - redpanda
      - start
      - --mode=dev-container
      - --smp=1
      - --memory=1G
      - --kafka-addr=internal://0.0.0.0:9092,external://0.0.0.0:19092
      - --advertise-kafka-addr=internal://redpanda:9092,external://localhost:19092
      - --default-log-level=warn
    ports:
      - "19092:19092"   # Kafka API for tools on the host
      - "9644:9644"     # admin API and Prometheus metrics
    volumes:
      - redpanda-data:/var/lib/redpanda/data
    healthcheck:
      test: ["CMD-SHELL", "rpk cluster health | grep -E 'Healthy:.+true'"]
      interval: 5s
      timeout: 5s
      retries: 20

  topics:
    image: redpandadata/redpanda:v24.2.7
    depends_on:
      redpanda:
        condition: service_healthy
    entrypoint: ["bash", "/create-topics.sh"]
    environment:
      TOPIC_PARTITION_RETENTION_BYTES: ${TOPIC_PARTITION_RETENTION_BYTES:-268435456}
    volumes:
      - ./infra/create-topics.sh:/create-topics.sh:ro
    restart: "no"

  console:
    image: redpandadata/console:v2.7.2
    depends_on:
      redpanda:
        condition: service_healthy
    environment:
      KAFKA_BROKERS: redpanda:9092
    ports:
      - "8081:8080"

  gateway:
    build:
      context: .
      target: gateway
    depends_on:
      topics:
        condition: service_completed_successfully
    environment:
      KAFKA_BOOTSTRAP: redpanda:9092
      DEVICE_ID_SECRET: ${DEVICE_ID_SECRET:-local-dev-secret-change-me}
      # A fixed heap: ZGC otherwise grows towards MaxRAMPercentage of the whole host.
      JAVA_OPTS: -Xms1g -Xmx1g -XX:+UseZGC -XX:+ZGenerational
    ports:
      - "8080:8080"

  # --- Ad serving: the ad server runs an OpenRTB auction across three mock DSPs ------------------
  redis:
    image: redis:7.4-alpine
    command: ["redis-server", "--save", "", "--appendonly", "no", "--maxmemory", "256mb", "--maxmemory-policy",
  ↪ "volatile-ttl"]

  dsp-premium: &dsp
    build:
      context: .
      target: mock-dsp
    environment:
      DSP_PROFILE: premium
      PUBLIC_URL: http://dsp-premium:9000
      JAVA_OPTS: -Xmx192m -XX:+UseZGC -XX:+ZGenerational

```

```
  dsp-budget:
    <<: *dsp
    environment:
      DSP_PROFILE: budget
      PUBLIC_URL: http://dsp-budget:9000
      JAVA_OPTS: -Xmx192m -XX:+UseZGC -XX:+ZGenerational

  dsp-slow:
    <<: *dsp
    environment:
      DSP_PROFILE: slow
      PUBLIC_URL: http://dsp-slow:9000
      JAVA_OPTS: -Xmx192m -XX:+UseZGC -XX:+ZGenerational

  ad-server:
    build:
      context: .
      target: ad-server
    depends_on:
      topics:
        condition: service_completed_successfully
      redis:
        condition: service_started
    environment:
      KAFKA_BOOTSTRAP: redpanda:9092
      REDIS_URL: redis://redis:6379
      # Must match the gateway's, so frequency caps and event reporting see the same device ids.
      DEVICE_ID_SECRET: ${DEVICE_ID_SECRET:-local-dev-secret-change-me}
      DSP_ENDPOINTS: premium-dsp=http://dsp-premium:9000/openrtb2/bid,budget-dsp=http://dsp-budget:9000/openrtb2
  ↪ /bid,slow-dsp=http://dsp-slow:9000/openrtb2/bid
      TMAX_MS: ${TMAX_MS:-120}
      FLOOR_CPM: ${FLOOR_CPM:-5.00}
      AUCTION_TYPE: ${AUCTION_TYPE:-first}
      # Budget periods: must match the pacing service.
      PACING_PERIOD_MINUTES: ${PACING_PERIOD_MINUTES:-30}
      BUDGET_SCALE: ${BUDGET_SCALE:-1.0}
      JAVA_OPTS: -Xms512m -Xmx512m -XX:+UseZGC -XX:+ZGenerational
    ports:
      - "8090:8090"

  # Reads delivered spend from Flink's minute totals and line item budgets, and steers the ad
  # server through Redis: a throttle and a stop threshold per line item.
  pacing-service:
    build:
      context: .
      target: pacing-service
    depends_on:
      topics:
        condition: service_completed_successfully
      redis:
        condition: service_started
    environment:
      KAFKA_BOOTSTRAP: redpanda:9092
      REDIS_URL: redis://redis:6379
      PACING_PERIOD_MINUTES: ${PACING_PERIOD_MINUTES:-30}
      JAVA_OPTS: -Xmx256m -XX:+UseZGC -XX:+ZGenerational
    ports:
      - "8095:8095"

  # Continuous reconciliation between independent sources (Phase 5).
  audit-service:
    build:
      context: .
      target: audit-service
    depends_on:
      topics:
        condition: service_completed_successfully
      redis:
        condition: service_started
    environment:
      KAFKA_BOOTSTRAP: redpanda:9092
      REDIS_URL: redis://redis:6379
      DRUID_URL: http://router:8888
      PACING_PERIOD_MINUTES: ${PACING_PERIOD_MINUTES:-30}
      JAVA_OPTS: -Xmx384m -XX:+UseZGC -XX:+ZGenerational
    ports:
      - "8096:8096"

  simulator:
    build:
      context: .
      target: simulator
    depends_on:
      - gateway
      - ad-server
    environment:
      GATEWAY_URL: http://gateway:8080
      # Every ad break asks the ad server for its ads, like a real player.
```

```
      # Empty (SIM_AD_SERVER_URL=) plans ad breaks locally: load tests of the event path alone.
      AD_SERVER_URL: ${SIM_AD_SERVER_URL-http://ad-server:8090}
      TARGET_EPS: ${TARGET_EPS:-2000}
      DURATION_SECONDS: ${DURATION_SECONDS:-0}
      CONCURRENCY: ${CONCURRENCY:-16}
      # Event ids derive from the seed: reusing one replays ids the pipeline will de-duplicate.
      SEED: ${SEED:-42}
      # Holds every active session's plan: ~225k sessions at 50k events/s.
      JAVA_OPTS: -Xms2g -Xmx2g -XX:+UseZGC -XX:+ZGenerational

  # Flink in application mode: the job manager runs PipelineJob from the image's usrlib.
  jobmanager:
    build:
      context: .
      target: stream-processor
    command: standalone-job --job-classname dev.primetech.ads.stream.PipelineJob
    depends_on:
      topics:
        condition: service_completed_successfully
    environment:
      KAFKA_BOOTSTRAP: redpanda:9092
      FLINK_PROPERTIES: &flink-properties |
        jobmanager.rpc.address: jobmanager
        parallelism.default: 4
        state.backend.type: rocksdb
        execution.checkpointing.incremental: true
        execution.checkpointing.dir: file:///flink-data/checkpoints
        execution.checkpointing.savepoint-dir: file:///flink-data/savepoints
        execution.checkpointing.externalized-checkpoint-retention: RETAIN_ON_CANCELLATION
        jobmanager.memory.process.size: 1024m
        taskmanager.numberOfTaskSlots: 4
        taskmanager.memory.process.size: 3072m
        taskmanager.memory.managed.fraction: 0.3
        metrics.reporter.prom.factory.class: org.apache.flink.metrics.prometheus.PrometheusReporterFactory
        metrics.reporter.prom.port: 9249
    ports:
      - "8082:8081"   # Flink web UI and REST API
    volumes:
      - flink-data:/flink-data
    restart: unless-stopped

  taskmanager:
    build:
      context: .
      target: stream-processor
    command: taskmanager
    depends_on:
      - jobmanager
    environment:
      FLINK_PROPERTIES: *flink-properties
    volumes:
      - flink-data:/flink-data
    restart: unless-stopped

  # --- Druid: real-time OLAP over ad-events-clean.v1 -------------------------------------------
  # Kafka ingestion rolls events up to one row per minute and dimension combination.
  postgres:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: druid
      POSTGRES_PASSWORD: druid-local-only
      POSTGRES_DB: druid
    volumes:
      - druid-metadata:/var/lib/postgresql/data

  zookeeper:
    image: zookeeper:3.9.6
    environment:
      ZOO_MY_ID: 1

  coordinator:
    image: apache/druid:37.0.0
    command: coordinator
    depends_on: [postgres, zookeeper]
    env_file: infra/druid/druid.env
    environment:
      DRUID_XMX: 384m
      DRUID_XMS: 384m
    volumes:
      - druid-shared:/opt/shared

  broker:
    image: apache/druid:37.0.0
    command: broker
    depends_on: [coordinator]
    env_file: infra/druid/druid.env
    environment:
      DRUID_XMX: 512m
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
252       DRUID_XMS: 512m
253       DRUID_MAXDIRECTMEMORYSIZE: 384m
254
255   historical:
256     image: apache/druid:37.0.0
257     command: historical
258     depends_on: [coordinator]
259     env_file: infra/druid/druid.env
260     environment:
261       DRUID_XMX: 512m
262       DRUID_XMS: 512m
263       DRUID_MAXDIRECTMEMORYSIZE: 384m
264     volumes:
265       - druid-shared:/opt/shared
266
267   middlemanager:
268     image: apache/druid:37.0.0
269     command: middleManager
270     depends_on: [coordinator]
271     env_file: infra/druid/druid.env
272     environment:
273       DRUID_XMX: 128m
274       DRUID_XMS: 128m
275     volumes:
276       - druid-shared:/opt/shared
277
278   router:
279     image: apache/druid:37.0.0
280     command: router
281     depends_on: [coordinator]
282     env_file: infra/druid/druid.env
283     environment:
284       DRUID_XMX: 128m
285       DRUID_XMS: 128m
286     ports:
287       - "8888:8888"   # Druid console and SQL API
288
289   druid-setup:
290     image: apache/druid:37.0.0
291     depends_on:
292       router:
293         condition: service_started
294       topics:
295         condition: service_completed_successfully
296     entrypoint: ["sh", "/specs/submit-supervisor.sh"]
297     volumes:
298       - ./infra/druid:/specs:ro
299     restart: "no"
300
301   dashboard:
302     build:
303       context: ./dashboard
304     environment:
305       # Server-side routes reach these over the Compose network; the browser only talks to the dashboard.
306       NUXT_GATEWAY_URL: http://gateway:8080
307       NUXT_FLINK_URL: http://jobmanager:8081
308       NUXT_DRUID_URL: http://router:8888
309       NUXT_AD_SERVER_URL: http://ad-server:8090
310       NUXT_PACING_URL: http://pacing-service:8095
311       NUXT_AUDIT_URL: http://audit-service:8096
312       NUXT_ALERTMANAGER_URL: http://alertmanager:9093
313       NUXT_ALERT_WEBHOOK_TOKEN: ${ALERT_WEBHOOK_TOKEN:-local-dev-alert-token-change-me}
314       # Alert notifications on installed apps (optional): `npx web-push generate-vapid-keys`.
315       NUXT_VAPID_PUBLIC_KEY: ${VAPID_PUBLIC_KEY:-}
316       NUXT_VAPID_PRIVATE_KEY: ${VAPID_PRIVATE_KEY:-}
317       NUXT_VAPID_SUBJECT: ${VAPID_SUBJECT:-}
318       # Sign-in is off for local development. deploy/docker-compose.prod.yml turns it on; to try it
319       # locally, set DASHBOARD_AUTH_DISABLED=false plus DASHBOARD_USERS and DASHBOARD_SESSION_SECRET.
320       NUXT_AUTH_DISABLED: ${DASHBOARD_AUTH_DISABLED:-true}
321       NUXT_AUTH_USERS: ${DASHBOARD_USERS:-}
322       NUXT_SESSION_SECRET: ${DASHBOARD_SESSION_SECRET:-}
323     volumes:
324       - dashboard-data:/app/.data
325     ports:
326       - "3000:3000"
327
328   # Observability (Phase 5): metrics, alerting rules, notifications and dashboards.
329   prometheus:
330     image: prom/prometheus:v3.5.0
331     command:
332       - --config.file=/etc/prometheus/prometheus.yml
333       - --storage.tsdb.retention.time=7d
334       - --web.enable-lifecycle
335     volumes:
336       - ./infra/prometheus:/etc/prometheus:ro
337       - prometheus-data:/prometheus
338     ports:
339       - "9090:9090"
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```

  alertmanager:
    image: prom/alertmanager:v0.28.1
    # The webhook token is written to a file at start: Alertmanager reads credentials from files,
    # not environment variables.
    entrypoint: ["/bin/sh", "-c"]
    command:
      - printf %s "$$ALERT_WEBHOOK_TOKEN" > /tmp/webhook-token &&
        exec /bin/alertmanager --config.file=/etc/alertmanager/alertmanager.yml --storage.path=/alertmanager
    environment:
      ALERT_WEBHOOK_TOKEN: ${ALERT_WEBHOOK_TOKEN:-local-dev-alert-token-change-me}
    volumes:
      - ./infra/alertmanager:/etc/alertmanager:ro
    ports:
      - "9093:9093"

  grafana:
    image: grafana/grafana:12.1.1
    depends_on: [prometheus]
    environment:
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD:-admin}
      # Local convenience: dashboards are viewable without signing in.
      GF_AUTH_ANONYMOUS_ENABLED: ${GRAFANA_ANONYMOUS:-true}
      GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
      GF_USERS_DEFAULT_THEME: system
      GF_ANALYTICS_REPORTING_ENABLED: "false"
      GF_ANALYTICS_CHECK_FOR_UPDATES: "false"
    volumes:
      - ./infra/grafana/provisioning:/etc/grafana/provisioning:ro
      - ./infra/grafana/dashboards:/var/lib/grafana/dashboards:ro
    ports:
      - "3001:3000"

volumes:
  dashboard-data:
  prometheus-data:
  redpanda-data:
  flink-data:
  druid-shared:
  druid-metadata:
```

`pom.xml`

156 lines · Maven parent build: modules and pinned dependency versions.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>dev.primetech.ads</groupId>
  <artifactId>ads-eventing</artifactId>
  <version>0.1.0-SNAPSHOT</version>
  <packaging>pom</packaging>
  <name>Ads Eventing Platform</name>

  <modules>
    <module>ads-common</module>
    <module>ingest-gateway</module>
    <module>event-simulator</module>
    <module>stream-processor</module>
    <module>openrtb</module>
    <module>mock-dsp</module>
    <module>ad-server</module>
    <module>pacing-service</module>
    <module>audit-service</module>
  </modules>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

    <kafka.version>3.8.0</kafka.version>
    <jackson.version>2.17.2</jackson.version>
    <micrometer.version>1.13.4</micrometer.version>
    <slf4j.version>2.0.16</slf4j.version>
    <jedis.version>5.2.0</jedis.version>
    <junit.version>5.11.0</junit.version>
    <assertj.version>3.26.3</assertj.version>
  </properties>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>dev.primetech.ads</groupId>
        <artifactId>ads-common</artifactId>
        <version>${project.version}</version>
      </dependency>
```

```
      <dependency>
        <groupId>dev.primetech.ads</groupId>
        <artifactId>openrtb</artifactId>
        <version>${project.version}</version>
      </dependency>
      <dependency>
        <groupId>redis.clients</groupId>
        <artifactId>jedis</artifactId>
        <version>${jedis.version}</version>
      </dependency>
      <dependency>
        <groupId>com.fasterxml.jackson</groupId>
        <artifactId>jackson-bom</artifactId>
        <version>${jackson.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
      <dependency>
        <groupId>org.apache.kafka</groupId>
        <artifactId>kafka-clients</artifactId>
        <version>${kafka.version}</version>
      </dependency>
      <dependency>
        <groupId>io.micrometer</groupId>
        <artifactId>micrometer-registry-prometheus</artifactId>
        <version>${micrometer.version}</version>
      </dependency>
      <dependency>
        <!-- kafka-clients brings slf4j-api 1.7, which cannot see slf4j-simple 2.x and drops all logs. -->
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-api</artifactId>
        <version>${slf4j.version}</version>
      </dependency>
      <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>${slf4j.version}</version>
      </dependency>
      <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>${junit.version}</version>
      </dependency>
      <dependency>
        <groupId>org.assertj</groupId>
        <artifactId>assertj-core</artifactId>
        <version>${assertj.version}</version>
      </dependency>
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.assertj</groupId>
      <artifactId>assertj-core</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <pluginManagement>
      <plugins>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-surefire-plugin</artifactId>
          <version>3.5.0</version>
        </plugin>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
        </plugin>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-shade-plugin</artifactId>
          <version>3.6.0</version>
          <executions>
            <execution>
              <phase>package</phase>
              <goals><goal>shade</goal></goals>
              <configuration>
                <createDependencyReducedPom>false</createDependencyReducedPom>
                <transformers>
```

```
                  <transformer implementation="org.apache.maven.plugins.shade.resource.ServicesResourceTransform
  ↪ er"/>
                  <transformer implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransform
  ↪ er">
                    <mainClass>${main.class}</mainClass>
                  </transformer>
                </transformers>
                <filters>
                  <filter>
                    <artifact>*:*</artifact>
                    <excludes>
                      <exclude>META-INF/*.SF</exclude>
                      <exclude>META-INF/*.DSA</exclude>
                      <exclude>META-INF/*.RSA</exclude>
                      <exclude>module-info.class</exclude>
                    </excludes>
                  </filter>
                </filters>
              </configuration>
            </execution>
          </executions>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```

.github

.github/workflows/ci.yml

71 lines

```yaml
name: CI

on:
  push:
    branches: ["main"]
  pull_request:

permissions:
  contents: read

jobs:
  java:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: Build and test
        run: mvn -B verify

  dashboard:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: dashboard
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
          cache-dependency-path: dashboard/package-lock.json
      - run: npm ci
      - run: npx nuxt typecheck
      - run: npm test
      - run: npm run build
      - run: npm run generate

  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build all images
        run: docker compose build
      - name: Validate the production overlay
        run: >
          DASHBOARD_DOMAIN=ci.example DASHBOARD_USERS=ci=x DASHBOARD_SESSION_SECRET=x
          ALERT_WEBHOOK_TOKEN=x GRAFANA_ADMIN_PASSWORD=x
          docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml config --quiet

  observability:
```

```
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Prometheus config and alert rules
        run: >
          docker run --rm -v "$PWD/infra/prometheus:/etc/prometheus:ro" --entrypoint promtool
          prom/prometheus:v3.5.0 check config /etc/prometheus/prometheus.yml
      - name: Alert rule unit tests
        run: >
          docker run --rm -v "$PWD/infra/prometheus:/etc/prometheus:ro" -w /etc/prometheus/tests --entrypoint pr
↪ omtool
          prom/prometheus:v3.5.0 test rules ads-eventing-test.yml
      - name: Load test scripts compile
        run: python3 -m py_compile loadtest/run.py loadtest/report.py
      - name: Alertmanager config
        run: >
          docker run --rm -v "$PWD/infra/alertmanager:/etc/alertmanager:ro" --entrypoint amtool
          prom/alertmanager:v0.28.1 check-config /etc/alertmanager/alertmanager.yml
```

`.github/workflows/nuxtjs.yml`

63 lines · Builds the dashboard in dashboard/ as a static site and deploys it to GitHub Pages.

```
# Builds the dashboard in dashboard/ as a static site and deploys it to GitHub Pages.
# The hosted page polls whichever gateway you point it at: ?gateway=http://localhost:8080 by default.
# Requires Settings -> Pages -> Source: "GitHub Actions".
name: Deploy dashboard to Pages

on:
  push:
    branches: ["main"]
    paths: ["dashboard/**", ".github/workflows/nuxtjs.yml"]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

# Allow only one concurrent deployment, skipping runs queued between the run in-progress and latest queued.
concurrency:
  group: "pages"
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: dashboard
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: "22"
          cache: npm
          cache-dependency-path: dashboard/package-lock.json
      - name: Setup Pages
        id: pages
        uses: actions/configure-pages@v5
      - name: Install dependencies
        run: npm ci
      - name: Static HTML export with Nuxt
        run: npm run generate
        env:
          # Project pages are served under /<repo>/.
          NUXT_APP_BASE_URL: ${{ steps.pages.outputs.base_path }}/
          # No server on Pages: the browser calls a gateway itself (?gateway=<url>).
          NUXT_PUBLIC_GATEWAY_MODE: direct
      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: ./dashboard/.output/public

  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    needs: build
    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v5
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

ad-server

ad-server/pom.xml

59 lines

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.primetech.ads</groupId>
    <artifactId>ads-eventing</artifactId>
    <version>0.1.0-SNAPSHOT</version>
  </parent>

  <artifactId>ad-server</artifactId>
  <name>Ad Server (decisioning and OpenRTB auctions)</name>

  <properties>
    <main.class>dev.primetech.ads.adserver.AdServerApp</main.class>
  </properties>

  <dependencies>
    <dependency>
      <groupId>dev.primetech.ads</groupId>
      <artifactId>openrtb</artifactId>
    </dependency>
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
    </dependency>
    <dependency>
      <groupId>redis.clients</groupId>
      <artifactId>jedis</artifactId>
    </dependency>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
    </dependency>

    <!-- Tests run real auctions against in-process mock DSPs. -->
    <dependency>
      <groupId>dev.primetech.ads</groupId>
      <artifactId>mock-dsp</artifactId>
      <version>${project.version}</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <finalName>ad-server</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-shade-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

ad-server/src/main/java/dev/primetech/ads/adserver/AdDecisionService.java

250 lines · Answers one ad request: pseudonymise the device id (same keyed hash as the event pipeline, so caps and reporting agree); send one OpenRTB request to every DSP in parallel, with a hard deadline; meanwhile, filter direct-sold line items by targeting; drop candidates over their frequency cap (one Redis read for all of them); run the pod auction, record caps, notify winning DSPs, and log the decision.

```java
package dev.primetech.ads.adserver;

import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.DeviceIdHasher;
import dev.primetech.ads.common.adserving.AdDecision;
import dev.primetech.ads.common.adserving.AdRequest;
import dev.primetech.ads.common.adserving.AdResponse;
import dev.primetech.ads.openrtb.BidRequest;
import dev.primetech.ads.openrtb.BidResponse;
import dev.primetech.ads.openrtb.OpenRtb;

import java.time.Duration;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
```

```
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.UUID;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ThreadLocalRandom;

/**
 * Answers one ad request:
 * <ol>
 *   <li>pseudonymise the device id (same keyed hash as the event pipeline, so caps and reporting agree);</li>
 *   <li>send one OpenRTB request to every DSP in parallel, with a hard deadline;</li>
 *   <li>meanwhile, filter direct-sold line items by targeting;</li>
 *   <li>drop candidates over their frequency cap (one Redis read for all of them);</li>
 *   <li>run the pod auction, record caps, notify winning DSPs, and log the decision.</li>
 * </ol>
 * A failing DSP, Redis or Kafka never fails the request: each degrades to fewer candidates,
 * no capping, or a missing log record, and is counted.
 */
public final class AdDecisionService {
    public record Settings(int maxSlots, Duration tmax, long floorCpmMicros, int auctionType, int dspFrequencyCa
↪ pPerHour) {
    }

    private static final BidRequest.App APP = new BidRequest.App("primetech-ctv", "PrimeTech Streaming",
            "dev.primetech.tv", new BidRequest.Publisher("primetech", "PrimeTech"));

    private final Settings settings;
    private final DirectDemand direct;
    private final List<DspClient> dsps;
    private final FrequencyCaps caps;
    private final DecisionSink sink;
    private final DeviceIdHasher hasher;
    private final AdServerMetrics metrics;
    private final Set<String> announcedCampaigns = ConcurrentHashMap.newKeySet();

    public AdDecisionService(Settings settings, DirectDemand direct, List<DspClient> dsps, FrequencyCaps caps,
                             DecisionSink sink, DeviceIdHasher hasher, AdServerMetrics metrics) {
        this.settings = settings;
        this.direct = direct;
        this.dsps = dsps;
        this.caps = caps;
        this.sink = sink;
        this.hasher = hasher;
        this.metrics = metrics;
    }

    /**
     * Announces campaigns so the stream processor can enrich their events. A campaign counts as
     * announced only once the write succeeds; a failed one is forgotten and retried on its next win.
     */
    public CompletableFuture<Void> announce(List<CampaignMetadata> campaigns) {
        var writes = new ArrayList<CompletableFuture<Void>>();
        for (CampaignMetadata c : campaigns) {
            if (announcedCampaigns.add(c.campaignId())) {
                writes.add(sink.campaign(c).whenComplete((ok, error) -> {
                    if (error != null) {
                        announcedCampaigns.remove(c.campaignId());
                    }
                }));
            }
        }
        return CompletableFuture.allOf(writes.toArray(CompletableFuture[]::new));
    }

    /** Checks a request; returns a problem description or null if it is acceptable. */
    public String validate(AdRequest r) {
        if (r.sessionId() == null || r.sessionId().isBlank() || r.sessionId().length() > 64) {
            return "sessionId is required (at most 64 characters)";
        }
        if (r.deviceType() == null) {
            return "deviceType is required";
        }
        if (r.country() == null || !r.country().matches("[A-Z]{2}")) {
            return "country must be an ISO 3166-1 alpha-2 code";
        }
        if (r.consent() == null) {
            return "consent is required";
        }
        if (r.slots() != null && (r.slots() < 1 || r.slots() > settings.maxSlots())) {
            return "slots must be between 1 and " + settings.maxSlots();
        }
        if (r.maxAdSeconds() != null && r.maxAdSeconds() < 5) {
            return "maxAdSeconds must be at least 5";
        }
        return null;
    }
```

```
103
104     public AdResponse decide(AdRequest r) {
105         long start = System.nanoTime();
106         String requestId = UUID.randomUUID().toString();
107         int slots = r.slots() == null ? 3 : r.slots();
108         int maxAdSeconds = r.maxAdSeconds() == null ? 30 : r.maxAdSeconds();
109         int podSeconds = r.podSeconds() == null ? slots * maxAdSeconds : r.podSeconds();
110         String deviceIdHash = r.consent() && r.deviceId() != null && !r.deviceId().isBlank()
111                 ? hasher.hash(r.deviceId()) : null;
112
113         // 1. Programmatic demand, in parallel and bounded by tmax.
114         byte[] bidRequest = DspClient.serialize(bidRequest(requestId, r, deviceIdHash, slots, maxAdSeconds, podS
    ↪ econds));
115         List<CompletableFuture<DspClient.Result>> calls = dsps.stream()
116                 .map(d -> d.bid(bidRequest, settings.tmax())).toList();
117
118         // 2. Direct demand while the DSPs think.
119         List<Candidate> directCandidates = direct.eligible(r.country(), r.deviceType(), ThreadLocalRandom.curren
    ↪ t());
120
121         List<DspClient.Result> results = calls.stream().map(CompletableFuture::join).toList();
122         Map<Integer, List<Candidate>> dspCandidates = new HashMap<>();
123         for (DspClient.Result result : results) {
124             metrics.dsp(result);
125             for (BidResponse.Bid bid : result.bids()) {
126                 Candidate c = fromBid(result.dsp(), bid);
127                 if (c != null) {
128                     dspCandidates.computeIfAbsent(Integer.parseInt(c.impId()), k -> new ArrayList<>()).add(c);
129                 }
130             }
131         }
132
133         // 3. Frequency caps, one read for every campaign in play.
134         Set<String> cappedCampaigns = new HashSet<>();
135         int cappedCandidates = 0;
136         if (deviceIdHash != null) {
137             Set<String> campaignIds = new HashSet<>();
138             directCandidates.forEach(c -> campaignIds.add(c.campaignId()));
139             dspCandidates.values().forEach(list -> list.forEach(c -> campaignIds.add(c.campaignId())));
140             Map<String, Integer> limits = new HashMap<>();
141             directCandidates.forEach(c -> limits.merge(c.campaignId(), c.frequencyCapPerHour(), Math::min));
142             dspCandidates.values().forEach(list -> list.forEach(c -> limits.merge(c.campaignId(), c.frequencyCap
    ↪ PerHour(), Math::min)));
143             try {
144                 caps.counts(deviceIdHash, campaignIds).forEach((campaign, seen) -> {
145                     if (seen >= limits.getOrDefault(campaign, Integer.MAX_VALUE)) {
146                         cappedCampaigns.add(campaign);
147                     }
148                 });
149             } catch (Exception e) {
150                 // Fail open: serving an ad a little too often beats serving nothing.
151                 metrics.capStoreError();
152             }
153             for (Candidate c : directCandidates) {
154                 if (cappedCampaigns.contains(c.campaignId())) {
155                     cappedCandidates++;
156                 }
157             }
158             for (List<Candidate> list : dspCandidates.values()) {
159                 for (Candidate c : list) {
160                     if (cappedCampaigns.contains(c.campaignId())) {
161                         cappedCandidates++;
162                     }
163                 }
164             }
165         }
166         List<Candidate> directOk = directCandidates.stream().filter(c -> !cappedCampaigns.contains(c.campaignId(
    ↪ ))).toList();
167
168         // 4. The auction.
169         List<Auction.Winner> winners = Auction.run(slots, maxAdSeconds, podSeconds, position -> {
170             List<Candidate> all = new ArrayList<>(directOk);
171             for (Candidate c : dspCandidates.getOrDefault(position, List.of())) {
172                 if (!cappedCampaigns.contains(c.campaignId())) {
173                     all.add(c);
174                 }
175             }
176             return all;
177         }, settings.auctionType(), settings.floorCpmMicros());
178
179         // 5. Side effects of winning.
180         var served = new ArrayList<AdResponse.ServedAd>();
181         var slotLog = new ArrayList<AdDecision.Slot>();
182         List<String> wonCampaigns = new ArrayList<>();
183         for (Auction.Winner w : winners) {
184             Candidate c = w.candidate();
185             long priceMicros = w.clearingCpmMicros() / 1000;
186             served.add(new AdResponse.ServedAd(served.size() + 1, requestId, c.adId(), c.creativeId(), c.campaig
```

```
    ↪ nId(),
187                     c.lineItemId(), c.advertiserId(), c.durationSeconds(), priceMicros, c.source()));
188             slotLog.add(new AdDecision.Slot(w.position(), c.source(), c.campaignId(), c.lineItemId(), c.advertis
    ↪ erId(),
189                     c.durationSeconds(), c.bidCpmMicros(), w.clearingCpmMicros(), w.eligibleCandidates()));
190             wonCampaigns.add(c.campaignId());
191             metrics.win(c.source(), priceMicros);
192             if (c.isDirect()) {
193                 direct.served(c.lineItemId(), priceMicros);
194             }
195             if (!c.isDirect()) {
196                 // DSPs only tell us the campaign id and advertiser domain; name the campaign by its id.
197                 String dspCampaign = c.campaignId().substring(c.campaignId().indexOf(':') + 1);
198                 announce(List.of(new CampaignMetadata(c.campaignId(), dspCampaign + " via " + c.source(),
199                         c.advertiserId(), c.advertiserId())));
200                 if (c.nurl() != null) {
201                     dsps.stream().filter(d -> d.name().equals(c.source())).findFirst()
202                             .ifPresent(d -> d.notify(OpenRtb.expandMacros(c.nurl(), requestId, w.clearingCpmMicr
    ↪ os())));
203                 }
204             }
205         }
206         if (deviceIdHash != null && !wonCampaigns.isEmpty()) {
207             try {
208                 caps.record(deviceIdHash, wonCampaigns);
209             } catch (Exception e) {
210                 metrics.capStoreError();
211             }
212         }
213
214         long latencyNanos = System.nanoTime() - start;
215         metrics.request(slots, served.size(), latencyNanos, cappedCandidates);
216         sink.decision(new AdDecision(requestId, System.currentTimeMillis(), r.sessionId(), r.deviceType(), r.cou
    ↪ ntry(),
217                 r.consent(), slots, served.size(), settings.auctionType(), settings.floorCpmMicros(),
218                 latencyNanos / 1_000_000, directCandidates.size(), cappedCandidates, slotLog,
219                 results.stream().map(x -> new AdDecision.Dsp(x.dsp(), x.status().name(), x.bids().size(), x.late
    ↪ ncyMs())).toList()));
220         return new AdResponse(requestId, served);
221     }
222
223     private BidRequest bidRequest(String requestId, AdRequest r, String deviceIdHash, int slots, int maxAdSecond
    ↪ s,
224                                   int podSeconds) {
225         var imps = new ArrayList<BidRequest.Imp>();
226         for (int i = 1; i <= slots; i++) {
227             int slotInPod = i == 1 ? 1 : (i == slots ? -1 : 0);
228             imps.add(new BidRequest.Imp(String.valueOf(i),
229                     new BidRequest.Video(List.of("video/mp4"), 5, maxAdSeconds, OpenRtb.VAST_PROTOCOLS, 1920, 10
    ↪ 80,
230                             OpenRtb.PLACEMENT_INSTREAM, requestId, slotInPod, 0, podSeconds),
231                     OpenRtb.microsToCpm(settings.floorCpmMicros()), "USD", 1));
232         }
233         var device = new BidRequest.Device(OpenRtb.deviceType(r.deviceType()), new BidRequest.Geo(OpenRtb.alpha3
    ↪ (r.country())),
234                 deviceIdHash, deviceIdHash == null ? 1 : 0);
235         return new BidRequest(requestId, imps, APP, device, settings.auctionType(), (int) settings.tmax().toMill
    ↪ is(),
236                 List.of("USD"), new BidRequest.Source(requestId));
237     }
238
239     /** Maps a bid onto our ids; bids we cannot use (bad imp id, no advertiser, no duration) are ignored. */
240     private Candidate fromBid(String dsp, BidResponse.Bid bid) {
241         if (bid.impid() == null || !bid.impid().matches("\\d+") || bid.adomain() == null || bid.adomain().isEmpt
    ↪ y()
242                 || bid.cid() == null || bid.crid() == null || bid.dur() == null || bid.price() <= 0) {
243             return null;
244         }
245         String campaignId = dsp + ":" + bid.cid();
246         String creativeId = dsp + ":" + bid.crid();
247         return new Candidate(dsp, bid.impid(), campaignId, campaignId, creativeId, creativeId, bid.adomain().get
    ↪ (0),
248                 bid.dur(), OpenRtb.cpmToMicros(bid.price()), settings.dspFrequencyCapPerHour(), bid.nurl(), fals
    ↪ e);
249     }
250 }
```

ad-server/src/main/java/dev/primetech/ads/adserver/AdServerApp.java

89 lines · Entry point: wires the ad server (DSP clients, Redis, Kafka, pacing view, HTTP).

```
1  package dev.primetech.ads.adserver;
2
3  import dev.primetech.ads.common.DeviceIdHasher;
4  import dev.primetech.ads.common.catalog.AdCatalog;
5  import dev.primetech.ads.common.catalog.LineItemConfig;
6  import io.micrometer.core.instrument.binder.jvm.JvmGcMetrics;
7  import io.micrometer.core.instrument.binder.jvm.JvmMemoryMetrics;
8  import io.micrometer.prometheusmetrics.PrometheusConfig;
```

```
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import dev.primetech.ads.common.pacing.PacingPeriod;

import java.time.Duration;
import java.util.ArrayList;
import java.util.SplittableRandom;
import java.util.concurrent.CompletableFuture;

public final class AdServerApp {
    private static final Logger log = LoggerFactory.getLogger(AdServerApp.class);

    public static void main(String[] args) throws Exception {
        AdServerConfig config = AdServerConfig.fromEnv(System.getenv());

        var registry = new PrometheusMeterRegistry(PrometheusConfig.DEFAULT);
        registry.config().commonTags("service", "ad-server");
        new JvmMemoryMetrics().bindTo(registry);
        new JvmGcMetrics().bindTo(registry);

        var http = DspClient.sharedHttpClient();
        var dsps = new ArrayList<DspClient>();
        config.dsps().forEach((name, uri) -> dsps.add(new DspClient(name, uri, http)));
        var metrics = new AdServerMetrics(registry, new ArrayList<>(config.dsps().keySet()));

        FrequencyCaps caps;
        if (config.redisUrl() == null || config.redisUrl().isBlank()) {
            log.warn("REDIS_URL not set: frequency caps are per process");
            caps = new InMemoryFrequencyCaps();
        } else {
            caps = new RedisFrequencyCaps(config.redisUrl());
        }
        var sink = new KafkaDecisionSink(config.kafkaBootstrap(), registry);
        var catalog = AdCatalog.generate(config.campaigns(), new SplittableRandom(config.seed()));
        // Budgets as the pacing service and this server will both see them.
        var lineItems = catalog.lineItems().stream()
                .map(l -> new LineItemConfig(l.lineItemId(), l.campaignId(), l.advertiserId(), l.tier(), l.cpmMi
    ↪ cros(),
                        Math.round(l.periodBudgetMicros() * config.budgetScale())))
                .toList();
        PacingView pacing;
        if (config.redisUrl() == null || config.redisUrl().isBlank()) {
            log.warn("REDIS_URL not set: no budgets, direct line items at fixed participation");
            pacing = PacingView.fixed(config.directParticipation());
        } else {
            pacing = new RedisPacingView(config.redisUrl(), PacingPeriod.of(Duration.ofMinutes(config.pacingPeri
    ↪ odMinutes())),
                    lineItems, config.directParticipation(), System::currentTimeMillis, registry);
        }
        if (pacing instanceof RedisPacingView view) {
            metrics.pacingSummary(view::summary);
        }
        var direct = new DirectDemand(catalog, pacing);
        var service = new AdDecisionService(config.settings(), direct, dsps, caps, sink,
                new DeviceIdHasher(config.deviceIdSecret()), metrics);
        // Enrichment depends on the catalog being in Kafka: do not serve until it is.
        service.announce(catalog.campaigns()).get(60, java.util.concurrent.TimeUnit.SECONDS);
        // The pacing service learns budgets from here.
        CompletableFuture.allOf(lineItems.stream().map(sink::lineItem).toArray(CompletableFuture[]::new))
                .get(60, java.util.concurrent.TimeUnit.SECONDS);

        var server = new AdServerHttp(config.port(), service, metrics, registry, config.corsOrigin(),
                config.maxInFlight());
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            server.close();
            sink.close();
            if (caps instanceof AutoCloseable c) {
                try {
                    c.close();
                } catch (Exception ignored) {
                    // shutting down
                }
            }
        }, "shutdown"));
        server.start();
        log.info("{} direct campaigns, DSPs {}, {}", catalog.campaigns().size(), config.dsps().keySet(), config.
    ↪ settings());
    }

    private AdServerApp() {
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/AdServerConfig.java

74 lines · Ad server settings read from environment variables.

```java
package dev.primetech.ads.adserver;

import dev.primetech.ads.openrtb.BidRequest;
import dev.primetech.ads.openrtb.OpenRtb;

import java.net.URI;
import java.time.Duration;
import java.util.LinkedHashMap;
import java.util.Map;

public record AdServerConfig(
        int port,
        String kafkaBootstrap,
        String redisUrl,
        String deviceIdSecret,
        Map<String, URI> dsps,
        AdDecisionService.Settings settings,
        int campaigns,
        double directParticipation,
        long pacingPeriodMinutes,
        double budgetScale,
        int maxInFlight,
        long seed,
        String corsOrigin) {

    public static AdServerConfig fromEnv(Map<String, String> env) {
        String auction = env.getOrDefault("AUCTION_TYPE", "first");
        if (!auction.equals("first") && !auction.equals("second")) {
            throw new IllegalArgumentException("AUCTION_TYPE must be first or second");
        }
        String secret = env.get("DEVICE_ID_SECRET");
        if (secret == null || secret.isBlank()) {
            throw new IllegalStateException("DEVICE_ID_SECRET must be set (the same value as the gateway's)");
        }
        return new AdServerConfig(
                Integer.parseInt(env.getOrDefault("PORT", "8090")),
                env.getOrDefault("KAFKA_BOOTSTRAP", "localhost:19092"),
                // Unset runs with in-process caps: fine for one instance, wrong for several.
                env.get("REDIS_URL"),
                secret,
                parseDsps(env.getOrDefault("DSP_ENDPOINTS", "")),
                new AdDecisionService.Settings(
                        Integer.parseInt(env.getOrDefault("MAX_SLOTS", "5")),
                        Duration.ofMillis(Long.parseLong(env.getOrDefault("TMAX_MS", "120"))),
                        OpenRtb.cpmToMicros(Double.parseDouble(env.getOrDefault("FLOOR_CPM", "5.00"))),
                        auction.equals("first") ? BidRequest.FIRST_PRICE : BidRequest.SECOND_PRICE,
                        Integer.parseInt(env.getOrDefault("DSP_FREQUENCY_CAP_PER_HOUR", "4"))),
                Integer.parseInt(env.getOrDefault("CAMPAIGNS", "50")),
                // Without pacing state (no Redis, or the pacing service is down): fixed participation.
                Double.parseDouble(env.getOrDefault("DIRECT_PARTICIPATION", "0.015")),
                Long.parseLong(env.getOrDefault("PACING_PERIOD_MINUTES", "30")),
                // Catalog budgets are sized for a 30-minute period at ~2,000 events/s; scale to taste.
                Double.parseDouble(env.getOrDefault("BUDGET_SCALE", "1.0")),
                Integer.parseInt(env.getOrDefault("MAX_IN_FLIGHT", "256")),
                Long.parseLong(env.getOrDefault("SEED", "42")),
                env.getOrDefault("CORS_ORIGIN", "*"));
    }

    /** {@code name=url,name=url}, e.g. {@code premium-dsp=http://dsp-premium:9000/openrtb2/bid}. */
    static Map<String, URI> parseDsps(String value) {
        var out = new LinkedHashMap<String, URI>();
        for (String entry : value.split(",")) {
            if (entry.isBlank()) {
                continue;
            }
            int eq = entry.indexOf('=');
            if (eq <= 0) {
                throw new IllegalArgumentException("DSP_ENDPOINTS entries must be name=url: " + entry);
            }
            out.put(entry.substring(0, eq).trim(), URI.create(entry.substring(eq + 1).trim()));
        }
        return out;
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/AdServerHttp.java

127 lines · HTTP front of the ad server, on virtual threads like the gateway.

```java
package dev.primetech.ads.adserver;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.adserving.AdRequest;
import dev.primetech.ads.common.http.JsonHttp;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;
import org.slf4j.Logger;
```

```java
import org.slf4j.LoggerFactory;

import java.io.IOException;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.Map;
import java.util.concurrent.Executors;
import java.util.concurrent.Semaphore;

/**
 * HTTP front of the ad server, on virtual threads like the gateway.
 *
 * <p>Admission control: at most {@code maxInFlight} decisions run at once; beyond that a request
 * gets 503 straight away. Without it, an overloaded server keeps accepting work it can no longer
 * finish before clients give up, and abandoned connections pile up until it runs out of file
 * descriptors and stops answering altogether (this happened in testing). Shedding turns overload
 * into unfilled breaks, which players already handle.
 */
public final class AdServerHttp implements AutoCloseable {
    private static final Logger log = LoggerFactory.getLogger(AdServerHttp.class);

    private final HttpServer server;
    private final AdDecisionService service;
    private final AdServerMetrics metrics;
    private final String corsOrigin;
    private final Semaphore inFlight;

    public AdServerHttp(int port, AdDecisionService service, AdServerMetrics metrics, PrometheusMeterRegistry re
↪ gistry,
                        String corsOrigin, int maxInFlight) throws IOException {
        this.service = service;
        this.metrics = metrics;
        this.corsOrigin = corsOrigin;
        this.inFlight = new Semaphore(maxInFlight);
        server = HttpServer.create(new InetSocketAddress(port), 2048);
        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        server.createContext("/v1/ad-requests", this::handleAdRequest);
        server.createContext("/stats", ex -> get(ex, () -> JsonHttp.json(ex, 200, metrics.snapshot())));
        server.createContext("/metrics", ex -> get(ex, () -> JsonHttp.send(ex, 200, "text/plain; version=0.0.4",
                registry.scrape().getBytes(StandardCharsets.UTF_8))));
        server.createContext("/health", ex -> get(ex, () -> JsonHttp.json(ex, 200, Map.of("status", "UP"))));
    }

    public void start() {
        server.start();
        log.info("ad server listening on :{}", port());
    }

    public int port() {
        return server.getAddress().getPort();
    }

    @Override
    public void close() {
        server.stop(2);
    }

    private void handleAdRequest(HttpExchange ex) throws IOException {
        try (ex) {
            JsonHttp.cors(ex, corsOrigin);
            if (ex.getRequestMethod().equals("OPTIONS")) {
                JsonHttp.noContent(ex);
                return;
            }
            if (!ex.getRequestMethod().equals("POST")) {
                JsonHttp.error(ex, 405, "use POST");
                return;
            }
            byte[] body = JsonHttp.readLimited(ex.getRequestBody(), 16 * 1024);
            if (body == null) {
                JsonHttp.error(ex, 413, "request too large");
                return;
            }
            AdRequest request;
            try {
                request = Json.MAPPER.readValue(body, AdRequest.class);
            } catch (JsonProcessingException e) {
                JsonHttp.error(ex, 400, "malformed JSON");
                return;
            }
            String problem = service.validate(request);
            if (problem != null) {
                JsonHttp.error(ex, 400, problem);
                return;
            }
            if (!inFlight.tryAcquire()) {
                metrics.shed();
                ex.getResponseHeaders().set("Retry-After", "1");
```

```
                JsonHttp.error(ex, 503, "overloaded");
                return;
            }
            try {
                JsonHttp.json(ex, 200, service.decide(request));
            } finally {
                inFlight.release();
            }
        } catch (RuntimeException e) {
            log.error("unexpected error handling ad request", e);
            JsonHttp.error(ex, 500, "internal error");
        }
    }

    private void get(HttpExchange ex, IoAction action) throws IOException {
        try (ex) {
            JsonHttp.cors(ex, corsOrigin);
            if (!ex.getRequestMethod().equals("GET")) {
                JsonHttp.error(ex, 405, "use GET");
                return;
            }
            action.run();
        }
    }

    @FunctionalInterface
    private interface IoAction {
        void run() throws IOException;
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/AdServerMetrics.java

143 lines · Ad server counters, exposed to Prometheus and summarised on /stats for the dashboard.

```
package dev.primetech.ads.adserver;

import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Timer;
import io.micrometer.core.instrument.distribution.ValueAtPercentile;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.TimeUnit;

/** Ad server counters, exposed to Prometheus and summarised on {@code /stats} for the dashboard. */
public final class AdServerMetrics {
    private final MeterRegistry registry;
    private final Counter requests;
    private final Counter slotsRequested;
    private final Counter slotsFilled;
    private final Counter frequencyCapped;
    private final Counter capStoreErrors;
    private final Counter spendMicros;
    private final Counter shed;
    private final Timer latency;
    private final Map<String, Counter> winsBySource = new ConcurrentHashMap<>();
    private final Map<String, DspMeters> dsps = new LinkedHashMap<>();
    private volatile java.util.function.Supplier<Map<String, Object>> pacingSummary = Map::of;

    private record DspMeters(Counter requests, Counter bids, Counter noBids, Counter timeouts, Counter errors,
                             Counter wins, Timer latency) {
    }

    public AdServerMetrics(MeterRegistry registry, List<String> dspNames) {
        this.registry = registry;
        requests = registry.counter("adserver.requests");
        slotsRequested = registry.counter("adserver.slots.requested");
        slotsFilled = registry.counter("adserver.slots.filled");
        frequencyCapped = registry.counter("adserver.frequency.capped");
        capStoreErrors = registry.counter("adserver.frequency.store.errors");
        spendMicros = registry.counter("adserver.spend.micros");
        shed = registry.counter("adserver.requests.shed");
        latency = Timer.builder("adserver.decision.latency")
                .description("Time to answer an ad request, including the DSP auction")
                .publishPercentiles(0.5, 0.99).publishPercentileHistogram().register(registry);
        for (String dsp : dspNames) {
            dsps.put(dsp, new DspMeters(
                    registry.counter("adserver.dsp.requests", "dsp", dsp),
                    registry.counter("adserver.dsp.bids", "dsp", dsp),
                    registry.counter("adserver.dsp.nobids", "dsp", dsp),
                    registry.counter("adserver.dsp.timeouts", "dsp", dsp),
                    registry.counter("adserver.dsp.errors", "dsp", dsp),
                    registry.counter("adserver.dsp.wins", "dsp", dsp),
                    Timer.builder("adserver.dsp.latency").tag("dsp", dsp)
                            .publishPercentiles(0.5, 0.99).register(registry)));
```

```
        }
    }

    void request(int slots, int filled, long latencyNanos, int capped) {
        requests.increment();
        slotsRequested.increment(slots);
        slotsFilled.increment(filled);
        frequencyCapped.increment(capped);
        latency.record(latencyNanos, TimeUnit.NANOSECONDS);
    }

    void pacingSummary(java.util.function.Supplier<Map<String, Object>> summary) {
        this.pacingSummary = summary;
    }

    void shed() {
        shed.increment();
    }

    void capStoreError() {
        capStoreErrors.increment();
    }

    void win(String source, long priceMicros) {
        winsBySource.computeIfAbsent(source, s -> registry.counter("adserver.wins", "source", s)).increment();
        spendMicros.increment(priceMicros);
        DspMeters d = dsps.get(source);
        if (d != null) {
            d.wins.increment();
        }
    }

    void dsp(DspClient.Result r) {
        DspMeters d = dsps.get(r.dsp());
        if (d == null) {
            return;
        }
        d.requests.increment();
        d.latency.record(r.latencyMs(), TimeUnit.MILLISECONDS);
        switch (r.status()) {
            case BID -> d.bids.increment(r.bids().size());
            case NO_BID -> d.noBids.increment();
            case TIMEOUT -> d.timeouts.increment();
            case ERROR -> d.errors.increment();
        }
    }

    public Map<String, Object> snapshot() {
        var out = new LinkedHashMap<String, Object>();
        out.put("timestampMs", System.currentTimeMillis());
        out.put("requests", (long) requests.count());
        out.put("slotsRequested", (long) slotsRequested.count());
        out.put("slotsFilled", (long) slotsFilled.count());
        out.put("shed", (long) shed.count());
        out.put("frequencyCapped", (long) frequencyCapped.count());
        out.put("capStoreErrors", (long) capStoreErrors.count());
        out.put("spendMicros", (long) spendMicros.count());
        out.put("latencyP50Ms", percentile(latency, 0.5));
        out.put("latencyP99Ms", percentile(latency, 0.99));
        var wins = new LinkedHashMap<String, Long>();
        winsBySource.forEach((k, v) -> wins.put(k, (long) v.count()));
        out.put("winsBySource", wins);
        var dspStats = new LinkedHashMap<String, Object>();
        dsps.forEach((name, d) -> {
            var m = new LinkedHashMap<String, Object>();
            m.put("requests", (long) d.requests.count());
            m.put("bids", (long) d.bids.count());
            m.put("noBids", (long) d.noBids.count());
            m.put("timeouts", (long) d.timeouts.count());
            m.put("errors", (long) d.errors.count());
            m.put("wins", (long) d.wins.count());
            m.put("latencyP50Ms", percentile(d.latency, 0.5));
            m.put("latencyP99Ms", percentile(d.latency, 0.99));
            dspStats.put(name, m);
        });
        out.put("dsps", dspStats);
        out.put("pacing", pacingSummary.get());
        return out;
    }

    private static double percentile(Timer timer, double p) {
        for (ValueAtPercentile v : timer.takeSnapshot().percentileValues()) {
            if (v.percentile() == p) {
                return v.value(TimeUnit.MILLISECONDS);
            }
        }
        return 0;
    }
```

```
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/Auction.java

86 lines · Fills an ad pod slot by slot.

```
package dev.primetech.ads.adserver;

import dev.primetech.ads.openrtb.BidRequest;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.function.IntFunction;

/**
 * Fills an ad pod slot by slot. Each slot is a unified auction between direct-sold line items and
 * the DSP bids for that slot, under the rules a CTV break needs:
 *
 * <ul>
 *   <li>bids below the floor are ignored;</li>
 *   <li>competitive separation: an advertiser (and so a campaign) appears at most once per break;</li>
 *   <li>the ads must fit the break's total length.</li>
 * </ul>
 *
 * Guaranteed line items (sponsorships) take a slot ahead of the auction whenever pacing lets them
 * compete; the auction only prices what is left. Direct line items pay their contracted CPM. DSP winners pay th
↳ eir bid in a first-price auction,
 * or the next highest eligible bid plus one cent (never less than the floor, never more than their
 * bid) in a second-price auction.
 */
public final class Auction {
    static final long ONE_CENT_CPM_MICROS = 10_000;

    /**
     * Guaranteed line items first (priority tier, regardless of price), then highest offer; ties go
     * to direct demand, then a stable order so results are repeatable.
     */
    private static final Comparator<Candidate> RANKING = Comparator
            .comparing((Candidate c) -> !c.guaranteed())
            .thenComparing(Comparator.comparingLong(Candidate::bidCpmMicros).reversed())
            .thenComparing(c -> !c.isDirect())
            .thenComparing(Candidate::creativeId);

    public record Winner(int position, Candidate candidate, long clearingCpmMicros, int eligibleCandidates) {
    }

    private Auction() {
    }

    /**
     * @param candidatesForSlot every candidate for a 1-based slot position (direct and that slot's bids)
     */
    public static List<Winner> run(int slots, int maxAdSeconds, int podSeconds, IntFunction<List<Candidate>> can
↳ didatesForSlot,
                                   int auctionType, long floorCpmMicros) {
        var winners = new ArrayList<Winner>();
        Set<String> advertisersInPod = new HashSet<>();
        int secondsLeft = podSeconds;

        for (int position = 1; position <= slots; position++) {
            int fits = Math.min(maxAdSeconds, secondsLeft);
            List<Candidate> eligible = new ArrayList<>();
            for (Candidate c : candidatesForSlot.apply(position)) {
                if (c.bidCpmMicros() >= floorCpmMicros
                        && c.durationSeconds() <= fits
                        && !advertisersInPod.contains(c.advertiserId())) {
                    eligible.add(c);
                }
            }
            if (eligible.isEmpty()) {
                continue;
            }
            eligible.sort(RANKING);
            Candidate winner = eligible.get(0);
            winners.add(new Winner(position, winner, clearingPrice(winner, eligible, auctionType, floorCpmMicros
↳ ),
                    eligible.size()));
            advertisersInPod.add(winner.advertiserId());
            secondsLeft -= winner.durationSeconds();
        }
        return winners;
    }

    private static long clearingPrice(Candidate winner, List<Candidate> ranked, int auctionType, long floor) {
        if (winner.isDirect() || auctionType == BidRequest.FIRST_PRICE) {
            return winner.bidCpmMicros();
```

```
        }
        // Runner-up plus a cent; a sole bidder pays the floor.
        long price = ranked.size() > 1 ? Math.max(floor, ranked.get(1).bidCpmMicros() + ONE_CENT_CPM_MICROS) : floor;
        return Math.min(winner.bidCpmMicros(), price);
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/Candidate.java

31 lines · Something that could fill a slot: a direct-sold creative or a DSP bid.

```
package dev.primetech.ads.adserver;

/**
 * Something that could fill a slot: a direct-sold creative or a DSP bid.
 *
 * @param source       {@link #DIRECT} or the DSP name
 * @param impId        the OpenRTB imp a DSP bid is for (slot position as a string); null for direct
 * @param bidCpmMicros what the candidate offers: a direct line item's fixed CPM, or the DSP bid
 * @param nurl         DSP win notice URL with macros; null for direct
 * @param guaranteed   a guaranteed direct line item: wins ahead of the auction (priority tier)
 */
public record Candidate(
        String source,
        String impId,
        String campaignId,
        String lineItemId,
        String adId,
        String creativeId,
        String advertiserId,
        int durationSeconds,
        long bidCpmMicros,
        int frequencyCapPerHour,
        String nurl,
        boolean guaranteed) {

    public static final String DIRECT = "direct";

    public boolean isDirect() {
        return DIRECT.equals(source);
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/DecisionSink.java

27 lines · Where the ad server records decisions and announces campaigns.

```
package dev.primetech.ads.adserver;

import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.adserving.AdDecision;
import dev.primetech.ads.common.catalog.LineItemConfig;

import java.util.concurrent.CompletableFuture;

/** Where the ad server records decisions and announces campaigns. Calls must not block. */
public interface DecisionSink extends AutoCloseable {

    /** Best effort: a lost audit record is counted, never retried by the caller. */
    void decision(AdDecision decision);

    /**
     * Must eventually succeed, or the stream processor cannot enrich the campaign's events; the
     * returned future completes when the broker has the record, so callers can retry.
     */
    CompletableFuture<Void> campaign(CampaignMetadata campaign);

    /** Line item terms for the pacing service; completes when the broker has the record. */
    CompletableFuture<Void> lineItem(LineItemConfig lineItem);

    @Override
    default void close() {
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/DirectDemand.java

56 lines · Direct-sold line items from the campaign catalog, filtered by targeting and by pacing: each line item enters an auction with the probability its pacer sets, and not at all once its budget is spent.

```
package dev.primetech.ads.adserver;

import dev.primetech.ads.common.DeviceType;
import dev.primetech.ads.common.catalog.AdCatalog;
import dev.primetech.ads.common.catalog.LineItemConfig;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
```

```java
import java.util.random.RandomGenerator;

/**
 * Direct-sold line items from the campaign catalog, filtered by targeting and by pacing: each line
 * item enters an auction with the probability its pacer sets, and not at all once its budget is
 * spent.
 */
public final class DirectDemand {
    private final Map<String, List<AdCatalog.AdSpec>> byLineItem = new HashMap<>();
    private final Map<String, LineItemConfig.Tier> tiers = new HashMap<>();
    private final PacingView pacing;

    public DirectDemand(AdCatalog catalog, PacingView pacing) {
        for (AdCatalog.AdSpec ad : catalog.ads()) {
            byLineItem.computeIfAbsent(ad.lineItemId(), k -> new ArrayList<>()).add(ad);
        }
        catalog.lineItems().forEach(l -> tiers.put(l.lineItemId(), l.tier()));
        this.pacing = pacing;
    }

    public List<Candidate> eligible(String country, DeviceType deviceType, RandomGenerator random) {
        var out = new ArrayList<Candidate>();
        for (List<AdCatalog.AdSpec> creatives : byLineItem.values()) {
            AdCatalog.AdSpec first = creatives.get(0);
            if (!first.targets(country, deviceType)) {
                continue;
            }
            double throttle = pacing.throttle(first.lineItemId());
            if (throttle <= 0 || (throttle < 1 && random.nextDouble() >= throttle)) {
                continue;
            }
            boolean guaranteed = tiers.get(first.lineItemId()) == LineItemConfig.Tier.GUARANTEED;
            for (AdCatalog.AdSpec ad : creatives) {
                out.add(new Candidate(Candidate.DIRECT, null, ad.campaignId(), ad.lineItemId(), ad.adId(),
                        ad.creativeId(), ad.advertiserId(), ad.durationSeconds(), ad.cpmMicros(),
                        ad.frequencyCapPerHour(), null, guaranteed));
            }
        }
        return out;
    }

    /** Charges a served direct ad to its line item's budget. */
    public void served(String lineItemId, long priceMicros) {
        pacing.served(lineItemId, priceMicros);
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/DspClient.java

116 lines · Calls one DSP's OpenRTB endpoint.

```java
package dev.primetech.ads.adserver;

import dev.primetech.ads.common.Json;
import dev.primetech.ads.openrtb.BidRequest;
import dev.primetech.ads.openrtb.BidResponse;

import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.net.http.HttpTimeoutException;
import java.time.Duration;
import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionException;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeoutException;

/**
 * Calls one DSP's OpenRTB endpoint. Never fails: every outcome, including a missed deadline,
 * becomes a {@link Result}, so one bad bidder cannot break or slow down an auction.
 *
 * <p>The auction deadline and the HTTP request lifetime are deliberately different. The auction
 * stops waiting at {@code tmax}, but the HTTP exchange may run on (up to {@link #REQUEST_TIMEOUT})
 * so a late response returns its connection to the pool. Aborting at {@code tmax} closes the
 * connection instead, and under load the reconnects cost enough CPU to cause more timeouts: in
 * profiling, connection churn in the HTTP client was the largest single cost.
 */
public final class DspClient {
    public enum Status { BID, NO_BID, TIMEOUT, ERROR }

    public record Result(String dsp, Status status, List<BidResponse.Bid> bids, long latencyMs) {
    }

    static final Duration REQUEST_TIMEOUT = Duration.ofSeconds(1);

    private final String name;
```

```
    private final URI endpoint;
    private final HttpClient http;

    public DspClient(String name, URI endpoint, HttpClient http) {
        this.name = name;
        this.endpoint = endpoint;
        this.http = http;
    }

    /** One client per process: HTTP/1.1 keep-alive connections, virtual threads for the work. */
    public static HttpClient sharedHttpClient() {
        return HttpClient.newBuilder()
                .version(HttpClient.Version.HTTP_1_1)
                .connectTimeout(Duration.ofMillis(500))
                .executor(Executors.newVirtualThreadPerTaskExecutor())
                .build();
    }

    public String name() {
        return name;
    }

    /** Serializes once; the same bytes go to every DSP. */
    public static byte[] serialize(BidRequest request) {
        try {
            return Json.MAPPER.writeValueAsBytes(request);
        } catch (IOException e) {
            throw new IllegalStateException("cannot serialize bid request", e);
        }
    }

    /** Completes within {@code deadline}, as TIMEOUT if the DSP has not answered by then. */
    public CompletableFuture<Result> bid(byte[] body, Duration deadline) {
        long start = System.nanoTime();
        var httpRequest = HttpRequest.newBuilder(endpoint)
                .timeout(REQUEST_TIMEOUT)
                .header("Content-Type", "application/json")
                .header("x-openrtb-version", "2.6")
                .POST(HttpRequest.BodyPublishers.ofByteArray(body))
                .build();
        // copy(): the timeout completes our copy, leaving the HTTP exchange itself running.
        return http.sendAsync(httpRequest, HttpResponse.BodyHandlers.ofByteArray())
                .copy()
                .orTimeout(deadline.toMillis(), java.util.concurrent.TimeUnit.MILLISECONDS)
                .handle((response, error) -> {
                    long latency = (System.nanoTime() - start) / 1_000_000;
                    if (error != null) {
                        Throwable cause = error instanceof CompletionException ? error.getCause() : error;
                        boolean timeout = cause instanceof HttpTimeoutException || cause instanceof TimeoutExcep
  ↪ tion;
                        return new Result(name, timeout ? Status.TIMEOUT : Status.ERROR, List.of(), latency);
                    }
                    return parse(response, latency);
                });
    }

    private Result parse(HttpResponse<byte[]> response, long latency) {
        if (response.statusCode() == 204) {
            return new Result(name, Status.NO_BID, List.of(), latency);
        }
        if (response.statusCode() != 200) {
            return new Result(name, Status.ERROR, List.of(), latency);
        }
        try {
            BidResponse parsed = Json.MAPPER.readValue(response.body(), BidResponse.class);
            List<BidResponse.Bid> bids = parsed.seatbid() == null ? List.of()
                    : parsed.seatbid().stream().flatMap(s -> s.bid() == null ? java.util.stream.Stream.empty() :
  ↪  s.bid().stream()).toList();
            return new Result(name, bids.isEmpty() ? Status.NO_BID : Status.BID, bids, latency);
        } catch (IOException e) {
            return new Result(name, Status.ERROR, List.of(), latency);
        }
    }

    /** Fire-and-forget GET of a win notice. */
    public void notify(String url) {
        http.sendAsync(HttpRequest.newBuilder(URI.create(url)).timeout(Duration.ofSeconds(2)).GET().build(),
                HttpResponse.BodyHandlers.discarding());
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/FrequencyCaps.java

21 lines · Per device, per campaign impression counts in fixed clock-hour windows.

```
package dev.primetech.ads.adserver;

import java.util.Collection;
import java.util.Map;
```

```

/**
 * Per device, per campaign impression counts in fixed clock-hour windows. Counting happens when an
 * ad is served, not when the player reports the impression: simpler and immediate, at the cost of
 * counting the occasional served-but-not-played ad.
 */
public interface FrequencyCaps {

    /** Impressions this hour for each campaign; campaigns never served are absent or zero. */
    Map<String, Long> counts(String deviceIdHash, Collection<String> campaignIds) throws Exception;

    void record(String deviceIdHash, Collection<String> campaignIds) throws Exception;

    static long currentHour() {
        return System.currentTimeMillis() / 3_600_000L;
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/InMemoryFrequencyCaps.java

32 lines · Single-instance frequency caps, for tests and for running without Redis.

```
package dev.primetech.ads.adserver;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/** Single-instance frequency caps, for tests and for running without Redis. */
public final class InMemoryFrequencyCaps implements FrequencyCaps {
    private final Map<String, Long> counters = new ConcurrentHashMap<>();

    @Override
    public Map<String, Long> counts(String deviceIdHash, Collection<String> campaignIds) {
        long hour = FrequencyCaps.currentHour();
        var out = new HashMap<String, Long>();
        for (String c : campaignIds) {
            Long n = counters.get(RedisFrequencyCaps.key(deviceIdHash, c, hour));
            if (n != null) {
                out.put(c, n);
            }
        }
        return out;
    }

    @Override
    public void record(String deviceIdHash, Collection<String> campaignIds) {
        long hour = FrequencyCaps.currentHour();
        for (String c : campaignIds) {
            counters.merge(RedisFrequencyCaps.key(deviceIdHash, c, hour), 1L, Long::sum);
        }
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/KafkaDecisionSink.java

121 lines · Writes the decision log and campaign metadata to Kafka without waiting for acks: the ad request must not pay for an audit write.

```
package dev.primetech.ads.adserver;

import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.Topics;
import dev.primetech.ads.common.adserving.AdDecision;
import dev.primetech.ads.common.catalog.LineItemConfig;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.serialization.ByteArraySerializer;
import org.apache.kafka.common.serialization.StringSerializer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.io.IOException;
import java.time.Duration;
import java.util.Map;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.atomic.AtomicBoolean;

/**
 * Writes the decision log and campaign metadata to Kafka without waiting for acks: the ad request
 * must not pay for an audit write.
 *
 * <p>{@code max.block.ms} is small but not zero, and topic metadata is fetched before the first
 * request. With zero, every send fails until the producer has learned the topic's partitions,
 * which silently dropped the whole campaign catalog at start-up in testing.
```

```
 */
public final class KafkaDecisionSink implements DecisionSink {
    private static final Logger log = LoggerFactory.getLogger(KafkaDecisionSink.class);

    private final KafkaProducer<String, byte[]> producer;
    private final Counter failures;
    private final AtomicBoolean warnedOnce = new AtomicBoolean();

    public KafkaDecisionSink(String bootstrap, MeterRegistry registry) {
        producer = new KafkaProducer<>(Map.of(
                ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrap,
                ProducerConfig.CLIENT_ID_CONFIG, "ad-server",
                ProducerConfig.ACKS_CONFIG, "all",
                ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true,
                ProducerConfig.LINGER_MS_CONFIG, 20,
                ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4",
                // Stall a request by 50 ms at most: if the buffer stays full, drop the record and count it.
                ProducerConfig.MAX_BLOCK_MS_CONFIG, 50),
                new StringSerializer(), new ByteArraySerializer());
        failures = registry.counter("adserver.log.failures");
        warmUp(Topics.AD_DECISIONS, Duration.ofSeconds(60));
        warmUp(Topics.CAMPAIGN_METADATA, Duration.ofSeconds(60));
        warmUp(Topics.LINE_ITEMS, Duration.ofSeconds(60));
    }

    /** Fetches a topic's partitions before serving, retrying while the broker comes up. */
    private void warmUp(String topic, Duration patience) {
        long deadline = System.nanoTime() + patience.toNanos();
        while (true) {
            try {
                producer.partitionsFor(topic);
                return;
            } catch (RuntimeException e) {
                if (System.nanoTime() > deadline) {
                    throw new IllegalStateException("no metadata for " + topic + " after " + patience, e);
                }
                try {
                    Thread.sleep(250);
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                    throw new IllegalStateException(ie);
                }
            }
        }
    }

    @Override
    public void decision(AdDecision decision) {
        send(Topics.AD_DECISIONS, decision.requestId(), decision);
    }

    @Override
    public CompletableFuture<Void> campaign(CampaignMetadata campaign) {
        return send(Topics.CAMPAIGN_METADATA, campaign.campaignId(), campaign);
    }

    @Override
    public CompletableFuture<Void> lineItem(LineItemConfig lineItem) {
        return send(Topics.LINE_ITEMS, lineItem.lineItemId(), lineItem);
    }

    private CompletableFuture<Void> send(String topic, String key, Object value) {
        var result = new CompletableFuture<Void>();
        try {
            producer.send(new ProducerRecord<>(topic, key, Json.MAPPER.writeValueAsBytes(value)), (m, e) -> {
                if (e != null) {
                    failed(topic, e);
                    result.completeExceptionally(e);
                } else {
                    result.complete(null);
                }
            });
        } catch (IOException | RuntimeException e) {
            failed(topic, e);
            result.completeExceptionally(e);
        }
        return result;
    }

    private void failed(String topic, Exception e) {
        failures.increment();
        if (warnedOnce.compareAndSet(false, true)) {
            log.warn("write to {} failed (further failures are only counted): {}", topic, e.toString());
        }
    }

    @Override
    public void close() {
```

```
119          producer.close(Duration.ofSeconds(10));
120      }
121  }
```

ad-server/src/main/java/dev/primetech/ads/adserver/PacingView.java

28 lines · The ad server's view of budget pacing: how often each direct line item may compete, and the running total of what it has served.

```
 1  package dev.primetech.ads.adserver;
 2
 3  /**
 4   * The ad server's view of budget pacing: how often each direct line item may compete, and the
 5   * running total of what it has served. Must be cheap to call per request.
 6   */
 7  public interface PacingView {
 8
 9      /** Probability the line item enters this auction; 0 once its budget is spent. */
10      double throttle(String lineItemId);
11
12      /** Records a served impression's price against the line item's budget. */
13      void served(String lineItemId, long priceMicros);
14
15      /** Every line item at the same fixed participation and no budget: for tests and running without Redis. */
16      static PacingView fixed(double participation) {
17          return new PacingView() {
18              @Override
19              public double throttle(String lineItemId) {
20                  return participation;
21              }
22
23              @Override
24              public void served(String lineItemId, long priceMicros) {
25              }
26          };
27      }
28  }
```

ad-server/src/main/java/dev/primetech/ads/adserver/RedisFrequencyCaps.java

83 lines · Frequency caps in Redis: one counter per device, campaign and hour, so every ad server instance shares them.

```
 1  package dev.primetech.ads.adserver;
 2
 3  import org.apache.commons.pool2.impl.GenericObjectPoolConfig;
 4  import redis.clients.jedis.Connection;
 5  import redis.clients.jedis.DefaultJedisClientConfig;
 6  import redis.clients.jedis.HostAndPort;
 7  import redis.clients.jedis.JedisPooled;
 8
 9  import java.net.URI;
10  import java.time.Duration;
11
12  import java.util.ArrayList;
13  import java.util.Collection;
14  import java.util.HashMap;
15  import java.util.List;
16  import java.util.Map;
17
18  /**
19   * Frequency caps in Redis: one counter per device, campaign and hour, so every ad server instance
20   * shares them. Reading is one MGET per request; writing is one pipelined INCR + EXPIRE per win.
21   *
22   * <p>Tight timeouts on purpose: a cap check is worth a few milliseconds, not the request. When
23   * Redis is slow or the pool is busy, calls fail fast and the caller serves uncapped.
24   */
25  public final class RedisFrequencyCaps implements FrequencyCaps, AutoCloseable {
26      private static final long KEY_TTL_SECONDS = 2 * 3600;
27
28      private final JedisPooled redis;
29
30      public RedisFrequencyCaps(String url) {
31          URI uri = URI.create(url);
32          var pool = new GenericObjectPoolConfig<Connection>();
33          pool.setMaxTotal(32);
34          pool.setMaxIdle(32);
35          pool.setMinIdle(8);
36          pool.setMaxWait(Duration.ofMillis(20));
37          var client = DefaultJedisClientConfig.builder()
38                  .connectionTimeoutMillis(200)
39                  .socketTimeoutMillis(50)
40                  .build();
41          this.redis = new JedisPooled(pool, new HostAndPort(uri.getHost(), uri.getPort() > 0 ? uri.getPort() : 63
    ↪ 79), client);
42      }
43
44      static String key(String deviceIdHash, String campaignId, long hour) {
45          return "fc:" + deviceIdHash + ":" + campaignId + ":" + hour;
46      }
47
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
    @Override
    public Map<String, Long> counts(String deviceIdHash, Collection<String> campaignIds) {
        if (campaignIds.isEmpty()) {
            return Map.of();
        }
        long hour = FrequencyCaps.currentHour();
        List<String> ids = new ArrayList<>(campaignIds);
        String[] keys = ids.stream().map(c -> key(deviceIdHash, c, hour)).toArray(String[]::new);
        List<String> values = redis.mget(keys);
        var counts = new HashMap<String, Long>();
        for (int i = 0; i < ids.size(); i++) {
            if (values.get(i) != null) {
                counts.put(ids.get(i), Long.parseLong(values.get(i)));
            }
        }
        return counts;
    }

    @Override
    public void record(String deviceIdHash, Collection<String> campaignIds) {
        long hour = FrequencyCaps.currentHour();
        try (var pipeline = redis.pipelined()) {
            for (String campaignId : campaignIds) {
                String key = key(deviceIdHash, campaignId, hour);
                pipeline.incr(key);
                pipeline.expire(key, KEY_TTL_SECONDS);
            }
            pipeline.sync();
        }
    }

    @Override
    public void close() {
        redis.close();
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/RedisPacingView.java

207 lines · Pacing state from Redis, refreshed in the background once a second so requests never wait on it.

```
package dev.primetech.ads.adserver;

import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.catalog.LineItemConfig;
import dev.primetech.ads.common.pacing.PacingPeriod;
import dev.primetech.ads.common.pacing.PacingState;
import dev.primetech.ads.common.pacing.ServedSpend;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import org.apache.commons.pool2.impl.GenericObjectPoolConfig;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import redis.clients.jedis.Connection;
import redis.clients.jedis.DefaultJedisClientConfig;
import redis.clients.jedis.HostAndPort;
import redis.clients.jedis.JedisPooled;

import java.io.IOException;
import java.net.URI;
import java.time.Duration;
import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
import java.util.function.LongSupplier;

/**
 * Pacing state from Redis, refreshed in the background once a second so requests never wait on it.
 *
 * <p>Served spend is shared through Redis without touching Redis on the request path: requests add
 * to this instance's totals ({@link ServedLedger}), and the background thread writes them as
 * absolute values every 250 ms and reads every instance's totals every second. An instance sees
 * its own spending immediately and another instance's within about a second.
 *
 * <p>If pacing state is missing or stale (pacing service down), each line item falls back to a
 * small fixed throttle and a hard stop once its served spend reaches its budget. Every delivered
 * impression was served first, so that stop cannot overspend, whatever share of served ads goes
 * unplayed. Under-delivering for a while is recoverable; overspending a client's budget is not.
 */
public final class RedisPacingView implements PacingView, AutoCloseable {
    private static final Logger log = LoggerFactory.getLogger(RedisPacingView.class);
    static final Duration STALE_AFTER = Duration.ofSeconds(60);

    private final JedisPooled redis;
    private final PacingPeriod period;
    private final Map<String, LineItemConfig> configs;
    private final double fallbackThrottle;
    private final LongSupplier clock;
    private final Counter fallbacks;
```

```
49      private final Counter errors;
50
51      static final Duration WRITE_EVERY = Duration.ofMillis(250);
52      static final Duration READ_EVERY = Duration.ofSeconds(1);
53
54      private volatile Map<String, PacingState> states = Map.of();
55      private final ServedLedger ledger = new ServedLedger();
56      /** This instance's field in each served-spend hash: unique per process start. */
57      private final String instanceId;
58      private final Thread refresher;
59
60      public RedisPacingView(String url, PacingPeriod period, Collection<LineItemConfig> lineItems, double fallbac
  ↪ kThrottle,
61                             LongSupplier clock, MeterRegistry registry) {
62          URI uri = URI.create(url);
63          var pool = new GenericObjectPoolConfig<Connection>();
64          pool.setMaxTotal(16);
65          pool.setMaxWait(Duration.ofMillis(20));
66          this.redis = new JedisPooled(pool, new HostAndPort(uri.getHost(), uri.getPort() > 0 ? uri.getPort() : 63
  ↪ 79),
67                  DefaultJedisClientConfig.builder().connectionTimeoutMillis(200).socketTimeoutMillis(200).build()
  ↪ );
68          this.period = period;
69          var byId = new HashMap<String, LineItemConfig>();
70          lineItems.forEach(l -> byId.put(l.lineItemId(), l));
71          this.configs = Map.copyOf(byId);
72          this.fallbackThrottle = fallbackThrottle;
73          this.clock = clock;
74          this.fallbacks = registry.counter("adserver.pacing.fallback");
75          this.errors = registry.counter("adserver.pacing.errors");
76          this.instanceId = hostName() + "-" + Long.toHexString(System.nanoTime() ^ ProcessHandle.current().pid()
  ↪ << 32);
77          registry.gauge("adserver.served.unwritten.micros", ledger, ServedLedger::unwrittenMicros);
78          refresh();
79          this.refresher = Thread.ofPlatform().name("pacing-refresh").daemon().start(this::refreshLoop);
80      }
81
82      @Override
83      public double throttle(String lineItemId) {
84          LineItemConfig config = configs.get(lineItemId);
85          if (config == null) {
86              return 0;
87          }
88          long now = clock.getAsLong();
89          long periodIndex = period.index(now);
90          PacingState s = states.get(lineItemId);
91          double throttle;
92          long stopAt;
93          if (s == null || s.periodIndex() != periodIndex || now - s.updatedAtMs() > STALE_AFTER.toMillis()) {
94              fallbacks.increment();
95              throttle = fallbackThrottle;
96              stopAt = config.periodBudgetMicros();
97          } else {
98              throttle = s.throttle();
99              stopAt = s.stopAtServedMicros();
100         }
101         return servedNow(lineItemId, periodIndex) >= stopAt ? 0 : throttle;
102     }
103
104     /** Never touches Redis: the background thread writes the totals. */
105     @Override
106     public void served(String lineItemId, long priceMicros) {
107         ledger.add(PacingState.servedKey(period.index(clock.getAsLong()), lineItemId), priceMicros);
108     }
109
110     long servedNow(String lineItemId, long periodIndex) {
111         return ledger.served(periodIndex, lineItemId, PacingState.servedKey(periodIndex, lineItemId));
112     }
113
114     private void refreshLoop() {
115         long lastRead = System.nanoTime();
116         while (!Thread.currentThread().isInterrupted()) {
117             try {
118                 Thread.sleep(WRITE_EVERY.toMillis());
119             } catch (InterruptedException e) {
120                 flush(); // what was served last must not be lost on shutdown
121                 return;
122             }
123             flush();
124             if (System.nanoTime() - lastRead >= READ_EVERY.toNanos()) {
125                 refresh();
126                 lastRead = System.nanoTime();
127             }
128         }
129     }
130
131     /** Writes this instance's totals as absolute values: retrying after a failure is always safe. */
132     void flush() {
```

```
        Map<String, Long> totals = ledger.unwritten();
        if (totals.isEmpty()) {
            return;
        }
        try (var p = redis.pipelined()) {
            long ttlSeconds = period.lengthMs() / 1000 * 2;
            totals.forEach((key, value) -> {
                p.hset(key, instanceId, Long.toString(value));
                p.expire(key, ttlSeconds);
            });
            p.sync();
            ledger.written(totals);
        } catch (RuntimeException e) {
            errors.increment(); // kept in the ledger; the next write carries the full totals
        }
    }

    /** Reads pacing state and every instance's served totals. Runs on the writing thread, right after a write. */
    void refresh() {
        try {
            var parsed = new HashMap<String, PacingState>();
            for (var e : redis.hgetAll(PacingState.REDIS_HASH).entrySet()) {
                try {
                    parsed.put(e.getKey(), Json.MAPPER.readValue(e.getValue(), PacingState.class));
                } catch (IOException ignored) {
                    // one bad field must not hide the others
                }
            }
            long periodIndex = period.index(clock.getAsLong());
            Map<String, Long> totals = ServedSpend.read(redis, periodIndex, configs.keySet());
            states = parsed;
            String prefix = PacingState.servedKey(periodIndex, "");
            ledger.snapshot(periodIndex, totals, prefix);
            ledger.prune(prefix);
        } catch (RuntimeException e) {
            errors.increment();
            log.debug("pacing refresh failed: {}", e.toString());
        }
    }

    /** For the stats page: how many line items have fresh state, and how many are spent. */
    Map<String, Object> summary() {
        long now = clock.getAsLong();
        long periodIndex = period.index(now);
        int fresh = 0, exhausted = 0;
        for (PacingState s : states.values()) {
            if (s.periodIndex() == periodIndex && now - s.updatedAtMs() <= STALE_AFTER.toMillis()) {
                fresh++;
                if (s.status() == PacingState.Status.EXHAUSTED) {
                    exhausted++;
                }
            }
        }
        return Map.of("lineItems", configs.size(), "fresh", fresh, "exhausted", exhausted);
    }

    @Override
    public void close() {
        refresher.interrupt();
        try {
            refresher.join(2_000);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        redis.close();
    }

    private static String hostName() {
        try {
            return java.net.InetAddress.getLocalHost().getHostName();
        } catch (java.io.IOException e) {
            return "ad-server";
        }
    }
}
```

ad-server/src/main/java/dev/primetech/ads/adserver/ServedLedger.java

95 lines · This ad server instance's served spend per period and line item, and how it combines with the shared totals in Redis.

```
package dev.primetech.ads.adserver;

import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

/**
```

```
 * This ad server instance's served spend per period and line item, and how it combines with the
 * shared totals in Redis.
 *
 * <p>Requests only add to an in-memory total ({@link #add}); a background thread writes each
 * total to Redis as an absolute value and reads everyone's totals back. Served spend as seen by
 * this instance is the shared total minus what Redis holds for this instance, plus this
 * instance's own up-to-date total. So it never lags this instance's own spending, it counts other
 * instances' spending within a refresh, and a failed write loses nothing: the next write carries
 * the full total.
 *
 * <p>The first version incremented Redis on every request and gave up after 20 ms waiting for a
 * connection. While the stack warmed up, 4,744 increments were dropped in three minutes, and the
 * audit found every line item's counter 3–6% low and 21 line items over budget.
 */
final class ServedLedger {
    /** Keyed by {@code PacingState.servedKey(period, lineItemId)}; monotonic. */
    private final Map<String, AtomicLong> mine = new ConcurrentHashMap<>();
    /** What Redis holds for this instance, per key: the last value written successfully. */
    private final Map<String, Long> written = new ConcurrentHashMap<>();

    private volatile Snapshot snapshot = new Snapshot(Long.MIN_VALUE, Map.of(), Map.of());

    /** Shared totals per line item for one period, and this instance's share of them at the time. */
    private record Snapshot(long periodIndex, Map<String, Long> totals, Map<String, Long> writtenAtRead) {
    }

    void add(String key, long micros) {
        mine.computeIfAbsent(key, k -> new AtomicLong()).addAndGet(micros);
    }

    /** Totals that differ from what Redis holds for this instance: what the next write should send. */
    Map<String, Long> unwritten() {
        var out = new HashMap<String, Long>();
        mine.forEach((key, total) -> {
            long value = total.get();
            if (value != written.getOrDefault(key, 0L)) {
                out.put(key, value);
            }
        });
        return out;
    }

    /** Records a successful write of these absolute totals. */
    void written(Map<String, Long> values) {
        written.putAll(values);
    }

    /**
     * Installs totals read from Redis. Must be called by the same thread that writes, right after
     * reading, so {@code written} is exactly what Redis held for this instance during the read.
     */
    void snapshot(long periodIndex, Map<String, Long> totalsByLineItem, String keyPrefix) {
        var share = new HashMap<String, Long>();
        written.forEach((key, value) -> {
            if (key.startsWith(keyPrefix)) {
                share.put(key.substring(keyPrefix.length()), value);
            }
        });
        snapshot = new Snapshot(periodIndex, Map.copyOf(totalsByLineItem), share);
    }

    /** Served spend for a line item in a period, all instances included. */
    long served(long periodIndex, String lineItemId, String key) {
        AtomicLong own = mine.get(key);
        long ownNow = own == null ? 0 : own.get();
        Snapshot s = snapshot;
        if (s.periodIndex() != periodIndex) {
            return ownNow;
        }
        return s.totals().getOrDefault(lineItemId, 0L) - s.writtenAtRead().getOrDefault(lineItemId, 0L) + ownNow
;
    }

    /** Drops keys of finished periods once they have been written. */
    void prune(String currentKeyPrefix) {
        mine.entrySet().removeIf(e -> !e.getKey().startsWith(currentKeyPrefix)
                && e.getValue().get() == written.getOrDefault(e.getKey(), 0L));
        written.keySet().removeIf(k -> !k.startsWith(currentKeyPrefix) && !mine.containsKey(k));
    }

    long unwrittenMicros() {
        long sum = 0;
        for (var e : mine.entrySet()) {
            sum += e.getValue().get() - written.getOrDefault(e.getKey(), 0L);
        }
        return sum;
    }
}
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

ad-server/src/main/resources/simplelogger.properties

7 lines · The Kafka client logs its full config at INFO on startup; keep only warnings.

```
org.slf4j.simpleLogger.defaultLogLevel=info
org.slf4j.simpleLogger.showDateTime=true
org.slf4j.simpleLogger.dateTimeFormat=yyyy-MM-dd'T'HH:mm:ss.SSSZ
org.slf4j.simpleLogger.showThreadName=false
org.slf4j.simpleLogger.showShortLogName=true
# The Kafka client logs its full config at INFO on startup; keep only warnings.
org.slf4j.simpleLogger.log.org.apache.kafka=warn
```

ad-server/src/test/java/dev/primetech/ads/adserver/AdDecisionServiceTest.java

265 lines · Real OpenRTB round trips against in-process DSPs: a good one, a slow one and a broken one.

```
package dev.primetech.ads.adserver;

import com.sun.net.httpserver.HttpServer;
import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.DeviceIdHasher;
import dev.primetech.ads.common.DeviceType;
import dev.primetech.ads.common.adserving.AdDecision;
import dev.primetech.ads.common.adserving.AdRequest;
import dev.primetech.ads.common.adserving.AdResponse;
import dev.primetech.ads.common.catalog.AdCatalog;
import dev.primetech.ads.dsp.Bidder;
import dev.primetech.ads.dsp.DspProfile;
import dev.primetech.ads.dsp.DspServer;
import dev.primetech.ads.openrtb.BidRequest;
import io.micrometer.prometheusmetrics.PrometheusConfig;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.Test;

import java.io.IOException;
import java.net.InetSocketAddress;
import java.net.URI;
import java.time.Duration;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.SplittableRandom;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CopyOnWriteArrayList;

import static org.assertj.core.api.Assertions.assertThat;

/** Real OpenRTB round trips against in-process DSPs: a good one, a slow one and a broken one. */
class AdDecisionServiceTest {
    private final List<AutoCloseable> servers = new ArrayList<>();
    private final RecordingSink sink = new RecordingSink();
    private AdServerMetrics metrics;

    static final class RecordingSink implements DecisionSink {
        final List<AdDecision> decisions = new CopyOnWriteArrayList<>();
        final List<CampaignMetadata> campaigns = new CopyOnWriteArrayList<>();
        volatile int failNextCampaigns;

        @Override
        public void decision(AdDecision decision) {
            decisions.add(decision);
        }

        @Override
        public CompletableFuture<Void> lineItem(dev.primetech.ads.common.catalog.LineItemConfig lineItem) {
            return CompletableFuture.completedFuture(null);
        }

        @Override
        public CompletableFuture<Void> campaign(CampaignMetadata campaign) {
            if (failNextCampaigns > 0) {
                failNextCampaigns--;
                return CompletableFuture.failedFuture(new RuntimeException("broker down"));
            }
            campaigns.add(campaign);
            return CompletableFuture.completedFuture(null);
        }
    }

    @AfterEach
    void stop() throws Exception {
        for (AutoCloseable s : servers) {
            s.close();
        }
    }

    private URI goodDsp(double cpm) throws IOException {
        var profile = new DspProfile("good-dsp", 1.0, cpm, 0.05, 1, 0, 1.0, List.of("brand.example", "other.exam
```

```
  ↪ ple"));
 75          var server = new DspServer(0, new Bidder(profile, "http://localhost"),
 76                  new PrometheusMeterRegistry(PrometheusConfig.DEFAULT), 1, false);
 77          server.start();
 78          servers.add(server);
 79          return URI.create("http://localhost:" + server.port() + "/openrtb2/bid");
 80      }
 81
 82      /** Answers 500 or sleeps past any deadline. */
 83      private URI badDsp(boolean slow) throws IOException {
 84          var server = HttpServer.create(new InetSocketAddress(0), 0);
 85          server.createContext("/", ex -> {
 86              if (slow) {
 87                  try {
 88                      Thread.sleep(1_000);
 89                  } catch (InterruptedException ignored) {
 90                      Thread.currentThread().interrupt();
 91                  }
 92              }
 93              ex.sendResponseHeaders(500, -1);
 94              ex.close();
 95          });
 96          server.setExecutor(java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor());
 97          server.start();
 98          servers.add(() -> server.stop(0));
 99          return URI.create("http://localhost:" + server.getAddress().getPort() + "/");
100      }
101
102      private AdDecisionService service(Map<String, URI> dsps, int auctionType, int campaigns) {
103          var catalog = AdCatalog.generate(campaigns, new SplittableRandom(42));
104          // Standard line items always compete; guaranteed ones (which would take every slot) only when
105          // the catalog is too small to have standard ones (the first five campaigns are guaranteed).
106          if (campaigns <= 5) {
107              return service(dsps, auctionType, catalog, new RecordingPacing(id -> 1.0));
108          }
109          var guaranteed = catalog.lineItems().stream()
110                  .filter(l -> l.tier() == dev.primetech.ads.common.catalog.LineItemConfig.Tier.GUARANTEED)
111                  .map(dev.primetech.ads.common.catalog.LineItemConfig::lineItemId).collect(java.util.stream.Colle
  ↪ ctors.toSet());
112          return service(dsps, auctionType, catalog, new RecordingPacing(id -> guaranteed.contains(id) ? 0.0 : 1.0
  ↪ ));
113      }
114
115      /** Pacing with fixed throttles that records what was charged. */
116      static final class RecordingPacing implements PacingView {
117          final java.util.function.ToDoubleFunction<String> throttles;
118          final Map<String, Long> charged = new java.util.concurrent.ConcurrentHashMap<>();
119
120          RecordingPacing(java.util.function.ToDoubleFunction<String> throttles) {
121              this.throttles = throttles;
122          }
123
124          @Override
125          public double throttle(String lineItemId) {
126              return throttles.applyAsDouble(lineItemId);
127          }
128
129          @Override
130          public void served(String lineItemId, long priceMicros) {
131              charged.merge(lineItemId, priceMicros, Long::sum);
132          }
133      }
134
135      private AdDecisionService service(Map<String, URI> dsps, int auctionType, AdCatalog catalog, PacingView paci
  ↪ ng) {
136          var http = DspClient.sharedHttpClient();
137          var clients = new ArrayList<DspClient>();
138          dsps.forEach((name, uri) -> clients.add(new DspClient(name, uri, http)));
139          metrics = new AdServerMetrics(new PrometheusMeterRegistry(PrometheusConfig.DEFAULT), new ArrayList<>(dsp
  ↪ s.keySet()));
140          return new AdDecisionService(new AdDecisionService.Settings(5, Duration.ofMillis(150), 5_000_000, auctio
  ↪ nType, 2),
141                  new DirectDemand(catalog, pacing), clients, new InMemoryFrequencyCaps(), sink,
142                  new DeviceIdHasher("test-secret-0123456789"), metrics);
143      }
144
145      private static AdRequest request(String device, int slots) {
146          return new AdRequest("sess-1", device, true, DeviceType.CTV, "US", slots, 30, slots * 30);
147      }
148
149      @Test
150      void fillsAPodFromDirectAndProgrammaticDemand() throws Exception {
151          // DSP bids around $80 CPM, above every direct line item ($20-60), so it wins where it bids.
152          var service = service(Map.of("good-dsp", goodDsp(80)), BidRequest.FIRST_PRICE, 20);
153
154          AdResponse response = service.decide(request("device-1", 3));
155
156          assertThat(response.ads()).hasSize(3);
```

```
157          assertThat(response.ads()).extracting(AdResponse.ServedAd::position).containsExactly(1, 2, 3);
158          assertThat(response.ads()).allSatisfy(ad -> assertThat(ad.auctionId()).isEqualTo(response.requestId()));
159          assertThat(response.ads()).extracting(AdResponse.ServedAd::advertiserId).doesNotHaveDuplicates();
160          assertThat(response.ads()).extracting(AdResponse.ServedAd::source).contains("good-dsp", Candidate.DIRECT
     ↪ );
161          var dspAd = response.ads().stream().filter(a -> a.source().equals("good-dsp")).findFirst().orElseThrow()
     ↪ ;
162          assertThat(dspAd.campaignId()).startsWith("good-dsp:");
163          assertThat(dspAd.priceMicros()).as("per-impression price from an ~$80 CPM").isBetween(70_000L, 90_000L);
164
165          AdDecision decision = sink.decisions.get(0);
166          assertThat(decision.requestId()).isEqualTo(response.requestId());
167          assertThat(decision.slotsFilled()).isEqualTo(3);
168          assertThat(decision.dsps()).singleElement().satisfies(d -> assertThat(d.status()).isEqualTo("BID"));
169          assertThat(sink.campaigns).filteredOn(c -> c.campaignId().equals(dspAd.campaignId())).singleElement()
170                  .satisfies(c -> assertThat(c.campaignName()).isEqualTo(dspAd.campaignId().substring("good-dsp:".
     ↪ length()) + " via good-dsp"));
171      }
172
173      @Test
174      void aSlowOrBrokenDspCostsNothingButItsBids() throws Exception {
175          var service = service(Map.of("slow", badDsp(true), "broken", badDsp(false)), BidRequest.FIRST_PRICE, 20)
     ↪ ;
176
177          long start = System.nanoTime();
178          AdResponse response = service.decide(request("device-2", 2));
179          long ms = (System.nanoTime() - start) / 1_000_000;
180
181          assertThat(ms).as("bounded by tmax, not by the slow DSP").isLessThan(600);
182          assertThat(response.ads()).as("direct demand still fills").hasSize(2);
183          assertThat(sink.decisions.get(0).dsps()).extracting(AdDecision.Dsp::status)
184                  .containsExactlyInAnyOrder("TIMEOUT", "ERROR");
185          @SuppressWarnings("unchecked")
186          var dsps = (Map<String, Map<String, Long>>) metrics.snapshot().get("dsps");
187          assertThat(dsps.get("slow").get("timeouts")).isEqualTo(1);
188          assertThat(dsps.get("broken").get("errors")).isEqualTo(1);
189      }
190
191      @Test
192      void frequencyCapsStopACampaignForOneDeviceOnly() {
193          // One campaign, capped at 2-4 per hour: the same device runs out, another device does not.
194          var service = service(Map.of(), BidRequest.FIRST_PRICE, 1);
195          var served = new ArrayList<String>();
196          for (int i = 0; i < 6; i++) {
197              service.decide(request("device-3", 1)).ads().forEach(a -> served.add(a.campaignId()));
198          }
199          assertThat(served).hasSizeBetween(2, 4);
200          assertThat(sink.decisions.get(5).frequencyCapped()).isPositive();
201
202          assertThat(service.decide(request("device-4", 1)).ads()).hasSize(1);
203      }
204
205      @Test
206      void withoutConsentNothingIsCappedOrForwarded() throws Exception {
207          var service = service(Map.of("good-dsp", goodDsp(80)), BidRequest.FIRST_PRICE, 1);
208          var anon = new AdRequest("sess-anon", "device-5", false, DeviceType.CTV, "US", 1, 30, 30);
209          var seen = new HashSet<Integer>();
210          for (int i = 0; i < 6; i++) {
211              seen.add(service.decide(anon).ads().size());
212          }
213          assertThat(seen).containsExactly(1);
214          assertThat(sink.decisions).allSatisfy(d -> assertThat(d.frequencyCapped()).isZero());
215      }
216
217      @Test
218      void retriesCampaignAnnouncementsThatFailed() {
219          var service = service(Map.of(), BidRequest.FIRST_PRICE, 1);
220          var campaign = new CampaignMetadata("c1", "Campaign", "adv", "Advertiser");
221
222          sink.failNextCampaigns = 1;
223          assertThat(service.announce(List.of(campaign))).isCompletedExceptionally();
224          assertThat(sink.campaigns).isEmpty();
225
226          assertThat(service.announce(List.of(campaign))).isCompleted();
227          service.announce(List.of(campaign));
228          assertThat(sink.campaigns).as("announced once it succeeds, then not again").hasSize(1);
229      }
230
231      @Test
232      void guaranteedLineItemsTakeTheFirstSlotsAndAreChargedToTheirBudget() throws Exception {
233          var catalog = AdCatalog.generate(20, new SplittableRandom(42));
234          var pacing = new RecordingPacing(id -> 1.0);
235          var service = service(Map.of("good-dsp", goodDsp(80)), BidRequest.FIRST_PRICE, catalog, pacing);
236
237          AdResponse response = service.decide(request("device-g", 2));
238
239          var guaranteedIds = catalog.lineItems().stream()
240                  .filter(l -> l.tier() == dev.primetech.ads.common.catalog.LineItemConfig.Tier.GUARANTEED)
```

```
                .map(dev.primetech.ads.common.catalog.LineItemConfig::lineItemId).toList();
        assertThat(response.ads()).as("sponsors beat $80 bids").allSatisfy(a -> assertThat(guaranteedIds).contai
    ↪ ns(a.lineItemId()));
        for (var ad : response.ads()) {
            assertThat(pacing.charged.get(ad.lineItemId())).isEqualTo(ad.priceMicros());
        }
    }

    @Test
    void aSpentLineItemNoLongerCompetes() {
        var catalog = AdCatalog.generate(1, new SplittableRandom(42));
        var service = service(Map.of(), BidRequest.FIRST_PRICE, catalog, new RecordingPacing(id -> 0.0));

        assertThat(service.decide(request("device-x", 2)).ads()).isEmpty();
    }

    @Test
    void validatesRequests() {
        var service = service(Map.of(), BidRequest.FIRST_PRICE, 5);
        assertThat(service.validate(request("d", 3))).isNull();
        assertThat(service.validate(new AdRequest(null, "d", true, DeviceType.CTV, "US", 3, 30, 90))).contains("
    ↪ sessionId");
        assertThat(service.validate(new AdRequest("s", "d", true, DeviceType.CTV, "usa", 3, 30, 90))).contains("
    ↪ country");
        assertThat(service.validate(new AdRequest("s", "d", true, DeviceType.CTV, "US", 9, 30, 90))).contains("s
    ↪ lots");
        assertThat(service.validate(new AdRequest("s", "d", null, DeviceType.CTV, "US", 3, 30, 90))).contains("c
    ↪ onsent");
    }
}
```

ad-server/src/test/java/dev/primetech/ads/adserver/AdServerHttpTest.java

94 lines · Test.

```
package dev.primetech.ads.adserver;

import com.sun.net.httpserver.HttpServer;
import dev.primetech.ads.common.DeviceIdHasher;
import dev.primetech.ads.common.catalog.AdCatalog;
import dev.primetech.ads.openrtb.BidRequest;
import io.micrometer.prometheusmetrics.PrometheusConfig;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.Test;

import java.net.InetSocketAddress;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.util.List;
import java.util.SplittableRandom;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.Executors;

import static org.assertj.core.api.Assertions.assertThat;

class AdServerHttpTest {
    private static final String BODY = """
            {"sessionId":"s1","deviceId":"d1","consent":true,"deviceType":"CTV","country":"US","slots":2}
            """;

    private HttpServer slowDsp;
    private AdServerHttp server;
    private final HttpClient http = HttpClient.newHttpClient();

    @AfterEach
    void stop() {
        server.close();
        slowDsp.stop(0);
    }

    private void start(int maxInFlight) throws Exception {
        slowDsp = HttpServer.create(new InetSocketAddress(0), 0);
        slowDsp.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        slowDsp.createContext("/", ex -> {
            try {
                Thread.sleep(800);
            } catch (InterruptedException ignored) {
                Thread.currentThread().interrupt();
            }
            ex.sendResponseHeaders(204, -1);
            ex.close();
        });
        slowDsp.start();

        var registry = new PrometheusMeterRegistry(PrometheusConfig.DEFAULT);
```

```
        var metrics = new AdServerMetrics(registry, List.of("slow"));
        var dsp = new DspClient("slow", URI.create("http://localhost:" + slowDsp.getAddress().getPort() + "/"),
                DspClient.sharedHttpClient());
        var service = new AdDecisionService(
                new AdDecisionService.Settings(5, Duration.ofMillis(700), 5_000_000, BidRequest.FIRST_PRICE, 4),
                new DirectDemand(AdCatalog.generate(10, new SplittableRandom(1)), PacingView.fixed(1.0)), List.of(dsp),
                new InMemoryFrequencyCaps(), new AdDecisionServiceTest.RecordingSink(),
                new DeviceIdHasher("test-secret-0123456789"), metrics);
        server = new AdServerHttp(0, service, metrics, registry, "*", maxInFlight);
        server.start();
    }

    private CompletableFuture<HttpResponse<String>> post(String body) {
        return http.sendAsync(HttpRequest.newBuilder(URI.create("http://localhost:" + server.port() + "/v1/ad-requests"))
                .POST(HttpRequest.BodyPublishers.ofString(body)).build(), HttpResponse.BodyHandlers.ofString());
    }

    @Test
    void shedsLoadInsteadOfQueueing() throws Exception {
        start(1);
        var first = post(BODY);
        Thread.sleep(150); // the first request is now waiting on the slow DSP
        long start = System.nanoTime();
        var second = post(BODY).get();
        long ms = (System.nanoTime() - start) / 1_000_000;

        assertThat(second.statusCode()).isEqualTo(503);
        assertThat(second.headers().firstValue("Retry-After")).contains("1");
        assertThat(ms).as("rejected immediately, not after the slow DSP").isLessThan(300);
        assertThat(first.get().statusCode()).isEqualTo(200);
    }

    @Test
    void rejectsInvalidRequests() throws Exception {
        start(4);
        assertThat(post("{not json").get().statusCode()).isEqualTo(400);
        assertThat(post(BODY.replace("\"US\"", "\"usa\"")).get().statusCode()).isEqualTo(400);
        assertThat(post(BODY.replace("\"slots\":2", "\"slots\":9")).get().statusCode()).isEqualTo(400);
    }
}
```

ad-server/src/test/java/dev/primetech/ads/adserver/AuctionTest.java

110 lines · Test.

```
package dev.primetech.ads.adserver;

import dev.primetech.ads.openrtb.BidRequest;
import org.junit.jupiter.api.Test;

import java.util.List;
import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;

class AuctionTest {
    private static final long FLOOR = 5_000_000;

    static Candidate direct(String advertiser, long cpm, int seconds) {
        return new Candidate(Candidate.DIRECT, null, "camp-" + advertiser, "li-" + advertiser, "ad-" + advertiser,
                "cr-" + advertiser + "-" + seconds, advertiser, seconds, cpm, 3, null, false);
    }

    static Candidate dsp(String dsp, int slot, String advertiser, long cpm, int seconds) {
        return new Candidate(dsp, String.valueOf(slot), dsp + ":" + advertiser, dsp + ":" + advertiser,
                dsp + ":cr-" + advertiser, dsp + ":cr-" + advertiser + slot, advertiser, seconds, cpm, 4, "http://win", false);
    }

    private static List<Auction.Winner> run(int slots, int podSeconds, Map<Integer, List<Candidate>> bySlot, int type) {
        return Auction.run(slots, 30, podSeconds, p -> bySlot.getOrDefault(p, List.of()), type, FLOOR);
    }

    @Test
    void highestOfferWinsAndPaysItsBidInFirstPrice() {
        var winners = run(1, 30, Map.of(1, List.of(
                direct("a", 20_000_000, 30), dsp("dsp", 1, "b", 31_500_000, 15), dsp("dsp2", 1, "c", 25_000_000, 15))),
                BidRequest.FIRST_PRICE);

        assertThat(winners).singleElement().satisfies(w -> {
            assertThat(w.candidate().advertiserId()).isEqualTo("b");
            assertThat(w.clearingCpmMicros()).isEqualTo(31_500_000);
            assertThat(w.eligibleCandidates()).isEqualTo(3);
        });
```

```
    }

    @Test
    void secondPricePaysRunnerUpPlusOneCentButNeverMoreThanTheBid() {
        var winners = run(1, 30, Map.of(1, List.of(dsp("dsp", 1, "b", 31_500_000, 15), dsp("dsp2", 1, "c", 25_000_000, 15))),
                BidRequest.SECOND_PRICE);
        assertThat(winners.get(0).clearingCpmMicros()).isEqualTo(25_010_000);

        var tied = run(1, 30, Map.of(1, List.of(dsp("dsp", 1, "b", 25_000_000, 15), dsp("dsp2", 1, "c", 25_000_000, 15))),
                BidRequest.SECOND_PRICE);
        assertThat(tied.get(0).clearingCpmMicros()).as("capped at the winner's bid").isEqualTo(25_000_000);

        var alone = run(1, 30, Map.of(1, List.of(dsp("dsp", 1, "b", 31_500_000, 15))), BidRequest.SECOND_PRICE);
        assertThat(alone.get(0).clearingCpmMicros()).as("sole bidder pays the floor").isEqualTo(FLOOR);
    }

    @Test
    void directLineItemsPayTheirContractedCpm() {
        var winners = run(1, 30, Map.of(1, List.of(direct("a", 40_000_000, 30), dsp("dsp", 1, "b", 10_000_000, 15))),
                BidRequest.SECOND_PRICE);
        assertThat(winners.get(0).candidate().isDirect()).isTrue();
        assertThat(winners.get(0).clearingCpmMicros()).isEqualTo(40_000_000);
    }

    @Test
    void ignoresBidsUnderTheFloor() {
        assertThat(run(1, 30, Map.of(1, List.of(dsp("dsp", 1, "b", 4_990_000, 15))), BidRequest.FIRST_PRICE)).isEmpty();
    }

    @Test
    void anAdvertiserAppearsAtMostOncePerBreak() {
        var a = direct("a", 50_000_000, 15);
        var winners = run(3, 90, Map.of(
                1, List.of(a, dsp("dsp", 1, "b", 30_000_000, 15)),
                2, List.of(a, dsp("dsp", 2, "a", 45_000_000, 15), dsp("dsp", 2, "c", 20_000_000, 15)),
                3, List.of(a)), BidRequest.FIRST_PRICE);

        assertThat(winners).extracting(w -> w.candidate().advertiserId()).containsExactly("a", "c");
        assertThat(winners).extracting(Auction.Winner::position).containsExactly(1, 2);
    }

    @Test
    void adsMustFitTheRemainingBreak() {
        var winners = run(3, 45, Map.of(
                1, List.of(direct("a", 50_000_000, 30)),
                2, List.of(direct("b", 40_000_000, 30), dsp("dsp", 2, "c", 10_000_000, 15)),
                3, List.of(direct("d", 40_000_000, 15))), BidRequest.FIRST_PRICE);

        assertThat(winners).extracting(w -> w.candidate().advertiserId()).containsExactly("a", "c");
        assertThat(winners.stream().mapToInt(w -> w.candidate().durationSeconds()).sum()).isLessThanOrEqualTo(45);
    }

    @Test
    void guaranteedLineItemsWinAheadOfHigherBids() {
        var sponsor = new Candidate(Candidate.DIRECT, null, "camp-g", "li-g", "ad-g", "cr-g", "g", 15, 25_000_000, 3,
                null, true);
        var winners = run(2, 60, Map.of(
                1, List.of(dsp("dsp", 1, "b", 80_000_000, 15), sponsor),
                2, List.of(dsp("dsp", 2, "b", 80_000_000, 15), sponsor)), BidRequest.FIRST_PRICE);

        assertThat(winners.get(0).candidate().guaranteed()).isTrue();
        assertThat(winners.get(0).clearingCpmMicros()).as("pays its contracted CPM").isEqualTo(25_000_000);
        assertThat(winners.get(1).candidate().advertiserId()).as("separation still applies").isEqualTo("b");
    }

    @Test
    void tiesGoToDirectDemand() {
        var winners = run(1, 30, Map.of(1, List.of(dsp("dsp", 1, "b", 30_000_000, 15), direct("a", 30_000_000, 15))),
                BidRequest.FIRST_PRICE);
        assertThat(winners.get(0).candidate().isDirect()).isTrue();
    }
}
```

ad-server/src/test/java/dev/primetech/ads/adserver/RedisPacingViewTest.java

31 lines · Test.

```
package dev.primetech.ads.adserver;

import dev.primetech.ads.common.catalog.LineItemConfig;
import dev.primetech.ads.common.pacing.PacingPeriod;
import io.micrometer.core.instrument.simple.SimpleMeterRegistry;
```

```
import org.junit.jupiter.api.Test;

import java.time.Duration;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;

class RedisPacingViewTest {
    @Test
    void withoutPacingStateFallsBackToASmallThrottleAndStillStopsAtTheBudget() {
        var li = new LineItemConfig("li-1", "camp", "adv", LineItemConfig.Tier.STANDARD, 30_000_000, 8_000_000);
        // Nothing listens on port 1: every Redis call fails, as when Redis is down.
        try (var view = new RedisPacingView("redis://localhost:1", PacingPeriod.of(Duration.ofMinutes(30)), List
 ↪ .of(li),
                0.01, System::currentTimeMillis, new SimpleMeterRegistry())) {
            assertThat(view.throttle("li-1")).isEqualTo(0.01);
            assertThat(view.throttle("unknown")).isZero();

            // Without pacing state the stop is the budget itself in served spend ($8): delivery
            // never exceeds what was served, so this cannot overspend.
            view.served("li-1", 7_990_000);
            assertThat(view.throttle("li-1")).isEqualTo(0.01);
            view.served("li-1", 10_000);
            assertThat(view.throttle("li-1")).isZero();
        }
    }
}
```

ad-server/src/test/java/dev/primetech/ads/adserver/ServedLedgerTest.java

80 lines · Test.

```
package dev.primetech.ads.adserver;

import org.junit.jupiter.api.Test;

import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;

class ServedLedgerTest {
    private static final long P = 42;
    private static final String PREFIX = "pacing:served:42:";
    private static final String KEY = PREFIX + "li-1";

    private final ServedLedger ledger = new ServedLedger();

    @Test
    void seesItsOwnSpendingImmediately() {
        ledger.add(KEY, 100);
        ledger.add(KEY, 50);
        assertThat(ledger.served(P, "li-1", KEY)).isEqualTo(150);
    }

    @Test
    void failedWritesLoseNothing() {
        // Found live: per-request increments that failed while the stack warmed up were simply
        // dropped, and the counters ended 3-6% low. Totals that fail to write stay unwritten,
        // and the next write carries all of them.
        ledger.add(KEY, 100);
        Map<String, Long> attempt1 = ledger.unwritten(); // ... and Redis times out: not marked written
        ledger.add(KEY, 30);
        Map<String, Long> attempt2 = ledger.unwritten();

        assertThat(attempt1).containsEntry(KEY, 100L);
        assertThat(attempt2).containsEntry(KEY, 130L);
        ledger.written(attempt2);
        assertThat(ledger.unwritten()).isEmpty();
        assertThat(ledger.unwrittenMicros()).isZero();
    }

    @Test
    void combinesOtherInstancesWithItsOwnLatestTotal() {
        ledger.add(KEY, 100);
        ledger.written(ledger.unwritten()); // Redis now holds 100 for this instance
        // Read back: this instance's 100 plus another instance's 400.
        ledger.snapshot(P, Map.of("li-1", 500L), PREFIX);
        ledger.add(KEY, 20); // served after the read

        assertThat(ledger.served(P, "li-1", KEY)).isEqualTo(520);
    }

    @Test
    void aWriteThatFailedBeforeTheReadIsNotCountedTwiceOrMissed() {
        ledger.add(KEY, 100);
        ledger.written(ledger.unwritten()); // 100 written
        ledger.add(KEY, 50); // 150 total, but this write fails: Redis still holds 100 for us
        ledger.snapshot(P, Map.of("li-1", 100L + 400L), PREFIX);

```

```
58          assertThat(ledger.served(P, "li-1", KEY)).isEqualTo(550);
59      }
60
61      @Test
62      void anotherPeriodStartsFromThisInstancesOwnSpend() {
63          ledger.snapshot(P - 1, Map.of("li-1", 999L), "pacing:served:41:");
64          ledger.add(KEY, 10);
65          assertThat(ledger.served(P, "li-1", KEY)).isEqualTo(10);
66      }
67
68      @Test
69      void forgetsFinishedPeriodsOnlyOnceWritten() {
70          String old = "pacing:served:41:li-1";
71          ledger.add(old, 70);
72          ledger.prune(PREFIX);
73          assertThat(ledger.unwritten()).containsEntry(old, 70L); // not written yet: kept
74
75          ledger.written(ledger.unwritten());
76          ledger.prune(PREFIX);
77          assertThat(ledger.unwritten()).isEmpty();
78          assertThat(ledger.unwrittenMicros()).isZero();
79      }
80  }
```

ads-common

ads-common/pom.xml

27 lines

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4           xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
5    <modelVersion>4.0.0</modelVersion>
6    <parent>
7      <groupId>dev.primetech.ads</groupId>
8      <artifactId>ads-eventing</artifactId>
9      <version>0.1.0-SNAPSHOT</version>
10   </parent>
11
12   <artifactId>ads-common</artifactId>
13   <name>Ads Common (event model, validation, privacy)</name>
14
15   <dependencies>
16     <dependency>
17       <groupId>com.fasterxml.jackson.core</groupId>
18       <artifactId>jackson-databind</artifactId>
19     </dependency>
20     <!-- Only for pacing.ServedSpend. Optional, so it does not end up in the Flink job's jar. -->
21     <dependency>
22       <groupId>redis.clients</groupId>
23       <artifactId>jedis</artifactId>
24       <optional>true</optional>
25     </dependency>
26   </dependencies>
27 </project>
```

ads-common/src/main/java/dev/primetech/ads/common/AdEvent.java

33 lines · Canonical ad event written to the Topics#AD_EVENTS topic.

```
1  package dev.primetech.ads.common;
2
3  /**
4   * Canonical ad event written to the {@link Topics#AD_EVENTS} topic. It never contains a raw device
5   * identifier: {@code deviceIdHash} is a keyed hash and is null when the viewer did not consent.
6   */
7  public record AdEvent(
8          int schemaVersion,
9          String eventId,
10         EventType type,
11         long eventTimeMs,
12         long receivedTimeMs,
13         String sessionId,
14         String adId,
15         String creativeId,
16         String campaignId,
17         String lineItemId,
18         String deviceIdHash,
19         DeviceType deviceType,
20         String country,
21         boolean consent,
22         Long priceMicros,
23         String auctionId) {
24
25     public static final int SCHEMA_VERSION = 1;
26
```

```
    public AdEvent(int schemaVersion, String eventId, EventType type, long eventTimeMs, long receivedTimeMs,
                   String sessionId, String adId, String creativeId, String campaignId, String lineItemId,
                   String deviceIdHash, DeviceType deviceType, String country, boolean consent, Long priceMicros
) {
        this(schemaVersion, eventId, type, eventTimeMs, receivedTimeMs, sessionId, adId, creativeId, campaignId,
                lineItemId, deviceIdHash, deviceType, country, consent, priceMicros, null);
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/CampaignMetadata.java

12 lines · Reference data the stream processor joins onto every event.

```
package dev.primetech.ads.common;

/**
 * Reference data the stream processor joins onto every event. Published to the compacted
 * {@link Topics#CAMPAIGN_METADATA} topic keyed by campaign id, so the latest value per campaign wins.
 */
public record CampaignMetadata(
        String campaignId,
        String campaignName,
        String advertiserId,
        String advertiserName) {
}
```

ads-common/src/main/java/dev/primetech/ads/common/ClientEvent.java

45 lines · An event as a player sends it to the ingest gateway.

```
package dev.primetech.ads.common;

/**
 * An event as a player sends it to the ingest gateway. It may carry a raw device id, so it
 * must never be written to Kafka as-is; {@link EventNormalizer} turns it into an {@link AdEvent}.
 *
 * @param eventId       client-generated UUID; the idempotency key used for de-duplication downstream
 * @param type          playback event type
 * @param eventTimeMs   when the event happened on the device (epoch millis)
 * @param sessionId     playback session; all events of one ad break share it
 * @param adId          ad (creative placement) that was shown
 * @param creativeId    creative asset
 * @param campaignId    advertiser campaign
 * @param lineItemId    line item inside the campaign that won the decision
 * @param deviceId      raw device identifier, only used when {@code consent} is true
 * @param deviceType    device class
 * @param country       ISO 3166-1 alpha-2 country code
 * @param consent       whether the viewer allows identifier-based ad tracking
 * @param priceMicros   clearing price in micro-currency units; only set on impressions
 * @param auctionId     the ad server decision that served this ad, when the ad came from one
 */
public record ClientEvent(
        String eventId,
        EventType type,
        Long eventTimeMs,
        String sessionId,
        String adId,
        String creativeId,
        String campaignId,
        String lineItemId,
        String deviceId,
        DeviceType deviceType,
        String country,
        Boolean consent,
        Long priceMicros,
        String auctionId) {

    /** Event without an auction id (ads not served by the ad server). */
    public ClientEvent(String eventId, EventType type, Long eventTimeMs, String sessionId, String adId,
                       String creativeId, String campaignId, String lineItemId, String deviceId,
                       DeviceType deviceType, String country, Boolean consent, Long priceMicros) {
        this(eventId, type, eventTimeMs, sessionId, adId, creativeId, campaignId, lineItemId, deviceId, deviceTy
pe,
                country, consent, priceMicros, null);
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/DeadLetter.java

16 lines · A rejected event written to Topics#AD_EVENTS_DLQ.

```
package dev.primetech.ads.common;

/**
 * A rejected event written to {@link Topics#AD_EVENTS_DLQ}. The original event is kept for
 * debugging and replay, but with its raw device id removed: the DLQ gets the same privacy rules
 * as the main topic.
 */
public record DeadLetter(RejectReason reason, long receivedTimeMs, ClientEvent event) {

    public static DeadLetter of(RejectReason reason, long receivedTimeMs, ClientEvent e) {
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
        var scrubbed = new ClientEvent(e.eventId(), e.type(), e.eventTimeMs(), e.sessionId(), e.adId(),
                e.creativeId(), e.campaignId(), e.lineItemId(), null, e.deviceType(), e.country(), e.consent(),
                e.priceMicros(), e.auctionId());
        return new DeadLetter(reason, receivedTimeMs, scrubbed);
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/DeviceIdHasher.java

36 lines · Pseudonymises device ids with HMAC-SHA256.

```
package dev.primetech.ads.common;

import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.security.GeneralSecurityException;
import java.util.HexFormat;

/**
 * Pseudonymises device ids with HMAC-SHA256. A plain hash of a device id can be reversed by
 * hashing every known id; a keyed hash cannot without the secret, and rotating the secret
 * unlinks old and new data.
 */
public final class DeviceIdHasher {
    private static final String ALGORITHM = "HmacSHA256";

    private final SecretKeySpec key;

    public DeviceIdHasher(String secret) {
        if (secret == null || secret.length() < 16) {
            throw new IllegalArgumentException("device id hashing secret must be at least 16 characters");
        }
        this.key = new SecretKeySpec(secret.getBytes(StandardCharsets.UTF_8), ALGORITHM);
    }

    public String hash(String deviceId) {
        try {
            // Mac instances are not thread-safe and cheap to create relative to a network request.
            Mac mac = Mac.getInstance(ALGORITHM);
            mac.init(key);
            return HexFormat.of().formatHex(mac.doFinal(deviceId.getBytes(StandardCharsets.UTF_8)));
        } catch (GeneralSecurityException ex) {
            throw new IllegalStateException("HmacSHA256 unavailable", ex);
        }
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/DeviceType.java

8 lines · Device types accepted on events (CTV, mobile, web, ...).

```
package dev.primetech.ads.common;

public enum DeviceType {
    CTV,
    MOBILE,
    WEB,
    GAME_CONSOLE
}
```

ads-common/src/main/java/dev/primetech/ads/common/EnrichedAdEvent.java

64 lines · A de-duplicated event with campaign metadata attached and an invalid-traffic verdict.

```
package dev.primetech.ads.common;

/**
 * A de-duplicated event with campaign metadata attached and an invalid-traffic verdict. Written
 * to {@link Topics#AD_EVENTS_CLEAN}, the topic reporting (Druid) and billing read from.
 *
 * <p>Flat rather than wrapping {@link AdEvent} so analytics stores can ingest it without
 * flattening rules. IVT events are kept and flagged, not dropped: advertisers are entitled to see
 * what was filtered, and the decision can be revisited.
 *
 * @param campaignName   null if metadata for the campaign never arrived
 * @param ivtReason      null for valid traffic, otherwise why the event is not billable
 * @param processedTimeMs when the stream processor emitted this record (processing time)
 * @param auctionId      the ad server decision that served the ad, if any (joins to ad-decisions.v1)
 */
public record EnrichedAdEvent(
        int schemaVersion,
        String eventId,
        EventType type,
        long eventTimeMs,
        long receivedTimeMs,
        long processedTimeMs,
        String sessionId,
        String adId,
        String creativeId,
        String campaignId,
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
        String lineItemId,
        String campaignName,
        String advertiserId,
        String advertiserName,
        String deviceIdHash,
        DeviceType deviceType,
        String country,
        boolean consent,
        Long priceMicros,
        String ivtReason,
        String auctionId) {

    public static final int SCHEMA_VERSION = 1;

    public static EnrichedAdEvent of(AdEvent e, CampaignMetadata meta, long processedTimeMs) {
        return new EnrichedAdEvent(SCHEMA_VERSION, e.eventId(), e.type(), e.eventTimeMs(), e.receivedTimeMs(),
                processedTimeMs, e.sessionId(), e.adId(), e.creativeId(), e.campaignId(), e.lineItemId(),
                meta == null ? null : meta.campaignName(),
                meta == null ? null : meta.advertiserId(),
                meta == null ? null : meta.advertiserName(),
                e.deviceIdHash(), e.deviceType(), e.country(), e.consent(), e.priceMicros(), null, e.auctionId()
    );
    }

    public boolean isIvt() {
        return ivtReason != null;
    }

    /** Billable spend: impressions only, and never invalid traffic. */
    public long billableMicros() {
        return !isIvt() && priceMicros != null ? priceMicros : 0;
    }

    public EnrichedAdEvent withIvt(String reason) {
        return new EnrichedAdEvent(schemaVersion, eventId, type, eventTimeMs, receivedTimeMs, processedTimeMs,
                sessionId, adId, creativeId, campaignId, lineItemId, campaignName, advertiserId, advertiserName,
                deviceIdHash, deviceType, country, consent, priceMicros, reason, auctionId);
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/EventNormalizer.java

34 lines · Converts a validated ClientEvent to the canonical AdEvent, applying privacy rules.

```
package dev.primetech.ads.common;

/** Converts a validated {@link ClientEvent} to the canonical {@link AdEvent}, applying privacy rules. */
public final class EventNormalizer {
    private final DeviceIdHasher hasher;

    public EventNormalizer(DeviceIdHasher hasher) {
        this.hasher = hasher;
    }

    public AdEvent normalize(ClientEvent e, long receivedTimeMs) {
        // Without consent the identifier is dropped entirely, not just hashed.
        String deviceIdHash = e.consent() && e.deviceId() != null && !e.deviceId().isBlank()
                ? hasher.hash(e.deviceId())
                : null;
        return new AdEvent(
                AdEvent.SCHEMA_VERSION,
                e.eventId(),
                e.type(),
                e.eventTimeMs(),
                receivedTimeMs,
                e.sessionId(),
                e.adId(),
                e.creativeId(),
                e.campaignId(),
                e.lineItemId(),
                deviceIdHash,
                e.deviceType(),
                e.country(),
                e.consent(),
                e.priceMicros(),
                e.auctionId());
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/EventType.java

19 lines · Client-side ad playback events, modelled on the VAST tracking events a CTV player fires.

```
package dev.primetech.ads.common;

/**
 * Client-side ad playback events, modelled on the VAST tracking events a CTV player fires.
 * Quartiles are cumulative: a COMPLETE implies the player also passed every earlier quartile.
 */
public enum EventType {
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
    IMPRESSION,
    FIRST_QUARTILE,
    MIDPOINT,
    THIRD_QUARTILE,
    COMPLETE,
    CLICK;

    /** Only impressions are billable, so only they carry a clearing price. */
    public boolean isBillable() {
        return this == IMPRESSION;
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/EventValidator.java

79 lines · Stateless, per-event checks done at the edge.

```
package dev.primetech.ads.common;

import java.time.Duration;
import java.util.Optional;

/**
 * Stateless, per-event checks done at the edge. Anything that needs state (duplicates,
 * invalid-traffic patterns) is left to the stream processor.
 */
public final class EventValidator {
    static final int MAX_ID_LENGTH = 64;

    private final long maxFutureSkewMs;
    private final long maxAgeMs;

    /**
     * @param maxFutureSkew how far ahead of server time a device clock may be
     * @param maxAge        oldest event accepted; late events within this bound are kept so the
     *                      stream processor can still count them
     */
    public EventValidator(Duration maxFutureSkew, Duration maxAge) {
        this.maxFutureSkewMs = maxFutureSkew.toMillis();
        this.maxAgeMs = maxAge.toMillis();
    }

    public static EventValidator withDefaults() {
        return new EventValidator(Duration.ofMinutes(5), Duration.ofHours(24));
    }

    /** Returns the first problem found, or empty if the event is acceptable. */
    public Optional<RejectReason> validate(ClientEvent e, long nowMs) {
        if (e.type() == null) {
            return Optional.of(RejectReason.UNKNOWN_EVENT_TYPE);
        }
        if (isBlank(e.eventId()) || isBlank(e.sessionId()) || isBlank(e.adId())
                || isBlank(e.creativeId()) || isBlank(e.campaignId()) || isBlank(e.lineItemId())
                || e.eventTimeMs() == null || e.deviceType() == null || e.country() == null
                || e.consent() == null) {
            return Optional.of(RejectReason.MISSING_FIELD);
        }
        if (tooLong(e.eventId()) || tooLong(e.sessionId()) || tooLong(e.adId())
                || tooLong(e.creativeId()) || tooLong(e.campaignId()) || tooLong(e.lineItemId())
                || tooLong(e.deviceId()) || tooLong(e.auctionId())) {
            return Optional.of(RejectReason.FIELD_TOO_LONG);
        }
        if (!isCountryCode(e.country())) {
            return Optional.of(RejectReason.INVALID_COUNTRY);
        }
        if (e.eventTimeMs() > nowMs + maxFutureSkewMs) {
            return Optional.of(RejectReason.EVENT_TIME_IN_FUTURE);
        }
        if (e.eventTimeMs() < nowMs - maxAgeMs) {
            return Optional.of(RejectReason.EVENT_TIME_TOO_OLD);
        }
        if (e.type().isBillable()) {
            if (e.priceMicros() == null) {
                return Optional.of(RejectReason.MISSING_PRICE);
            }
            if (e.priceMicros() < 0) {
                return Optional.of(RejectReason.NEGATIVE_PRICE);
            }
        } else if (e.priceMicros() != null) {
            return Optional.of(RejectReason.PRICE_ON_NON_BILLABLE);
        }
        return Optional.empty();
    }

    private static boolean isBlank(String s) {
        return s == null || s.isBlank();
    }

    private static boolean tooLong(String s) {
```

```
73          return s != null && s.length() > MAX_ID_LENGTH;
74      }
75
76      private static boolean isCountryCode(String s) {
77          return s.length() == 2 && Character.isUpperCase(s.charAt(0)) && Character.isUpperCase(s.charAt(1));
78      }
79  }
```

ads-common/src/main/java/dev/primetech/ads/common/IngestRequest.java

7 lines · Body of POST /v1/events.

```
1  package dev.primetech.ads.common;
2
3  import java.util.List;
4
5  /** Body of {@code POST /v1/events}. Players batch events to save radio and connection overhead. */
6  public record IngestRequest(List<ClientEvent> events) {
7  }
```

ads-common/src/main/java/dev/primetech/ads/common/IngestResponse.java

13 lines · Per-batch result.

```
 1  package dev.primetech.ads.common;
 2
 3  import java.util.List;
 4
 5  /**
 6   * Per-batch result. Rejections are permanent (the client must not retry them); a non-2xx status
 7   * means nothing in the batch is guaranteed durable and the whole batch should be retried.
 8   */
 9  public record IngestResponse(int accepted, int rejected, List<Rejection> errors) {
10
11      public record Rejection(int index, String eventId, RejectReason reason) {
12      }
13  }
```

ads-common/src/main/java/dev/primetech/ads/common/Json.java

18 lines · One shared, thread-safe mapper so every service reads and writes events the same way.

```
 1  package dev.primetech.ads.common;
 2
 3  import com.fasterxml.jackson.annotation.JsonInclude;
 4  import com.fasterxml.jackson.databind.DeserializationFeature;
 5  import com.fasterxml.jackson.databind.ObjectMapper;
 6
 7  /** One shared, thread-safe mapper so every service reads and writes events the same way. */
 8  public final class Json {
 9      public static final ObjectMapper MAPPER = new ObjectMapper()
10              .setSerializationInclusion(JsonInclude.Include.NON_NULL)
11              // Clients may be newer than the gateway; extra fields must not fail a whole batch.
12              .configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false)
13              // An unknown event type becomes null and is rejected per event by the validator.
14              .configure(DeserializationFeature.READ_UNKNOWN_ENUM_VALUES_AS_NULL, true);
15
16      private Json() {
17      }
18  }
```

ads-common/src/main/java/dev/primetech/ads/common/LineItemMinute.java

30 lines · Per line item, per event-time minute delivery totals, written to Topics#LINE_ITEM_MINUTES.

```
 1  package dev.primetech.ads.common;
 2
 3  /**
 4   * Per line item, per event-time minute delivery totals, written to {@link Topics#LINE_ITEM_MINUTES}.
 5   *
 6   * <p>The topic is an upsert stream: when late events arrive inside the allowed lateness, the
 7   * minute is emitted again with a higher {@code revision} and the full corrected totals (not a
 8   * delta). Readers keep the highest revision per {@link #key()}; the topic is compacted on that key.
 9   */
10  public record LineItemMinute(
11          String lineItemId,
12          String campaignId,
13          String advertiserId,
14          long windowStartMs,
15          long windowEndMs,
16          int revision,
17          long impressions,
18          long firstQuartiles,
19          long midpoints,
20          long thirdQuartiles,
21          long completes,
22          long clicks,
23          long spendMicros,
24          long ivtEvents,
25          long emittedAtMs) {
26
27      public String key() {
```

```
28          return lineItemId + "|" + windowStartMs;
29      }
30  }
```

ads-common/src/main/java/dev/primetech/ads/common/RejectReason.java

13 lines · Reasons an event is rejected at the gateway (sent to the dead-letter topic).

```
 1  package dev.primetech.ads.common;
 2
 3  public enum RejectReason {
 4      MISSING_FIELD,
 5      UNKNOWN_EVENT_TYPE,
 6      FIELD_TOO_LONG,
 7      INVALID_COUNTRY,
 8      EVENT_TIME_IN_FUTURE,
 9      EVENT_TIME_TOO_OLD,
10      PRICE_ON_NON_BILLABLE,
11      MISSING_PRICE,
12      NEGATIVE_PRICE
13  }
```

ads-common/src/main/java/dev/primetech/ads/common/Topics.java

23 lines · Names of every Kafka topic, shared by all services.

```
 1  package dev.primetech.ads.common;
 2
 3  public final class Topics {
 4      /** Validated, privacy-scrubbed events, keyed by session id. May contain duplicates. */
 5      public static final String AD_EVENTS = "ad-events.v1";
 6      /** Events that failed validation, kept with the reason for auditing and replay. */
 7      public static final String AD_EVENTS_DLQ = "ad-events-dlq.v1";
 8      /** Campaign reference data, compacted, keyed by campaign id. */
 9      public static final String CAMPAIGN_METADATA = "campaign-metadata.v1";
10      /** De-duplicated, enriched, IVT-flagged events, written transactionally. Keyed by session id. */
11      public static final String AD_EVENTS_CLEAN = "ad-events-clean.v1";
12      /** Per line item per minute delivery totals, an upsert stream keyed by line item and minute. */
13      public static final String LINE_ITEM_MINUTES = "line-item-minutes.v1";
14      /** One record per ad request: candidates, bids, winners and clearing prices. Keyed by request id. */
15      public static final String AD_DECISIONS = "ad-decisions.v1";
16      /** Direct line item terms (tier, budget), compacted, keyed by line item id. */
17      public static final String LINE_ITEMS = "line-items.v1";
18      /** Results of the audit service's reconciliations, one record per check run. */
19      public static final String AUDIT_RESULTS = "audit-results.v1";
20
21      private Topics() {
22      }
23  }
```

ads-common/src/main/java/dev/primetech/ads/common/adserving/AdDecision.java

37 lines · Audit record of one ad request, written to ad-decisions.v1: what was asked for, who bid, who won each slot and at what price.

```
 1  package dev.primetech.ads.common.adserving;
 2
 3  import dev.primetech.ads.common.DeviceType;
 4
 5  import java.util.List;
 6
 7  /**
 8   * Audit record of one ad request, written to {@code ad-decisions.v1}: what was asked for, who bid,
 9   * who won each slot and at what price. With the auction id on tracking events, it lets reporting
10   * compare what was sold with what was actually played.
11   */
12  public record AdDecision(
13          String requestId,
14          long timestampMs,
15          String sessionId,
16          DeviceType deviceType,
17          String country,
18          boolean consent,
19          int slotsRequested,
20          int slotsFilled,
21          int auctionType,
22          long floorCpmMicros,
23          long latencyMs,
24          int directCandidates,
25          int frequencyCapped,
26          List<Slot> slots,
27          List<Dsp> dsps) {
28
29      /** @param bidCpmMicros the winner's bid; {@code clearingCpmMicros} is what it pays */
30      public record Slot(int position, String source, String campaignId, String lineItemId, String advertiserId,
31                         int durationSeconds, long bidCpmMicros, long clearingCpmMicros, int candidates) {
32      }
33
34      /** @param status BID, NO_BID, TIMEOUT or ERROR */
35      public record Dsp(String name, String status, int bids, long latencyMs) {
36      }
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
}
```

ads-common/src/main/java/dev/primetech/ads/common/adserving/AdRequest.java

22 lines · A player asking for the ads of one ad break (pod).

```
package dev.primetech.ads.common.adserving;

import dev.primetech.ads.common.DeviceType;

/**
 * A player asking for the ads of one ad break (pod). Like tracking events, it may carry a raw
 * device id; the ad server pseudonymises it before using or forwarding it.
 *
 * @param slots              most ads the player will show in this break
 * @param maxAdSeconds       longest single ad that fits a slot
 * @param podSeconds         total length of the break
 */
public record AdRequest(
        String sessionId,
        String deviceId,
        Boolean consent,
        DeviceType deviceType,
        String country,
        Integer slots,
        Integer maxAdSeconds,
        Integer podSeconds) {
}
```

ads-common/src/main/java/dev/primetech/ads/common/adserving/AdResponse.java

27 lines · The ads to play, in order.

```
package dev.primetech.ads.common.adserving;

import java.util.List;

/**
 * The ads to play, in order. Fewer ads than slots requested (possibly none) means the rest went
 * unfilled. Players report {@link ServedAd#auctionId()} and the price on their tracking events.
 */
public record AdResponse(String requestId, List<ServedAd> ads) {

    /**
     * @param source       {@code direct} for a direct-sold line item, otherwise the winning DSP's name
     * @param priceMicros  clearing price of one impression, in micro-dollars
     */
    public record ServedAd(
            int position,
            String auctionId,
            String adId,
            String creativeId,
            String campaignId,
            String lineItemId,
            String advertiserId,
            int durationSeconds,
            long priceMicros,
            String source) {
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/catalog/AdCatalog.java

135 lines · Synthetic direct-sold campaigns, line items and creatives: the stand-in for a campaign management system.

```
package dev.primetech.ads.common.catalog;

import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.DeviceType;

import java.util.ArrayList;
import java.util.EnumSet;
import java.util.List;
import java.util.Set;
import java.util.random.RandomGenerator;

/**
 * Synthetic direct-sold campaigns, line items and creatives: the stand-in for a campaign
 * management system. Ids are deterministic for a given seed. Traffic weights follow a power law,
 * like real campaigns where a few large advertisers take most impressions.
 */
public final class AdCatalog {

    /**
     * One creative of a line item, with the line item's terms and targeting.
     *
     * @param countries           ISO alpha-2 countries the line item targets; empty means all
     * @param deviceTypes         device types it targets; empty means all
     * @param frequencyCapPerHour most impressions of the campaign one device may see per hour
     */
    public record AdSpec(String adId, String creativeId, String campaignId, String lineItemId, String advertiser
```

```
  ↪ Id,
27                          long cpmMicros, int durationSeconds, Set<String> countries, Set<DeviceType> deviceTypes
  ↪ ,
28                          int frequencyCapPerHour) {
29          /** Price of one impression: CPM is per thousand. */
30          public long impressionPriceMicros() {
31              return cpmMicros / 1000;
32          }
33
34          public boolean targets(String country, DeviceType deviceType) {
35              return (countries.isEmpty() || countries.contains(country))
36                      && (deviceTypes.isEmpty() || deviceTypes.contains(deviceType));
37          }
38      }
39
40      private static final String[] ADVERTISERS = {
41              "Brightline Auto", "Harbor Bank", "Juniper Foods", "Northpeak Outdoors", "Lumen Mobile",
42              "Copperleaf Home", "Tidewater Travel", "Orbit Games", "Fernway Pharmacy", "Summit Insurance",
43              "Kestrel Apparel", "Bluestem Grocers"};
44      private static final String[] OBJECTIVES = {"Awareness", "Launch", "Seasonal", "Retargeting", "Brand Lift"};
45
46      /** Campaigns of the largest advertisers carry guaranteed (sponsorship) line items. */
47      static final int GUARANTEED_CAMPAIGNS = 5;
48
49      private final List<AdSpec> ads;
50      private final List<CampaignMetadata> campaigns;
51      private final List<LineItemConfig> lineItems;
52      private final double[] cumulativeWeights;
53
54      private AdCatalog(List<AdSpec> ads, List<CampaignMetadata> campaigns, List<LineItemConfig> lineItems,
55                        double[] cumulativeWeights) {
56          this.ads = ads;
57          this.campaigns = campaigns;
58          this.lineItems = lineItems;
59          this.cumulativeWeights = cumulativeWeights;
60      }
61
62      public static AdCatalog generate(int campaigns, RandomGenerator random) {
63          var ads = new ArrayList<AdSpec>();
64          var metadata = new ArrayList<CampaignMetadata>();
65          var lineItemConfigs = new ArrayList<LineItemConfig>();
66          var weights = new ArrayList<Double>();
67          for (int c = 0; c < campaigns; c++) {
68              String campaignId = "camp-%04d".formatted(c);
69              int advertiser = c % ADVERTISERS.length;
70              metadata.add(new CampaignMetadata(campaignId,
71                      "%s %s %d".formatted(ADVERTISERS[advertiser], OBJECTIVES[c % OBJECTIVES.length], c / ADVERTI
  ↪ SERS.length + 1),
72                      "adv-%02d".formatted(advertiser), ADVERTISERS[advertiser]));
73              double campaignWeight = 1.0 / Math.pow(c + 1, 0.8);
74              String advertiserId = "adv-%02d".formatted(advertiser);
75              int lineItems = 1 + random.nextInt(3);
76              for (int l = 0; l < lineItems; l++) {
77                  String lineItemId = "%s-li-%d".formatted(campaignId, l);
78                  // CTV CPMs typically sit in the $20-$60 range.
79                  long cpmMicros = (20 + random.nextInt(41)) * 1_000_000L;
80                  int creatives = 1 + random.nextInt(2);
81                  // Some line items are narrowly targeted, as real direct deals often are.
82                  Set<String> countries = random.nextDouble() < 0.3 ? Set.of("US") : Set.of();
83                  Set<DeviceType> devices = random.nextDouble() < 0.2 ? EnumSet.of(DeviceType.CTV) : EnumSet.noneO
  ↪ f(DeviceType.class);
84                  int frequencyCap = 2 + random.nextInt(3);
85                  // Budgets per pacing period, in dollars: sponsorships are larger and fewer.
86                  boolean guaranteed = c < GUARANTEED_CAMPAIGNS;
87                  long budgetDollars = guaranteed ? 150 + random.nextInt(151) : 20 + random.nextInt(61);
88                  lineItemConfigs.add(new LineItemConfig(lineItemId, campaignId, advertiserId,
89                          guaranteed ? LineItemConfig.Tier.GUARANTEED : LineItemConfig.Tier.STANDARD,
90                          cpmMicros, budgetDollars * 1_000_000L));
91                  for (int k = 0; k < creatives; k++) {
92                      String creativeId = "%s-cr-%d".formatted(lineItemId, k);
93                      String adId = "ad-%s".formatted(creativeId.substring("camp-".length()));
94                      int duration = random.nextBoolean() ? 15 : 30;
95                      ads.add(new AdSpec(adId, creativeId, campaignId, lineItemId, advertiserId, cpmMicros, durati
  ↪ on,
96                              countries, devices, frequencyCap));
97                      weights.add(campaignWeight / (lineItems * creatives));
98                  }
99              }
100         }
101         double[] cumulative = new double[weights.size()];
102         double sum = 0;
103         for (int i = 0; i < cumulative.length; i++) {
104             sum += weights.get(i);
105             cumulative[i] = sum;
106         }
107         return new AdCatalog(List.copyOf(ads), List.copyOf(metadata), List.copyOf(lineItemConfigs), cumulative);
108     }
109
```

```
    public AdSpec pick(RandomGenerator random) {
        double r = random.nextDouble() * cumulativeWeights[cumulativeWeights.length - 1];
        int lo = 0, hi = cumulativeWeights.length - 1;
        while (lo < hi) {
            int mid = (lo + hi) >>> 1;
            if (cumulativeWeights[mid] < r) {
                lo = mid + 1;
            } else {
                hi = mid;
            }
        }
        return ads.get(lo);
    }

    public List<AdSpec> ads() {
        return ads;
    }

    public List<CampaignMetadata> campaigns() {
        return campaigns;
    }

    public List<LineItemConfig> lineItems() {
        return lineItems;
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/catalog/LineItemConfig.java

22 lines · The commercial terms of a direct-sold line item, as the pacing service needs them.

```
package dev.primetech.ads.common.catalog;

/**
 * The commercial terms of a direct-sold line item, as the pacing service needs them. Published by
 * the ad server to the compacted {@code line-items.v1} topic.
 *
 * @param periodBudgetMicros spend allowed per pacing period, in micro-dollars
 */
public record LineItemConfig(
        String lineItemId,
        String campaignId,
        String advertiserId,
        Tier tier,
        long cpmMicros,
        long periodBudgetMicros) {

    /**
     * GUARANTEED line items (sponsorships, upfront deals) are served ahead of the auction whenever
     * pacing allows; STANDARD ones compete on price with programmatic demand.
     */
    public enum Tier { GUARANTEED, STANDARD }
}
```

ads-common/src/main/java/dev/primetech/ads/common/http/JsonHttp.java

64 lines · Small helpers for JSON services on the JDK HTTP server.

```
package dev.primetech.ads.common.http;

import com.sun.net.httpserver.HttpExchange;
import dev.primetech.ads.common.Json;

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.util.Map;

/** Small helpers for JSON services on the JDK HTTP server. */
public final class JsonHttp {
    private JsonHttp() {
    }

    /** Reads at most {@code limit} bytes; returns null if the body is larger. */
    public static byte[] readLimited(InputStream in, int limit) throws IOException {
        byte[] bytes = in.readNBytes(limit + 1);
        return bytes.length > limit ? null : bytes;
    }

    public static void json(HttpExchange ex, int status, Object body) throws IOException {
        send(ex, status, "application/json", Json.MAPPER.writeValueAsBytes(body));
    }

    public static void error(HttpExchange ex, int status, String message) throws IOException {
        json(ex, status, Map.of("error", message));
    }

    public static void noContent(HttpExchange ex) throws IOException {
        ex.sendResponseHeaders(204, -1);
    }
```

```

    public static void send(HttpExchange ex, int status, String contentType, byte[] body) throws IOException {
        ex.getResponseHeaders().set("Content-Type", contentType);
        ex.sendResponseHeaders(status, body.length);
        try (OutputStream out = ex.getResponseBody()) {
            out.write(body);
        }
    }

    public static void cors(HttpExchange ex, String origin) {
        var h = ex.getResponseHeaders();
        h.set("Access-Control-Allow-Origin", origin);
        h.set("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
        h.set("Access-Control-Allow-Headers", "Content-Type");
    }

    /** Query string parameter, or null. Values are URL-decoded. */
    public static String query(HttpExchange ex, String name) {
        String raw = ex.getRequestURI().getRawQuery();
        if (raw == null) {
            return null;
        }
        for (String pair : raw.split("&")) {
            int eq = pair.indexOf('=');
            String key = eq < 0 ? pair : pair.substring(0, eq);
            if (key.equals(name)) {
                return eq < 0 ? "" : java.net.URLDecoder.decode(pair.substring(eq + 1), java.nio.charset.Standar
↪ dCharsets.UTF_8);
            }
        }
        return null;
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/pacing/PacingPeriod.java

33 lines · Budgets reset every period.

```
package dev.primetech.ads.common.pacing;

import java.time.Duration;

/**
 * Budgets reset every period. Periods are aligned to the epoch, so every service computes the same
 * period from the clock alone. Real deals pace per day; the period is configurable so a whole
 * "day" of pacing can be watched in a short demo.
 */
public record PacingPeriod(long lengthMs) {

    public static PacingPeriod of(Duration length) {
        return new PacingPeriod(length.toMillis());
    }

    public long index(long timeMs) {
        return Math.floorDiv(timeMs, lengthMs);
    }

    public long start(long index) {
        return index * lengthMs;
    }

    public long end(long index) {
        return (index + 1) * lengthMs;
    }

    /** Fraction of the period elapsed at {@code timeMs}, between 0 and 1. */
    public double elapsed(long timeMs) {
        long index = index(timeMs);
        return (double) (timeMs - start(index)) / lengthMs;
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/pacing/PacingState.java

42 lines · The pacing service's instruction for one line item in the current period, read by the ad server.

```
package dev.primetech.ads.common.pacing;

/**
 * The pacing service's instruction for one line item in the current period, read by the ad server.
 *
 * @param throttle           probability the line item enters an auction (0..1)
 * @param stopAtServedMicros stop serving once the ad server's served-spend counter reaches this:
 *                           the authoritative remaining budget translated into served spend
 * @param deliveredMicros    billable spend reported by the stream processor (closed minutes only)
 * @param targetMicros       what even pacing would have delivered by the same point in time
 */
public record PacingState(
        String lineItemId,
        long periodIndex,
```

```
        long budgetMicros,
        long deliveredMicros,
        long targetMicros,
        double throttle,
        long stopAtServedMicros,
        Status status,
        long updatedAtMs) {

    public enum Status {
        /** Within tolerance of the even-delivery curve. */
        ON_PACE,
        /** Ahead of the curve: throttled down. */
        AHEAD,
        /** Behind the curve, but the throttle can still rise. */
        BEHIND,
        /** Behind with the throttle already at its maximum: there is not enough eligible traffic. */
        UNDER_DELIVERING,
        /** Budget spent (or committed by ads already served): no more serving this period. */
        EXHAUSTED
    }

    /** Redis keys shared by the pacing service (writer) and the ad server (reader). */
    public static final String REDIS_HASH = "pacing:state";

    public static String servedKey(long periodIndex, String lineItemId) {
        return "pacing:served:" + periodIndex + ":" + lineItemId;
    }
}
```

ads-common/src/main/java/dev/primetech/ads/common/pacing/ServedSpend.java

47 lines · Served spend per period and line item, as kept in Redis by the ad servers.

```
package dev.primetech.ads.common.pacing;

import redis.clients.jedis.Response;
import redis.clients.jedis.UnifiedJedis;

import java.util.ArrayList;
import java.util.Collection;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Served spend per period and line item, as kept in Redis by the ad servers.
 *
 * <p>Each line item has a hash ({@link PacingState#servedKey}) with one field per ad server
 * instance, holding that instance's cumulative served spend for the period. Instances write their
 * absolute total ({@code HSET}), never an increment, so a write that fails, times out or is retried
 * can neither lose nor double-count spend. The line item's served spend is the sum of the fields.
 */
public final class ServedSpend {

    /** Sums every instance's field for each line item; line items with no hash are absent. */
    public static Map<String, Long> read(UnifiedJedis redis, long periodIndex, Collection<String> lineItemIds) {
        var out = new HashMap<String, Long>();
        if (lineItemIds.isEmpty()) {
            return out;
        }
        List<String> ids = new ArrayList<>(lineItemIds);
        var responses = new ArrayList<Response<List<String>>>(ids.size());
        try (var pipeline = redis.pipelined()) {
            for (String id : ids) {
                responses.add(pipeline.hvals(PacingState.servedKey(periodIndex, id)));
            }
            pipeline.sync();
        }
        for (int i = 0; i < ids.size(); i++) {
            List<String> values = responses.get(i).get();
            if (values != null && !values.isEmpty()) {
                out.put(ids.get(i), values.stream().mapToLong(Long::parseLong).sum());
            }
        }
        return out;
    }

    private ServedSpend() {
    }
}
```

ads-common/src/test/java/dev/primetech/ads/common/EnrichedAdEventTest.java

51 lines · Test.

```
package dev.primetech.ads.common;

import com.fasterxml.jackson.databind.JsonNode;
import org.junit.jupiter.api.Test;

```

```java
import static org.assertj.core.api.Assertions.assertThat;

class EnrichedAdEventTest {
    private static final AdEvent IMPRESSION = new AdEvent(1, "e1", EventType.IMPRESSION, 1_000L, 1_100L, "s1",
            "ad", "cr", "camp-1", "li-1", "hash", DeviceType.CTV, "US", true, 40_000L);
    private static final CampaignMetadata META = new CampaignMetadata("camp-1", "Spring Launch", "adv-1", "Harbo
↪ r Bank");

    @Test
    void enrichmentCopiesEventAndMetadata() {
        var e = EnrichedAdEvent.of(IMPRESSION, META, 2_000L);

        assertThat(e.campaignName()).isEqualTo("Spring Launch");
        assertThat(e.advertiserName()).isEqualTo("Harbor Bank");
        assertThat(e.processedTimeMs()).isEqualTo(2_000L);
        assertThat(e.billableMicros()).isEqualTo(40_000L);
        assertThat(e.isIvt()).isFalse();
    }

    @Test
    void ivtEventsAreNotBillable() {
        var e = EnrichedAdEvent.of(IMPRESSION, META, 2_000L).withIvt("DEVICE_RATE");

        assertThat(e.isIvt()).isTrue();
        assertThat(e.billableMicros()).isZero();
        assertThat(e.priceMicros()).as("the price is kept for reporting").isEqualTo(40_000L);
    }

    @Test
    void missingMetadataLeavesFieldsNull() {
        var e = EnrichedAdEvent.of(IMPRESSION, null, 2_000L);
        assertThat(e.campaignName()).isNull();
        assertThat(e.advertiserId()).isNull();
    }

    @Test
    void jsonIsFlatWithAnIvtFlagForAnalytics() throws Exception {
        var e = EnrichedAdEvent.of(IMPRESSION, META, 2_000L).withIvt("DEVICE_RATE");
        JsonNode json = Json.MAPPER.readTree(Json.MAPPER.writeValueAsBytes(e));

        assertThat(json.get("ivt").asBoolean()).isTrue();
        assertThat(json.get("ivtReason").asText()).isEqualTo("DEVICE_RATE");
        assertThat(json.get("campaignName").asText()).isEqualTo("Spring Launch");
        assertThat(json.has("billableMicros")).isFalse();
        assertThat(Json.MAPPER.treeToValue(json, EnrichedAdEvent.class)).isEqualTo(e);
    }
}
```

ads-common/src/test/java/dev/primetech/ads/common/EventNormalizerTest.java

53 lines · Test.

```java
package dev.primetech.ads.common;

import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class EventNormalizerTest {
    private static final String SECRET = "test-secret-0123456789";
    private final EventNormalizer normalizer = new EventNormalizer(new DeviceIdHasher(SECRET));

    @Test
    void hashesDeviceIdWhenConsented() {
        AdEvent out = normalizer.normalize(TestEvents.impression(), 42L);

        assertThat(out.deviceIdHash()).hasSize(64).isNotEqualTo("device-abc");
        assertThat(out.receivedTimeMs()).isEqualTo(42L);
        assertThat(out.schemaVersion()).isEqualTo(AdEvent.SCHEMA_VERSION);
        assertThat(out.priceMicros()).isEqualTo(25_000L);
    }

    @Test
    void hashIsStableForSameSecretAndDiffersAcrossSecrets() {
        var other = new DeviceIdHasher("another-secret-0123456789");
        var same = new DeviceIdHasher(SECRET);
        assertThat(same.hash("device-abc")).isEqualTo(new DeviceIdHasher(SECRET).hash("device-abc"));
        assertThat(other.hash("device-abc")).isNotEqualTo(same.hash("device-abc"));
    }

    @Test
    void dropsDeviceIdWithoutConsent() {
        var e = TestEvents.impression();
        var noConsent = new ClientEvent(e.eventId(), e.type(), e.eventTimeMs(), e.sessionId(), e.adId(),
                e.creativeId(), e.campaignId(), e.lineItemId(), e.deviceId(), e.deviceType(), e.country(), false
↪ ,
                e.priceMicros());

```

```
        AdEvent out = normalizer.normalize(noConsent, 42L);

        assertThat(out.deviceIdHash()).isNull();
        assertThat(out.consent()).isFalse();
    }

    @Test
    void serializedEventNeverContainsRawDeviceId() throws Exception {
        String json = Json.MAPPER.writeValueAsString(normalizer.normalize(TestEvents.impression(), 42L));
        assertThat(json).doesNotContain("device-abc");
    }

    @Test
    void rejectsWeakSecret() {
        assertThatThrownBy(() -> new DeviceIdHasher("short")).isInstanceOf(IllegalArgumentException.class);
    }
}
```

ads-common/src/test/java/dev/primetech/ads/common/EventValidatorTest.java

95 lines · Test.

```
package dev.primetech.ads.common;

import org.junit.jupiter.api.Test;

import java.time.Duration;

import static dev.primetech.ads.common.TestEvents.NOW;
import static org.assertj.core.api.Assertions.assertThat;

class EventValidatorTest {
    private final EventValidator validator = new EventValidator(Duration.ofMinutes(5), Duration.ofHours(24));

    @Test
    void acceptsWellFormedEvents() {
        assertThat(validator.validate(TestEvents.impression(), NOW)).isEmpty();
        assertThat(validator.validate(TestEvents.midpoint(), NOW)).isEmpty();
    }

    @Test
    void acceptsEventWithoutDeviceId() {
        var e = TestEvents.midpoint();
        var noDevice = new ClientEvent(e.eventId(), e.type(), e.eventTimeMs(), e.sessionId(), e.adId(),
                e.creativeId(), e.campaignId(), e.lineItemId(), null, e.deviceType(), e.country(), false, null);
        assertThat(validator.validate(noDevice, NOW)).isEmpty();
    }

    @Test
    void rejectsUnknownType() {
        var e = TestEvents.midpoint();
        var bad = new ClientEvent(e.eventId(), null, e.eventTimeMs(), e.sessionId(), e.adId(), e.creativeId(),
                e.campaignId(), e.lineItemId(), e.deviceId(), e.deviceType(), e.country(), e.consent(), null);
        assertThat(validator.validate(bad, NOW)).contains(RejectReason.UNKNOWN_EVENT_TYPE);
    }

    @Test
    void rejectsMissingAndBlankFields() {
        var e = TestEvents.midpoint();
        var blankSession = new ClientEvent(e.eventId(), e.type(), e.eventTimeMs(), " ", e.adId(), e.creativeId()
 ↪ ,
                e.campaignId(), e.lineItemId(), e.deviceId(), e.deviceType(), e.country(), e.consent(), null);
        var noConsent = new ClientEvent(e.eventId(), e.type(), e.eventTimeMs(), e.sessionId(), e.adId(),
                e.creativeId(), e.campaignId(), e.lineItemId(), e.deviceId(), e.deviceType(), e.country(), null,
 ↪  null);
        assertThat(validator.validate(blankSession, NOW)).contains(RejectReason.MISSING_FIELD);
        assertThat(validator.validate(noConsent, NOW)).contains(RejectReason.MISSING_FIELD);
    }

    @Test
    void rejectsOverlongIds() {
        var e = TestEvents.midpoint();
        var longId = "x".repeat(EventValidator.MAX_ID_LENGTH + 1);
        var bad = new ClientEvent(longId, e.type(), e.eventTimeMs(), e.sessionId(), e.adId(), e.creativeId(),
                e.campaignId(), e.lineItemId(), e.deviceId(), e.deviceType(), e.country(), e.consent(), null);
        assertThat(validator.validate(bad, NOW)).contains(RejectReason.FIELD_TOO_LONG);
    }

    @Test
    void rejectsBadCountry() {
        var e = TestEvents.midpoint();
        var bad = new ClientEvent(e.eventId(), e.type(), e.eventTimeMs(), e.sessionId(), e.adId(), e.creativeId(
 ↪ ),
                e.campaignId(), e.lineItemId(), e.deviceId(), e.deviceType(), "usa", e.consent(), null);
        assertThat(validator.validate(bad, NOW)).contains(RejectReason.INVALID_COUNTRY);
    }

    @Test
    void enforcesEventTimeBounds() {
```

```
        var e = TestEvents.midpoint();
        long justInsideFuture = NOW + Duration.ofMinutes(5).toMillis();
        long justInsidePast = NOW - Duration.ofHours(24).toMillis();
        assertThat(validator.validate(withTime(e, justInsideFuture), NOW)).isEmpty();
        assertThat(validator.validate(withTime(e, justInsidePast), NOW)).isEmpty();
        assertThat(validator.validate(withTime(e, justInsideFuture + 1), NOW))
                .contains(RejectReason.EVENT_TIME_IN_FUTURE);
        assertThat(validator.validate(withTime(e, justInsidePast - 1), NOW))
                .contains(RejectReason.EVENT_TIME_TOO_OLD);
    }

    @Test
    void enforcesPriceRules() {
        var imp = TestEvents.impression();
        var mid = TestEvents.midpoint();
        assertThat(validator.validate(withPrice(imp, null), NOW)).contains(RejectReason.MISSING_PRICE);
        assertThat(validator.validate(withPrice(imp, -1L), NOW)).contains(RejectReason.NEGATIVE_PRICE);
        assertThat(validator.validate(withPrice(imp, 0L), NOW)).isEmpty();
        assertThat(validator.validate(withPrice(mid, 10L), NOW)).contains(RejectReason.PRICE_ON_NON_BILLABLE);
    }

    private static ClientEvent withTime(ClientEvent e, long t) {
        return new ClientEvent(e.eventId(), e.type(), t, e.sessionId(), e.adId(), e.creativeId(), e.campaignId()
↪ ,
                e.lineItemId(), e.deviceId(), e.deviceType(), e.country(), e.consent(), e.priceMicros());
    }

    private static ClientEvent withPrice(ClientEvent e, Long price) {
        return new ClientEvent(e.eventId(), e.type(), e.eventTimeMs(), e.sessionId(), e.adId(), e.creativeId(),
                e.campaignId(), e.lineItemId(), e.deviceId(), e.deviceType(), e.country(), e.consent(), price);
    }
}
```

ads-common/src/test/java/dev/primetech/ads/common/JsonTest.java

29 lines · Test.

```
package dev.primetech.ads.common;

import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.assertThat;

class JsonTest {
    @Test
    void unknownEnumAndExtraFieldsDoNotFailTheBatch() throws Exception {
        String body = """
                {"events":[{"eventId":"e1","type":"SKIP","newField":1},{"eventId":"e2","type":"CLICK"}]}
                """;
        IngestRequest req = Json.MAPPER.readValue(body, IngestRequest.class);

        assertThat(req.events()).hasSize(2);
        assertThat(req.events().get(0).type()).isNull();
        assertThat(req.events().get(1).type()).isEqualTo(EventType.CLICK);
    }

    @Test
    void roundTripsAdEvent() throws Exception {
        var event = new AdEvent(1, "e1", EventType.IMPRESSION, 1L, 2L, "s", "a", "c", "camp", "li",
                null, DeviceType.CTV, "US", false, 100L);
        String json = Json.MAPPER.writeValueAsString(event);

        assertThat(json).doesNotContain("deviceIdHash");
        assertThat(Json.MAPPER.readValue(json, AdEvent.class)).isEqualTo(event);
    }
}
```

ads-common/src/test/java/dev/primetech/ads/common/PacingPeriodTest.java

39 lines · Test.

```
package dev.primetech.ads.common;

import dev.primetech.ads.common.catalog.AdCatalog;
import dev.primetech.ads.common.catalog.LineItemConfig;
import dev.primetech.ads.common.pacing.PacingPeriod;
import org.junit.jupiter.api.Test;

import java.time.Duration;
import java.util.SplittableRandom;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.within;

class PacingPeriodTest {
    @Test
    void periodsAreEpochAligned() {
        var period = PacingPeriod.of(Duration.ofMinutes(30));
        long t = 1_800_000L * 1000 + 900_000;

```

```
        assertThat(period.index(t)).isEqualTo(1000);
        assertThat(period.start(1000)).isEqualTo(1_800_000L * 1000);
        assertThat(period.end(1000)).isEqualTo(1_800_000L * 1001);
        assertThat(period.elapsed(t)).isCloseTo(0.5, within(1e-9));
    }

    @Test
    void catalogGivesEveryLineItemATierAndBudget() {
        var catalog = AdCatalog.generate(20, new SplittableRandom(1));

        assertThat(catalog.lineItems()).isNotEmpty()
                .extracting(LineItemConfig::lineItemId).doesNotHaveDuplicates()
                .containsExactlyInAnyOrderElementsOf(catalog.ads().stream().map(AdCatalog.AdSpec::lineItemId).di
  ↪ stinct().toList());
        assertThat(catalog.lineItems()).filteredOn(l -> l.tier() == LineItemConfig.Tier.GUARANTEED)
                .isNotEmpty()
                .allSatisfy(l -> assertThat(l.periodBudgetMicros()).isBetween(150_000_000L, 300_000_000L));
        assertThat(catalog.lineItems()).filteredOn(l -> l.tier() == LineItemConfig.Tier.STANDARD)
                .allSatisfy(l -> assertThat(l.periodBudgetMicros()).isBetween(20_000_000L, 80_000_000L));
    }
}
```

ads-common/src/test/java/dev/primetech/ads/common/TestEvents.java

18 lines · Test.

```
package dev.primetech.ads.common;

public final class TestEvents {
    public static final long NOW = 1_760_000_000_000L;

    private TestEvents() {
    }

    public static ClientEvent impression() {
        return new ClientEvent("evt-1", EventType.IMPRESSION, NOW, "sess-1", "ad-1", "cr-1",
                "camp-1", "li-1", "device-abc", DeviceType.CTV, "US", true, 25_000L);
    }

    public static ClientEvent midpoint() {
        return new ClientEvent("evt-2", EventType.MIDPOINT, NOW, "sess-1", "ad-1", "cr-1",
                "camp-1", "li-1", "device-abc", DeviceType.CTV, "US", true, null);
    }
}
```

audit-service

audit-service/pom.xml

51 lines

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.primetech.ads</groupId>
    <artifactId>ads-eventing</artifactId>
    <version>0.1.0-SNAPSHOT</version>
  </parent>

  <artifactId>audit-service</artifactId>
  <name>Audit Service (continuous reconciliation)</name>

  <properties>
    <main.class>dev.primetech.ads.audit.AuditApp</main.class>
  </properties>

  <dependencies>
    <dependency>
      <groupId>dev.primetech.ads</groupId>
      <artifactId>ads-common</artifactId>
    </dependency>
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
    </dependency>
    <dependency>
      <groupId>redis.clients</groupId>
      <artifactId>jedis</artifactId>
    </dependency>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
```

```
      <artifactId>slf4j-simple</artifactId>
    </dependency>
  </dependencies>

  <build>
    <finalName>audit-service</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-shade-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

audit-service/src/main/java/dev/primetech/ads/audit/AuditApp.java

168 lines · Wires the audit to Kafka (decision log, minute totals and budgets in; results out), Druid, Redis and HTTP.

```
package dev.primetech.ads.audit;

import com.sun.net.httpserver.HttpServer;
import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.LineItemMinute;
import dev.primetech.ads.common.Topics;
import dev.primetech.ads.common.catalog.LineItemConfig;
import dev.primetech.ads.common.http.JsonHttp;
import dev.primetech.ads.common.pacing.PacingPeriod;
import io.micrometer.prometheusmetrics.PrometheusConfig;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.TopicPartition;
import org.apache.kafka.common.serialization.ByteArrayDeserializer;
import org.apache.kafka.common.serialization.ByteArraySerializer;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.apache.kafka.common.serialization.StringSerializer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.io.IOException;
import java.io.UncheckedIOException;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicBoolean;

/**
 * Wires the audit to Kafka (decision log, minute totals and budgets in; results out), Druid, Redis
 * and HTTP. Like the pacing service it rebuilds its state from the topics on start.
 */
public final class AuditApp {
    private static final Logger log = LoggerFactory.getLogger(AuditApp.class);

    public static void main(String[] args) throws Exception {
        var env = System.getenv();
        String bootstrap = env.getOrDefault("KAFKA_BOOTSTRAP", "localhost:19092");
        var period = PacingPeriod.of(Duration.ofMinutes(Long.parseLong(env.getOrDefault("PACING_PERIOD_MINUTES",
 "30"))));
        long intervalSeconds = Long.parseLong(env.getOrDefault("AUDIT_INTERVAL_SECONDS", "60"));
        int port = Integer.parseInt(env.getOrDefault("PORT", "8096"));

        var registry = new PrometheusMeterRegistry(PrometheusConfig.DEFAULT);
        registry.config().commonTags("service", "audit-service");
        var ledger = new AuditLedger(period);
        var caughtUp = new AtomicBoolean(false);
        var sources = new Sources.Live(env.getOrDefault("DRUID_URL", "http://localhost:8888"),
                env.getOrDefault("REDIS_URL", "redis://localhost:6379"));
        var producer = new KafkaProducer<>(Map.<String, Object>of(
                ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrap,
                ProducerConfig.ACKS_CONFIG, "all",
                ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true,
                ProducerConfig.CLIENT_ID_CONFIG, "audit-service"), new StringSerializer(), new ByteArraySerializ
er());
        var service = new AuditService(ledger, sources, period, System::currentTimeMillis, caughtUp::get, r -> {
            try {
                producer.send(new ProducerRecord<>(Topics.AUDIT_RESULTS, r.check(), Json.MAPPER.writeValueAsByte
s(r)));
            } catch (IOException e) {
                throw new UncheckedIOException(e);
```

```
            }
        }, registry);

        Thread.ofPlatform().name("audit-consumer").daemon().start(() -> consume(bootstrap, ledger, caughtUp));
        var scheduler = Executors.newSingleThreadScheduledExecutor();
        scheduler.scheduleWithFixedDelay(() -> {
            try {
                service.run();
            } catch (RuntimeException e) {
                log.error("audit round failed", e);
            }
        }, 15, intervalSeconds, TimeUnit.SECONDS);

        HttpServer http = HttpServer.create(new InetSocketAddress(port), 64);
        http.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        http.createContext("/stats", ex -> {
            try (ex) {
                JsonHttp.json(ex, 200, Map.of("caughtUp", caughtUp.get(), "periodMinutes", period.lengthMs() / 60_000,
                        "checks", service.stats()));
            }
        });
        http.createContext("/metrics", ex -> {
            try (ex) {
                JsonHttp.send(ex, 200, "text/plain; version=0.0.4", registry.scrape().getBytes(StandardCharsets.UTF_8));
            }
        });
        http.createContext("/health", ex -> {
            try (ex) {
                JsonHttp.json(ex, 200, Map.of("status", "UP", "caughtUp", caughtUp.get()));
            }
        });
        http.start();
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            http.stop(1);
            scheduler.shutdownNow();
            producer.close(Duration.ofSeconds(5));
            sources.close();
        }, "shutdown"));
        log.info("audit service on :{}, every {} s, period {} min", port, intervalSeconds, period.lengthMs() / 60_000);
    }

    /**
     * Reads the three topics from the beginning, forever, with no consumer group. Marks the ledger
     * caught up once it has read every partition up to where it ended at startup.
     */
    private static void consume(String bootstrap, AuditLedger ledger, AtomicBoolean caughtUp) {
        var config = Map.<String, Object>of(
                ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrap,
                ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed",
                ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false,
                ConsumerConfig.MAX_POLL_RECORDS_CONFIG, 5_000,
                ConsumerConfig.CLIENT_ID_CONFIG, "audit-service");
        while (true) {
            try (var consumer = new KafkaConsumer<>(config, new StringDeserializer(), new ByteArrayDeserializer())) {
                List<TopicPartition> partitions = new ArrayList<>();
                for (String topic : List.of(Topics.LINE_ITEMS, Topics.LINE_ITEM_MINUTES, Topics.AD_DECISIONS)) {
                    consumer.partitionsFor(topic, Duration.ofSeconds(30))
                            .forEach(p -> partitions.add(new TopicPartition(topic, p.partition())));
                }
                consumer.assign(partitions);
                consumer.seekToBeginning(partitions);
                // Re-reading the decision log into a ledger that already counted it would double
                // every line item's spend: start clean, and report pending until caught up again.
                caughtUp.set(false);
                ledger.reset();
                Map<TopicPartition, Long> endAtStart = consumer.endOffsets(partitions);
                while (true) {
                    for (ConsumerRecord<String, byte[]> r : consumer.poll(Duration.ofMillis(500))) {
                        if (r.value() == null) {
                            continue;
                        }
                        try {
                            switch (r.topic()) {
                                case Topics.LINE_ITEMS -> ledger.lineItem(Json.MAPPER.readValue(r.value(), LineItemConfig.class));
                                case Topics.LINE_ITEM_MINUTES -> ledger.minute(Json.MAPPER.readValue(r.value(), LineItemMinute.class));
                                default -> ledger.decision(Json.MAPPER.readTree(r.value()));
                            }
                        } catch (IOException e) {
                            log.warn("skipping malformed record on {}: {}", r.topic(), e.toString());
                        }
                    }
                    if (!caughtUp.get() && endAtStart.entrySet().stream().allMatch(e -> consumer.position(e.getK
```

```
   ↪ ey()) >= e.getValue())) {
150                         caughtUp.set(true);
151                         log.info("caught up with the topics");
152                     }
153                 }
154             } catch (RuntimeException e) {
155                 log.warn("consumer failed, restarting in 5 s: {}", e.toString());
156                 try {
157                     Thread.sleep(5_000);
158                 } catch (InterruptedException ie) {
159                     Thread.currentThread().interrupt();
160                     return;
161                 }
162             }
163         }
164     }
165
166     private AuditApp() {
167     }
168 }
```

audit-service/src/main/java/dev/primetech/ads/audit/AuditLedger.java

96 lines · What the audit has read from Kafka: Flink's minute totals (latest revision of each), direct spend per period and line item from the ad server's decision log, and line item budgets.

```
 1  package dev.primetech.ads.audit;
 2
 3  import com.fasterxml.jackson.databind.JsonNode;
 4  import dev.primetech.ads.common.LineItemMinute;
 5  import dev.primetech.ads.common.catalog.LineItemConfig;
 6  import dev.primetech.ads.common.pacing.PacingPeriod;
 7
 8  import java.util.HashMap;
 9  import java.util.Map;
10  import java.util.concurrent.ConcurrentHashMap;
11  import java.util.concurrent.atomic.AtomicLong;
12
13  /**
14   * What the audit has read from Kafka: Flink's minute totals (latest revision of each), direct spend
15   * per period and line item from the ad server's decision log, and line item budgets. Bounded:
16   * only recent minutes and periods are kept.
17   */
18  public final class AuditLedger {
19      static final long KEEP_MINUTES_MS = 3 * 3_600_000L;
20      static final int KEEP_PERIODS = 12;
21
22      private final PacingPeriod period;
23      private final Map<String, LineItemMinute> minutes = new ConcurrentHashMap<>();
24      private final Map<Long, Map<String, AtomicLong>> servedByPeriod = new ConcurrentHashMap<>();
25      private final Map<String, LineItemConfig> lineItems = new ConcurrentHashMap<>();
26      private final AtomicLong latestDecisionMs = new AtomicLong(Long.MIN_VALUE);
27
28      public AuditLedger(PacingPeriod period) {
29          this.period = period;
30      }
31
32      public void minute(LineItemMinute m) {
33          minutes.merge(m.key(), m, (old, now) -> now.revision() >= old.revision() ? now : old);
34      }
35
36      public void lineItem(LineItemConfig config) {
37          lineItems.put(config.lineItemId(), config);
38      }
39
40      /** One record from the decision log: counts direct wins at the price the ad server charged. */
41      public void decision(JsonNode d) {
42          long ts = d.path("timestampMs").asLong();
43          latestDecisionMs.accumulateAndGet(ts, Math::max);
44          for (JsonNode slot : d.path("slots")) {
45              if ("direct".equals(slot.path("source").asText()) && slot.hasNonNull("lineItemId")) {
46                  // The ad server charges clearing CPM / 1000 per impression, rounded down.
47                  long price = slot.path("clearingCpmMicros").asLong() / 1000;
48                  servedByPeriod.computeIfAbsent(period.index(ts), k -> new ConcurrentHashMap<>())
49                          .computeIfAbsent(slot.path("lineItemId").asText(), k -> new AtomicLong())
50                          .addAndGet(price);
51              }
52          }
53      }
54
55      /** Flink totals for minutes starting in [from, to). */
56      public Map<String, Checks.Totals> minutes(long from, long to) {
57          var out = new HashMap<String, Checks.Totals>();
58          for (LineItemMinute m : minutes.values()) {
59              if (m.windowStartMs() >= from && m.windowStartMs() < to) {
60                  out.put(m.key(), new Checks.Totals(m.impressions(), m.spendMicros()));
61              }
62          }
63          return out;
```

```
    }

    public Map<String, Long> served(long periodIndex) {
        var out = new HashMap<String, Long>();
        servedByPeriod.getOrDefault(periodIndex, Map.of()).forEach((li, v) -> out.put(li, v.get()));
        return out;
    }

    public Map<String, Long> budgets() {
        var out = new HashMap<String, Long>();
        lineItems.forEach((id, c) -> out.put(id, c.periodBudgetMicros()));
        return out;
    }

    /** Timestamp of the newest decision read: the log is complete up to about here. */
    public long latestDecisionMs() {
        return latestDecisionMs.get();
    }

    /** Forgets everything, before the topics are read again from the beginning. */
    public void reset() {
        minutes.clear();
        servedByPeriod.clear();
        lineItems.clear();
        latestDecisionMs.set(Long.MIN_VALUE);
    }

    public void prune(long now) {
        minutes.values().removeIf(m -> m.windowStartMs() < now - KEEP_MINUTES_MS);
        long current = period.index(now);
        servedByPeriod.keySet().removeIf(p -> p < current - KEEP_PERIODS);
    }
}
```

audit-service/src/main/java/dev/primetech/ads/audit/AuditService.java

196 lines · Runs every reconciliation over data that has settled, and exposes the results as metrics (for alerts), as recent history (for the dashboard) and as records (for the audit trail).

```
package dev.primetech.ads.audit;

import dev.primetech.ads.common.pacing.PacingPeriod;
import io.micrometer.core.instrument.Gauge;
import io.micrometer.core.instrument.MeterRegistry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.function.BooleanSupplier;
import java.util.function.Consumer;
import java.util.function.LongSupplier;

/**
 * Runs every reconciliation over data that has settled, and exposes the results as metrics (for
 * alerts), as recent history (for the dashboard) and as records (for the audit trail).
 *
 * <p>Settled means no source can still change it: minutes older than the stream processor's
 * allowed lateness (10 min) plus a margin, and pacing periods that ended a few minutes ago.
 */
public final class AuditService {
    private static final Logger log = LoggerFactory.getLogger(AuditService.class);
    private static final long MINUTE_MS = 60_000;
    /** Minutes this old can no longer be revised: 10 min allowed lateness plus 2 min margin. */
    static final long MINUTE_SETTLE_MS = 12 * MINUTE_MS;
    /** How many settled minutes each run compares. */
    static final long MINUTE_WINDOW_MS = 30 * MINUTE_MS;
    /** A period is checked once it ended this long ago: its last minutes are closed and ingested. */
    static final long PERIOD_SETTLE_MS = 3 * MINUTE_MS;
    /** The decision log must have been read at least this far past the period's end. */
    static final long DECISION_MARGIN_MS = 30_000;
    static final int HISTORY = 30;
    static final List<String> CHECKS = List.of(Checks.FLINK_VS_DRUID, Checks.SERVED_VS_DECISIONS, Checks.BUDGETS
↳ ,
            Checks.DELIVERY_RATIO);

    private final AuditLedger ledger;
    private final Sources sources;
    private final PacingPeriod period;
    private final LongSupplier clock;
    private final BooleanSupplier caughtUp;
    private final Consumer<CheckResult> publisher;
    private final Map<String, Deque<CheckResult>> history = new ConcurrentHashMap<>();
```

```
    /** The time the audit started: windows never reach back before data it could have read. */
    private final long startedMs;

    public AuditService(AuditLedger ledger, Sources sources, PacingPeriod period, LongSupplier clock,
                        BooleanSupplier caughtUp, Consumer<CheckResult> publisher, MeterRegistry registry) {
        this.ledger = ledger;
        this.sources = sources;
        this.period = period;
        this.clock = clock;
        this.caughtUp = caughtUp;
        this.publisher = publisher;
        this.startedMs = clock.getAsLong();
        for (String check : CHECKS) {
            history.put(check, new ArrayDeque<>());
            Gauge.builder("audit.check.ok", () -> okValue(latest(check)))
                    .description("1 if the latest completed run agreed, 0 if it found a mismatch, NaN while pend
ing")
                    .tag("check", check).register(registry);
            Gauge.builder("audit.check.mismatches", () -> latest(check) == null ? Double.NaN : latest(check).mis
matches())
                    .tag("check", check).register(registry);
            Gauge.builder("audit.check.compared", () -> latest(check) == null ? Double.NaN : latest(check).compa
red())
                    .tag("check", check).register(registry);
            Gauge.builder("audit.check.last.completed.seconds", () -> lastCompletedMs(check) / 1000.0)
                    .description("When the check last completed (OK or MISMATCH), as a Unix timestamp")
                    .tag("check", check).register(registry);
        }
    }

    /** One audit round. Each check is independent: one source being down does not stop the others. */
    public List<CheckResult> run() {
        long now = clock.getAsLong();
        ledger.prune(now);
        var results = new ArrayList<CheckResult>();
        if (!caughtUp.getAsBoolean()) {
            for (String check : CHECKS) {
                results.add(CheckResult.pending(check, "reading the topics from the beginning", now));
            }
        } else {
            results.add(flinkVsDruid(now));
            results.addAll(periodChecks(now));
        }
        for (CheckResult r : results) {
            record(r);
        }
        return results;
    }

    private CheckResult flinkVsDruid(long now) {
        long to = Math.floorDiv(now - MINUTE_SETTLE_MS, MINUTE_MS) * MINUTE_MS;
        long from = to - MINUTE_WINDOW_MS; // well inside the ledger's 3 h of minutes
        try {
            return Checks.flinkVsDruid(ledger.minutes(from, to), sources.druidMinutes(from, to), from, to, now);
        } catch (Exception e) {
            return CheckResult.error(Checks.FLINK_VS_DRUID, "Druid unavailable: " + e.getMessage(), now);
        }
    }

    private List<CheckResult> periodChecks(long now) {
        long p = period.index(now - PERIOD_SETTLE_MS) - 1;
        long from = period.start(p);
        long to = period.end(p);
        Map<String, Long> served = ledger.served(p);
        if (ledger.latestDecisionMs() < to + DECISION_MARGIN_MS || served.isEmpty()) {
            String why = served.isEmpty() ? "no direct spend served in the last settled period" : "decision log
not read up to the period end yet";
            return List.of(CheckResult.pending(Checks.SERVED_VS_DECISIONS, why, now),
                    CheckResult.pending(Checks.BUDGETS, why, now), CheckResult.pending(Checks.DELIVERY_RATIO, wh
y, now));
        }
        var results = new ArrayList<CheckResult>();
        try {
            var ids = new HashSet<>(served.keySet());
            ids.addAll(ledger.budgets().keySet());
            results.add(Checks.servedVsDecisions(served, sources.redisServed(p, ids), from, to, now));
        } catch (Exception e) {
            results.add(CheckResult.error(Checks.SERVED_VS_DECISIONS, "Redis unavailable: " + e.getMessage(), no
w));
        }
        try {
            Map<String, Long> budgets = ledger.budgets();
            Map<String, Long> delivered = sources.druidSpend(from, to);
            results.add(Checks.budgets(budgets, delivered, from, to, now));
            long deliveredDirect = budgets.keySet().stream().mapToLong(id -> delivered.getOrDefault(id, 0L)).sum
();
            long servedDirect = served.values().stream().mapToLong(Long::longValue).sum();
            results.add(Checks.deliveryRatio(deliveredDirect, servedDirect, from, to, now));
```

```
131          } catch (Exception e) {
132              results.add(CheckResult.error(Checks.BUDGETS, "Druid unavailable: " + e.getMessage(), now));
133              results.add(CheckResult.error(Checks.DELIVERY_RATIO, "Druid unavailable: " + e.getMessage(), now));
134          }
135          return results;
136      }
137
138      private void record(CheckResult r) {
139          Deque<CheckResult> h = history.get(r.check());
140          synchronized (h) {
141              h.addFirst(r);
142              while (h.size() > HISTORY) {
143                  h.removeLast();
144              }
145          }
146          if (r.status() == CheckResult.Status.MISMATCH) {
147              log.warn("{}: {} {}", r.check(), r.summary(), r.examples());
148          } else if (r.status() == CheckResult.Status.ERROR) {
149              log.warn("{}: {}", r.check(), r.summary());
150          }
151          if (r.status() == CheckResult.Status.OK || r.status() == CheckResult.Status.MISMATCH) {
152              try {
153                  publisher.accept(r);
154              } catch (RuntimeException e) {
155                  log.warn("cannot publish audit result: {}", e.toString());
156              }
157          }
158      }
159
160      /** The latest OK or MISMATCH result: pending and error runs say nothing about the data. */
161      CheckResult latest(String check) {
162          Deque<CheckResult> h = history.get(check);
163          synchronized (h) {
164              return h.stream().filter(r -> r.status() == CheckResult.Status.OK || r.status() == CheckResult.Statu
    ↪ s.MISMATCH)
165                      .findFirst().orElse(null);
166          }
167      }
168
169      private long lastCompletedMs(String check) {
170          CheckResult r = latest(check);
171          return r == null ? startedMs : r.ranAtMs();
172      }
173
174      private static double okValue(CheckResult r) {
175          return r == null ? Double.NaN : r.status() == CheckResult.Status.OK ? 1 : 0;
176      }
177
178      /** For the stats endpoint: per check, the latest run of any status and the latest completed one. */
179      public Map<String, Object> stats() {
180          var out = new LinkedHashMap<String, Object>();
181          for (String check : CHECKS) {
182              Deque<CheckResult> h = history.get(check);
183              List<CheckResult> runs;
184              synchronized (h) {
185                  runs = List.copyOf(h);
186              }
187              var m = new LinkedHashMap<String, Object>();
188              m.put("latest", runs.isEmpty() ? null : runs.get(0));
189              m.put("latestCompleted", latest(check));
190              m.put("recent", runs.stream().map(r -> Map.of("status", r.status(), "ranAtMs", r.ranAtMs(),
191                      "mismatches", r.mismatches())).toList());
192              out.put(check, m);
193          }
194          return out;
195      }
196  }
```

audit-service/src/main/java/dev/primetech/ads/audit/CheckResult.java

49 lines · The outcome of one run of one reconciliation.

```
 1  package dev.primetech.ads.audit;
 2
 3  import java.util.List;
 4
 5  /**
 6   * The outcome of one run of one reconciliation.
 7   *
 8   * @param compared   how many items (line item minutes, line items) were compared
 9   * @param mismatches how many disagreed
10   * @param summary    one line for people: what was compared and what was found
11   * @param examples   up to a few disagreeing items, for whoever investigates
12   */
13  public record CheckResult(
14          String check,
15          Status status,
16          long windowStartMs,
17          long windowEndMs,
```

```
        long compared,
        long mismatches,
        String summary,
        List<String> examples,
        long ranAtMs) {

    public enum Status {
        /** Every compared item agreed. */
        OK,
        /** At least one item disagreed: the sources tell different stories. */
        MISMATCH,
        /** Not enough settled data yet (e.g. just started); not a failure. */
        PENDING,
        /** A source could not be read; the check says nothing about the data. */
        ERROR
    }

    static final int MAX_EXAMPLES = 5;

    static CheckResult of(String check, long from, long to, long compared, List<String> problems, String summary
 , long now) {
        return new CheckResult(check, problems.isEmpty() ? Status.OK : Status.MISMATCH, from, to, compared,
                problems.size(), summary, List.copyOf(problems.subList(0, Math.min(MAX_EXAMPLES, problems.size()
 ))), now);
    }

    static CheckResult pending(String check, String why, long now) {
        return new CheckResult(check, Status.PENDING, 0, 0, 0, 0, why, List.of(), now);
    }

    static CheckResult error(String check, String why, long now) {
        return new CheckResult(check, Status.ERROR, 0, 0, 0, 0, why, List.of(), now);
    }
}
```

audit-service/src/main/java/dev/primetech/ads/audit/Checks.java

127 lines · The reconciliations, as pure functions over data already read from each source.

```
package dev.primetech.ads.audit;

import java.util.ArrayList;
import java.util.Map;
import java.util.TreeSet;

/**
 * The reconciliations, as pure functions over data already read from each source. Each compares
 * two records of the same fact that are produced independently, so a disagreement means one of
 * them is wrong: lost or double-counted events, a broken counter, or spend beyond a budget.
 */
public final class Checks {
    public static final String FLINK_VS_DRUID = "flink-vs-druid";
    public static final String SERVED_VS_DECISIONS = "served-vs-decision-log";
    public static final String BUDGETS = "budget-compliance";
    public static final String DELIVERY_RATIO = "delivered-vs-served";

    /** Overspend allowed before a line item counts as over budget. */
    static final double BUDGET_TOLERANCE = 1.01;
    /** Expected range of delivered / served spend: some served ads are never played, none are invented. */
    static final double MIN_DELIVERY_RATIO = 0.85;
    static final double MAX_DELIVERY_RATIO = 1.02;

    /** Billable impressions and spend for one line item in one minute. */
    public record Totals(long impressions, long spendMicros) {
    }

    /**
     * Flink's exactly-once minute totals against Druid's rollup of the clean event stream, for
     * minutes that can no longer change. Both derive from the same events through different code
     * (Flink windows vs Druid ingestion and SQL), so they must agree exactly.
     *
     * @param flink keyed by {@code lineItemId|minuteStartMs}
     * @param druid keyed the same way
     */
    public static CheckResult flinkVsDruid(Map<String, Totals> flink, Map<String, Totals> druid, long from, long
 to, long now) {
        var keys = new TreeSet<>(flink.keySet());
        keys.addAll(druid.keySet());
        var problems = new ArrayList<String>();
        long spend = 0;
        for (String key : keys) {
            Totals f = flink.getOrDefault(key, new Totals(0, 0));
            Totals d = druid.getOrDefault(key, new Totals(0, 0));
            spend += f.spendMicros();
            if (!f.equals(d)) {
                problems.add(String.format("%s: Flink %d imps / %d µ$, Druid %d imps / %d µ$",
                        key, f.impressions(), f.spendMicros(), d.impressions(), d.spendMicros()));
            }
        }
```

```
        String summary = problems.isEmpty()
                ? String.format("%d line item minutes agree exactly (%s of spend)", keys.size(), dollars(spend))
                : String.format("%d of %d line item minutes disagree", problems.size(), keys.size());
        return CheckResult.of(FLINK_VS_DRUID, from, to, keys.size(), problems, summary, now);
    }

    /**
     * The ad server's served-spend counters in Redis (what pacing steers by) against the sum of
     * direct wins in its decision log. Both are written by the ad server for every direct win, one
     * to Redis and one to Kafka, so they must agree exactly for a settled period.
     */
    public static CheckResult servedVsDecisions(Map<String, Long> decisionLog, Map<String, Long> redis,
                                                long from, long to, long now) {
        var ids = new TreeSet<>(decisionLog.keySet());
        ids.addAll(redis.keySet());
        var problems = new ArrayList<String>();
        long total = 0;
        for (String id : ids) {
            long logged = decisionLog.getOrDefault(id, 0L);
            long counted = redis.getOrDefault(id, 0L);
            total += logged;
            if (logged != counted) {
                problems.add(String.format("%s: decision log %s, Redis %s", id, dollars(logged), dollars(counted ));
            }
        }
        String summary = problems.isEmpty()
                ? String.format("%d line items agree exactly (%s served)", ids.size(), dollars(total))
                : String.format("%d of %d line items disagree", problems.size(), ids.size());
        return CheckResult.of(SERVED_VS_DECISIONS, from, to, ids.size(), problems, summary, now);
    }

    /** Delivered spend per line item (from Druid) against its budget for a settled period. */
    public static CheckResult budgets(Map<String, Long> budgets, Map<String, Long> delivered, long from, long to , long now) {
        var problems = new ArrayList<String>();
        double worst = 0;
        for (var e : new java.util.TreeMap<>(budgets).entrySet()) {
            long budget = e.getValue();
            long spent = delivered.getOrDefault(e.getKey(), 0L);
            if (budget <= 0) {
                continue;
            }
            worst = Math.max(worst, (double) spent / budget);
            if (spent > budget * BUDGET_TOLERANCE) {
                problems.add(String.format("%s: delivered %s of %s (%.1f%%)", e.getKey(), dollars(spent), dollar s(budget),
                        100.0 * spent / budget));
            }
        }
        String summary = problems.isEmpty()
                ? String.format("%d line items within budget, highest at %.1f%%", budgets.size(), 100 * worst)
                : String.format("%d of %d line items over budget by more than 1%%", problems.size(), budgets.siz e());
        return CheckResult.of(BUDGETS, from, to, budgets.size(), problems, summary, now);
    }

    /**
     * Total delivered (played, valid) against served direct spend for a settled period. Below the
     * range, tracking events are going missing; above it, something counts spend that was never served.
     */
    public static CheckResult deliveryRatio(long deliveredMicros, long servedMicros, long from, long to, long no w) {
        if (servedMicros == 0) {
            return CheckResult.pending(DELIVERY_RATIO, "nothing served in the period", now);
        }
        double ratio = (double) deliveredMicros / servedMicros;
        var problems = new ArrayList<String>();
        if (ratio < MIN_DELIVERY_RATIO || ratio > MAX_DELIVERY_RATIO) {
            problems.add(String.format("delivered/served %.3f outside [%.2f, %.2f]", ratio, MIN_DELIVERY_RATIO, MAX_DELIVERY_RATIO));
        }
        String summary = String.format("delivered %s of %s served (%.1f%%)", dollars(deliveredMicros), dollars(s ervedMicros),
                100 * ratio);
        return CheckResult.of(DELIVERY_RATIO, from, to, 1, problems, summary, now);
    }

    static String dollars(long micros) {
        return String.format("$%,.2f", micros / 1e6);
    }

    private Checks() {
    }
}
```

audit-service/src/main/java/dev/primetech/ads/audit/Sources.java

101 lines · The two sources the audit queries on demand: Druid (SQL over HTTP) and Redis.

```java
package dev.primetech.ads.audit;

import com.fasterxml.jackson.databind.JsonNode;
import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.pacing.ServedSpend;
import redis.clients.jedis.JedisPooled;

import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.time.Instant;
import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

/** The two sources the audit queries on demand: Druid (SQL over HTTP) and Redis. */
public interface Sources {

    /** Billable impressions and spend per line item and minute, keyed {@code lineItemId|minuteStartMs}. */
    Map<String, Checks.Totals> druidMinutes(long from, long to) throws IOException;

    /** Billable spend per line item over [from, to). */
    Map<String, Long> druidSpend(long from, long to) throws IOException;

    /** The ad server's served-spend counters for a period. */
    Map<String, Long> redisServed(long periodIndex, Collection<String> lineItemIds);

    final class Live implements Sources, AutoCloseable {
        private final URI druidSql;
        private final HttpClient http = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(5)).build();
        private final JedisPooled redis;

        public Live(String druidUrl, String redisUrl) {
            this.druidSql = URI.create(druidUrl.replaceAll("/+$", "") + "/druid/v2/sql");
            URI r = URI.create(redisUrl);
            this.redis = new JedisPooled(r.getHost(), r.getPort() > 0 ? r.getPort() : 6379);
        }

        @Override
        public Map<String, Checks.Totals> druidMinutes(long from, long to) throws IOException {
            var out = new HashMap<String, Checks.Totals>();
            for (JsonNode row : sql("""
                    SELECT lineItemId, TIME_FLOOR(__time, 'PT1M') AS minute_start,
                           SUM(events) FILTER (WHERE type = 'IMPRESSION' AND ivt = 0) AS impressions,
                           SUM(priceMicros) FILTER (WHERE ivt = 0) AS spend
                    FROM ad_events
                    WHERE __time >= MILLIS_TO_TIMESTAMP(%d) AND __time < MILLIS_TO_TIMESTAMP(%d)
                      AND lineItemId IS NOT NULL AND lineItemId <> ''
                    GROUP BY 1, 2""".formatted(from, to))) {
                // ("minute" is a reserved word in Druid SQL: the first version failed with 400.)
                long minute = Instant.parse(row.path("minute_start").asText()).toEpochMilli();
                out.put(row.path("lineItemId").asText() + "|" + minute,
                        new Checks.Totals(row.path("impressions").asLong(), row.path("spend").asLong()));
            }
            return out;
        }

        @Override
        public Map<String, Long> druidSpend(long from, long to) throws IOException {
            var out = new HashMap<String, Long>();
            for (JsonNode row : sql("""
                    SELECT lineItemId, SUM(priceMicros) FILTER (WHERE ivt = 0) AS spend
                    FROM ad_events
                    WHERE __time >= MILLIS_TO_TIMESTAMP(%d) AND __time < MILLIS_TO_TIMESTAMP(%d)
                      AND lineItemId IS NOT NULL AND lineItemId <> ''
                    GROUP BY 1""".formatted(from, to))) {
                out.put(row.path("lineItemId").asText(), row.path("spend").asLong());
            }
            return out;
        }

        @Override
        public Map<String, Long> redisServed(long periodIndex, Collection<String> lineItemIds) {
            return ServedSpend.read(redis, periodIndex, lineItemIds);
        }

        private JsonNode sql(String query) throws IOException {
            var body = Json.MAPPER.writeValueAsBytes(Map.of("query", query));
            var request = HttpRequest.newBuilder(druidSql).timeout(Duration.ofSeconds(30))
                    .header("Content-Type", "application/json").POST(HttpRequest.BodyPublishers.ofByteArray(body
↪ )).build();
            try {
```

```
                HttpResponse<byte[]> response = http.send(request, HttpResponse.BodyHandlers.ofByteArray());
                if (response.statusCode() != 200) {
                    throw new IOException("Druid SQL returned " + response.statusCode());
                }
                return Json.MAPPER.readTree(response.body());
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                throw new IOException("interrupted", e);
            }
        }

        @Override
        public void close() {
            redis.close();
        }
    }
}
```

audit-service/src/main/resources/simplelogger.properties

7 lines · The Kafka client logs its full config at INFO on startup; keep only warnings.

```
org.slf4j.simpleLogger.defaultLogLevel=info
org.slf4j.simpleLogger.showDateTime=true
org.slf4j.simpleLogger.dateTimeFormat=yyyy-MM-dd'T'HH:mm:ss.SSSZ
org.slf4j.simpleLogger.showThreadName=false
org.slf4j.simpleLogger.showShortLogName=true
# The Kafka client logs its full config at INFO on startup; keep only warnings.
org.slf4j.simpleLogger.log.org.apache.kafka=warn
```

audit-service/src/test/java/dev/primetech/ads/audit/AuditServiceTest.java

171 lines · Sources that answer from maps, or fail like a down Druid or Redis.

```
package dev.primetech.ads.audit;

import com.fasterxml.jackson.databind.JsonNode;
import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.LineItemMinute;
import dev.primetech.ads.common.catalog.LineItemConfig;
import dev.primetech.ads.common.pacing.PacingPeriod;
import io.micrometer.core.instrument.simple.SimpleMeterRegistry;
import org.junit.jupiter.api.Test;

import java.io.IOException;
import java.time.Duration;
import java.util.ArrayList;
import java.util.Collection;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicLong;

import static org.assertj.core.api.Assertions.assertThat;

class AuditServiceTest {
    private static final PacingPeriod PERIOD = PacingPeriod.of(Duration.ofMinutes(10));
    private static final long P = 3_000_000;
    private static final long START = PERIOD.start(P);

    /** Sources that answer from maps, or fail like a down Druid or Redis. */
    static final class FakeSources implements Sources {
        final Map<String, Checks.Totals> druidMinutes = new HashMap<>();
        final Map<String, Long> druidSpend = new HashMap<>();
        final Map<String, Long> redis = new HashMap<>();
        boolean druidDown;
        long minutesFrom, minutesTo, spendFrom, spendTo;

        @Override
        public Map<String, Checks.Totals> druidMinutes(long from, long to) throws IOException {
            if (druidDown) throw new IOException("connection refused");
            minutesFrom = from;
            minutesTo = to;
            return druidMinutes;
        }

        @Override
        public Map<String, Long> druidSpend(long from, long to) throws IOException {
            if (druidDown) throw new IOException("connection refused");
            spendFrom = from;
            spendTo = to;
            return druidSpend;
        }

        @Override
        public Map<String, Long> redisServed(long periodIndex, Collection<String> ids) {
            return redis;
        }
    }
```

```
57
58      private final AuditLedger ledger = new AuditLedger(PERIOD);
59      private final FakeSources sources = new FakeSources();
60      // Five minutes after period P ended: P is settled.
61      private final AtomicLong now = new AtomicLong(PERIOD.end(P) + 5 * 60_000);
62      private final AtomicBoolean caughtUp = new AtomicBoolean(true);
63      private final List<CheckResult> published = new ArrayList<>();
64      private final SimpleMeterRegistry registry = new SimpleMeterRegistry();
65      private final AuditService service = new AuditService(ledger, sources, PERIOD, now::get, caughtUp::get,
66              published::add, registry);
67
68      private static JsonNode decision(long ts, String lineItemId, long clearingCpmMicros) throws IOException {
69          return Json.MAPPER.readTree("""
70                  {"timestampMs": %d, "slots": [
71                    {"source": "direct", "lineItemId": "%s", "clearingCpmMicros": %d},
72                    {"source": "premium-dsp", "lineItemId": null, "clearingCpmMicros": 9000000}]}""
73                  .formatted(ts, lineItemId, clearingCpmMicros));
74      }
75
76      private CheckResult result(List<CheckResult> results, String check) {
77          return results.stream().filter(r -> r.check().equals(check)).findFirst().orElseThrow();
78      }
79
80      private void healthyPeriod() throws IOException {
81          ledger.lineItem(new LineItemConfig("li-1", "c", "a", LineItemConfig.Tier.STANDARD, 30_000_000, 10_000_00
  ↪ 0));
82          ledger.decision(decision(START + 1_000, "li-1", 30_000_000)); // $0.03
83          ledger.decision(decision(START + 2_000, "li-1", 30_000_000));
84          ledger.decision(decision(PERIOD.end(P) + 60_000, "li-1", 30_000_000)); // next period: log read past the
  ↪  end
85          sources.redis.put("li-1", 60_000L);
86          sources.druidSpend.put("li-1", 58_000L);
87      }
88
89      @Test
90      void reportsPendingUntilTheTopicsHaveBeenRead() {
91          caughtUp.set(false);
92          assertThat(service.run()).allMatch(r -> r.status() == CheckResult.Status.PENDING);
93          assertThat(published).isEmpty();
94          assertThat(registry.get("audit.check.ok").tag("check", Checks.BUDGETS).gauge().value()).isNaN();
95      }
96
97      @Test
98      void checksTheLastSettledPeriodAndPublishesTheResults() throws IOException {
99          healthyPeriod();
100         List<CheckResult> results = service.run();
101
102         assertThat(results).allMatch(r -> r.status() == CheckResult.Status.OK);
103         assertThat(sources.spendFrom).isEqualTo(START);
104         assertThat(sources.spendTo).isEqualTo(PERIOD.end(P));
105         assertThat(published).hasSize(4);
106         assertThat(registry.get("audit.check.ok").tag("check", Checks.SERVED_VS_DECISIONS).gauge().value()).isEq
  ↪ ualTo(1);
107     }
108
109     @Test
110     void comparesOnlyMinutesThatCanNoLongerChange() throws IOException {
111         healthyPeriod();
112         service.run();
113         // The newest minute compared ended at least 12 minutes ago (10 min allowed lateness + margin).
114         assertThat(now.get() - sources.minutesTo).isGreaterThanOrEqualTo(12 * 60_000);
115         assertThat(sources.minutesTo - sources.minutesFrom).isEqualTo(30 * 60_000);
116     }
117
118     @Test
119     void aBrokenCounterIsAMismatchAndShowsInTheMetrics() throws IOException {
120         healthyPeriod();
121         sources.redis.put("li-1", 30_000L); // lost one increment
122         CheckResult r = result(service.run(), Checks.SERVED_VS_DECISIONS);
123
124         assertThat(r.status()).isEqualTo(CheckResult.Status.MISMATCH);
125         assertThat(registry.get("audit.check.ok").tag("check", Checks.SERVED_VS_DECISIONS).gauge().value()).isZe
  ↪ ro();
126         assertThat(registry.get("audit.check.mismatches").tag("check", Checks.SERVED_VS_DECISIONS).gauge().value
  ↪ ()).isEqualTo(1);
127     }
128
129     @Test
130     void aDownSourceIsAnErrorThatKeepsTheLastVerdict() throws IOException {
131         healthyPeriod();
132         service.run();
133         sources.druidDown = true;
134         List<CheckResult> results = service.run();
135
136         assertThat(result(results, Checks.FLINK_VS_DRUID).status()).isEqualTo(CheckResult.Status.ERROR);
137         assertThat(result(results, Checks.BUDGETS).status()).isEqualTo(CheckResult.Status.ERROR);
138         // Redis still answers, so that check still runs.
139         assertThat(result(results, Checks.SERVED_VS_DECISIONS).status()).isEqualTo(CheckResult.Status.OK);
```

```
        // An error says nothing about the data: the gauge keeps the last real verdict, and the
        // last-completed timestamp stops advancing, which is what the staleness alert watches.
        assertThat(registry.get("audit.check.ok").tag("check", Checks.BUDGETS).gauge().value()).isEqualTo(1);
    }

    @Test
    void waitsForTheDecisionLogToPassThePeriodEnd() throws IOException {
        ledger.lineItem(new LineItemConfig("li-1", "c", "a", LineItemConfig.Tier.STANDARD, 30_000_000, 10_000_00
↪ 0));
        ledger.decision(decision(START + 1_000, "li-1", 30_000_000)); // nothing read after P yet
        assertThat(result(service.run(), Checks.SERVED_VS_DECISIONS).status()).isEqualTo(CheckResult.Status.PEND
↪ ING);
    }

    @Test
    void keepsTheLatestRevisionOfEachMinute() {
        ledger.minute(minute(2, 7));
        ledger.minute(minute(1, 5)); // an older revision arriving late
        assertThat(ledger.minutes(0, Long.MAX_VALUE).values()).containsExactly(new Checks.Totals(7, 7_000));
    }

    @Test
    void resetForgetsDecisionsSoAReReadDoesNotDoubleCount() throws IOException {
        ledger.decision(decision(START + 1_000, "li-1", 30_000_000));
        ledger.reset();
        ledger.decision(decision(START + 1_000, "li-1", 30_000_000));
        assertThat(ledger.served(P)).containsEntry("li-1", 30_000L);
    }

    private static LineItemMinute minute(int revision, long impressions) {
        return new LineItemMinute("li-1", "c", "a", START, START + 60_000, revision, impressions, 0, 0, 0, 0, 0,
                impressions * 1_000, 0, START);
    }
}
```

audit-service/src/test/java/dev/primetech/ads/audit/ChecksTest.java

58 lines · Test.

```
package dev.primetech.ads.audit;

import dev.primetech.ads.audit.Checks.Totals;
import org.junit.jupiter.api.Test;

import java.util.Map;

import static org.assertj.core.api.Assertions.assertThat;

class ChecksTest {

    @Test
    void flinkAndDruidMustAgreeExactly() {
        var flink = Map.of("li-1|0", new Totals(10, 300_000), "li-2|0", new Totals(5, 150_000));
        assertThat(Checks.flinkVsDruid(flink, Map.copyOf(flink), 0, 60_000, 1).status()).isEqualTo(CheckResult.S
↪ tatus.OK);

        // One impression missing from Druid, and a minute Flink never produced.
        var druid = Map.of("li-1|0", new Totals(9, 270_000), "li-2|0", new Totals(5, 150_000), "li-3|0", new Tot
↪ als(1, 30_000));
        CheckResult r = Checks.flinkVsDruid(flink, druid, 0, 60_000, 1);
        assertThat(r.status()).isEqualTo(CheckResult.Status.MISMATCH);
        assertThat(r.compared()).isEqualTo(3);
        assertThat(r.mismatches()).isEqualTo(2);
        assertThat(r.examples()).anyMatch(e -> e.startsWith("li-1|0: Flink 10 imps"));
    }

    @Test
    void minutesWithNothingBillableOnEitherSideAgree() {
        // Flink emits a minute for quartile events alone; Druid may have no row for it.
        var flink = Map.of("li-1|0", new Totals(0, 0));
        assertThat(Checks.flinkVsDruid(flink, Map.of(), 0, 60_000, 1).status()).isEqualTo(CheckResult.Status.OK)
↪ ;
    }

    @Test
    void servedCountersMustMatchTheDecisionLog() {
        assertThat(Checks.servedVsDecisions(Map.of("a", 5L), Map.of("a", 5L), 0, 1, 1).status())
                .isEqualTo(CheckResult.Status.OK);
        CheckResult r = Checks.servedVsDecisions(Map.of("a", 5L, "b", 7L), Map.of("a", 5L), 0, 1, 1);
        assertThat(r.status()).isEqualTo(CheckResult.Status.MISMATCH);
        assertThat(r.examples()).containsExactly("b: decision log $0.00, Redis $0.00");
    }

    @Test
    void flagsOnlyLineItemsMoreThanOnePercentOverBudget() {
        var budgets = Map.of("a", 100_000_000L, "b", 100_000_000L, "c", 100_000_000L);
        var delivered = Map.of("a", 100_900_000L, "b", 101_100_000L, "c", 50_000_000L);
        CheckResult r = Checks.budgets(budgets, delivered, 0, 1, 1);
        assertThat(r.mismatches()).isEqualTo(1);
```

```
48          assertThat(r.examples()).containsExactly("b: delivered $101.10 of $100.00 (101.1%)");
49      }
50
51      @Test
52      void deliveryRatioOutsideTheExpectedRangeIsAMismatch() {
53          assertThat(Checks.deliveryRatio(96, 100, 0, 1, 1).status()).isEqualTo(CheckResult.Status.OK);
54          assertThat(Checks.deliveryRatio(60, 100, 0, 1, 1).status()).isEqualTo(CheckResult.Status.MISMATCH);
55          assertThat(Checks.deliveryRatio(110, 100, 0, 1, 1).status()).isEqualTo(CheckResult.Status.MISMATCH);
56          assertThat(Checks.deliveryRatio(0, 0, 0, 1, 1).status()).isEqualTo(CheckResult.Status.PENDING);
57      }
58  }
```

dashboard

dashboard/.dockerignore

4 lines

```
1  node_modules
2  .nuxt
3  .output
4  .data
```

dashboard/Dockerfile

22 lines · Optional "cacert" build secret: extra CA bundle for builds behind a TLS-inspecting proxy.

```
 1  FROM node:22-alpine AS build
 2  WORKDIR /app
 3  COPY package.json package-lock.json ./
 4  # Optional "cacert" build secret: extra CA bundle for builds behind a TLS-inspecting proxy.
 5  RUN --mount=type=secret,id=cacert,required=false \
 6      if [ -f /run/secrets/cacert ]; then export NODE_EXTRA_CA_CERTS=/run/secrets/cacert; fi; \
 7      npm ci --ignore-scripts
 8  COPY . .
 9  RUN npx nuxt prepare && npm run build
10
11  FROM node:22-alpine
12  WORKDIR /app
13  COPY --from=build /app/.output ./.output
14  # The password-hash tool, for creating NUXT_AUTH_USERS entries without Node installed locally.
15  COPY --from=build /app/scripts/hash-password.ts ./tools/scripts/
16  COPY --from=build /app/server/utils/auth.ts ./tools/server/utils/
17  ENV HOST=0.0.0.0 PORT=3000 NODE_ENV=production
18  # Push subscriptions live here; a volume mounted on it inherits this ownership.
19  RUN mkdir -p /app/.data && chown node:node /app/.data
20  USER node
21  EXPOSE 3000
22  CMD ["node", ".output/server/index.mjs"]
```

dashboard/app/app.vue

114 lines

```
  1  <script setup lang="ts">
  2  const session = useSession()
  3  const online = useOnline()
  4  const { canPrompt, iosHint, install } = useInstall()
  5  const showIosHint = ref(false)
  6
  7  onMounted(() => session.check())
  8  // Back online, or back from the background: the session may have expired meanwhile.
  9  watch(online, (now) => now && session.check())
 10  </script>
 11
 12  <template>
 13    <main>
 14      <header>
 15        <div class="title">
 16          <BrandLogo :height="48" />
 17          <div>
 18            <h1>Ads Eventing</h1>
 19            <p class="muted">Live pipeline · ad server → player → gateway → Kafka → Flink → Druid</p>
 20          </div>
 21        </div>
 22        <div class="actions">
 23          <button v-if="canPrompt" type="button" class="ghost" @click="install">Install app</button>
 24          <button v-else-if="iosHint" type="button" class="ghost" :aria-expanded="showIosHint"
 25            @click="showIosHint = !showIosHint">Install app</button>
 26          <template v-if="session.status.value === 'signed-in'">
 27            <span class="muted">{{ session.user.value }}</span>
 28            <button type="button" class="ghost" @click="session.signOut()">Sign out</button>
 29          </template>
 30          <span v-else-if="session.status.value === 'open'" class="muted" title="NUXT_AUTH_DISABLED=true">
 31            Sign-in off (local)
 32          </span>
 33        </div>
 34      </header>
 35
```

```
    <p v-if="showIosHint" class="card hint">
      In Safari, tap <strong>Share</strong>, then <strong>Add to Home Screen</strong>.
    </p>

    <p v-if="!online" class="card banner" role="status">
      <span aria-hidden="true">▲</span> You're offline. Figures resume when the connection returns.
    </p>

    <p v-if="session.status.value === 'checking'" class="muted">Loading…</p>
    <LoginForm v-else-if="session.status.value === 'signed-out'" />
    <section v-else-if="session.status.value === 'misconfigured'" class="card">
      <h2>Sign-in is not configured</h2>
      <p class="muted">
        The server requires sign-in but has no valid users or session secret. Set
        <code>NUXT_AUTH_USERS</code> and <code>NUXT_SESSION_SECRET</code> (see the README, "Hosting").
      </p>
    </section>
    <DashboardView v-else />
  </main>
</template>

<style scoped>
main {
  max-width: 1200px;
  margin: 0 auto;
  /* Safe-area insets keep content clear of notches when installed as an app. */
  padding: max(24px, env(safe-area-inset-top)) max(16px, env(safe-area-inset-right))
    max(48px, env(safe-area-inset-bottom)) max(16px, env(safe-area-inset-left));
  display: grid;
  gap: 16px;
}
header {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: flex-end;
  gap: 12px;
}
.title {
  display: flex;
  align-items: center;
  gap: 14px;
}
h1 {
  margin: 0;
  font-size: 22px;
}
header p {
  margin: 2px 0 0;
}
.actions {
  display: flex;
  align-items: center;
  gap: 8px;
  flex-wrap: wrap;
  font-size: 13px;
}
.ghost {
  font: inherit;
  min-height: 36px;
  padding: 6px 12px;
  border: 1px solid var(--axis);
  border-radius: 8px;
  background: var(--surface);
  color: var(--ink-1);
  cursor: pointer;
}
.ghost:focus-visible {
  outline: 2px solid var(--series-1);
  outline-offset: 2px;
}
.hint,
.banner {
  margin: 0;
}
.banner span {
  color: var(--warning);
}
</style>
```

dashboard/app/assets/main.css

84 lines

```
:root {
  color-scheme: light;
  --page: #f9f9f7;
  --surface: #fcfcfb;
  --ink-1: #0b0b0b;
```

```
  --ink-2: #52514e;
  --ink-muted: #898781;
  --grid: #e1e0d9;
  --axis: #c3c2b7;
  --border: rgba(11, 11, 11, 0.1);
  --series-1: #2a78d6;
  --good: #0ca30c;
  --critical: #d03b3b;
  --warning: #fab219;
  --tooltip-bg: #0b0b0b;
  --tooltip-ink: #f0efec;
}

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    color-scheme: dark;
    --page: #0d0d0d;
    --surface: #1a1a19;
    --ink-1: #f0efec;
    --ink-2: #c3c2b7;
    --ink-muted: #898781;
    --grid: #2c2c2a;
    --axis: #383835;
    --border: rgba(255, 255, 255, 0.1);
    --series-1: #3987e5;
    --tooltip-bg: #f0efec;
    --tooltip-ink: #0b0b0b;
  }
}

:root[data-theme="dark"] {
  color-scheme: dark;
  --page: #0d0d0d;
  --surface: #1a1a19;
  --ink-1: #f0efec;
  --ink-2: #c3c2b7;
  --ink-muted: #898781;
  --grid: #2c2c2a;
  --axis: #383835;
  --border: rgba(255, 255, 255, 0.1);
  --series-1: #3987e5;
  --tooltip-bg: #f0efec;
  --tooltip-ink: #0b0b0b;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  background: var(--page);
  color: var(--ink-1);
  font: 14px/1.45 ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  font-variant-numeric: tabular-nums;
}

.card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 16px;
  min-width: 0;
}

.card h2 {
  margin: 0 0 12px;
  font-size: 14px;
  font-weight: 600;
}

.muted {
  color: var(--ink-muted);
}

code {
  font-family: ui-monospace, SFMono-Regular, Menlo, monospace;
  font-size: 12px;
}
```

dashboard/app/components/AdServingPanel.vue

187 lines

```
<script setup lang="ts">
interface DspStats {
  requests: number
  bids: number
  noBids: number
```

```
  timeouts: number
  errors: number
  wins: number
  latencyP50Ms: number
  latencyP99Ms: number
}
interface AdServerStats {
  available: boolean
  error?: string
  timestampMs: number
  requests: number
  shed: number
  slotsRequested: number
  slotsFilled: number
  frequencyCapped: number
  spendMicros: number
  latencyP50Ms: number
  latencyP99Ms: number
  winsBySource: Record<string, number>
  dsps: Record<string, DspStats>
}

const { data, reachable } = usePolled<AdServerStats>('/api/adserver', 2000)

// Auctions per second from successive counter readings.
const rate = ref(0)
let previous: { t: number; n: number } | null = null
watch(data, (s) => {
  if (!s?.available) return
  if (previous && s.timestampMs > previous.t && s.requests >= previous.n) {
    rate.value = ((s.requests - previous.n) * 1000) / (s.timestampMs - previous.t)
  }
  previous = { t: s.timestampMs, n: s.requests }
})

const fill = computed(() => (data.value?.slotsRequested ? data.value.slotsFilled / data.value.slotsRequested : 0
↪ ))
const totalWins = computed(() => Object.values(data.value?.winsBySource ?? {}).reduce((a, b) => a + b, 0))
const rows = computed(() => {
  const s = data.value
  if (!s?.available) return []
  const out = Object.entries(s.dsps ?? {}).map(([name, d]) => ({
    name,
    bidRate: d.requests ? (d.requests - d.noBids - d.timeouts - d.errors) / d.requests : 0,
    timeoutRate: d.requests ? d.timeouts / d.requests : 0,
    winShare: totalWins.value ? d.wins / totalWins.value : 0,
    p99: d.latencyP99Ms,
  }))
  out.push({
    name: 'direct (line items)',
    bidRate: NaN,
    timeoutRate: NaN,
    winShare: totalWins.value ? (s.winsBySource.direct ?? 0) / totalWins.value : 0,
    p99: NaN,
  })
  return out.sort((a, b) => b.winShare - a.winShare)
})
</script>

<template>
  <section class="card">
    <h2>Ad serving · OpenRTB auction</h2>
    <p v-if="!reachable" class="muted">
      Needs the server routes from the Docker stack (<code>docker compose up</code>); not available on the stati
↪ c site.
    </p>
    <p v-else-if="data && !data.available" class="muted">The ad server is not reachable yet. <code>{{ data.error
↪  }}</code></p>
    <template v-else-if="data">
      <div class="tiles">
        <StatTile label="Auctions/s" :value="formatWhole(rate)" :detail="`${formatCompact(data.requests)} total`
↪ " />
        <StatTile label="Fill rate" :value="formatPercent(fill, 1)" detail="slots served / requested" />
        <StatTile
          label="Decision latency p99"
          :value="`${data.latencyP99Ms.toFixed(0)} ms`"
          :detail="`p50 ${data.latencyP50Ms.toFixed(0)} ms · includes DSP wait`"
        />
        <StatTile
          label="Requests shed"
          :value="formatWhole(data.shed)"
          :detail="data.shed === 0 ? 'within capacity' : 'answered 503 under overload'"
          :tone="data.shed === 0 ? 'good' : 'critical'"
        />
        <StatTile label="Frequency-capped" :value="formatCompact(data.frequencyCapped)" detail="candidates remov
↪ ed" />
      </div>
      <div class="scroll">
```

```
        <table>
          <thead>
            <tr>
              <th>Demand</th>
              <th class="share">Win share</th>
              <th class="num">Bid rate</th>
              <th class="num">Timeouts</th>
              <th class="num">p99</th>
            </tr>
          </thead>
          <tbody>
            <tr v-for="r in rows" :key="r.name">
              <td><code>{{ r.name }}</code></td>
              <td class="share">
                <div class="share-cell">
                  <span class="track" aria-hidden="true"><span class="bar" :style="{ width: `${r.winShare * 100}%` }" /></span>
                  <span class="value">{{ formatPercent(r.winShare, 1) }}</span>
                </div>
              </td>
              <td class="num">{{ Number.isNaN(r.bidRate) ? '—' : formatPercent(r.bidRate, 0) }}</td>
              <td class="num">{{ Number.isNaN(r.timeoutRate) ? '—' : formatPercent(r.timeoutRate, 1) }}</td>
              <td class="num">{{ Number.isNaN(r.p99) ? '—' : `${r.p99.toFixed(0)} ms` }}</td>
            </tr>
          </tbody>
        </table>
      </div>
    </template>
  </section>
</template>

<style scoped>
p {
  margin: 0;
}
.tiles {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(170px, 1fr));
  gap: 12px;
  margin-bottom: 16px;
}
/* Phones: two tiles per row instead of one, halving the scroll to the charts. */
@media (max-width: 520px) {
  .tiles {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}
.tiles :deep(.card) {
  background: var(--page);
}
.scroll {
  overflow-x: auto;
}
table {
  width: 100%;
  min-width: 460px;
  border-collapse: collapse;
  font-size: 13px;
}
th {
  text-align: left;
  font-weight: 500;
  color: var(--ink-muted);
  border-bottom: 1px solid var(--axis);
  padding: 6px 8px;
  white-space: nowrap;
}
td {
  padding: 8px;
  border-bottom: 1px solid var(--grid);
}
.num {
  text-align: right;
  white-space: nowrap;
}
.share {
  width: 40%;
}
.share-cell {
  display: flex;
  align-items: center;
  gap: 8px;
}
.track {
  flex: 1;
  min-width: 48px;
}
.bar {
```

```
  display: block;
  height: 10px;
  min-width: 2px;
  background: var(--series-1);
  border-radius: 0 4px 4px 0;
}
.value {
  width: 48px;
  text-align: right;
  white-space: nowrap;
}
</style>
```

dashboard/app/components/AlertsPanel.vue

176 lines

```
<script setup lang="ts">
interface ActiveAlert {
  id: string
  alertname: string
  severity: 'critical' | 'warning' | string
  check: string | null
  summary: string
  description: string
  startsAt: string
}
interface Sent {
  status: 'firing' | 'resolved'
  alertname: string
  severity: string
  summary: string
  atMs: number
}
interface AlertsResponse {
  available: boolean
  error?: string
  push?: { enabled: boolean; devices: number }
  active?: ActiveAlert[]
  recent?: Sent[]
}

const { data, reachable } = usePolled<AlertsResponse>('/api/alerts', 10_000)
const push = usePush()
const now = ref(Date.now())
useVisibleInterval(() => (now.value = Date.now()), 10_000)

const active = computed(() => data.value?.active ?? [])
const recent = computed(() => (data.value?.recent ?? []).slice(0, 5))
const since = (iso: string) => formatDuration(Math.max(0, now.value - Date.parse(iso)))

const PUSH_TEXT: Record<string, string> = {
  checking: '',
  unsupported: 'This browser cannot receive notifications. On iPhone, add the app to the Home Screen first.',
  unavailable: 'Notifications are not configured on the server (VAPID keys).',
  blocked: 'Notifications are blocked for this site in the browser settings.',
  off: 'Get critical and warning alerts on this device, even when the app is closed.',
  on: 'This device gets a notification when an alert fires or resolves.',
}
</script>

<template>
  <section id="alerts" class="card" aria-labelledby="alerts-title">
    <div class="head">
      <h2 id="alerts-title">Alerts</h2>
      <div v-if="reachable && data?.available" class="push">
        <span class="muted">{{ PUSH_TEXT[push.state.value] }}</span>
        <button v-if="push.state.value === 'off'" type="button" :disabled="push.busy.value" @click="push.enable(
  ↪ )">
          {{ push.busy.value ? 'Turning on…' : 'Turn on notifications' }}
        </button>
        <button v-else-if="push.state.value === 'on'" type="button" class="ghost" :disabled="push.busy.value"
          @click="push.disable()">Turn off</button>
      </div>
    </div>
    <p v-if="push.error.value" class="critical-text" role="alert">▲ {{ push.error.value }}</p>

    <p v-if="!reachable" class="muted">
      Needs the server routes from the Docker stack (<code>docker compose up</code>); not available on the stati
  ↪ c site.
    </p>
    <p v-else-if="data && !data.available" class="muted">Alertmanager is not reachable. <code>{{ data.error }}</
  ↪ code></p>
    <template v-else-if="data">
      <p v-if="!active.length" class="all-clear"><span class="icon good" aria-hidden="true">✓</span> No active a
  ↪ lerts.</p>
      <ul v-else class="alerts">
        <li v-for="a in active" :key="a.id" :class="a.severity">
          <span class="icon" aria-hidden="true">{{ a.severity === 'critical' ? '!' : '▲' }}</span>
```

```
          <div>
            <strong>{{ a.summary }}</strong>
            <span class="muted"> · {{ a.severity }} · for {{ since(a.startsAt) }}</span>
            <p class="muted">{{ a.description }}</p>
          </div>
        </li>
      </ul>
      <details v-if="recent.length" class="recent">
        <summary class="muted">Recent notifications</summary>
        <ul>
          <li v-for="(n, i) in recent" :key="i" class="muted">
            {{ formatClock(n.atMs) }} · {{ n.status === 'firing' ? 'Fired' : 'Resolved' }}: {{ n.summary }}
          </li>
        </ul>
      </details>
    </template>
  </section>
</template>

<style scoped>
.head {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: baseline;
  gap: 8px 16px;
}
.head h2 {
  margin-bottom: 8px;
}
.push {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 8px;
  font-size: 13px;
}
button {
  font: inherit;
  font-weight: 600;
  min-height: 36px;
  padding: 6px 12px;
  border: 0;
  border-radius: 8px;
  background: var(--series-1);
  color: #fff;
  cursor: pointer;
}
button.ghost {
  font-weight: 400;
  border: 1px solid var(--axis);
  background: var(--surface);
  color: var(--ink-1);
}
button:focus-visible {
  outline: 2px solid var(--series-1);
  outline-offset: 2px;
}
.all-clear {
  margin: 4px 0 0;
}
.icon.good {
  color: var(--good);
  font-weight: 700;
}
.alerts {
  list-style: none;
  margin: 4px 0 0;
  padding: 0;
  display: grid;
  gap: 10px;
}
.alerts li {
  display: grid;
  grid-template-columns: 20px 1fr;
  gap: 8px;
  padding: 10px 12px;
  border-radius: 8px;
  border: 1px solid var(--border);
  border-left: 4px solid var(--warning);
}
.alerts li.critical {
  border-left-color: var(--critical);
}
.alerts li .icon {
  font-weight: 700;
  color: var(--warning);
}
```

```
.alerts li.critical .icon {
  color: var(--critical);
}
.alerts p {
  margin: 4px 0 0;
  font-size: 13px;
}
.critical-text {
  color: var(--critical);
  font-size: 13px;
}
.recent {
  margin-top: 12px;
  font-size: 13px;
}
.recent ul {
  margin: 6px 0 0;
  padding-left: 18px;
}
</style>
```

dashboard/app/components/AuditPanel.vue

188 lines

```
<script setup lang="ts">
type Status = 'OK' | 'MISMATCH' | 'PENDING' | 'ERROR'
interface Result {
  check: string
  status: Status
  windowStartMs: number
  windowEndMs: number
  compared: number
  mismatches: number
  summary: string
  examples: string[]
  ranAtMs: number
}
interface CheckStats {
  latest: Result | null
  latestCompleted: Result | null
  recent: { status: Status; ranAtMs: number; mismatches: number }[]
}
interface AuditStats {
  available: boolean
  error?: string
  caughtUp?: boolean
  checks?: Record<string, CheckStats>
}

const { data, reachable } = usePolled<AuditStats>('/api/audit', 15_000)

const CHECKS: Record<string, { title: string; what: string }> = {
  'flink-vs-druid': {
    title: 'Stream totals vs analytics store',
    what: 'Flink’s exactly-once minute totals against Druid’s rollup, per line item and minute, once minutes can no longer change.',
  },
  'served-vs-decision-log': {
    title: 'Pacing counters vs decision log',
    what: 'Served spend in Redis, which pacing steers by, against every direct win in the ad server’s decision log.',
  },
  'budget-compliance': {
    title: 'Budget compliance',
    what: 'Delivered spend per line item, from Druid, against its budget for the last settled period.',
  },
  'delivered-vs-served': {
    title: 'Delivered vs served',
    what: 'Played, valid spend against served spend: too low means tracking is being lost, too high means invented spend.',
  },
}
const STATUS: Record<Status, { label: string; icon: string; tone: string }> = {
  OK: { label: 'Agrees', icon: '✓', tone: 'good' },
  MISMATCH: { label: 'Mismatch', icon: '!', tone: 'critical' },
  PENDING: { label: 'Waiting for data', icon: '○', tone: 'muted' },
  ERROR: { label: 'Source unavailable', icon: '▲', tone: 'warning' },
}

const rows = computed(() =>
  Object.entries(CHECKS).map(([key, meta]) => {
    const s = data.value?.checks?.[key]
    // Show the latest real verdict; mention a newer pending or error run separately.
    const shown = s?.latestCompleted ?? s?.latest ?? null
    const newer = s?.latest && s.latestCompleted && s.latest.ranAtMs > s.latestCompleted.ranAtMs && s.latest.status !== 'OK'
      ? s.latest : null
    const history = (s?.recent ?? []).slice(0, 12).reverse()
```

```
    return { key, ...meta, shown, newer, history }
  }),
)
const windowLabel = (r: Result) =>
  r.windowEndMs ? `${formatClock(r.windowStartMs)}–${formatClock(r.windowEndMs)}` : ''
</script>

<template>
  <section class="card" aria-labelledby="audit-title">
    <h2 id="audit-title">Audit · continuous reconciliation</h2>
    <p v-if="!reachable" class="muted">
      Needs the server routes from the Docker stack (<code>docker compose up</code>); not available on the stati
↪ c site.
    </p>
    <p v-else-if="data && !data.available" class="muted">The audit service is not reachable yet. <code>{{ data.e
↪ rror }}</code></p>
    <p v-else-if="data && !data.caughtUp" class="muted">The audit service is reading the topics from the beginni
↪ ng…</p>
    <ul v-else-if="data" class="checks">
      <li v-for="r in rows" :key="r.key">
        <div class="title">
          <strong>{{ r.title }}</strong>
          <span v-if="r.shown" :class="['status', STATUS[r.shown.status].tone]">
            <span class="icon" aria-hidden="true">{{ STATUS[r.shown.status].icon }}</span>
            {{ STATUS[r.shown.status].label }}
          </span>
        </div>
        <p class="muted what">{{ r.what }}</p>
        <template v-if="r.shown">
          <p class="summary">{{ r.shown.summary }}</p>
          <p class="muted meta">
            <template v-if="windowLabel(r.shown)">Window {{ windowLabel(r.shown) }} · </template>checked {{ form
↪ atClock(r.shown.ranAtMs) }}
          </p>
          <ul v-if="r.shown.examples.length" class="examples">
            <li v-for="e in r.shown.examples" :key="e"><code>{{ e }}</code></li>
          </ul>
        </template>
        <p v-if="r.newer" :class="['meta', STATUS[r.newer.status].tone]">
          <span aria-hidden="true">{{ STATUS[r.newer.status].icon }}</span> Latest run: {{ r.newer.summary }}
        </p>
        <div v-if="r.history.length" class="history" :aria-label="`Last ${r.history.length} runs`">
          <span v-for="(h, i) in r.history" :key="i" :class="['tick', STATUS[h.status].tone]"
            :title="`${formatClock(h.ranAtMs)}: ${STATUS[h.status].label}`" />
        </div>
      </li>
    </ul>
  </section>
</template>

<style scoped>
.checks {
  list-style: none;
  margin: 0;
  padding: 0;
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 260px), 1fr));
  gap: 12px;
}
.checks > li {
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 12px;
  min-width: 0;
}
.title {
  display: flex;
  justify-content: space-between;
  align-items: baseline;
  gap: 8px;
  flex-wrap: wrap;
}
.status {
  font-size: 13px;
  white-space: nowrap;
}
.status .icon {
  font-weight: 700;
}
.good {
  color: var(--good);
}
.critical {
  color: var(--critical);
}
.warning {
  color: var(--ink-2);
}
```

```
.warning > span,
.warning .icon {
  color: var(--warning);
}
.muted {
  color: var(--ink-muted);
}
p {
  margin: 6px 0 0;
}
.what,
.meta {
  font-size: 12px;
}
.summary {
  font-size: 13px;
}
.examples {
  margin: 6px 0 0;
  padding-left: 16px;
  font-size: 12px;
  overflow-wrap: anywhere;
}
.history {
  display: flex;
  gap: 3px;
  margin-top: 10px;
}
.tick {
  width: 10px;
  height: 10px;
  border-radius: 2px;
  background: var(--grid);
}
.tick.good {
  background: var(--good);
}
.tick.critical {
  background: var(--critical);
}
.tick.warning {
  background: var(--warning);
}
</style>
```

dashboard/app/components/BrandLogo.vue

27 lines · The Quila Visuals logo. The PNG is used as a mask so the logo takes the current text colour and

```
<script setup lang="ts">
// The Quila Visuals logo. The PNG is used as a mask so the logo takes the current text colour and
// follows the light and dark themes from a single file.
withDefaults(defineProps<{ height?: number }>(), { height: 44 })
const src = `${useRuntimeConfig().app.baseURL}brand-logo.png`
</script>

<template>
  <span
    class="brand-logo"
    role="img"
    aria-label="Quila Visuals"
    :style="{ height: `${height}px`, '--logo': `url(${src})` }"
  />
</template>

<style scoped>
.brand-logo {
  display: inline-block;
  /* brand-logo.png is 686 × 746. */
  aspect-ratio: 686 / 746;
  flex: none;
  background: var(--ink-1);
  -webkit-mask: var(--logo) center / contain no-repeat;
  mask: var(--logo) center / contain no-repeat;
}
</style>
```

dashboard/app/components/CampaignTable.vue

125 lines

```
<script setup lang="ts">
interface CampaignRow {
  advertiserName: string
  campaignName: string
  campaignId: string
  impressions: number
  completes: number
  spendMicros: number
  ivtEvents: number
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
  events: number
}
interface Campaigns {
  available: boolean
  minutes: number
  rows?: CampaignRow[]
  error?: string
}

const { data, reachable } = usePolled<Campaigns>('/api/campaigns?minutes=60', 10_000)
const rows = computed(() => data.value?.rows ?? [])
const maxSpend = computed(() => Math.max(1, ...rows.value.map((r) => r.spendMicros)))
</script>

<template>
  <section class="card">
    <h2>Campaign delivery · last {{ data?.minutes ?? 60 }} minutes · Druid</h2>
    <p v-if="!reachable" class="muted">
      Needs the server routes from the Docker stack (<code>docker compose up</code>); not available on the stati
↪ c site.
    </p>
    <p v-else-if="data && !data.available" class="muted">Druid is not reachable yet. <code>{{ data.error }}</cod
↪ e></p>
    <p v-else-if="data && !rows.length" class="muted">No delivery in this period.</p>
    <div v-else-if="rows.length" class="scroll">
      <table>
        <thead>
          <tr>
            <th>Campaign</th>
            <th class="num">Impressions</th>
            <th class="spend">Spend</th>
            <th class="num">Completion</th>
            <th class="num">IVT</th>
          </tr>
        </thead>
        <tbody>
          <tr v-for="r in rows" :key="r.campaignId">
            <td>
              <div>{{ r.campaignName }}</div>
              <div class="muted small">{{ r.advertiserName }}</div>
            </td>
            <td class="num">{{ formatWhole(r.impressions) }}</td>
            <td class="spend">
              <div class="spend-cell">
                <span class="track" aria-hidden="true">
                  <span class="bar" :style="{ width: `${(r.spendMicros / maxSpend) * 100}%` }" />
                </span>
                <span class="value">{{ formatUsd(r.spendMicros) }}</span>
              </div>
            </td>
            <td class="num">{{ r.impressions ? formatPercent(r.completes / r.impressions, 1) : '—' }}</td>
            <td class="num">{{ r.events ? formatPercent(r.ivtEvents / r.events, 1) : '—' }}</td>
          </tr>
        </tbody>
      </table>
    </div>
  </section>
</template>

<style scoped>
p {
  margin: 0;
}
.scroll {
  overflow-x: auto;
}
table {
  width: 100%;
  min-width: 480px;
  border-collapse: collapse;
  font-size: 13px;
}
th {
  text-align: left;
  font-weight: 500;
  color: var(--ink-muted);
  border-bottom: 1px solid var(--axis);
  padding: 6px 8px;
  white-space: nowrap;
}
td {
  padding: 6px 8px;
  border-bottom: 1px solid var(--grid);
  vertical-align: middle;
}
.num {
  text-align: right;
  white-space: nowrap;
```

```
}
.small {
  font-size: 12px;
}
.spend {
  width: 30%;
  min-width: 140px;
}
.spend-cell {
  display: flex;
  align-items: center;
  gap: 8px;
}
.track {
  flex: 1;
  min-width: 48px;
}
.bar {
  display: block;
  height: 10px;
  min-width: 2px;
  background: var(--series-1);
  border-radius: 0 4px 4px 0;
}
.value {
  width: 56px;
  text-align: right;
  white-space: nowrap;
}
</style>
```

dashboard/app/components/DashboardView.vue

155 lines

```
<script setup lang="ts">
const online = useOnline()
const { direct, gatewayUrl, latest, history, status, lastError, currentEps, rejectionRate } = useGatewayStats()

const rejectionRows = computed(() =>
  Object.entries(latest.value?.rejectedByReason ?? {})
    .filter(([, n]) => n > 0)
    .sort((a, b) => b[1] - a[1]),
)
</script>

<template>
  <div class="dashboard">
    <div class="status" :class="status" role="status">
      <span class="icon" aria-hidden="true">{{ status === 'up' ? '●' : status === 'down' ? '▲' : '○' }}</span>
      {{ status === 'up' ? 'Gateway connected' : status === 'down' ? 'Gateway unreachable' : 'Connecting' }}
      <code v-if="direct" class="muted">{{ gatewayUrl }}</code>
    </div>

    <AlertsPanel />

    <section v-if="status === 'down' && !latest && online" class="card notice">
      <h2>Can't reach the gateway</h2>
      <p v-if="direct">
        Start the stack with <code>docker compose up --build</code>, or point the dashboard at another gateway
        with <code>?gateway=http://host:8080</code>.
      </p>
      <p v-else>The dashboard server can't reach the ingest gateway. The stack may be stopped or restarting.</p>
      <p class="muted"><code>{{ lastError }}</code></p>
    </section>

    <section class="tiles">
      <StatTile label="Accepted events/s" :value="formatWhole(currentEps)" detail="2s window" />
      <StatTile
        label="Events accepted"
        :value="formatCompact(latest?.accepted ?? 0)"
        :detail="`${formatCompact(latest?.batches ?? 0)} batches`"
      />
      <StatTile
        label="Rejection rate"
        :value="formatPercent(rejectionRate)"
        :detail="`${formatCompact(latest?.rejected ?? 0)} sent to DLQ`"
      />
      <StatTile
        label="Request latency p99"
        :value="`${(latest?.latencyP99Ms ?? 0).toFixed(1)} ms`"
        :detail="`p50 ${(latest?.latencyP50Ms ?? 0).toFixed(1)} ms · includes Kafka ack`"
      />
      <StatTile
        label="Publish failures"
        :value="formatWhole(latest?.publishFailures ?? 0)"
        :detail="(latest?.publishFailures ?? 0) === 0 ? 'Kafka healthy' : 'batches returned 503'"
        :tone="(latest?.publishFailures ?? 0) === 0 ? 'good' : 'critical'"
      />
```

```
    </section>

    <AdServingPanel />

    <PacingPanel />

    <AuditPanel />

    <PipelinePanel />

    <section class="card">
      <h2>Accepted events per second · last 5 minutes</h2>
      <ThroughputChart :points="history" />
    </section>

    <div class="split">
      <section class="card">
        <h2>Playback funnel</h2>
        <FunnelChart :by-type="latest?.acceptedByType ?? {}" />
      </section>

      <section class="card">
        <h2>Rejections by reason</h2>
        <table v-if="rejectionRows.length">
          <thead>
            <tr><th>Reason</th><th class="num">Events</th><th class="num">Share</th></tr>
          </thead>
          <tbody>
            <tr v-for="[reason, n] in rejectionRows" :key="reason">
              <td><code>{{ reason }}</code></td>
              <td class="num">{{ formatWhole(n) }}</td>
              <td class="num">{{ formatPercent(n / (latest?.rejected || 1), 1) }}</td>
            </tr>
          </tbody>
        </table>
        <p v-else class="muted">No rejected events.</p>
      </section>
    </div>

    <CampaignTable />
  </div>
</template>

<style scoped>
.dashboard {
  display: grid;
  gap: 16px;
}
.status {
  display: flex;
  align-items: center;
  gap: 8px;
  flex-wrap: wrap;
  font-size: 13px;
  color: var(--ink-2);
}
.status.up .icon {
  color: var(--good);
}
.status.down .icon {
  color: var(--critical);
}
.notice p {
  margin: 0 0 8px;
}
.tiles {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 12px;
}
/* Phones: two tiles per row instead of one, halving the scroll to the charts. */
@media (max-width: 520px) {
  .tiles {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}
.split {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
  gap: 16px;
}
table {
  width: 100%;
  border-collapse: collapse;
  font-size: 13px;
}
th {
  text-align: left;
```

```
  font-weight: 500;
  color: var(--ink-muted);
  border-bottom: 1px solid var(--axis);
  padding: 6px 4px;
}
td {
  padding: 6px 4px;
  border-bottom: 1px solid var(--grid);
}
.num {
  text-align: right;
}
</style>
```

dashboard/app/components/FunnelChart.vue

111 lines · Playback funnel: how many impressions went on to each quartile. Bars share one hue because

```
<script setup lang="ts">
// Playback funnel: how many impressions went on to each quartile. Bars share one hue because
// they are one series (counts of a single measure), not categories needing identity.
const props = defineProps<{ byType: Record<string, number> }>()

const STEPS = [
  ['IMPRESSION', 'Impression'],
  ['FIRST_QUARTILE', '25%'],
  ['MIDPOINT', '50%'],
  ['THIRD_QUARTILE', '75%'],
  ['COMPLETE', 'Complete'],
  ['CLICK', 'Click'],
] as const

const hover = ref<string | null>(null)

const rows = computed(() => {
  const impressions = props.byType.IMPRESSION ?? 0
  const max = Math.max(1, ...STEPS.map(([k]) => props.byType[k] ?? 0))
  return STEPS.map(([key, label]) => {
    const count = props.byType[key] ?? 0
    return {
      key,
      label,
      count,
      width: (count / max) * 100,
      ofImpressions: impressions > 0 ? count / impressions : 0,
    }
  })
})
</script>

<template>
  <div class="funnel" role="table" aria-label="Accepted events by type">
    <div
      v-for="r in rows"
      :key="r.key"
      class="row"
      role="row"
      @pointerenter="hover = r.key"
      @pointerleave="hover = null"
    >
      <span class="name" role="rowheader">{{ r.label }}</span>
      <span class="track" role="cell">
        <span class="bar" :style="{ width: `${Math.max(r.width, r.count ? 0.5 : 0)}%` }" />
      </span>
      <span class="num" role="cell">
        {{ formatCompact(r.count) }}
        <span v-if="r.key !== 'IMPRESSION'" class="muted">· {{ formatPercent(r.ofImpressions, 1) }}</span>
      </span>
      <span v-if="hover === r.key" class="tip">
        {{ formatWhole(r.count) }} {{ r.label.toLowerCase() }} events
        <template v-if="r.key !== 'IMPRESSION'">({{ formatPercent(r.ofImpressions, 2) }} of impressions)</templa
  ↪ te>
      </span>
    </div>
  </div>
</template>

<style scoped>
.funnel {
  display: grid;
  gap: 8px;
}
.row {
  position: relative;
  display: grid;
  grid-template-columns: 88px 1fr 128px;
  align-items: center;
  gap: 10px;
  padding: 2px 0;
```

```
}
.name {
  color: var(--ink-2);
  font-size: 13px;
}
.track {
  height: 14px;
}
.bar {
  display: block;
  height: 100%;
  background: var(--series-1);
  border-radius: 0 4px 4px 0;
  transition: width 0.4s ease;
}
.row:hover .bar {
  filter: brightness(1.1);
}
.num {
  text-align: right;
  font-size: 13px;
}
.tip {
  position: absolute;
  right: 0;
  bottom: 100%;
  background: var(--tooltip-bg);
  color: var(--tooltip-ink);
  padding: 4px 8px;
  border-radius: 6px;
  font-size: 12px;
  pointer-events: none;
  white-space: nowrap;
  z-index: 1;
}
@media (max-width: 480px) {
  .row {
    grid-template-columns: 72px 1fr 104px;
  }
}
</style>
```

dashboard/app/components/LoginForm.vue

106 lines

```
<script setup lang="ts">
const session = useSession()
const username = ref('')
const password = ref('')
const error = ref('')
const busy = ref(false)

async function submit() {
  error.value = ''
  busy.value = true
  try {
    await session.signIn(username.value.trim(), password.value)
    password.value = ''
  } catch (err) {
    const code = session.statusOf(err)
    error.value =
      code === 401 ? 'Wrong user name or password.'
      : code === 429 ? 'Too many failed attempts. Try again in 15 minutes.'
      : code === 503 ? 'Sign-in is not configured on the server.'
      : 'Could not reach the server. Check your connection.'
  } finally {
    busy.value = false
  }
}
</script>

<template>
  <section class="card login" aria-labelledby="login-title">
    <BrandLogo class="logo" :height="96" />
    <h2 id="login-title">Sign in</h2>
    <p class="muted">The dashboard shows live delivery and spend data. Sign in to continue.</p>
    <form @submit.prevent="submit">
      <label>
        User name
        <input v-model="username" name="username" autocomplete="username" autocapitalize="none"
          spellcheck="false" required maxlength="64">
      </label>
      <label>
        Password
        <input v-model="password" name="password" type="password" autocomplete="current-password" required>
      </label>
      <p v-if="error" class="error" role="alert"><span aria-hidden="true">▲</span> {{ error }}</p>
      <button type="submit" :disabled="busy">{{ busy ? 'Signing in…' : 'Sign in' }}</button>
```

```
    </form>
  </section>
</template>

<style scoped>
.login {
  max-width: 380px;
  width: 100%;
  margin: 8vh auto 0;
}
.logo {
  display: block;
  margin: 4px auto 16px;
}
.login > p {
  margin: 0 0 16px;
}
form {
  display: grid;
  gap: 12px;
}
label {
  display: grid;
  gap: 4px;
  font-size: 13px;
  color: var(--ink-2);
}
input {
  font: inherit;
  /* 16px stops iOS Safari zooming into the field. */
  font-size: 16px;
  padding: 10px 12px;
  border: 1px solid var(--axis);
  border-radius: 8px;
  background: var(--page);
  color: var(--ink-1);
}
input:focus-visible,
button:focus-visible {
  outline: 2px solid var(--series-1);
  outline-offset: 2px;
}
button {
  font: inherit;
  font-weight: 600;
  padding: 10px 12px;
  min-height: 44px;
  border: 0;
  border-radius: 8px;
  background: var(--series-1);
  color: #fff;
  cursor: pointer;
}
button:disabled {
  opacity: 0.6;
  cursor: progress;
}
.error {
  margin: 0;
  color: var(--critical);
  font-size: 13px;
}
</style>
```

dashboard/app/components/PacingPanel.vue

271 lines

```
<script setup lang="ts">
type Status = 'ON_PACE' | 'AHEAD' | 'BEHIND' | 'UNDER_DELIVERING' | 'EXHAUSTED'
interface LineItem {
  lineItemId: string
  campaignId: string | null
  tier: 'GUARANTEED' | 'STANDARD' | null
  budgetMicros: number
  deliveredMicros: number
  targetMicros: number
  throttle: number
  status: Status
}
interface PacingStats {
  available: boolean
  error?: string
  periodStartMs: number
  periodEndMs: number
  elapsed: number
  lineItems: LineItem[]
}

```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
const { data, reachable } = usePolled<PacingStats>('/api/pacing', 5000)

// Status uses the reserved status colours, always with an icon and a label.
const STATUS: Record<Status, { label: string; icon: string; tone: string }> = {
  ON_PACE: { label: 'On pace', icon: '✓', tone: 'good' },
  AHEAD: { label: 'Ahead', icon: '▲', tone: 'warning' },
  BEHIND: { label: 'Behind', icon: '▼', tone: 'warning' },
  UNDER_DELIVERING: { label: 'Under-delivering', icon: '!', tone: 'critical' },
  EXHAUSTED: { label: 'Budget spent', icon: '■', tone: 'muted' },
}

const items = computed(() => data.value?.lineItems ?? [])
const counts = computed(() => {
  const c: Partial<Record<Status, number>> = {}
  for (const li of items.value) c[li.status] = (c[li.status] ?? 0) + 1
  return c
})
const totals = computed(() => {
  let budget = 0, delivered = 0, target = 0
  for (const li of items.value) {
    budget += li.budgetMicros
    delivered += li.deliveredMicros
    target += li.targetMicros
  }
  return { budget, delivered, target }
})
const rows = computed(() =>
  [...items.value]
    .sort((a, b) => (a.tier === b.tier ? b.budgetMicros - a.budgetMicros : a.tier === 'GUARANTEED' ? -1 : 1))
    .slice(0, 12),
)
const pct = (part: number, whole: number) => (whole > 0 ? Math.min(100, (part / whole) * 100) : 0)
</script>

<template>
  <section class="card">
    <h2>Budget pacing</h2>
    <p v-if="!reachable" class="muted">
      Needs the server routes from the Docker stack (<code>docker compose up</code>); not available on the stati
↪ c site.
    </p>
    <p v-else-if="data && !data.available" class="muted">The pacing service is not reachable yet. <code>{{ data.
↪ error }}</code></p>
    <p v-else-if="data && !items.length" class="muted">No line items yet: waiting for the ad server to publish b
↪ udgets.</p>
    <template v-else-if="data">
      <div class="tiles">
        <StatTile
          label="Period elapsed"
          :value="formatPercent(data.elapsed, 0)"
          :detail="`${formatClock(data.periodStartMs)}–${formatClock(data.periodEndMs)}`"
        />
        <StatTile
          label="Delivered / budget"
          :value="formatPercent(totals.delivered / (totals.budget || 1), 1)"
          :detail="`${formatUsd(totals.delivered)} of ${formatUsd(totals.budget)} · target ${formatPercent(total
↪ s.target / (totals.budget || 1), 0)}`"
        />
        <div class="card status-tile">
          <div class="label">Line items by status</div>
          <ul>
            <li v-for="(meta, key) in STATUS" :key="key" :class="meta.tone">
              <span class="icon" aria-hidden="true">{{ meta.icon }}</span>
              <span>{{ meta.label }}</span>
              <strong>{{ counts[key] ?? 0 }}</strong>
            </li>
          </ul>
        </div>
      </div>

      <div class="legend muted">
        <span><span class="swatch bar-swatch" aria-hidden="true" /> delivered</span>
        <span><span class="swatch target-swatch" aria-hidden="true" /> even-pacing target</span>
        <span>bar length = budget</span>
      </div>
      <div class="scroll">
        <table>
          <thead>
            <tr>
              <th>Line item</th>
              <th class="bullet-col">Delivered vs target</th>
              <th class="num">Budget</th>
              <th class="num">Throttle</th>
              <th>Status</th>
            </tr>
          </thead>
          <tbody>
            <tr v-for="li in rows" :key="li.lineItemId">
```

```
              <td>
                <code>{{ li.lineItemId }}</code>
                <span v-if="li.tier === 'GUARANTEED'" class="tier">guaranteed</span>
              </td>
              <td class="bullet-col">
                <div
                  class="bullet"
                  :title="`${formatUsd(li.deliveredMicros)} delivered, target ${formatUsd(li.targetMicros)}`"
                >
                  <span class="fill" :style="{ width: `${pct(li.deliveredMicros, li.budgetMicros)}%` }" />
                  <span class="target" :style="{ left: `${pct(li.targetMicros, li.budgetMicros)}%` }" />
                </div>
              </td>
              <td class="num">{{ formatUsd(li.budgetMicros) }}</td>
              <td class="num">{{ li.throttle >= 0.1 ? li.throttle.toFixed(2) : li.throttle.toFixed(3) }}</td>
              <td :class="['status', STATUS[li.status].tone]">
                <span class="icon" aria-hidden="true">{{ STATUS[li.status].icon }}</span>
                {{ STATUS[li.status].label }}
              </td>
            </tr>
          </tbody>
        </table>
      </div>
      <p v-if="items.length > rows.length" class="muted more">
        Showing {{ rows.length }} of {{ items.length }} line items: guaranteed first, then by budget.
      </p>
    </template>
  </section>
</template>

<style scoped>
p {
  margin: 0;
}
.tiles {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 12px;
  margin-bottom: 16px;
}
.tiles :deep(.card),
.status-tile {
  background: var(--page);
}
.status-tile .label {
  color: var(--ink-2);
  font-size: 13px;
  margin-bottom: 6px;
}
.status-tile ul {
  list-style: none;
  margin: 0;
  padding: 0;
  display: grid;
  gap: 2px;
  font-size: 13px;
}
.status-tile li {
  display: grid;
  grid-template-columns: 16px 1fr auto;
  gap: 6px;
  color: var(--ink-2);
}
.icon {
  font-weight: 700;
  text-align: center;
}
.good .icon {
  color: var(--good);
}
.warning .icon {
  color: var(--warning);
}
.critical .icon {
  color: var(--critical);
}
.muted .icon {
  color: var(--ink-muted);
}
.legend {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
  font-size: 12px;
  margin-bottom: 6px;
}
.swatch {
  display: inline-block;
```

```
  vertical-align: middle;
  margin-right: 4px;
}
.bar-swatch {
  width: 14px;
  height: 8px;
  background: var(--series-1);
  border-radius: 2px;
}
.target-swatch {
  width: 2px;
  height: 12px;
  background: var(--ink-1);
}
.scroll {
  overflow-x: auto;
}
table {
  width: 100%;
  min-width: 560px;
  border-collapse: collapse;
  font-size: 13px;
}
th {
  text-align: left;
  font-weight: 500;
  color: var(--ink-muted);
  border-bottom: 1px solid var(--axis);
  padding: 6px 8px;
  white-space: nowrap;
}
td {
  padding: 8px;
  border-bottom: 1px solid var(--grid);
  white-space: nowrap;
}
.num {
  text-align: right;
}
.tier {
  margin-left: 6px;
  font-size: 11px;
  color: var(--ink-2);
  border: 1px solid var(--border);
  border-radius: 4px;
  padding: 0 4px;
}
.bullet-col {
  width: 36%;
}
.bullet {
  position: relative;
  height: 12px;
  background: var(--grid);
  border-radius: 0 4px 4px 0;
}
.fill {
  position: absolute;
  inset: 0 auto 0 0;
  background: var(--series-1);
  border-radius: 0 4px 4px 0;
}
.target {
  position: absolute;
  top: -3px;
  bottom: -3px;
  width: 2px;
  margin-left: -1px;
  background: var(--ink-1);
}
.status {
  color: var(--ink-2);
}
.more {
  margin-top: 8px;
  font-size: 12px;
}
</style>
```

dashboard/app/components/PipelinePanel.vue

96 lines

```
<script setup lang="ts">
interface Pipeline {
  available: boolean
  error?: string
  job: { name: string; state: string; uptimeMs: number } | null
  checkpoints?: {
```

```
    completed: number
    failed: number
    restored: number
    lastCompletedAt: number | null
    lastDurationMs: number | null
    lastStateBytes: number | null
  }
  metrics?: {
    cleanPerSecond: number | null
    duplicatesDropped: number | null
    ivtFlagged: number | null
    metadataMissing: number | null
    lateDropped: number | null
  }
}

const { data, reachable } = usePolled<Pipeline>('/api/pipeline', 5000)
const now = ref(Date.now())
let clock: ReturnType<typeof setInterval> | undefined
onMounted(() => (clock = setInterval(() => (now.value = Date.now()), 1000)))
onBeforeUnmount(() => clearInterval(clock))

const running = computed(() => data.value?.job?.state === 'RUNNING')
const checkpointAge = computed(() => {
  const at = data.value?.checkpoints?.lastCompletedAt
  return at ? now.value - at : null
})
// Checkpoints run every 10s; one older than a minute means output is no longer being committed.
const checkpointStale = computed(() => checkpointAge.value !== null && checkpointAge.value > 60_000)
</script>

<template>
  <section class="card">
    <h2>Stream processing · Flink</h2>
    <p v-if="!reachable" class="muted">
      Needs the server routes from the Docker stack (<code>docker compose up</code>); not available on the stati
↪ c site.
    </p>
    <p v-else-if="data && !data.available" class="muted">Flink is not reachable yet. <code>{{ data.error }}</cod
↪ e></p>
    <p v-else-if="data && !data.job" class="muted">No job submitted yet.</p>
    <div v-else-if="data?.job" class="tiles">
      <StatTile
        label="Job"
        :value="data.job.state"
        :detail="`up ${formatDuration(data.job.uptimeMs)}`"
        :tone="running ? 'good' : 'critical'"
      />
      <StatTile
        label="Clean events/s"
        :value="formatWhole(data.metrics?.cleanPerSecond ?? 0)"
        detail="deduplicated, enriched, IVT-checked"
      />
      <StatTile
        label="Duplicates dropped"
        :value="formatCompact(data.metrics?.duplicatesDropped ?? 0)"
        detail="by event id, 30 min window"
      />
      <StatTile
        label="Invalid traffic flagged"
        :value="formatCompact(data.metrics?.ivtFlagged ?? 0)"
        detail="kept, not billed"
      />
      <StatTile
        label="Last checkpoint"
        :value="checkpointAge === null ? '—' : `${formatDuration(checkpointAge)} ago`"
        :detail="`${data.checkpoints?.completed ?? 0} ok · ${data.checkpoints?.failed ?? 0} failed · ${data.chec
↪ kpoints?.restored ?? 0} restores`"
        :tone="checkpointStale ? 'critical' : 'good'"
      />
    </div>
  </section>
</template>

<style scoped>
.tiles {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(170px, 1fr));
  gap: 12px;
}
/* Phones: two tiles per row instead of one, halving the scroll to the charts. */
@media (max-width: 520px) {
  .tiles {
    grid-template-columns: repeat(2, minmax(0, 1fr));
  }
}
.tiles :deep(.card) {
  background: var(--page);
```

```
}
p {
  margin: 0;
}
</style>
```

dashboard/app/components/StatTile.vue

50 lines

```
<script setup lang="ts">
defineProps<{
  label: string
  value: string
  detail?: string
  tone?: 'good' | 'critical'
}>()
</script>

<template>
  <div class="card tile">
    <div class="label">{{ label }}</div>
    <div class="value">{{ value }}</div>
    <div v-if="detail" class="detail" :class="tone">
      <span v-if="tone === 'good'" aria-hidden="true">✓</span>
      <span v-else-if="tone === 'critical'" aria-hidden="true">!</span>
      {{ detail }}
    </div>
  </div>
</template>

<style scoped>
.tile {
  display: flex;
  flex-direction: column;
  gap: 4px;
}
.label {
  color: var(--ink-2);
  font-size: 13px;
}
.value {
  font-size: clamp(22px, 6.5vw, 28px);
  overflow-wrap: anywhere;
  font-weight: 600;
  letter-spacing: -0.01em;
}
.detail {
  color: var(--ink-muted);
  font-size: 12px;
}
.detail.good span {
  color: var(--good);
  font-weight: 700;
}
.detail.critical span {
  color: var(--critical);
  font-weight: 700;
}
</style>
```

dashboard/app/components/ThroughputChart.vue

169 lines

```
<script setup lang="ts">
import type { RatePoint } from '~/composables/useGatewayStats'

const props = defineProps<{ points: RatePoint[] }>()

const H = 220
const M = { top: 12, right: 12, bottom: 24, left: 44 }
const innerH = H - M.top - M.bottom
const WINDOW_MS = 5 * 60 * 1000

// Draw at the container's real pixel width so text and strokes keep their size on any screen.
const wrap = ref<HTMLElement | null>(null)
const W = ref(640)
const innerW = computed(() => W.value - M.left - M.right)
let observer: ResizeObserver | undefined
onMounted(() => {
  observer = new ResizeObserver(([entry]) => {
    if (entry) W.value = Math.max(240, Math.round(entry.contentRect.width))
  })
  if (wrap.value) observer.observe(wrap.value)
})
onBeforeUnmount(() => observer?.disconnect())

const hover = ref<number | null>(null)

```

```
const domain = computed(() => {
  const end = props.points.at(-1)?.t ?? Date.now()
  return { start: end - WINDOW_MS, end }
})
const yTicks = computed(() => niceTicks(Math.max(1, ...props.points.map((p) => p.eps)) * 1.05))
const yMax = computed(() => yTicks.value.at(-1)!)

const x = (t: number) => M.left + ((t - domain.value.start) / WINDOW_MS) * innerW.value
const y = (v: number) => M.top + innerH - (v / yMax.value) * innerH

const path = computed(() =>
  props.points.map((p, i) => `${i === 0 ? 'M' : 'L'}${x(p.t).toFixed(1)},${y(p.eps).toFixed(1)}`).join(''),
)
const xTicks = computed(() => {
  // One tick per minute, or every other minute when narrow.
  const step = W.value < 420 ? 2 : 1
  const ticks = []
  for (let m = 5; m >= 0; m -= step) ticks.push(domain.value.end - m * 60_000)
  if (ticks.at(-1) !== domain.value.end) ticks.push(domain.value.end)
  return ticks
})

function onMove(evt: PointerEvent) {
  if (!props.points.length) return
  const svg = evt.currentTarget as SVGSVGElement
  const rect = svg.getBoundingClientRect()
  const px = evt.clientX - rect.left
  let best = 0
  for (let i = 1; i < props.points.length; i++) {
    if (Math.abs(x(props.points[i]!.t) - px) < Math.abs(x(props.points[best]!.t) - px)) best = i
  }
  hover.value = best
}

const hovered = computed(() => (hover.value === null ? null : props.points[hover.value] ?? null))
const tooltipLeft = computed(() => {
  if (!hovered.value) return '0px'
  // Keep the tooltip inside the card near either edge.
  return `${Math.min(Math.max(x(hovered.value.t), 70), W.value - 70)}px`
})
</script>

<template>
  <div ref="wrap" class="wrap">
    <svg
      :width="W"
      :height="H"
      :viewBox="`0 0 ${W} ${H}`"
      role="img"
      aria-label="Accepted events per second over the last five minutes"
      @pointermove="onMove"
      @pointerleave="hover = null"
    >
      <g v-for="v in yTicks" :key="v">
        <line :x1="M.left" :x2="W - M.right" :y1="y(v)" :y2="y(v)" :class="v === 0 ? 'baseline' : 'grid'" />
        <text :x="M.left - 8" :y="y(v)" class="tick" text-anchor="end" dominant-baseline="middle">
          {{ formatCompact(v) }}
        </text>
      </g>
      <text
        v-for="t in xTicks"
        :key="t"
        :x="x(t)"
        :y="H - 6"
        class="tick"
        :text-anchor="t === xTicks[0] ? 'start' : t === xTicks.at(-1) ? 'end' : 'middle'"
      >
        {{ t === xTicks.at(-1) ? 'now' : `-${Math.round((domain.end - t) / 60000)}m` }}
      </text>
      <path :d="path" class="line" />
      <g v-if="hovered">
        <line :x1="x(hovered.t)" :x2="x(hovered.t)" :y1="M.top" :y2="M.top + innerH" class="crosshair" />
        <circle :cx="x(hovered.t)" :cy="y(hovered.eps)" r="4" class="dot" />
      </g>
      <rect :x="M.left" :y="M.top" :width="innerW" :height="innerH" fill="transparent" />
    </svg>
    <div v-if="hovered" class="tooltip" :style="{ left: tooltipLeft }">
      <strong>{{ formatWhole(hovered.eps) }}</strong> events/s
      <div class="muted">{{ formatClock(hovered.t) }}</div>
    </div>
    <p v-if="points.length < 2" class="empty muted">Waiting for data…</p>
  </div>
</template>

<style scoped>
.wrap {
  position: relative;
}
```

```
 svg {
   display: block;
   touch-action: none;
 }
 .grid {
   stroke: var(--grid);
   stroke-width: 1;
 }
 .baseline {
   stroke: var(--axis);
   stroke-width: 1;
 }
 .tick {
   fill: var(--ink-muted);
   font-size: 11px;
 }
 .line {
   fill: none;
   stroke: var(--series-1);
   stroke-width: 2;
   stroke-linejoin: round;
   stroke-linecap: round;
 }
 .crosshair {
   stroke: var(--ink-muted);
   stroke-dasharray: 3 3;
 }
 .dot {
   fill: var(--series-1);
   stroke: var(--surface);
   stroke-width: 2;
 }
 .tooltip {
   position: absolute;
   top: 0;
   transform: translateX(-50%);
   background: var(--tooltip-bg);
   color: var(--tooltip-ink);
   padding: 6px 10px;
   border-radius: 6px;
   font-size: 12px;
   pointer-events: none;
   white-space: nowrap;
 }
 .tooltip .muted {
   color: inherit;
   opacity: 0.7;
 }
 .empty {
   position: absolute;
   inset: 0;
   display: grid;
   place-items: center;
   margin: 0;
 }
 </style>
```

dashboard/app/composables/useGatewayStats.ts

73 lines

```
export interface GatewaySnapshot {
  timestampMs: number
  uptimeSeconds: number
  accepted: number
  rejected: number
  batches: number
  publishFailures: number
  acceptedByType: Record<string, number>
  rejectedByReason: Record<string, number>
  latencyP50Ms: number
  latencyP99Ms: number
}

export interface RatePoint {
  t: number
  eps: number
}

const HISTORY_POINTS = 150 // 5 minutes at the default 2s poll

/**
 * Polls the gateway's counters and derives event rates from successive snapshots: through this
 * server's /api/gateway (Node builds), or from the gateway directly (static build, ?gateway=<url>).
 */
export function useGatewayStats() {
  const config = useRuntimeConfig().public
  const route = useRoute()
  const session = useSession()
```

```
  const direct = computed(() => Boolean(route.query.gateway) || config.gatewayMode === 'direct')
  const gatewayUrl = computed(() =>
    String(route.query.gateway || config.gatewayUrl).replace(/\/+$/, ''),
  )
  const statsUrl = computed(() => (direct.value ? `${gatewayUrl.value}/stats` : '/api/gateway'))

  const latest = ref<GatewaySnapshot | null>(null)
  const history = ref<RatePoint[]>([])
  const status = ref<'connecting' | 'up' | 'down'>('connecting')
  const lastError = ref('')
  let previous: GatewaySnapshot | null = null

  async function poll() {
    try {
      const snap = await $fetch<GatewaySnapshot>(statsUrl.value, { timeout: 2000 })
      if (previous) {
        const dt = (snap.timestampMs - previous.timestampMs) / 1000
        const delta = snap.accepted - previous.accepted
        // A negative delta means the gateway restarted and its counters reset: skip that point.
        if (dt > 0 && delta >= 0) {
          history.value = [...history.value, { t: snap.timestampMs, eps: delta / dt }].slice(-HISTORY_POINTS)
        }
      }
      previous = snap
      latest.value = snap
      status.value = 'up'
      lastError.value = ''
    } catch (err) {
      status.value = 'down'
      lastError.value = err instanceof Error ? err.message : String(err)
      session.noteError(err)
    }
  }

  useVisibleInterval(poll, Number(config.pollMs))

  const currentEps = computed(() => history.value.at(-1)?.eps ?? 0)
  const rejectionRate = computed(() => {
    const s = latest.value
    if (!s || s.accepted + s.rejected === 0) return 0
    return s.rejected / (s.accepted + s.rejected)
  })

  return { direct, gatewayUrl, latest, history, status, lastError, currentEps, rejectionRate }
}
```

dashboard/app/composables/useInstall.ts

50 lines

```
interface BeforeInstallPromptEvent extends Event {
  prompt(): Promise<void>
  userChoice: Promise<{ outcome: 'accepted' | 'dismissed' }>
}

/**
 * App installation. Chromium browsers (desktop, Android) offer a prompt we can trigger from a
 * button; iOS Safari has no prompt, so we explain Share → Add to Home Screen instead.
 */
export function useInstall() {
  const deferred = shallowRef<BeforeInstallPromptEvent | null>(null)
  const installed = ref(false)
  const iosHint = ref(false)

  function standalone() {
    return window.matchMedia('(display-mode: standalone)').matches
      || (navigator as Navigator & { standalone?: boolean }).standalone === true
  }

  function onPrompt(e: Event) {
    e.preventDefault()
    deferred.value = e as BeforeInstallPromptEvent
  }
  function onInstalled() {
    installed.value = true
    deferred.value = null
  }

  onMounted(() => {
    installed.value = standalone()
    iosHint.value = !installed.value && /iPhone|iPad|iPod/.test(navigator.userAgent)
    window.addEventListener('beforeinstallprompt', onPrompt)
    window.addEventListener('appinstalled', onInstalled)
  })
  onBeforeUnmount(() => {
    window.removeEventListener('beforeinstallprompt', onPrompt)
    window.removeEventListener('appinstalled', onInstalled)
  })

```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
  async function install() {
    const e = deferred.value
    if (!e) return
    deferred.value = null
    await e.prompt()
    if ((await e.userChoice).outcome === 'accepted') installed.value = true
  }

  const canPrompt = computed(() => deferred.value !== null && !installed.value)
  return { canPrompt, iosHint, installed, install }
}
```

dashboard/app/composables/useOnline.ts

15 lines

```
/** Whether the device reports a network connection. */
export function useOnline() {
  const online = ref(true)
  const update = () => (online.value = navigator.onLine)
  onMounted(() => {
    update()
    window.addEventListener('online', update)
    window.addEventListener('offline', update)
  })
  onBeforeUnmount(() => {
    window.removeEventListener('online', update)
    window.removeEventListener('offline', update)
  })
  return online
}
```

dashboard/app/composables/usePolled.ts

19 lines

```
/** Polls a same-origin server route. `data` keeps the last good response while a poll fails. */
export function usePolled<T>(path: string, everyMs: number) {
  const data = ref<T | null>(null)
  const reachable = ref(true)
  const session = useSession()

  useVisibleInterval(async () => {
    try {
      data.value = await $fetch<T>(path, { timeout: 5000 })
      reachable.value = true
    } catch (err) {
      // No server routes (the static GitHub Pages build), offline, or signed out.
      reachable.value = false
      session.noteError(err)
    }
  }, everyMs)

  return { data, reachable }
}
```

dashboard/app/composables/usePush.ts

94 lines

```
function base64UrlToBytes(value: string): Uint8Array<ArrayBuffer> {
  const padded = value.replace(/-/g, '+').replace(/_/g, '/') + '='.repeat((4 - (value.length % 4)) % 4)
  const raw = atob(padded)
  const bytes = new Uint8Array(new ArrayBuffer(raw.length))
  for (let i = 0; i < raw.length; i++) bytes[i] = raw.charCodeAt(i)
  return bytes
}

export type PushState =
  | 'checking'
  | 'unsupported' // this browser cannot receive push (e.g. iOS Safari outside a home-screen app)
  | 'unavailable' // the server has no VAPID keys
  | 'blocked' // the user denied notifications for this site
  | 'off'
  | 'on'

/** Alert notifications on this device: Web Push through the service worker. */
export function usePush() {
  const state = ref<PushState>('checking')
  const busy = ref(false)
  const error = ref('')
  let publicKey: string | null = null

  async function registration() {
    return navigator.serviceWorker.ready
  }

  async function refresh() {
    if (!('serviceWorker' in navigator) || !('PushManager' in window) || !('Notification' in window)) {
      state.value = 'unsupported'
      return
```

```
    }
    try {
      const key = await $fetch<{ enabled: boolean; publicKey: string | null }>('/api/push/key', { timeout: 5000 })
      publicKey = key.publicKey
      if (!key.enabled) {
        state.value = 'unavailable'
        return
      }
    } catch {
      state.value = 'unavailable'
      return
    }
    if (Notification.permission === 'denied') {
      state.value = 'blocked'
      return
    }
    const sub = await (await registration()).pushManager.getSubscription()
    state.value = sub ? 'on' : 'off'
  }

  async function enable() {
    if (!publicKey) return
    busy.value = true
    error.value = ''
    try {
      if ((await Notification.requestPermission()) !== 'granted') {
        state.value = Notification.permission === 'denied' ? 'blocked' : 'off'
        return
      }
      const reg = await registration()
      // The browser registers with its vendor's push service first; if that is unreachable the
      // promise can hang for minutes. Say so instead of waiting silently.
      const sub = await Promise.race([
        reg.pushManager.subscribe({ userVisibleOnly: true, applicationServerKey: base64UrlToBytes(publicKey) }),
        new Promise<never>((_, reject) =>
          setTimeout(() => reject(new Error("Couldn't reach the browser's push service. Try again later.")), 20_000)),
      ])
      await $fetch('/api/push/subscribe', { method: 'POST', body: sub.toJSON() })
      state.value = 'on'
    } catch (err) {
      error.value = err instanceof Error ? err.message : String(err)
    } finally {
      busy.value = false
    }
  }

  async function disable() {
    busy.value = true
    try {
      const sub = await (await registration()).pushManager.getSubscription()
      if (sub) {
        await $fetch('/api/push/unsubscribe', { method: 'POST', body: { endpoint: sub.endpoint } }).catch(() => {})
        await sub.unsubscribe()
      }
      state.value = 'off'
    } finally {
      busy.value = false
    }
  }

  onMounted(refresh)
  return { state, busy, error, enable, disable }
}
```

dashboard/app/composables/useSession.ts

56 lines

```
export type SessionStatus =
  | 'checking'
  | 'signed-in' // authenticated
  | 'open' // the server runs with authentication disabled
  | 'signed-out' // the server wants a login
  | 'misconfigured' // authentication is on but the server cannot check logins
  | 'no-server' // static build (GitHub Pages) or server unreachable: panels explain themselves

interface Me {
  user: string | null
  authDisabled: boolean
}

function statusOf(err: unknown): number | undefined {
  return (err as { statusCode?: number; response?: { status?: number } })?.statusCode
    ?? (err as { response?: { status?: number } })?.response?.status
}

```

```
/** Shared sign-in state. Any API call that answers 401 sends the app back to the sign-in screen. */
export function useSession() {
  const status = useState<SessionStatus>('session-status', () => 'checking')
  const user = useState<string | null>('session-user', () => null)

  async function check() {
    try {
      const me = await $fetch<Me>('/api/auth/me', { timeout: 5000 })
      user.value = me.user
      status.value = me.authDisabled ? 'open' : 'signed-in'
    } catch (err) {
      const code = statusOf(err)
      status.value = code === 401 ? 'signed-out' : code === 503 ? 'misconfigured' : 'no-server'
    }
  }

  async function signIn(username: string, password: string) {
    const me = await $fetch<Me>('/api/auth/login', { method: 'POST', body: { username, password } })
    user.value = me.user
    status.value = me.authDisabled ? 'open' : 'signed-in'
  }

  async function signOut() {
    await $fetch('/api/auth/logout', { method: 'POST' }).catch(() => {})
    user.value = null
    status.value = 'signed-out'
  }

  /** Called by pollers on any failed request. */
  function noteError(err: unknown) {
    if (statusOf(err) === 401 && status.value === 'signed-in') {
      user.value = null
      status.value = 'signed-out'
    }
  }

  return { status, user, check, signIn, signOut, noteError, statusOf }
}
```

dashboard/app/composables/useVisibleInterval.ts

36 lines

```
/**
 * Runs `fn` now and every `everyMs` while the page is visible and the device is online. Hidden
 * tabs, backgrounded phone apps and offline devices stop polling (saving battery and data) and
 * catch up as soon as they are shown again or reconnect.
 */
export function useVisibleInterval(task: () => void, everyMs: number) {
  let timer: ReturnType<typeof setInterval> | undefined
  const fn = () => {
    if (navigator.onLine) task()
  }

  function start() {
    stop()
    fn()
    timer = setInterval(fn, everyMs)
  }
  function stop() {
    clearInterval(timer)
    timer = undefined
  }
  function onVisibility() {
    if (document.visibilityState === 'visible') start()
    else stop()
  }

  onMounted(() => {
    if (document.visibilityState === 'visible') start()
    document.addEventListener('visibilitychange', onVisibility)
    window.addEventListener('online', onVisibility)
  })
  onBeforeUnmount(() => {
    stop()
    document.removeEventListener('visibilitychange', onVisibility)
    window.removeEventListener('online', onVisibility)
  })
}
```

dashboard/app/plugins/pwa.client.ts

11 lines · Registers the service worker (public/sw.js) for production builds, under the app's base URL so

```
// Registers the service worker (public/sw.js) for production builds, under the app's base URL so
// the same build works at / (Docker, hosted) and /primetech/ (GitHub Pages).
export default defineNuxtPlugin(() => {
  if (import.meta.dev || !('serviceWorker' in navigator) || !window.isSecureContext) return
  const base = useRuntimeConfig().app.baseURL
  window.addEventListener('load', () => {
```

```
    navigator.serviceWorker.register(`${base}sw.js`, { scope: base }).catch((err) => {
      console.warn('service worker registration failed', err)
    })
  })
})
```

dashboard/app/utils/format.ts

43 lines

```
const compact = new Intl.NumberFormat('en-US', { notation: 'compact', maximumFractionDigits: 1 })
const whole = new Intl.NumberFormat('en-US', { maximumFractionDigits: 0 })

export function formatCompact(n: number): string {
  return compact.format(n)
}

export function formatWhole(n: number): string {
  return whole.format(n)
}

export function formatPercent(fraction: number, digits = 2): string {
  return `${(fraction * 100).toFixed(digits)}%`
}

export function formatClock(ms: number): string {
  return new Date(ms).toLocaleTimeString('en-US', { hour12: false })
}

/** Evenly spaced axis ticks from 0 with a "nice" step (1, 2, 2.5 or 5 times a power of ten). */
export function niceTicks(max: number, count = 4): number[] {
  const raw = Math.max(max, 1) / count
  const pow = 10 ** Math.floor(Math.log10(raw))
  const step = ([1, 2, 2.5, 5, 10].find((s) => s * pow >= raw) ?? 10) * pow
  const ticks: number[] = []
  for (let v = 0; v <= max + step * 0.999; v += step) ticks.push(v)
  return ticks.length > 1 ? ticks : [0, step]
}

export function formatDuration(ms: number): string {
  const s = Math.floor(ms / 1000)
  if (s < 60) return `${s}s`
  const m = Math.floor(s / 60)
  if (m < 60) return `${m}m ${s % 60}s`
  const h = Math.floor(m / 60)
  return `${h}h ${m % 60}m`
}

export function formatUsd(micros: number): string {
  return new Intl.NumberFormat('en-US', { style: 'currency', currency: 'USD', maximumFractionDigits: 0 }).format
↳ (
    micros / 1_000_000,
  )
}
```

dashboard/nuxt.config.ts

68 lines · A client-side SPA: the same build is served by Node (Docker, or hosted behind Caddy) or as static

```
// A client-side SPA: the same build is served by Node (Docker, or hosted behind Caddy) or as static
// files (GitHub Pages). Under Node, the server routes in server/api proxy the gateway, Flink, Druid,
// the ad server and the pacing service, behind a login unless NUXT_AUTH_DISABLED=true. On Pages
// those panels say they need the Docker stack.
const base = process.env.NUXT_APP_BASE_URL || '/'

export default defineNuxtConfig({
  compatibilityDate: '2026-07-01',
  ssr: false,
  devtools: { enabled: false },
  css: ['~/assets/main.css'],
  app: {
    head: {
      title: 'Ads Eventing',
      meta: [
        // viewport-fit=cover lets the installed app draw under phone notches; main.css pads with safe-area inse
↳ ts.
        { name: 'viewport', content: 'width=device-width, initial-scale=1, viewport-fit=cover' },
        { name: 'description', content: 'Live ads eventing pipeline: ad serving, pacing, stream processing and d
↳ elivery. By Quila Visuals.' },
        { name: 'author', content: 'Quila Visuals' },
        { name: 'theme-color', content: '#f9f9f7', media: '(prefers-color-scheme: light)' },
        { name: 'theme-color', content: '#0d0d0d', media: '(prefers-color-scheme: dark)' },
        { name: 'apple-mobile-web-app-capable', content: 'yes' },
        { name: 'mobile-web-app-capable', content: 'yes' },
        { name: 'apple-mobile-web-app-status-bar-style', content: 'default' },
        { name: 'apple-mobile-web-app-title', content: 'Ads Eventing' },
      ],
      link: [
        { rel: 'manifest', href: `${base}manifest.webmanifest` },
        { rel: 'icon', type: 'image/png', sizes: '32x32', href: `${base}icons/favicon-32.png` },
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
        { rel: 'apple-touch-icon', href: `${base}icons/apple-touch-icon.png` },
      ],
    },
  },
  runtimeConfig: {
    // Server-only. Overridden with NUXT_GATEWAY_URL, NUXT_FLINK_URL, NUXT_DRUID_URL,
    // NUXT_AD_SERVER_URL, NUXT_PACING_URL.
    gatewayUrl: 'http://localhost:8080',
    flinkUrl: 'http://localhost:8082',
    druidUrl: 'http://localhost:8888',
    adServerUrl: 'http://localhost:8090',
    pacingUrl: 'http://localhost:8095',
    auditUrl: 'http://localhost:8096', // NUXT_AUDIT_URL
    alertmanagerUrl: 'http://localhost:9093', // NUXT_ALERTMANAGER_URL
    // Alertmanager's webhook (/api/alerts/webhook) must send this as a bearer token. NUXT_ALERT_WEBHOOK_TOKEN
    alertWebhookToken: '',
    // Web Push for alerts on installed apps. Generate keys with `npx web-push generate-vapid-keys`.
    vapidPublicKey: '', // NUXT_VAPID_PUBLIC_KEY
    vapidPrivateKey: '', // NUXT_VAPID_PRIVATE_KEY
    vapidSubject: '', // NUXT_VAPID_SUBJECT, e.g. mailto:ops@example.com
    pushStorePath: '.data/push-subscriptions.json', // NUXT_PUSH_STORE_PATH
    pushAllowedHosts: '', // NUXT_PUSH_ALLOWED_HOSTS: extra push service hosts (tests)
    // Authentication (see README "Hosting"). On unless NUXT_AUTH_DISABLED=true; if on and
    // misconfigured, the API answers 503 instead of serving data.
    authUsers: '', // NUXT_AUTH_USERS: user=scrypt-hash,... from `npm run hash-password -- <user>`
    sessionSecret: '', // NUXT_SESSION_SECRET: at least 32 random characters
    sessionTtlHours: 12, // NUXT_SESSION_TTL_HOURS
    authDisabled: false, // NUXT_AUTH_DISABLED
    trustProxy: false, // NUXT_TRUST_PROXY: true behind Caddy, to read the client address and scheme
    public: {
      // "proxy": the browser reads gateway stats through /api/gateway (Node builds).
      // "direct": it calls the gateway itself (the static Pages build, or ?gateway=<url>).
      gatewayMode: 'proxy', // NUXT_PUBLIC_GATEWAY_MODE
      gatewayUrl: 'http://localhost:8080', // NUXT_PUBLIC_GATEWAY_URL, used in direct mode
      pollMs: 2000,
    },
  },
  typescript: { strict: true },
})
```

dashboard/package.json

27 lines

```
{
  "name": "ads-eventing-dashboard",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "nuxt dev",
    "build": "nuxt build",
    "generate": "nuxt generate",
    "preview": "nuxt preview",
    "typecheck": "nuxt typecheck",
    "postinstall": "nuxt prepare",
    "test": "node --test --experimental-strip-types --no-warnings 'test/*.test.ts'",
    "hash-password": "node --experimental-strip-types --no-warnings scripts/hash-password.ts"
  },
  "dependencies": {
    "nuxt": "4.6.1",
    "vue": "3.5.44",
    "vue-router": "5.4.0",
    "web-push": "3.6.7"
  },
  "devDependencies": {
    "@types/node": "22.20.5",
    "@types/web-push": "3.6.4",
    "typescript": "5.9.3",
    "vue-tsc": "3.3.13"
  }
}
```

dashboard/public/manifest.webmanifest

18 lines

```
{
  "name": "Ads Eventing",
  "short_name": "Ads Eventing",
  "description": "Live ads eventing pipeline: ad serving, pacing, stream processing and delivery. By Quila Visua
↳ ls.",
  "id": "./",
  "start_url": "./",
  "scope": "./",
  "display": "standalone",
  "orientation": "any",
  "background_color": "#f9f9f7",
  "theme_color": "#11141e",
  "categories": ["business", "productivity"],
  "icons": [
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
    { "src": "icons/icon-192.png", "sizes": "192x192", "type": "image/png", "purpose": "any" },
    { "src": "icons/icon-512.png", "sizes": "512x512", "type": "image/png", "purpose": "any" },
    { "src": "icons/icon-maskable-512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable" }
  ]
}
```

dashboard/public/sw.js

119 lines · Service worker: makes the dashboard installable and lets the app shell open offline.

```
// Service worker: makes the dashboard installable and lets the app shell open offline.
//
// What is cached, and how:
//   pages (navigations)         network first, cached copy when offline
//   /_nuxt/* build assets       cache first: file names are content-hashed, so never stale
//   icons, logo, manifest       served from cache, refreshed in the background
//   /api/*                      NEVER cached: live, signed-in data must not outlive the session
//                               or be left on a shared device
const VERSION = 'ae-v3'
const SHELL = `${VERSION}-shell`
const ASSETS = `${VERSION}-assets`
const MAX_ASSETS = 80
const scope = new URL(self.registration.scope)
const PRECACHE = ['./', 'manifest.webmanifest', 'icons/icon-192.png', 'icons/icon-512.png', 'brand-logo.png'].map(
  (p) => new URL(p, scope).href,
)

self.addEventListener('install', (event) => {
  event.waitUntil(caches.open(SHELL).then((c) => c.addAll(PRECACHE)).then(() => self.skipWaiting()))
})

self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches
      .keys()
      .then((keys) => Promise.all(keys.filter((k) => !k.startsWith(VERSION)).map((k) => caches.delete(k))))
      .then(() => self.clients.claim()),
  )
})

async function trim(cacheName, max) {
  const cache = await caches.open(cacheName)
  const keys = await cache.keys()
  await Promise.all(keys.slice(0, Math.max(0, keys.length - max)).map((k) => cache.delete(k)))
}

self.addEventListener('fetch', (event) => {
  const request = event.request
  const url = new URL(request.url)
  if (request.method !== 'GET' || url.origin !== scope.origin || url.pathname.includes('/api/')) return

  if (request.mode === 'navigate') {
    event.respondWith(
      fetch(request)
        .then((response) => {
          if (response.ok) {
            const copy = response.clone()
            caches.open(SHELL).then((c) => c.put(scope.href, copy))
          }
          return response
        })
        .catch(() => caches.match(scope.href, { cacheName: SHELL }).then((r) => r || Response.error())),
    )
    return
  }

  if (url.pathname.includes('/_nuxt/')) {
    event.respondWith(
      caches.match(request, { cacheName: ASSETS }).then(
        (cached) =>
          cached ||
          fetch(request).then((response) => {
            if (response.ok) {
              const copy = response.clone()
              caches.open(ASSETS).then((c) => c.put(request, copy)).then(() => trim(ASSETS, MAX_ASSETS))
            }
            return response
          }),
      ),
    )
    return
  }

  if (PRECACHE.includes(url.href)) {
    event.respondWith(
      caches.match(request, { cacheName: SHELL }).then((cached) => {
        const refresh = fetch(request).then((response) => {
          if (response.ok) caches.open(SHELL).then((c) => c.put(request, response.clone()))
```

```
          return response
        })
        return cached || refresh
      }),
    )
  }
})

// Alert notifications (Web Push). The server sends {title, body, tag, url}; a notification with the
// same tag replaces the previous one, so "Resolved" replaces its own "Critical" on the lock screen.
self.addEventListener('push', (event) => {
  let data = {}
  try {
    data = event.data ? event.data.json() : {}
  } catch {
    data = { title: 'Ads Eventing', body: event.data ? event.data.text() : '' }
  }
  event.waitUntil(
    self.registration.showNotification(data.title || 'Ads Eventing alert', {
      body: data.body || '',
      tag: data.tag || 'ads-eventing',
      renotify: true,
      icon: new URL('icons/icon-192.png', scope).href,
      badge: new URL('icons/icon-192.png', scope).href,
      data: { url: new URL((data.url || './').replace(/^\//, ''), scope).href },
    }),
  )
})

// Tapping a notification opens the app (or focuses it) at the alerts panel.
self.addEventListener('notificationclick', (event) => {
  event.notification.close()
  const target = event.notification.data?.url || scope.href
  event.waitUntil(
    self.clients.matchAll({ type: 'window', includeUncontrolled: true }).then((windows) => {
      const open = windows.find((w) => w.url.startsWith(scope.href))
      if (open) return open.focus().then(() => open.navigate(target))
      return self.clients.openWindow(target)
    }),
  )
})
```

dashboard/scripts/hash-password.ts

49 lines · Prints a user entry for NUXT_AUTH_USERS. Usage: npm run hash-password -- <user>

```
// Prints a user entry for NUXT_AUTH_USERS. Usage: npm run hash-password -- <user>
// The password is read from stdin, never argv (other processes can read argv). On a terminal
// it is not echoed.
import { createInterface } from 'node:readline'
import { hashPassword } from '../server/utils/auth.ts'

const user = process.argv[2]
if (!user || !/^[A-Za-z0-9._-]{1,64}$/.test(user)) {
  console.error('usage: npm run hash-password -- <user>   (letters, digits, . _ -)')
  process.exit(1)
}

function done(password: string) {
  if (password.length < 12) {
    console.error('Use at least 12 characters.')
    process.exit(1)
  }
  console.log(`${user}=${hashPassword(password)}`)
  process.exit(0)
}

if (process.stdin.isTTY) {
  process.stderr.write(`Password for ${user}: `)
  process.stdin.setRawMode(true)
  process.stdin.setEncoding('utf8')
  let password = ''
  process.stdin.on('data', (chunk: string) => {
    for (const ch of chunk) {
      if (ch === '\r' || ch === '\n') {
        process.stdin.setRawMode(false)
        process.stderr.write('\n')
        done(password)
      } else if (ch === '\u0003') {
        process.stdin.setRawMode(false)
        process.exit(130)
      } else if (ch === '\u007f' || ch === '\b') {
        password = password.slice(0, -1)
      } else {
        password += ch
      }
    }
  })
} else {
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
  const rl = createInterface({ input: process.stdin })
  rl.once('line', (line) => {
    rl.close()
    done(line)
  })
}
```

dashboard/server/api/adserver.get.ts

11 lines · Ad server counters (auctions, fill, latency, per-DSP outcomes), proxied so the browser needs no

```
// Ad server counters (auctions, fill, latency, per-DSP outcomes), proxied so the browser needs no
// direct access to the ad server.
export default defineEventHandler(async () => {
  const url = useRuntimeConfig().adServerUrl.replace(/\/+$/, '')
  try {
    const stats = await $fetch<Record<string, unknown>>(`${url}/stats`, { timeout: 2000 })
    return { available: true, ...stats }
  } catch (err) {
    return { available: false, error: err instanceof Error ? err.message : String(err) }
  }
})
```

dashboard/server/api/alerts.get.ts

35 lines · Active alerts from Alertmanager (silenced and inhibited ones left out), plus recent notifications.

```
// Active alerts from Alertmanager (silenced and inhibited ones left out), plus recent notifications.
interface AmAlert {
  labels: Record<string, string>
  annotations: Record<string, string>
  startsAt: string
  fingerprint: string
}

export default defineEventHandler(async () => {
  const url = useRuntimeConfig().alertmanagerUrl.replace(/\/+$/, '')
  try {
    const alerts = await $fetch<AmAlert[]>(`${url}/api/v2/alerts`, {
      query: { active: true, silenced: false, inhibited: false },
      timeout: 3000,
    })
    return {
      available: true,
      push: { enabled: pushConfig() !== null, devices: await subscriptionCount() },
      active: alerts
        .map((a) => ({
          id: a.fingerprint,
          alertname: a.labels.alertname,
          severity: a.labels.severity ?? 'warning',
          check: a.labels.check ?? null,
          summary: a.annotations.summary ?? a.labels.alertname,
          description: a.annotations.description ?? '',
          startsAt: a.startsAt,
        }))
        .sort((a, b) => (a.severity === b.severity ? a.startsAt.localeCompare(b.startsAt) : a.severity === 'crit
↪ ical' ? -1 : 1)),
      recent: recentNotifications(),
    }
  } catch (err) {
    return { available: false, error: err instanceof Error ? err.message : String(err) }
  }
})
```

dashboard/server/api/alerts/webhook.post.ts

53 lines · Alertmanager's webhook: every firing or resolved alert becomes a push notification on each

```
// Alertmanager's webhook: every firing or resolved alert becomes a push notification on each
// device that turned notifications on, and joins the recent-notifications list.
// Authenticated by a shared bearer token, not a session; Caddy also blocks this path from outside.
import { timingSafeEqual } from 'node:crypto'

interface AmAlert {
  status: 'firing' | 'resolved'
  labels: Record<string, string>
  annotations: Record<string, string>
  startsAt: string
  endsAt: string
  fingerprint: string
}

function tokenMatches(given: string, expected: string) {
  const a = Buffer.from(given)
  const b = Buffer.from(expected)
  return a.length === b.length && timingSafeEqual(a, b)
}

export default defineEventHandler(async (event) => {
  const expected = String(useRuntimeConfig().alertWebhookToken || '')
  if (expected.length < 16) throw createError({ statusCode: 503, statusMessage: 'Alert webhook not configured' }
```

```
  ↪ )
 24    const given = (getRequestHeader(event, 'authorization') || '').replace(/^Bearer\s+/i, '')
 25    if (!tokenMatches(given, expected)) throw createError({ statusCode: 401, statusMessage: 'Bad token' })
 26
 27    if (Number(getRequestHeader(event, 'content-length') || 0) > 512 * 1024) {
 28      throw createError({ statusCode: 413, statusMessage: 'Request too large' })
 29    }
 30    const body = await readBody<{ alerts?: AmAlert[] }>(event)
 31    const alerts = Array.isArray(body?.alerts) ? body.alerts.slice(0, 50) : []
 32    let sent = 0
 33    for (const a of alerts) {
 34      const firing = a.status === 'firing'
 35      const name = a.labels.alertname ?? 'Alert'
 36      const summary = a.annotations.summary ?? name
 37      rememberNotification({
 38        status: a.status,
 39        alertname: name,
 40        severity: a.labels.severity ?? 'warning',
 41        summary,
 42        atMs: Date.now(),
 43      })
 44      const result = await pushToAll({
 45        title: firing ? `${a.labels.severity === 'critical' ? 'Critical' : 'Warning'}: ${summary}` : `Resolved: ${
  ↪ summary}`,
 46        body: (firing ? a.annotations.description ?? '' : `${name} is no longer firing.`).slice(0, 300),
 47        tag: a.fingerprint || name,
 48        url: '/#alerts',
 49      })
 50      sent += result.sent
 51    }
 52    return { received: alerts.length, pushed: sent }
 53  })
```

dashboard/server/api/audit.get.ts

9 lines · Reconciliation results, proxied from the audit service.

```
 1  // Reconciliation results, proxied from the audit service.
 2  export default defineEventHandler(async () => {
 3    const url = useRuntimeConfig().auditUrl.replace(/\/+$/, '')
 4    try {
 5      return { available: true, ...(await $fetch<Record<string, unknown>>(`${url}/stats`, { timeout: 3000 })) }
 6    } catch (err) {
 7      return { available: false, error: err instanceof Error ? err.message : String(err) }
 8    }
 9  })
```

dashboard/server/api/auth/login.post.ts

60 lines · Exchanges a user name and password for a session cookie (HttpOnly, SameSite=Strict, Secure on HTTPS).

```
 1  // Exchanges a user name and password for a session cookie (HttpOnly, SameSite=Strict, Secure on HTTPS).
 2  // Failed attempts are limited per client address and per user name; unknown users cost the same
 3  // time as known ones, so the response does not reveal which names exist.
 4  const byAddress = new RateLimiter(20, 15 * 60_000)
 5  const byUser = new RateLimiter(10, 15 * 60_000)
 6  let dummyHash: string | null = null
 7
 8  function originHost(origin: string): string | null {
 9    try {
10      return new URL(origin).host
11    } catch {
12      return null // e.g. "Origin: null" from a sandboxed frame
13    }
14  }
15
16  export default defineEventHandler(async (event) => {
17    const origin = getRequestHeader(event, 'origin')
18    if (origin && originHost(origin) !== getRequestHost(event, { xForwardedHost: authState().trustProxy })) {
19      throw createError({ statusCode: 403, statusMessage: 'Cross-origin login refused' })
20    }
21    // JSON only: a cross-site HTML form cannot send it without a CORS preflight, which is refused.
22    if (!getRequestHeader(event, 'content-type')?.startsWith('application/json')) {
23      throw createError({ statusCode: 415, statusMessage: 'Expected application/json' })
24    }
25    if (Number(getRequestHeader(event, 'content-length') || 0) > 4096) {
26      throw createError({ statusCode: 413, statusMessage: 'Request too large' })
27    }
28
29    const auth = authState()
30    if (auth.disabled) return { user: null, authDisabled: true }
31
32    const body = await readBody<{ username?: unknown; password?: unknown }>(event).catch(() => null)
33    const username = typeof body?.username === 'string' ? body.username.slice(0, 64) : ''
34    const password = typeof body?.password === 'string' ? body.password.slice(0, 1024) : ''
35    const address = clientAddress(event)
36    if (!byAddress.allowed(address) || !byUser.allowed(username)) {
37      setResponseHeader(event, 'Retry-After', 900)
38      throw createError({ statusCode: 429, statusMessage: 'Too many failed attempts, try again later' })
39    }
```

```

  const hash = auth.users.get(username)
  dummyHash ??= hashPassword('not-a-real-password')
  const ok = verifyPassword(password, hash ?? dummyHash) && hash !== undefined
  if (!ok) {
    byAddress.record(address)
    byUser.record(username)
    console.warn(`[auth] failed login from ${address}`)
    throw createError({ statusCode: 401, statusMessage: 'Wrong user name or password' })
  }

  byUser.reset(username)
  setCookie(event, SESSION_COOKIE, createSession(username, hash, auth.secret, auth.ttlMs), {
    httpOnly: true,
    sameSite: 'strict',
    secure: isHttps(event),
    path: '/',
    maxAge: Math.floor(auth.ttlMs / 1000),
  })
  return { user: username, authDisabled: false }
})
```

dashboard/server/api/auth/logout.post.ts

4 lines

```
export default defineEventHandler((event) => {
  deleteCookie(event, SESSION_COOKIE, { path: '/' })
  return { user: null }
})
```

dashboard/server/api/auth/me.get.ts

5 lines · Who is signed in. Reaching this handler means the middleware accepted the session (or auth is off).

```
// Who is signed in. Reaching this handler means the middleware accepted the session (or auth is off).
export default defineEventHandler((event) => ({
  user: (event.context.user as string | undefined) ?? null,
  authDisabled: authState().disabled,
}))
```

dashboard/server/api/campaigns.get.ts

52 lines · Campaign delivery from Druid. Billable numbers exclude invalid traffic (ivt = 1).

```
// Campaign delivery from Druid. Billable numbers exclude invalid traffic (ivt = 1).
interface Row {
  advertiserName: string | null
  campaignName: string | null
  campaignId: string
  impressions: number | null
  completes: number | null
  spendMicros: number | null
  ivtEvents: number | null
  events: number
}

export default defineEventHandler(async (event) => {
  const druid = useRuntimeConfig().druidUrl.replace(/\/+$/, '')
  // Whole minutes only, clamped, so the value is safe to inline into SQL.
  const minutes = Math.min(1440, Math.max(5, Math.trunc(Number(getQuery(event).minutes) || 60)))
  const query = `
    SELECT advertiserName, campaignName, campaignId,
      SUM(events) FILTER (WHERE type = 'IMPRESSION' AND ivt = 0) AS impressions,
      SUM(events) FILTER (WHERE type = 'COMPLETE' AND ivt = 0) AS completes,
      SUM(priceMicros) FILTER (WHERE type = 'IMPRESSION' AND ivt = 0) AS spendMicros,
      SUM(events) FILTER (WHERE ivt = 1) AS ivtEvents,
      SUM(events) AS events
    FROM ad_events
    WHERE __time >= CURRENT_TIMESTAMP - INTERVAL '${minutes}' MINUTE
    GROUP BY 1, 2, 3
    ORDER BY spendMicros DESC
    LIMIT 15`
  try {
    const rows = await $fetch<Row[]>(`${druid}/druid/v2/sql`, {
      method: 'POST',
      body: { query },
      timeout: 4000,
    })
    return {
      available: true,
      minutes,
      rows: rows.map((r) => ({
        advertiserName: r.advertiserName ?? 'Unknown advertiser',
        campaignName: r.campaignName ?? r.campaignId,
        campaignId: r.campaignId,
        impressions: r.impressions ?? 0,
        completes: r.completes ?? 0,
        spendMicros: r.spendMicros ?? 0,
        ivtEvents: r.ivtEvents ?? 0,
        events: r.events,
```

```
      })),
    }
  } catch (err) {
    return { available: false, minutes, error: err instanceof Error ? err.message : String(err) }
  }
})
```

dashboard/server/api/gateway.get.ts

9 lines · Ingest gateway counters, proxied so the browser only ever talks to this origin.

```
// Ingest gateway counters, proxied so the browser only ever talks to this origin.
export default defineEventHandler(async () => {
  const url = useRuntimeConfig().gatewayUrl.replace(/\/+$/, '')
  try {
    return await $fetch<Record<string, unknown>>(`${url}/stats`, { timeout: 1500 })
  } catch {
    throw createError({ statusCode: 502, statusMessage: 'Gateway unreachable' })
  }
})
```

dashboard/server/api/health.get.ts

2 lines · Liveness for load balancers and uptime checks. Unauthenticated, so it reveals nothing else.

```
// Liveness for load balancers and uptime checks. Unauthenticated, so it reveals nothing else.
export default defineEventHandler(() => ({ ok: true }))
```

dashboard/server/api/pacing.get.ts

10 lines · Budget pacing state per line item, proxied from the pacing service.

```
// Budget pacing state per line item, proxied from the pacing service.
export default defineEventHandler(async () => {
  const url = useRuntimeConfig().pacingUrl.replace(/\/+$/, '')
  try {
    const stats = await $fetch<Record<string, unknown>>(`${url}/stats`, { timeout: 2000 })
    return { available: true, ...stats }
  } catch (err) {
    return { available: false, error: err instanceof Error ? err.message : String(err) }
  }
})
```

dashboard/server/api/pipeline.get.ts

59 lines · Stream-processor health from the Flink REST API. Runs server-side, so the browser needs no

```
// Stream-processor health from the Flink REST API. Runs server-side, so the browser needs no
// access (or CORS) to Flink.
interface JobOverview { jobs: { jid: string; name: string; state: string; 'start-time': number; duration: number
 ↪ }[] }
interface JobDetail { vertices: { id: string; name: string }[] }
interface Checkpoints {
  counts: { completed: number; failed: number; restored: number }
  latest: { completed: { id: number; latest_ack_timestamp: number; end_to_end_duration: number; state_size: numb
 ↪ er } | null }
}
type MetricValues = { id: string; sum: number }[]

export default defineEventHandler(async () => {
  const flink = useRuntimeConfig().flinkUrl.replace(/\/+$/, '')
  try {
    const overview = await $fetch<JobOverview>(`${flink}/jobs/overview`, { timeout: 2000 })
    const job = overview.jobs.find((j) => j.state === 'RUNNING') ?? overview.jobs[0]
    if (!job) return { available: true, job: null }

    const [detail, checkpoints] = await Promise.all([
      $fetch<JobDetail>(`${flink}/jobs/${job.jid}`, { timeout: 2000 }),
      $fetch<Checkpoints>(`${flink}/jobs/${job.jid}/checkpoints`, { timeout: 2000 }),
    ])

    // Operator metrics are named "<operator name>.<metric>" and summed across parallel subtasks.
    async function metric(operator: string, name: string): Promise<number | null> {
      const vertex = detail.vertices.find((v) => v.name.includes(operator))
      if (!vertex) return null
      const values = await $fetch<MetricValues>(
        `${flink}/jobs/${job!.jid}/vertices/${vertex.id}/subtasks/metrics`,
        { query: { get: `${operator}.${name}`, agg: 'sum' }, timeout: 2000 },
      )
      return values[0]?.sum ?? null
    }

    const [cleanPerSecond, duplicatesDropped, ivtFlagged, metadataMissing, lateDropped] = await Promise.all([
      metric('restore-event-time', 'numRecordsOutPerSecond'),
      metric('dedup-by-event-id', 'duplicatesDropped'),
      metric('flag-invalid-traffic', 'ivtFlagged'),
      metric('enrich-with-campaign', 'metadataMissing'),
      metric('line-item-minute-totals', 'numLateRecordsDropped'),
    ])

    const latest = checkpoints.latest.completed
    return {
```

```
      available: true,
      job: { name: job.name, state: job.state, uptimeMs: job.duration },
      checkpoints: {
        completed: checkpoints.counts.completed,
        failed: checkpoints.counts.failed,
        restored: checkpoints.counts.restored,
        lastCompletedAt: latest?.latest_ack_timestamp ?? null,
        lastDurationMs: latest?.end_to_end_duration ?? null,
        lastStateBytes: latest?.state_size ?? null,
      },
      metrics: { cleanPerSecond, duplicatesDropped, ivtFlagged, metadataMissing, lateDropped },
    }
  } catch (err) {
    return { available: false, error: err instanceof Error ? err.message : String(err) }
  }
})
```

dashboard/server/api/push/key.get.ts

5 lines · The VAPID public key the browser needs to subscribe, or enabled: false if push is not set up.

```
// The VAPID public key the browser needs to subscribe, or enabled: false if push is not set up.
export default defineEventHandler(() => {
  const config = pushConfig()
  return config ? { enabled: true, publicKey: config.publicKey } : { enabled: false, publicKey: null }
})
```

dashboard/server/api/push/subscribe.post.ts

11 lines · Registers this device for alert notifications. Signed-in users only (the middleware checks).

```
// Registers this device for alert notifications. Signed-in users only (the middleware checks).
export default defineEventHandler(async (event) => {
  if (!pushConfig()) throw createError({ statusCode: 503, statusMessage: 'Push notifications are not configured' })
  if (!getRequestHeader(event, 'content-type')?.startsWith('application/json')) {
    throw createError({ statusCode: 415, statusMessage: 'Expected application/json' })
  }
  const sub = validSubscription(await readBody(event).catch(() => null))
  if (!sub) throw createError({ statusCode: 400, statusMessage: 'Not a push subscription from a supported browser' })
  await addSubscription(sub, (event.context.user as string | undefined) ?? null)
  return { subscribed: true }
})
```

dashboard/server/api/push/unsubscribe.post.ts

8 lines

```
export default defineEventHandler(async (event) => {
  if (!getRequestHeader(event, 'content-type')?.startsWith('application/json')) {
    throw createError({ statusCode: 415, statusMessage: 'Expected application/json' })
  }
  const body = await readBody<{ endpoint?: unknown }>(event).catch(() => null)
  if (typeof body?.endpoint === 'string') await removeSubscription(body.endpoint)
  return { subscribed: false }
})
```

dashboard/server/middleware/auth.ts

31 lines · Every data route is read-only and requires a signed session cookie. The only writes are signing in

```
// Every data route is read-only and requires a signed session cookie. The only writes are signing in
// and out, (un)subscribing this device to notifications, and Alertmanager's webhook.
// The alert webhook authenticates with its own bearer token (Alertmanager has no session).
const OPEN = new Set(['/api/health', '/api/auth/login', '/api/auth/logout', '/api/alerts/webhook'])
const WRITES = new Set(['/api/auth/login', '/api/auth/logout', '/api/alerts/webhook', '/api/push/subscribe', '/api/push/unsubscribe'])

export default defineEventHandler((event) => {
  setResponseHeaders(event, {
    'X-Content-Type-Options': 'nosniff',
    'Referrer-Policy': 'no-referrer',
    'X-Frame-Options': 'DENY',
    'Content-Security-Policy': "frame-ancestors 'none'",
  })
  const path = getRequestURL(event).pathname
  if (!path.startsWith('/api/')) return
  setResponseHeader(event, 'Cache-Control', 'no-store')

  const method = event.method
  if (WRITES.has(path) ? method !== 'POST' : method !== 'GET' && method !== 'HEAD') {
    throw createError({ statusCode: 405, statusMessage: 'Method Not Allowed' })
  }

  const auth = authState()
  if (auth.disabled || path === '/api/health') return
  if (auth.problem) throw createError({ statusCode: 503, statusMessage: 'Authentication is not configured' })
  if (OPEN.has(path)) return

  const user = verifySession(getCookie(event, SESSION_COOKIE), auth.users, auth.secret)
  if (!user) throw createError({ statusCode: 401, statusMessage: 'Sign in required' })
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
30    event.context.user = user
31  })
```

dashboard/server/plugins/auth-config.ts

7 lines · Reports the authentication mode once at startup, so a misconfiguration is visible in the logs.

```
1  // Reports the authentication mode once at startup, so a misconfiguration is visible in the logs.
2  export default defineNitroPlugin(() => {
3    const auth = authState()
4    if (auth.disabled) console.warn('[auth] DISABLED: the API is open to anyone who can reach this server')
5    else if (auth.problem) console.error(`[auth] misconfigured, API answers 503: ${auth.problem}`)
6    else console.info(`[auth] enabled for ${auth.users.size} user(s), sessions last ${auth.ttlMs / 3_600_000} h`)
7  })
```

dashboard/server/utils/auth.ts

129 lines · Password hashing, signed session tokens and login rate limiting for the hosted dashboard API.

```
  1  // Password hashing, signed session tokens and login rate limiting for the hosted dashboard API.
  2  // Self-contained (no Nuxt auto-imports) so it can be unit-tested with `node --test`.
  3  import { createHash, createHmac, randomBytes, scryptSync, timingSafeEqual } from 'node:crypto'
  4
  5  export const SESSION_COOKIE = 'ae_session'
  6
  7  // scrypt cost: ~50 ms per verification on a laptop core, and login is rate limited.
  8  const SCRYPT_N = 32768
  9  const SCRYPT_R = 8
 10  const SCRYPT_P = 1
 11  const SCRYPT_MAXMEM = 64 * 1024 * 1024
 12  const USERNAME = /^[A-Za-z0-9._-]{1,64}$/
 13
 14  const b64url = (buf: Buffer) => buf.toString('base64url')
 15
 16  /**
 17   * Hashes a password as `scrypt.N.r.p.salt.hash`. The format avoids `$`, `:` and `,` so it can
 18   * sit in an environment variable or a Compose .env file without quoting or interpolation.
 19   */
 20  export function hashPassword(password: string, salt: Buffer = randomBytes(16)): string {
 21    const hash = scryptSync(password, salt, 32, { N: SCRYPT_N, r: SCRYPT_R, p: SCRYPT_P, maxmem: SCRYPT_MAXMEM })
 22    return ['scrypt', SCRYPT_N, SCRYPT_R, SCRYPT_P, b64url(salt), b64url(hash)].join('.')
 23  }
 24
 25  export function verifyPassword(password: string, encoded: string): boolean {
 26    const parts = encoded.split('.')
 27    if (parts.length !== 6 || parts[0] !== 'scrypt') return false
 28    const [n, r, p] = parts.slice(1, 4).map(Number)
 29    if (!n || !r || !p || n > 1 << 20) return false
 30    const expected = Buffer.from(parts[5]!, 'base64url')
 31    if (expected.length !== 32) return false
 32    try {
 33      const actual = scryptSync(password, Buffer.from(parts[4]!, 'base64url'), 32, { N: n, r, p, maxmem: SCRYPT_MA
    ↪ XMEM })
 34      return timingSafeEqual(actual, expected)
 35    } catch {
 36      return false // invalid scrypt parameters in a configured hash
 37    }
 38  }
 39
 40  /** Parses `alice=scrypt...,bob=scrypt...`. Malformed entries are reported, not silently dropped. */
 41  export function parseUsers(spec: string): { users: Map<string, string>; errors: string[] } {
 42    const users = new Map<string, string>()
 43    const errors: string[] = []
 44    for (const entry of spec.split(',').map((e) => e.trim()).filter(Boolean)) {
 45      const eq = entry.indexOf('=')
 46      const name = eq > 0 ? entry.slice(0, eq) : ''
 47      const hash = eq > 0 ? entry.slice(eq + 1) : ''
 48      if (!USERNAME.test(name) || !hash.startsWith('scrypt.') || hash.split('.').length !== 6) {
 49        errors.push(`malformed user entry #${users.size + errors.length + 1}`)
 50      } else {
 51        users.set(name, hash)
 52      }
 53    }
 54    return { users, errors }
 55  }
 56
 57  interface SessionPayload {
 58    u: string
 59    exp: number
 60    /** Fingerprint of the user's password hash: changing the password ends existing sessions. */
 61    v: string
 62  }
 63
 64  const fingerprint = (passwordHash: string) => b64url(createHash('sha256').update(passwordHash).digest()).slice(0
    ↪ , 12)
 65  const sign = (data: string, secret: string) => b64url(createHmac('sha256', secret).update(data).digest())
 66
 67  export function createSession(user: string, passwordHash: string, secret: string, ttlMs: number, now = Date.now(
    ↪ )): string {
 68    const payload: SessionPayload = { u: user, exp: now + ttlMs, v: fingerprint(passwordHash) }
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
  const body = b64url(Buffer.from(JSON.stringify(payload)))
  return `${body}.${sign(body, secret)}`
}

/** Returns the user name for a valid, unexpired session whose password has not changed since. */
export function verifySession(
  token: string | undefined,
  users: Map<string, string>,
  secret: string,
  now = Date.now(),
): string | null {
  if (!token) return null
  const dot = token.indexOf('.')
  if (dot <= 0) return null
  const body = token.slice(0, dot)
  const mac = Buffer.from(token.slice(dot + 1), 'base64url')
  const expected = Buffer.from(sign(body, secret), 'base64url')
  if (mac.length !== expected.length || !timingSafeEqual(mac, expected)) return null
  let payload: SessionPayload
  try {
    payload = JSON.parse(Buffer.from(body, 'base64url').toString('utf8'))
  } catch {
    return null
  }
  const hash = typeof payload.u === 'string' ? users.get(payload.u) : undefined
  if (!hash || typeof payload.exp !== 'number' || payload.exp <= now || payload.v !== fingerprint(hash)) return
↪ null
  return payload.u
}

/** Fixed-window counter per key, e.g. failed logins per client address and per user name. */
export class RateLimiter {
  private readonly windows = new Map<string, { count: number; resetAt: number }>()
  private readonly limit: number
  private readonly windowMs: number

  constructor(limit: number, windowMs: number) {
    this.limit = limit
    this.windowMs = windowMs
  }

  /** True while the key is under its limit. */
  allowed(key: string, now = Date.now()): boolean {
    const w = this.windows.get(key)
    return !w || w.resetAt <= now || w.count < this.limit
  }

  record(key: string, now = Date.now()): void {
    if (this.windows.size > 10_000) this.prune(now)
    const w = this.windows.get(key)
    if (!w || w.resetAt <= now) this.windows.set(key, { count: 1, resetAt: now + this.windowMs })
    else w.count++
  }

  reset(key: string): void {
    this.windows.delete(key)
  }

  private prune(now: number) {
    for (const [k, w] of this.windows) if (w.resetAt <= now) this.windows.delete(k)
  }
}
```

dashboard/server/utils/notifications.ts

20 lines · The last notifications sent, for the app's alerts panel. In memory: the source of truth for

```
// The last notifications sent, for the app's alerts panel. In memory: the source of truth for
// active alerts is Alertmanager; this is only recent history.
export interface SentNotification {
  status: 'firing' | 'resolved'
  alertname: string
  severity: string
  summary: string
  atMs: number
}

const recent: SentNotification[] = []

export function rememberNotification(n: SentNotification) {
  recent.unshift(n)
  recent.length = Math.min(recent.length, 30)
}

export function recentNotifications(): SentNotification[] {
  return [...recent]
}
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

dashboard/server/utils/push.ts

144 lines · Web Push to installed copies of the app: VAPID keys from config, subscriptions kept in a small

```ts
// Web Push to installed copies of the app: VAPID keys from config, subscriptions kept in a small
// JSON file (one per device that turned notifications on).
import { mkdir, readFile, rename, writeFile } from 'node:fs/promises'
import { dirname } from 'node:path'
import webpush from 'web-push'

export interface StoredSubscription {
  endpoint: string
  keys: { p256dh: string; auth: string }
  user: string | null
  createdAt: number
}

export interface PushPayload {
  title: string
  body: string
  /** Notifications with the same tag replace each other: "resolved" replaces "firing". */
  tag: string
  url: string
}

// The browsers' push services. A subscription endpoint is a URL the server will POST to, so only
// these hosts are accepted, never an arbitrary (possibly internal) address.
const DEFAULT_PUSH_HOSTS = [
  'fcm.googleapis.com', // Chrome, Edge, Android
  'updates.push.services.mozilla.com', // Firefox
  '.push.apple.com', // Safari, iOS 16.4+ home-screen apps
  '.notify.windows.com', // legacy Edge
]

let configured: { publicKey: string } | null | undefined
let subscriptions: Map<string, StoredSubscription> | null = null

function settings() {
  const c = useRuntimeConfig()
  return {
    publicKey: String(c.vapidPublicKey || ''),
    privateKey: String(c.vapidPrivateKey || ''),
    subject: String(c.vapidSubject || ''),
    storePath: String(c.pushStorePath || '.data/push-subscriptions.json'),
    allowedHosts: String(c.pushAllowedHosts || '').split(',').map((h) => h.trim()).filter(Boolean),
  }
}

/** The VAPID public key when push is configured, else null (the app hides the option). */
export function pushConfig(): { publicKey: string } | null {
  if (configured !== undefined) return configured
  const s = settings()
  if (!s.publicKey || !s.privateKey || !s.subject) {
    configured = null
  } else {
    webpush.setVapidDetails(s.subject, s.publicKey, s.privateKey)
    configured = { publicKey: s.publicKey }
  }
  return configured
}

/** Accepts only push-service endpoints with well-formed keys. */
export function validSubscription(input: unknown): Omit<StoredSubscription, 'user' | 'createdAt'> | null {
  const sub = input as { endpoint?: unknown; keys?: { p256dh?: unknown; auth?: unknown } } | null
  if (typeof sub?.endpoint !== 'string' || sub.endpoint.length > 1024) return null
  const p256dh = sub.keys?.p256dh
  const auth = sub.keys?.auth
  if (typeof p256dh !== 'string' || !/^[A-Za-z0-9_-]{80,100}$/.test(p256dh)) return null
  if (typeof auth !== 'string' || !/^[A-Za-z0-9_-]{16,32}$/.test(auth)) return null
  let url: URL
  try {
    url = new URL(sub.endpoint)
  } catch {
    return null
  }
  if (url.protocol !== 'https:') return null
  const hosts = [...DEFAULT_PUSH_HOSTS, ...settings().allowedHosts]
  const allowed = hosts.some((h) => (h.startsWith('.') ? url.hostname.endsWith(h) : url.hostname === h))
  return allowed ? { endpoint: sub.endpoint, keys: { p256dh, auth } } : null
}

async function load(): Promise<Map<string, StoredSubscription>> {
  if (subscriptions) return subscriptions
  try {
    const list = JSON.parse(await readFile(settings().storePath, 'utf8')) as StoredSubscription[]
    subscriptions = new Map(list.map((s) => [s.endpoint, s]))
  } catch {
    subscriptions = new Map()
  }
```

```
  return subscriptions
}

async function save() {
  const path = settings().storePath
  await mkdir(dirname(path), { recursive: true })
  // Write then rename, so a crash never leaves a half-written file.
  await writeFile(`${path}.tmp`, JSON.stringify([...(await load()).values()]), { mode: 0o600 })
  await rename(`${path}.tmp`, path)
}

export async function addSubscription(sub: Omit<StoredSubscription, 'user' | 'createdAt'>, user: string | null)
  ↪ {
  const subs = await load()
  if (!subs.has(sub.endpoint) && subs.size >= 500) throw new Error('too many subscriptions')
  subs.set(sub.endpoint, { ...sub, user, createdAt: Date.now() })
  await save()
}

export async function removeSubscription(endpoint: string) {
  const subs = await load()
  if (subs.delete(endpoint)) await save()
}

export async function subscriptionCount() {
  return (await load()).size
}

/**
 * Sends to every subscribed device. Subscriptions the push service reports as gone (404, 410:
 * the app was uninstalled or notifications turned off) are dropped.
 */
export async function pushToAll(payload: PushPayload): Promise<{ sent: number; failed: number; removed: number }
  ↪ > {
  if (!pushConfig()) return { sent: 0, failed: 0, removed: 0 }
  const subs = await load()
  let sent = 0
  let failed = 0
  const gone: string[] = []
  await Promise.all(
    [...subs.values()].map(async (s) => {
      try {
        await webpush.sendNotification({ endpoint: s.endpoint, keys: s.keys }, JSON.stringify(payload), {
          TTL: 3600,
          urgency: 'high',
          timeout: 10_000,
        })
        sent++
      } catch (err) {
        const status = (err as { statusCode?: number }).statusCode
        if (status === 404 || status === 410) gone.push(s.endpoint)
        else failed++
      }
    }),
  )
  if (gone.length) {
    gone.forEach((e) => subs.delete(e))
    await save()
  }
  return { sent, failed, removed: gone.length }
}
```

dashboard/server/utils/session.ts

47 lines · Auth configuration from runtime config, resolved once. Fails closed: if authentication is on but

```
// Auth configuration from runtime config, resolved once. Fails closed: if authentication is on but
// misconfigured, every protected API call answers 503 rather than serving data unauthenticated.
import type { H3Event } from 'h3'

export interface AuthState {
  disabled: boolean
  users: Map<string, string>
  secret: string
  ttlMs: number
  trustProxy: boolean
  /** Why authentication cannot work, if it cannot. Never contains secrets. */
  problem: string | null
}

let cached: AuthState | null = null

export function authState(): AuthState {
  if (cached) return cached
  const config = useRuntimeConfig()
  const disabled = String(config.authDisabled) === 'true'
  const { users, errors } = parseUsers(String(config.authUsers || ''))
  const secret = String(config.sessionSecret || '')
  let problem: string | null = null
```

```
  if (!disabled) {
    if (errors.length) problem = `NUXT_AUTH_USERS has ${errors.length} malformed entr${errors.length === 1 ? 'y' : 'ies'}`
    else if (users.size === 0) problem = 'NUXT_AUTH_USERS is empty'
    else if (secret.length < 32) problem = 'NUXT_SESSION_SECRET must be at least 32 characters'
  }
  cached = {
    disabled,
    users,
    secret,
    ttlMs: Math.max(1, Number(config.sessionTtlHours) || 12) * 3_600_000,
    trustProxy: String(config.trustProxy) === 'true',
    problem,
  }
  return cached
}

/** The client address, from X-Forwarded-For only when a trusted proxy (Caddy) sets it. */
export function clientAddress(event: H3Event): string {
  return getRequestIP(event, { xForwardedFor: authState().trustProxy }) ?? 'unknown'
}

export function isHttps(event: H3Event): boolean {
  return getRequestProtocol(event, { xForwardedProto: authState().trustProxy }) === 'https'
}
```

dashboard/test/auth.test.ts

71 lines

```
import assert from 'node:assert/strict'
import { test } from 'node:test'
import { createSession, hashPassword, parseUsers, RateLimiter, verifyPassword, verifySession } from '../server/utils/auth.ts'

const SECRET = 'test-secret-that-is-at-least-32-characters'
const HOUR = 3_600_000

test('passwords verify only against their own hash', () => {
  const hash = hashPassword('correct horse')
  assert.ok(verifyPassword('correct horse', hash))
  assert.ok(!verifyPassword('correct horse ', hash))
  assert.ok(!verifyPassword('wrong', hash))
  assert.ok(!verifyPassword('correct horse', 'scrypt.1.1.1.x.y'), 'malformed hash')
  assert.notEqual(hashPassword('same'), hashPassword('same'), 'salted')
})

test('hashes survive a .env file: no $, : or , characters', () => {
  assert.doesNotMatch(hashPassword('pw'), /[$:,=]/)
})

test('parses users and reports malformed entries without revealing them', () => {
  const alice = hashPassword('a')
  const { users, errors } = parseUsers(` alice=${alice}, bad entry ,bob:${alice}`)
  assert.deepEqual([...users.keys()], ['alice'])
  assert.equal(errors.length, 2)
  assert.ok(errors.every((e) => !e.includes('scrypt')))
})

test('sessions are valid until they expire', () => {
  const hash = hashPassword('pw')
  const users = new Map([['alice', hash]])
  const t0 = 1_800_000_000_000
  const token = createSession('alice', hash, SECRET, 12 * HOUR, t0)
  assert.equal(verifySession(token, users, SECRET, t0 + HOUR), 'alice')
  assert.equal(verifySession(token, users, SECRET, t0 + 12 * HOUR), null)
})

test('tampered, foreign or orphaned sessions are rejected', () => {
  const hash = hashPassword('pw')
  const users = new Map([['alice', hash], ['bob', hashPassword('pw2')]])
  const token = createSession('alice', hash, SECRET, HOUR)
  const [body, mac] = token.split('.')
  const forged = Buffer.from(JSON.stringify({ u: 'bob', exp: Date.now() + HOUR, v: 'x' })).toString('base64url')

  assert.equal(verifySession(`${forged}.${mac}`, users, SECRET), null, 'body swapped')
  assert.equal(verifySession(`${body}.${mac}x`, users, SECRET), null, 'mac altered')
  assert.equal(verifySession(token, users, 'another-secret-another-secret-123'), null, 'other secret')
  assert.equal(verifySession(token, new Map(), SECRET), null, 'user removed')
  assert.equal(verifySession('', users, SECRET), null)
  assert.equal(verifySession('garbage', users, SECRET), null)
})

test('changing a password ends existing sessions', () => {
  const old = hashPassword('old')
  const token = createSession('alice', old, SECRET, HOUR)
  assert.equal(verifySession(token, new Map([['alice', old]]), SECRET), 'alice')
  assert.equal(verifySession(token, new Map([['alice', hashPassword('new')]]), SECRET), null)
})
```

```
59
60  test('rate limiter blocks after the limit until the window passes', () => {
61    const limiter = new RateLimiter(3, 60_000)
62    for (let i = 0; i < 3; i++) {
63      assert.ok(limiter.allowed('ip', 0))
64      limiter.record('ip', 0)
65    }
66    assert.ok(!limiter.allowed('ip', 1_000))
67    assert.ok(limiter.allowed('other', 1_000))
68    assert.ok(limiter.allowed('ip', 60_000))
69    limiter.reset('ip')
70    assert.ok(limiter.allowed('ip', 1_000))
71  })
```

dashboard/tsconfig.json

9 lines

```
1  {
2    "files": [],
3    "references": [
4      { "path": "./.nuxt/tsconfig.app.json" },
5      { "path": "./.nuxt/tsconfig.server.json" },
6      { "path": "./.nuxt/tsconfig.shared.json" },
7      { "path": "./.nuxt/tsconfig.node.json" }
8    ]
9  }
```

deploy

deploy/.env.example

23 lines · Copy to deploy/.env (git-ignored) and fill in. Values need no quoting.

```
1  # Copy to deploy/.env (git-ignored) and fill in. Values need no quoting.
2
3  # Public DNS name of this host. Caddy gets a Let's Encrypt certificate for it.
4  DASHBOARD_DOMAIN=ads.example.com
5
6  # Users, comma-separated. Create each entry with:
7  #   cd dashboard && npm run hash-password -- <user>
8  # or, without Node installed (after building the images):
9  #   docker compose run --rm --no-deps dashboard node --experimental-strip-types tools/scripts/hash-password.ts <
   ↳ user>
10 DASHBOARD_USERS=
11
12 # At least 32 random characters, e.g. `openssl rand -base64 48`. Changing it signs everyone out.
13 DASHBOARD_SESSION_SECRET=
14
15 # Shared secret between Alertmanager and the dashboard's alert webhook: `openssl rand -base64 32`.
16 ALERT_WEBHOOK_TOKEN=
17
18 # Grafana admin password (Grafana is not exposed publicly; see README "Observability").
19 GRAFANA_ADMIN_PASSWORD=
20
21 # Optional: alert notifications on installed apps. Generate with `npx web-push generate-vapid-keys`.
22 VAPID_PUBLIC_KEY=
23 VAPID_PRIVATE_KEY=
```

deploy/Caddyfile

17 lines · Automatic HTTPS: Caddy obtains and renews a certificate for DASHBOARD_DOMAIN (its DNS must point

```
1  # Automatic HTTPS: Caddy obtains and renews a certificate for DASHBOARD_DOMAIN (its DNS must point
2  # at this host, with ports 80 and 443 open). With DASHBOARD_DOMAIN=localhost it uses its own local CA.
3  {$DASHBOARD_DOMAIN} {
4      encode zstd gzip
5
6      header {
7          Strict-Transport-Security "max-age=31536000"
8          -Server
9      }
10
11     # Caddy replaces any client-supplied X-Forwarded-For/-Proto, so the dashboard
12     # (NUXT_TRUST_PROXY=true) sees the real client address for login rate limits.
13     # Alertmanager calls the alert webhook inside the Compose network; it is never public.
14     respond /api/alerts/webhook 404
15
16     reverse_proxy dashboard:3000
17 }
```

deploy/docker-compose.prod.yml

69 lines · Hosted deployment overlay: one public entry point (Caddy, HTTPS) in front of the dashboard,

```
1  # Hosted deployment overlay: one public entry point (Caddy, HTTPS) in front of the dashboard,
2  # which serves the installable app and the signed-in, read-only API. Nothing else is published:
3  # Kafka, Flink, Druid, the ad server and the pacing service are reachable only on the Compose
4  # network. See README "Hosting".
5  #
```

```
#   cp deploy/.env.example deploy/.env    # then fill it in
#   docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml --env-file deploy/.env up -d --build
services:
  caddy:
    image: caddy:2.10.2-alpine
    restart: unless-stopped
    depends_on: [dashboard]
    environment:
      DASHBOARD_DOMAIN: ${DASHBOARD_DOMAIN:?set DASHBOARD_DOMAIN in deploy/.env}
    ports:
      - "80:80"
      - "443:443"
      - "443:443/udp" # HTTP/3
    volumes:
      - ./deploy/Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy-data:/data # certificates: keep them, or Let's Encrypt rate limits bite on redeploys
      - caddy-config:/config

  dashboard:
    restart: unless-stopped
    environment:
      NUXT_AUTH_DISABLED: "false"
      NUXT_AUTH_USERS: ${DASHBOARD_USERS:?set DASHBOARD_USERS in deploy/.env}
      NUXT_SESSION_SECRET: ${DASHBOARD_SESSION_SECRET:?set DASHBOARD_SESSION_SECRET in deploy/.env}
      NUXT_TRUST_PROXY: "true"
      NUXT_ALERT_WEBHOOK_TOKEN: ${ALERT_WEBHOOK_TOKEN:?set ALERT_WEBHOOK_TOKEN in deploy/.env}
      NUXT_VAPID_SUBJECT: ${VAPID_SUBJECT:-mailto:ops@${DASHBOARD_DOMAIN}}
    ports: !reset []

  alertmanager:
    environment:
      ALERT_WEBHOOK_TOKEN: ${ALERT_WEBHOOK_TOKEN:?set ALERT_WEBHOOK_TOKEN in deploy/.env}
    ports: !reset []

  grafana:
    environment:
      # No anonymous access in production; reach Grafana through an SSH tunnel (see README).
      GF_AUTH_ANONYMOUS_ENABLED: "false"
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD:?set GRAFANA_ADMIN_PASSWORD in deploy/.env}
    ports: !reset []

  # Internal only in production.
  redpanda:
    ports: !reset []
  console:
    ports: !reset []
  gateway:
    ports: !reset []
  jobmanager:
    ports: !reset []
  router:
    ports: !reset []
  ad-server:
    ports: !reset []
  pacing-service:
    ports: !reset []
  audit-service:
    ports: !reset []
  prometheus:
    ports: !reset []

volumes:
  caddy-data:
  caddy-config:
```

docs

docs/report/build_report.py

636 lines

```
#!/usr/bin/env python3
"""Builds the Quila Visuals technical report for the Ads Eventing platform.

    python3 docs/report/build_report.py . docs/report/Quila-Visuals-Ads-Eventing-Report.pdf

Needs reportlab and the DejaVu fonts (fonts-dejavu-core). Logos come from docs/brand/.
"""
import collections
import datetime
import os
import re
import subprocess
import sys
import xml.etree.ElementTree as ET

from reportlab.lib import colors
```

```
from reportlab.lib.enums import TA_LEFT
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.units import mm
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import (BaseDocTemplate, CondPageBreak, Frame, KeepTogether, NextPageTemplate, PageBreak
,
                                PageTemplate, Paragraph, Preformatted, Spacer, Table, TableStyle)
from reportlab.platypus.tableofcontents import TableOfContents

REPO, OUT = sys.argv[1], sys.argv[2]
TODAY = datetime.date.today().strftime("%d %B %Y")

# ---------------------------------------------------------------- fonts and brand
F = "/usr/share/fonts/truetype/dejavu/"
pdfmetrics.registerFont(TTFont("Sans", F + "DejaVuSans.ttf"))
pdfmetrics.registerFont(TTFont("Sans-Bold", F + "DejaVuSans-Bold.ttf"))
pdfmetrics.registerFont(TTFont("Mono", F + "DejaVuSansMono.ttf"))
pdfmetrics.registerFont(TTFont("Mono-Bold", F + "DejaVuSansMono-Bold.ttf"))
pdfmetrics.registerFont(TTFont("Serif", F + "DejaVuSerif.ttf"))
from reportlab.pdfbase.pdfmetrics import registerFontFamily
registerFontFamily("Sans", normal="Sans", bold="Sans-Bold", italic="Sans", boldItalic="Sans-Bold")
registerFontFamily("Mono", normal="Mono", bold="Mono-Bold", italic="Mono", boldItalic="Mono-Bold")

INK = colors.HexColor("#141a26")
INK2 = colors.HexColor("#4a5262")
MUTED = colors.HexColor("#7a8292")
RULE = colors.HexColor("#d9dde4")
SOFT = colors.HexColor("#f3f4f6")
ACCENT = colors.HexColor("#e9a512")      # brand accent
ACCENT_INK = colors.HexColor("#8a5d00")
GOOD = colors.HexColor("#2c7a4b")

BRAND = "QUILA VISUALS"
DOC_TITLE = "Ads Eventing Platform"
DOC_SUB = "Technical report: design, implementation, verification and source code"


BRAND_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "brand")
LOGO_DARK = os.path.join(BRAND_DIR, "quila-visuals-logo.png")
LOGO_WHITE = os.path.join(BRAND_DIR, "quila-visuals-logo-white.png")


def draw_logo(c, x, y, height, light=False):
    """The QUILA VISUALS logo with its bottom-left corner at (x, y). Returns its width."""
    from reportlab.lib.utils import ImageReader
    img = ImageReader(LOGO_WHITE if light else LOGO_DARK)
    iw, ih = img.getSize()
    w = height * iw / ih
    c.drawImage(img, x, y, width=w, height=height, mask="auto")
    return w


# ---------------------------------------------------------------- styles
def S(name, **kw):
    base = dict(fontName="Sans", fontSize=9.6, leading=14, textColor=INK, alignment=TA_LEFT)
    base.update(kw)
    return ParagraphStyle(name, **base)


H1 = S("H1", fontName="Sans-Bold", fontSize=19, leading=24, spaceBefore=4, spaceAfter=10, textColor=INK)
H2 = S("H2", fontName="Sans-Bold", fontSize=12.5, leading=17, spaceBefore=12, spaceAfter=5, textColor=INK)
H3 = S("H3", fontName="Sans-Bold", fontSize=10.2, leading=14, spaceBefore=8, spaceAfter=3, textColor=INK2)
BODY = S("Body", spaceAfter=6)
SMALL = S("Small", fontSize=8.2, leading=11, textColor=INK2)
EYEBROW = S("Eyebrow", fontName="Sans-Bold", fontSize=7.6, leading=10, textColor=ACCENT_INK, spaceAfter=2)
BUL = S("Bul", leftIndent=12, bulletIndent=2, spaceAfter=3)
CELL = S("Cell", fontSize=8, leading=10.6)
CELLB = S("CellB", fontName="Sans-Bold", fontSize=8, leading=10.6)
CELLM = S("CellM", fontName="Mono", fontSize=7.4, leading=9.6)
HEADC = S("HeadC", fontName="Sans-Bold", fontSize=7.4, leading=9.5, textColor=MUTED)
H2A = S("H2A", fontName="Sans-Bold", fontSize=12.5, leading=17, spaceBefore=12, spaceAfter=5, textColor=INK)
FILEH = S("FileH", fontName="Mono-Bold", fontSize=8.4, leading=11, textColor=INK, spaceBefore=8, spaceAfter=1)
FILEMETA = S("FileMeta", fontSize=7.6, leading=10, textColor=MUTED, spaceAfter=3)
CODE = ParagraphStyle("Code", fontName="Mono", fontSize=6.5, leading=8.1, textColor=INK, backColor=SOFT,
                      borderPadding=(3, 4, 3, 4), leftIndent=4, rightIndent=4, spaceAfter=6)


def esc(s):
    return s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")


def md(s):
    """Tiny markdown: **bold**, `code` -> reportlab markup (input is trusted text written here)."""
    s = esc(s)
    s = re.sub(r"\*\*(.+?)\*\*", r"<b>\1</b>", s)
    s = re.sub(r"`(.+?)`", r'<font name="Mono" size="8.4">\1</font>', s)
```

```
    return s


def P(text, style=BODY):
    return Paragraph(md(text), style)


def bullets(items, style=BUL):
    return [Paragraph(md(i), style, bulletText="•") for i in items]


def table(rows, widths, head=True, mono_cols=(), bold_first=False, zebra=True):
    data = []
    for ri, row in enumerate(rows):
        cells = []
        for ci, v in enumerate(row):
            if isinstance(v, (Paragraph, Table)):
                cells.append(v)
                continue
            st = HEADC if (head and ri == 0) else (CELLM if ci in mono_cols else (CELLB if bold_first and ci == 0 else CELL))
            cells.append(Paragraph(md(str(v)) if not (head and ri == 0) else esc(str(v)).upper(), st))
        data.append(cells)
    t = Table(data, colWidths=widths, repeatRows=1 if head else 0)
    style = [("VALIGN", (0, 0), (-1, -1), "TOP"), ("LEFTPADDING", (0, 0), (-1, -1), 4),
             ("RIGHTPADDING", (0, 0), (-1, -1), 4), ("TOPPADDING", (0, 0), (-1, -1), 3.2),
             ("BOTTOMPADDING", (0, 0), (-1, -1), 3.2), ("LINEBELOW", (0, 0), (-1, -1), 0.4, RULE)]
    if head:
        style += [("LINEBELOW", (0, 0), (-1, 0), 0.9, INK2)]
    if zebra:
        for i in range(1 if head else 0, len(data)):
            if i % 2 == 0:
                style.append(("BACKGROUND", (0, i), (-1, i), SOFT))
    t.setStyle(TableStyle(style))
    return t


# ---------------------------------------------------------------- document with TOC and bookmarks
class Doc(BaseDocTemplate):
    def __init__(self, path, **kw):
        super().__init__(path, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm, topMargin=22 * mm,
                         bottomMargin=18 * mm, title=f"{DOC_TITLE}: technical report", author=BRAND,
                         subject=DOC_SUB, creator=BRAND, **kw)
        frame = Frame(self.leftMargin, self.bottomMargin, self.width, self.height, id="f")
        self.addPageTemplates([PageTemplate("cover", frames=[frame], onPage=cover_page),
                               PageTemplate("body", frames=[frame], onPage=body_page)])
        self._seq = 0

    def beforeDocument(self):
        self._seq = 0

    def afterFlowable(self, f):
        if not isinstance(f, Paragraph):
            return
        name = f.style.name
        if name in ("H1", "H2", "H2A", "FileH"):
            text = f.getPlainText()
            key = f"k{self._seq}"
            self._seq += 1
            self.canv.bookmarkPage(key)
            level = {"H1": 0, "H2": 1, "H2A": 1, "FileH": 2}[name]
            self.canv.addOutlineEntry(text, key, level=level, closed=level > 0)
            if name in ("H1", "H2"):
                self.notify("TOCEntry", (level, text, self.page, key))


def cover_page(c, doc):
    w, h = A4
    c.saveState()
    c.setFillColor(INK)
    c.rect(0, h * 0.42, w, h * 0.58, stroke=0, fill=1)
    c.setFillColor(ACCENT)
    c.rect(0, h * 0.42, w, 3.2, stroke=0, fill=1)
    draw_logo(c, 18 * mm, h - 70 * mm, 46 * mm, light=True)
    c.setFillColor(colors.HexColor("#c9cfd9"))
    c.setFont("Sans-Bold", 9)
    c.drawString(18 * mm, h * 0.42 + 120, "TECHNICAL REPORT")
    c.setFillColor(colors.white)
    c.setFont("Sans-Bold", 34)
    c.drawString(18 * mm, h * 0.42 + 78, "Ads Eventing")
    c.drawString(18 * mm, h * 0.42 + 40, "Platform")
    c.setFillColor(colors.HexColor("#c9cfd9"))
    c.setFont("Sans", 11)
    c.drawString(18 * mm, h * 0.42 + 16, "Design, implementation, verification and complete source code")
    # facts below the band
    jt = sum(tests.values())
    facts = [("Six phases", "plus an installable app"), (f"{TOTAL_LINES:,} lines", f"in {len(SOURCE)} source files"),
```

```
             (f"{jt + 7 + RULE_TESTS} automated tests", f"{jt} Java, 7 TypeScript, {RULE_TESTS} alert-rule"),
             ("20,000 events/s", "sustained, exactly-once")]
    col = (w - 36 * mm) / 2
    for i, (a, b) in enumerate(facts):
        x = 18 * mm + (i % 2) * col
        y0 = h * 0.42 - (28 + 26 * (i // 2)) * mm
        c.setFillColor(ACCENT)
        c.rect(x, y0 + 22, 18, 2.2, stroke=0, fill=1)
        c.setFillColor(INK)
        c.setFont("Sans-Bold", 15)
        c.drawString(x, y0 + 4, a)
        c.setFillColor(INK2)
        c.setFont("Sans", 8.4)
        c.drawString(x, y0 - 10, b)
    c.setFillColor(INK2)
    c.setFont("Sans", 9)
    c.drawString(18 * mm, 32 * mm, f"Prepared by {BRAND}")
    c.drawString(18 * mm, 27 * mm, f"Issued {TODAY}")
    c.drawString(18 * mm, 22 * mm, "Repository: github.com/michaelonidba-code/primetech")
    c.setStrokeColor(RULE)
    c.line(18 * mm, 38 * mm, w - 18 * mm, 38 * mm)
    c.restoreState()


def body_page(c, doc):
    w, h = A4
    c.saveState()
    draw_logo(c, 18 * mm, h - 16 * mm, 9 * mm)
    c.setFont("Sans", 7.6)
    c.setFillColor(MUTED)
    c.drawRightString(w - 18 * mm, h - 12.6 * mm, f"{DOC_TITLE} · Technical report")
    c.setStrokeColor(RULE)
    c.setLineWidth(0.5)
    c.line(18 * mm, h - 17 * mm, w - 18 * mm, h - 17 * mm)
    c.line(18 * mm, 13 * mm, w - 18 * mm, 13 * mm)
    c.drawString(18 * mm, 9 * mm, f"© {datetime.date.today().year} {BRAND}. Prepared for the Ads Eventing projec
↪ t.")
    c.setFillColor(INK)
    c.setFont("Sans-Bold", 7.6)
    c.drawRightString(w - 18 * mm, 9 * mm, str(doc.page))
    c.restoreState()


# ---------------------------------------------------------------- repository facts
def git(*args):
    return subprocess.run(["git", "-C", REPO, *args], capture_output=True, text=True, check=True).stdout


FILES = [f for f in git("ls-files").split() if not f.endswith((".png", ".jpg", ".pdf", "package-lock.json"))]
GENERATED = {"infra/grafana/dashboards/ads-eventing.json": "generated Grafana dashboard (33 panels)",
             "docs/loadtest/events.json": "load test measurements (data)", "docs/loadtest/full.json": "load test
↪  measurements (data)",
             "docs/overview.html": "one-page illustrated overview of the design (open in a browser)"}
SOURCE = [f for f in FILES if not f.endswith(".md")]
PURPOSE_OVERRIDES = {
    "ad-server/src/main/java/dev/primetech/ads/adserver/AdServerApp.java": "Entry point: wires the ad server (DS
↪ P clients, Redis, Kafka, pacing view, HTTP).",
    "ad-server/src/main/java/dev/primetech/ads/adserver/AdServerConfig.java": "Ad server settings read from envi
↪ ronment variables.",
    "ads-common/src/main/java/dev/primetech/ads/common/DeviceType.java": "Device types accepted on events (CTV,
↪ mobile, web, ...).",
    "ads-common/src/main/java/dev/primetech/ads/common/RejectReason.java": "Reasons an event is rejected at the
↪ gateway (sent to the dead-letter topic).",
    "ads-common/src/main/java/dev/primetech/ads/common/Topics.java": "Names of every Kafka topic, shared by all
↪ services.",
    "event-simulator/src/main/java/dev/primetech/ads/simulator/SimulatorApp.java": "Entry point: runs the traffi
↪ c simulator against the gateway and ad server.",
    "event-simulator/src/main/java/dev/primetech/ads/simulator/SimulatorConfig.java": "Simulator settings: rate,
↪  faults, campaigns, seed, ad server URL.",
    "ingest-gateway/src/main/java/dev/primetech/ads/gateway/GatewayApp.java": "Entry point: starts the ingest ga
↪ teway HTTP server and Kafka producer.",
    "ingest-gateway/src/main/java/dev/primetech/ads/gateway/GatewayConfig.java": "Gateway settings read from env
↪ ironment variables.",
    "ingest-gateway/src/main/java/dev/primetech/ads/gateway/KafkaEventSink.java": "Writes accepted events to Kaf
↪ ka (acks=all, idempotent) and rejected ones to the dead-letter topic.",
    "mock-dsp/src/main/java/dev/primetech/ads/dsp/DspApp.java": "Entry point: runs one mock DSP with a named bid
↪ ding preset.",
    "pom.xml": "Maven parent build: modules and pinned dependency versions.",
    "Dockerfile": "Multi-stage build producing one image per Java service.",
    "docker-compose.yml": "The whole stack for local runs: Kafka, Flink, Druid, services, observability, dashboa
↪ rd.",
}


def purpose(f):
    if f in PURPOSE_OVERRIDES:
        return PURPOSE_OVERRIDES[f]
    if f in GENERATED:
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
264          return GENERATED[f]
265      t = open(os.path.join(REPO, f), errors="replace").read()
266      if f.endswith(".java"):
267          m = re.search(r"/\*\*\s*(.*?)\*/\s*(?:@\w+(?:\([^)]*\))?\s*)*(?:public |final |abstract |sealed |static
   ↪ )*(?:class|record|interface|enum)", t, re.S)
268          if m:
269              s = re.sub(r"\s*\*\s?", " ", m.group(1))
270              s = re.sub(r"\{@\w+ ([^}]*)\}", r"\1", s)
271              s = re.sub(r"<[^>]+>", "", s)
272              s = " ".join(s.split())
273              return re.split(r"(?<=[.!?])\s", s)[0]
274          return "Test." if "/test/" in f else ""
275      m = re.match(r"\s*(?://|#|<!--)\s*(.+?)(?:-->)?\s*$", t.splitlines()[0] if t.strip() else "")
276      if m and not t.startswith("#!"):
277          return m.group(1).strip()
278      for line in t.splitlines()[:6]:
279          mm_ = re.match(r"\s*(?://|#)\s*(.+)", line)
280          if mm_ and not line.startswith("#!"):
281              return mm_.group(1).strip()
282      return ""
283
284
285  def lines_of(f):
286      return len(open(os.path.join(REPO, f), errors="replace").read().splitlines())
287
288
289  TOTAL_LINES = sum(lines_of(f) for f in SOURCE)
290
291  # Tests per module: @Test methods in each module's test sources (equals what Maven runs).
292  tests = collections.Counter()
293  for f in FILES:
294      if f.endswith(".java") and "/src/test/" in f:
295          tests[f.split("/")[0]] += len(re.findall(r"^\s*@Test\b", open(os.path.join(REPO, f)).read(), re.M))
296  tests = +tests
297
298  # Alert rules.
299  rules = []
300  cur = None
301  for line in open(os.path.join(REPO, "infra/prometheus/rules/ads-eventing.yml")):
302      if m := re.match(r"\s*- name: (\S+)", line):
303          group = m.group(1)
304      if m := re.match(r"\s*- alert: (\S+)", line):
305          cur = {"alert": m.group(1), "group": group, "for": "", "severity": "", "summary": ""}
306          rules.append(cur)
307      if cur and (m := re.match(r"\s*for: (\S+)", line)):
308          cur["for"] = m.group(1)
309      if cur and (m := re.search(r"severity: (\w+)", line)):
310          cur["severity"] = m.group(1)
311      if cur and (m := re.match(r'\s*summary: "?(.+?)"?\s*$', line)):
312          cur["summary"] = re.sub(r"\{\{.*?\}\}", "…", m.group(1))
313  RULE_TESTS = len(re.findall(r"^  - name:", open(os.path.join(REPO, "infra/prometheus/tests/ads-eventing-test.yml
   ↪ ")).read(), re.M))
314
315
316  def md_table(heading):
317      """Rows of the first Markdown table under a README heading."""
318      text = open(os.path.join(REPO, "README.md")).read()
319      i = text.index(heading)
320      rows = []
321      for line in text[i:].splitlines()[1:]:
322          if line.startswith("|"):
323              if re.match(r"^\|[-| :]+\|$", line):
324                  continue
325              rows.append([c.strip() for c in line.strip("|").split("|")])
326          elif rows:
327              break
328      return rows
329
330
331  # ---------------------------------------------------------------- content
332  W = A4[0] - 36 * mm
333  story = [NextPageTemplate("body"), PageBreak()]
334
335  # About + contents
336  story += [Paragraph("About this document", H1),
337            P(f"This report records everything built for the Ads Eventing platform: the architecture and the reaso
   ↪ ns "
338              f"for it, what each of the six phases delivered, how correctness and performance were verified, what
   ↪  "
339              f"failed and how it was fixed, how to run and operate the system, and, in Appendix B, the complete s
   ↪ ource "
340              f"code: {len(SOURCE)} files, {TOTAL_LINES:,} lines. Every figure in it was measured on the running s
   ↪ ystem."),
341            P(f"Prepared by **{BRAND}** on {TODAY}. Source: github.com/michaelonidba-code/primetech, pull requests
   ↪  "
342              f"#2 to #8. The PDF has bookmarks for every section and every source file."),
343            Spacer(1, 6)]
344  toc = TableOfContents()
```

```
toc.levelStyles = [S("T0", fontName="Sans-Bold", fontSize=9.6, leading=15, leftIndent=0),
                   S("T1", fontSize=8.6, leading=12, leftIndent=14, textColor=INK2)]
story += [Paragraph("Contents", H2), toc, PageBreak()]

# 1 Executive summary
story += [Paragraph("1. Executive summary", H1),
          P("The project builds, in one repository, a real-time advertising system for a connected-TV ad tier, "
            "modelled on the systems that serve and measure ads at streaming scale. A player asks the **ad serve
↪ r** "
            "for an ad break; the ad server runs an **OpenRTB 2.6 auction** across demand-side platforms and its
↪  own "
            "direct campaigns, within budgets set by a **pacing** service. The player reports playback to an **i
↪ ngest "
            "gateway**; events flow through **Kafka** to **Flink**, which removes duplicates, enriches, flags in
↪ valid "
            "traffic and totals delivery exactly once; **Druid** serves reporting. An **audit** service continuo
↪ usly "
            "reconciles independent records of the same money, and **alerts** reach phones through the installab
↪ le "
            "dashboard app."),
          table([["Outcome", "Result"],
                 ["Event path throughput", "**20,000 events/s** sustained on one 4-vCPU machine (1.7 billion a d
↪ ay), exactly-once"],
                 ["Ingest gateway alone", "**51,500 events/s**, p99 14–23 ms"],
                 ["Ad serving", "**660 auctions/s** across three DSPs, p99 459 ms, nothing shed"],
                 ["Correctness", "every reconciliation exact, including **956,446 = 956,446** events through a c
↪ rash"],
                 ["Budget pacing", "**0** line items over budget in measured periods; served-spend counters equa
↪ l the decision log to the cent"],
                 ["Alerting", "fault to phone notification in **49–87 s** in failure drills"],
                 ["Defects found by measurement", "**12** bugs no unit test caught, all fixed"],
                 ["Automated tests", f"**{sum(tests.values())} Java**, 7 TypeScript, {RULE_TESTS} alert-rule cas
↪ es; CI on every pull request"]],
                [48 * mm, W - 48 * mm], bold_first=True),
          Spacer(1, 8),
          P("Technology: Java 21 (virtual threads), Apache Flink 2.2, Redpanda (Kafka API), Apache Druid 37, Red
↪ is, "
            "Nuxt 4 / Vue 3 / TypeScript, Prometheus, Alertmanager, Grafana, Caddy, Docker Compose, GitHub Actio
↪ ns."),
          PageBreak()]

# 2 Architecture
story += [Paragraph("2. Architecture", H1),
          P("Kafka is the system of record between every stage. Druid and Redis are views that can be rebuilt, a
↪ nd the "
            "services that keep state (pacing, audit) rebuild it from the topics when they start, so nothing but
↪  Kafka "
            "needs backing up."),
          Paragraph("Data flow", H2),
          *bullets([
              "**Serving:** player → ad server (direct line items + OpenRTB 2.6 auction across 3 DSPs, frequency
↪  caps, floors) → ad pod back to the player. Every decision is logged to `ad-decisions.v1`.",
              "**Pacing:** the pacing service reads delivered spend per minute from Flink and writes a throttle
↪ and stop threshold per line item to Redis; the ad server reads them every second and writes its served spend bac
↪ k.",
              "**Measurement:** player → ingest gateway (validate, pseudonymise, acknowledge when durable) → `ad
↪ -events.v1` → Flink (de-duplicate, enrich, flag invalid traffic, minute totals, exactly-once) → `ad-events-clean
↪ .v1`, `line-item-minutes.v1` → Druid.",
              "**Assurance:** the audit service reconciles Flink with Druid, Redis with the decision log, and de
↪ livery with budgets; Prometheus scrapes every service; Alertmanager routes alerts to the dashboard, which pushes
↪  them to phones."]),
          Paragraph("Components", H2),
          table([["Component", "Built with", "Responsibility"],
                 ["Ingest gateway", "Java 21, JDK HTTP server, Kafka producer", "Validate events one by one, pse
↪ udonymise device IDs, acknowledge only when Kafka has the batch"],
                 ["Event bus", "Redpanda (Kafka API)", "Durable, partitioned, replayable log between every stage
↪ "],
                 ["Stream processor", "Apache Flink 2.2, RocksDB state", "De-duplicate, enrich, flag invalid tra
↪ ffic, minute totals per line item, exactly-once output"],
                 ["Analytics store", "Apache Druid 37", "Minute rollups with unique-device sketches, SQL"],
                 ["Ad server", "Java 21, OpenRTB 2.6", "Direct demand and DSP auction, frequency caps, floors, p
↪ acing, decision log"],
                 ["Mock DSPs", "Java 21", "Three bidders with realistic value and latency distributions"],
                 ["Pacing service", "Java 21", "A closed-loop controller per line item"],
                 ["Audit service", "Java 21", "Continuous reconciliation of independent records"],
                 ["Dashboard", "Nuxt 4, Vue 3, TypeScript, PWA", "Live view, sign-in, read-only API, alerts and
↪ push notifications"],
                 ["Observability", "Prometheus, Alertmanager, Grafana", f"{len(rules)} alert rules, an operation
↪ s dashboard"],
                 ["Edge", "Caddy", "Automatic HTTPS; the only published service in production"]],
                [32 * mm, 50 * mm, W - 82 * mm], bold_first=True),
          PageBreak()]

PHASES = [
    ("3. Phase 1: event backbone", "Pull request #2",
     ["**Event model and validation** (`ads-common`): `ClientEvent` in, `AdEvent` to Kafka; each event validated
↪  on its own so one bad event never fails a batch; rejected events go to `ad-events-dlq.v1` with the reason.",
      "**Privacy:** device IDs replaced at the gateway by an HMAC-SHA256 pseudonym when the viewer consented, dr
```

```
    ↪ opped when not. Raw IDs never reach Kafka.",
405         "**Ingest gateway:** JDK HTTP server on virtual threads; Kafka producer with `acks=all` and idempotence, k
    ↪ eyed by session; replies 200 only when every event is durable, otherwise 503 with `Retry-After` (at-least-once).
    ↪ ",
406         "**Traffic simulator:** realistic ad pods, quartile drop-off, clicks, skewed campaigns; injects duplicates
    ↪ , late and invalid events and keeps exact ground truth.",
407         "**Dashboard:** Nuxt 4 single-page app with live throughput, funnel and rejections."],
408       [["Measure", "Result"], ["Sustained ingest", "about **51,500 events/s** (4.4 billion a day), p50 ≈ 9 ms, p9
    ↪ 9 14–23 ms"],
409         ["Invalid events", "39,464 injected = 39,464 rejected = 39,464 in the dead-letter topic"],
410         ["Privacy", "0 raw device IDs in 3,000 sampled Kafka records"],
411         ["Tuning", "capping JVM heaps cut p99 from 55–80 ms to 14–23 ms"]]),
412       ("4. Phase 2: stream processing and reporting", "Pull request #3",
413        ["**De-duplication** by event ID in RocksDB state with a 30-minute TTL removes the duplicates at-least-once
    ↪  ingest lets through.",
414         "**Enrichment** with campaign metadata through broadcast state; events that arrive before their metadata a
    ↪ re buffered and released.",
415         "**Invalid traffic:** a device above 20 impressions in an event-time minute is flagged, excluded from spen
    ↪ d, and kept.",
416         "**Minute totals per line item:** event-time windows with 10 minutes of allowed lateness, emitted as revis
    ↪ ions; `CoalescingLateTrigger` limits late re-firings (217 revisions of one minute became 6).",
417         "**Exactly-once output:** checkpoints every 10 s, Kafka transactions on both sinks; `RestoreEventTime` res
    ↪ tores timestamps lost by broadcast state.",
418         "**Druid 37:** Kafka supervisor reading committed records only, minute rollup, HLL unique devices; dashboa
    ↪ rd panels for Flink health and campaign delivery."],
419       [["Measure", "Result"], ["Clean events vs ground truth", "**636,872 = 636,872**, identical per event type;
    ↪ 6,397 duplicates injected, 0 in output"],
420         ["Line item minutes vs recount", "306 / 306 windows match"],
421         ["Crash drill (TaskManager killed)", "**956,446 = 956,446**, 0 missing, 0 duplicated; 612 / 612 minutes ma
    ↪ tch"],
422         ["Druid vs clean topic", "identical totals, including 8,537,992,000 µ$ of spend"]]),
423       ("5. Phase 3: ad serving with OpenRTB auctions", "Pull request #4",
424        ["**OpenRTB 2.6 model** for connected-TV ad pods (`podid`, `slotinpod`, `poddur`), with price macros.",
425         "**Mock DSPs:** premium (selective, fast), budget (bids on most, cheap) and slow (high value, long latency
    ↪  tail).",
426         "**Ad server:** pseudonymises the device, fans out to DSPs in parallel within 120 ms while filtering direc
    ↪ t line items, applies frequency caps with one Redis `MGET`, runs a per-slot pod auction (floors, one ad per adve
    ↪ rtiser per break, duration fit, first or second price), sends win notices and logs every decision.",
427         "**Admission control:** bounded in-flight decisions with an immediate 503, so overload degrades instead of
    ↪  hanging.",
428         "The simulator now requests every break from the ad server and reports auction ID and clearing price on it
    ↪ s events."],
429       [["Measure", "Result"], ["Clean events vs ground truth", "**187,027 = 187,027**"],
430         ["Impressions joined to their auction at the clearing price", "**44,610 / 44,610**"],
431         ["Druid spend by source vs the auction join", "identical: $922.73 direct, $750.67 programmatic"],
432         ["Latency at 227 / 542 / 808 auctions/s", "p50/p99 91/129 ms, 104/318 ms, 149/333 ms"]]),
433       ("6. Phase 4: budgets and pacing", "Pull request #5",
434        ["**Line items** have a tier (guaranteed or standard) and a budget per pacing period; guaranteed line items
    ↪  take a slot ahead of the auction.",
435         "**Pacing service:** one controller per line item, learning its yield and setting `throttle = remaining bu
    ↪ dget ÷ remaining time ÷ yield`; cuts up to 4× per tick, raises at most 2×, catches up at most at 2× the even rat
    ↪ e.",
436         "**Two clocks:** delivered spend (Flink, exact, a minute old) budgets; served spend (ad server, immediate)
    ↪  steers; a stop threshold in served spend stops a line item the moment it is spent.",
437         "**Ad server** reads pacing state from Redis once a second; if it is stale it falls back to a small fixed
    ↪ throttle and a hard stop at budget, so it can under-deliver but never overspend."],
438       [["Measure", "Result"], ["Line items over budget by > 1%", "**0** in both measured periods"],
439         ["Median delivered / budget", "96.9% and 96.4% (stops early by design)"],
440         ["Served-spend counters vs decision log", "identical: $2,406.02 and $2,530.03"],
441         ["Closed-loop simulation", "98.9–99.2% of budget delivered, worst deviation 0.8–5.9%"]]),
442       ("7. Installable app and hosted API", "Pull request #6",
443        ["**Progressive web app:** manifest, icons (including maskable and Apple touch), service worker; installs o
    ↪ n Windows, macOS, Android and iOS and opens offline. API responses are never cached.",
444         "**Sign-in:** scrypt password hashes in configuration; HMAC-signed `HttpOnly; Secure; SameSite=Strict` ses
    ↪ sion cookies; changing a password ends its sessions.",
445         "**Read-only API:** every data route is GET and requires a session; login limited to 20 failures per 15 mi
    ↪ nutes per address and 10 per user name; fails closed with 503 when misconfigured.",
446         "**Hosting:** a production overlay with Caddy for automatic HTTPS and HSTS; only ports 80 and 443 publishe
    ↪ d."],
447       [["Check", "Result"], ["Internal ports from outside", "all closed"],
448         ["Data routes without / with a session", "401 / live data"],
449         ["Form login, cross-origin login, tampered cookie, POST to data", "415, 403, 401, 405"],
450         ["21 failed logins with spoofed X-Forwarded-For", "21st refused (429): spoofing has no effect"]]),
451       ("8. Phase 5: audit and observability", "Pull request #7",
452        ["**Audit service:** every minute compares Flink's totals with Druid's per line item and minute, Redis serv
    ↪ ed spend with the decision log, delivered spend with budgets, and delivered with served; results as metrics, rec
    ↪ ords on `audit-results.v1`, and a dashboard panel naming the line items.",
453         f"**Prometheus, Alertmanager, Grafana:** {len(rules)} alert rules with {RULE_TESTS} `promtool` unit tests,
    ↪  inhibition so one cause sends one notification, and a provisioned operations dashboard.",
454         "**Alerts to phones:** Alertmanager calls the dashboard's webhook (bearer token, blocked at Caddy), which
    ↪ sends encrypted Web Push (VAPID) to every subscribed device; push endpoints restricted to the browsers' push ser
    ↪ vices.",
455         "**Found by the audit's first run:** the ad server dropped served-spend increments when Redis was slow (4,
    ↪ 744 in three minutes; 21 line items over budget). Fixed by writing absolute per-instance totals in the backgroun
    ↪ d (`HSET`), so retries can neither lose nor double-count."],
456       [["Drill", "Fault → notification on a device"], ["Stop the pacing service", "**87 s**"],
457         ["Forge $5 of spend in Redis", "**64 s**, naming the line item and both amounts"],
458         ["Kill the Flink TaskManager", "**49 s**; 2,381 minutes across the failure still identical in Flink and Dr
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
   ↪ uid"]]),
459      ("9. Phase 6: load testing and write-up", "Pull request #8",
460       ["**Load test harness** (`loadtest/run.py`): steps the simulator through rising load and records throughput
   ↪ , latency, backlog, freshness, CPU per service and audit status from Prometheus and the running system; `report.
   ↪ py` renders the results.",
461        "**Found:** time-only topic retention filled the disk at 40,000 events/s; Redpanda crashed and Flink crash
   ↪ -looped with no alert naming the cause. Fixed with size-based retention, a disk alert on free bytes, and new cri
   ↪ tical alerts for failing checkpoints and crash loops.",
462        "**Profiled:** 14% of Flink's CPU went on compressing small Kafka batches; larger batches cost no visible
   ↪ latency under exactly-once and gave 7% more throughput.",
463        "**Design document** (`docs/DESIGN.md`) and a capacity model for 10 billion events a day."],
464       [["Offered events/s", "Gateway", "Flink", "Backlog", "Visible p50"],
465        ["5,000", "4,998/s", "4,971/s", "none", "5.7 s"], ["10,000", "9,867/s", "9,873/s", "none", "6.2 s"],
466        ["20,000", "20,083/s", "20,346/s", "none", "7.7 s"], ["25,000", "24,703/s", "20,513/s", "grows", "13.0 s"]
   ↪ ,
467        ["30,000", "29,601/s", "18,218/s", "grows", "27.1 s"]]),
468  ]
469  for title, pr, items, results in PHASES:
470      story += [Paragraph(title, H1), Paragraph(pr.upper(), EYEBROW), Paragraph("What was built", H2), *bullets(it
   ↪ ems),
471                Paragraph("Measured", H2)]
472      ncol = len(results[0])
473      widths = [62 * mm, W - 62 * mm] if ncol == 2 else [W / ncol] * ncol
474      story += [table(results, widths, bold_first=True), CondPageBreak(60 * mm)]
475  story.append(PageBreak())
476
477  # 10 Testing and verification
478  story += [Paragraph("10. Testing and verification", H1),
479            P("Unit tests cover the logic; end-to-end reconciliations cover the system. Continuous integration (Gi
   ↪ tHub "
480              "Actions) runs the Maven build and tests, the dashboard type check, unit tests and builds, every Doc
   ↪ ker "
481              "image, the production overlay, the Prometheus and Alertmanager configuration and the alert-rule tes
   ↪ ts."),
482            Paragraph("Automated tests by module", H2),
483            table([["Module", "Tests"]] + [[m, str(n)] for m, n in sorted(tests.items(), key=lambda x: -x[1])]
484                  + [["**Java total**", f"**{sum(tests.values())}**"], ["dashboard (TypeScript, node:test)", "7"],
485                     ["alert rules (promtool)", str(RULE_TESTS)]],
486                  [90 * mm, 30 * mm], mono_cols=(0,)),
487            Paragraph("End-to-end reconciliations", H2),
488            table([["Compared", "Result"],
489                   ["Clean events vs simulator ground truth", "636,872 = 636,872"],
490                   ["Exactly-once through a TaskManager kill", "956,446 = 956,446"],
491                   ["Druid vs the clean topic", "identical events, types, invalid traffic and spend"],
492                   ["Impressions joined to auctions at the clearing price", "44,610 / 44,610"],
493                   ["Served-spend counters vs the decision log, per period", "identical to the cent"],
494                   ["Flink minute totals vs Druid across a TaskManager kill", "2,381 minutes identical"]],
495                  [100 * mm, W - 100 * mm]),
496            PageBreak()]
497
498  # 11 Bugs
499  story += [Paragraph("11. Defects found by measurement", H1),
500            P("None of these failed a unit test. Each was found by reconciling, profiling or deliberately breaking
   ↪  the "
501              "running system, and each is fixed."),
502            table([["#", "Defect", "Found by", "Fix"],
503                   ["1", "One minute's total revised 217 times by late events", "reconciliation", "coalescing late
   ↪  trigger (6 revisions)"],
504                   ["2", "Flink lost event time on records from broadcast state", "reconciliation", "`RestoreEvent
   ↪ Time` operator"],
505                   ["3", "A replayed run looked like data loss", "crash drill", "pass the simulator seed through"]
   ↪ ,
506                   ["4", "Ad server hung under overload (13,062 stuck connections)", "load sweep", "admission cont
   ↪ rol, fail-fast Redis pool"],
507                   ["5", "Campaign catalog never published (82k events unenriched)", "reconciliation", "fetch meta
   ↪ data first, retry until acknowledged"],
508                   ["6", "Pacing overspent up to 51% after a cold start", "live run", "catch-up cap, reset at peri
   ↪ od rollover"],
509                   ["7", "A guessed constant under-delivered by 6%", "live run", "measured serve-to-play delay (15
   ↪  s)"],
510                   ["8", "Pacing fallback could overspend by 20%", "live measurement", "stop at budget in served s
   ↪ pend"],
511                   ["9", "Served-spend increments dropped under load", "audit", "absolute per-instance totals (`HS
   ↪ ET`)"],
512                   ["10", "A stale-checkpoint page during recovery", "failure drill", "2-minute restore grace, rul
   ↪ e test"],
513                   ["11", "A full disk took the system down unannounced", "load test", "size retention, disk and c
   ↪ rash-loop alerts"],
514                   ["12", "14% of Flink CPU compressing tiny batches", "profiling", "larger producer batches (+7%)
   ↪ "]],
515                  [8 * mm, 74 * mm, 27 * mm, W - 109 * mm]),
516            PageBreak()]
517
518  # 12 Security
519  story += [Paragraph("12. Security and privacy", H1),
520            *bullets([
521                "**Device privacy:** raw device IDs never leave the gateway; consented IDs become keyed HMAC pseud
   ↪ onyms, others are dropped; rotating the key unlinks history.",
```

```
522               "**Dashboard sign-in:** scrypt hashes; HMAC-signed session cookies (`HttpOnly`, `Secure`, `SameSit
   ↪ e=Strict`, 12 h); a password change ends existing sessions.",
523               "**Brute force:** failed logins limited per client address and per user name; unknown names cost t
   ↪ he same time as known ones.",
524               "**Least exposure:** in production only Caddy (80/443) is reachable; Kafka, Flink, Druid, Redis, P
   ↪ rometheus, Alertmanager and Grafana stay on the internal network.",
525               "**Read-only API:** data routes are GET only; the only writes are sign-in, sign-out, notification
   ↪ (un)subscription and the token-protected alert webhook, which Caddy blocks from outside.",
526               "**Server-side request forgery:** push subscription endpoints are restricted to the browsers' push
   ↪  services.",
527               "**Fails closed:** sign-in on but misconfigured → the API answers 503; it never serves data unauth
   ↪ enticated.",
528               "**No data at rest on devices:** the service worker caches only the app shell, never API responses
   ↪ ."]),
529           PageBreak()]
530
531  # 13 Operations
532  story += [Paragraph("13. Running and operating", H1),
533           Paragraph("Run locally", H2),
534           Preformatted("docker compose up --build            # whole stack, about 10 GB of memory\n"
535                        "open http://localhost:3000           # dashboard (sign-in off locally)\n"
536                        "open http://localhost:3001           # Grafana operations dashboard\n"
537                        f"mvn verify                           # Java build and {sum(tests.values())} tests", CO
   ↪ DE),
538           Paragraph("Deploy with HTTPS and sign-in", H2),
539           Preformatted("cp deploy/.env.example deploy/.env    # domain, users, secrets\n"
540                        "docker compose -f docker-compose.yml -f deploy/docker-compose.prod.yml \\\n"
541                        "  --env-file deploy/.env up -d --build", CODE),
542           Paragraph("Configuration", H2)]
543  cfg = md_table("## Configuration")
544  story.append(table(cfg, [62 * mm, 40 * mm, W - 102 * mm], mono_cols=(0,)))
545  story += [Paragraph("Kafka topics", H2)]
546  topics = md_table("### Topics")
547  story.append(table(topics, [W / len(topics[0])] * len(topics[0]), mono_cols=(0,)))
548  story += [Paragraph("Service endpoints", H2),
549           table([["Service", "Endpoints"],
550                  ["Ingest gateway :8080", "`POST /v1/events`, `GET /stats`, `/metrics`, `/health`"],
551                  ["Ad server :8090", "`POST /v1/ad-requests`, `GET /stats`, `/metrics`, `/health`"],
552                  ["Pacing service :8095", "`GET /stats`, `/metrics`, `/health`"],
553                  ["Audit service :8096", "`GET /stats`, `/metrics`, `/health`"],
554                  ["Dashboard API", "`/api/health`, `/api/auth/login`, `/api/auth/logout`, `/api/auth/me`, `/api/
   ↪ gateway`, `/api/pipeline`, `/api/campaigns`, `/api/adserver`, `/api/pacing`, `/api/audit`, `/api/alerts`, `/api/
   ↪ alerts/webhook`, `/api/push/key`, `/api/push/subscribe`, `/api/push/unsubscribe`"]],
555                 [40 * mm, W - 40 * mm], bold_first=True),
556           Paragraph("Alert rules", H2),
557           table([["Alert", "Group", "Severity", "For", "Summary"]] +
558                 [[r["alert"], r["group"], r["severity"], r["for"] or "—", r["summary"]] for r in rules],
559                 [44 * mm, 24 * mm, 16 * mm, 11 * mm, W - 95 * mm], mono_cols=(0,)),
560           PageBreak()]
561
562  # 14 Capacity and pull requests
563  story += [Paragraph("14. Capacity model", H1),
564           P("Per-core rates measured at saturation and rounded down, scaled to 10 billion events a day with a 3×
   ↪  peak "
565             "(350,000 events/s, 42,000 auctions/s). An order of magnitude from one small machine, not a quote.")
   ↪ ,
566           table([["Stage", "Per core", "Cores at peak", "Also needs"],
567                  ["Ingest gateway", "50,000 events/s", "7", "stateless, behind a load balancer"],
568                  ["Kafka", "50,000 events/s", "7", "~12 TB/day before compression; tiered storage; 96+ partition
   ↪ s"],
569                  ["Flink", "7,500 events/s", "47", "630 M event IDs of de-duplication state"],
570                  ["Druid ingestion", "40,000 events/s", "9", "historicals sized by queries and retention"],
571                  ["Ad server", "~750 auctions/s", "56", "regional, close to players"],
572                  ["**Total**", "", "**~125**", "**~250 with headroom and zone redundancy**"]],
573                 [34 * mm, 30 * mm, 24 * mm, W - 88 * mm], bold_first=True),
574           Paragraph("15. Pull requests delivered", H1),
575           table([["PR", "Branch", "Scope"],
576                  ["#2", "phase-1-event-backbone", "Event backbone: gateway, simulator, Kafka topics, dashboard"]
   ↪ ,
577                  ["#3", "phase-2-stream-processing", "Flink exactly-once processing and Druid reporting"],
578                  ["#4", "phase-3-ad-serving", "Ad server with OpenRTB 2.6 auctions and mock DSPs"],
579                  ["#5", "phase-4-pacing", "Budgets, tiers and closed-loop pacing"],
580                  ["#6", "pwa-hosted-api", "Installable app, signed-in API, HTTPS deployment"],
581                  ["#7", "phase-5-observability", "Audit service, Prometheus, Alertmanager, Grafana, push alerts"
   ↪ ],
582                  ["#8", "phase-6-load-and-writeup", "Load tests, profiling, capacity model, design document"]],
583                 [12 * mm, 52 * mm, W - 64 * mm], mono_cols=(1,)),
584           P("Each pull request is stacked on the previous one and passed CI (Java build and tests, dashboard che
   ↪ cks, "
585             "Docker images, and from Phase 5 the observability checks). Merge them in order, #2 to #8.", SMALL),
586           PageBreak()]
587
588  # Appendix A: inventory
589  story += [Paragraph("Appendix A. File inventory", H1),
590           P(f"Every file in the repository except images, the npm lock file and Markdown documentation: "
591             f"{len(SOURCE)} files, {TOTAL_LINES:,} lines. The purpose is taken from each file's own documentatio
   ↪ n "
592             "comment where it has one.")]
```

```
groups = collections.OrderedDict()
for f in sorted(SOURCE):
    top = f.split("/")[0] if "/" in f else "(repository root)"
    groups.setdefault(top, []).append(f)
for top, fs in groups.items():
    rows = [["File", "Lines", "Purpose"]]
    for f in fs:
        short = f[len(top) + 1:] if f.startswith(top + "/") else f
        short = re.sub(r"^src/(main|test)/java/dev/primetech/ads/\w+/", lambda m: f"{m.group(1)}/", short)
        rows.append([short, f"{lines_of(f):,}", purpose(f)])
    story += [Paragraph(f"{top}  ({sum(lines_of(f) for f in fs):,} lines)", H2A),
              table(rows, [62 * mm, 13 * mm, W - 75 * mm], mono_cols=(0,))]
story.append(PageBreak())

# Appendix B: full source
story += [Paragraph("Appendix B. Complete source code", H1),
          P("Every source file in full, grouped by module, with line numbers. Long lines are wrapped and marked with "
            "↪. The generated Grafana dashboard and load-test result files are listed in Appendix A but not reproduced. Use the PDF bookmarks to jump to any file.")]
MAXW = 112
for top, fs in groups.items():
    story.append(Paragraph(f"{top}", H2A))
    for f in fs:
        if f in GENERATED:
            continue
        text = open(os.path.join(REPO, f), errors="replace").read().expandtabs(4)
        lines = text.splitlines()
        width = len(str(len(lines)))
        out = []
        for i, line in enumerate(lines, 1):
            first = True
            while True:
                chunk, line = line[:MAXW], line[MAXW:]
                out.append(f"{i:>{width}}  {chunk}" if first else f"{'':>{width}}↪ {chunk}")
                first = False
                if not line:
                    break
        story += [KeepTogether([Paragraph(esc(f), FILEH),
                                Paragraph(f"{len(lines):,} lines" + (f" · {esc(purpose(f))}" if purpose(f) else ""), FILEMETA)]),
                  Preformatted("\n".join(out) if out else "(empty)", CODE)]

doc = Doc(OUT)
doc.multiBuild(story)
print(f"wrote {OUT}: {len(SOURCE)} files, {TOTAL_LINES} lines, {sum(tests.values())} Java tests, {len(rules)} rules, {RULE_TESTS} rule tests")
```

event-simulator

event-simulator/pom.xml

43 lines

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.primetech.ads</groupId>
    <artifactId>ads-eventing</artifactId>
    <version>0.1.0-SNAPSHOT</version>
  </parent>

  <artifactId>event-simulator</artifactId>
  <name>Event Simulator (synthetic ad playback traffic)</name>

  <properties>
    <main.class>dev.primetech.ads.simulator.SimulatorApp</main.class>
  </properties>

  <dependencies>
    <dependency>
      <groupId>dev.primetech.ads</groupId>
      <artifactId>ads-common</artifactId>
    </dependency>
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
    </dependency>
  </dependencies>
```

```

  <build>
    <finalName>event-simulator</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-shade-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

event-simulator/src/main/java/dev/primetech/ads/simulator/AdServerPodSource.java

96 lines · Asks the ad server for each break, asynchronously, like a player starting an ad pod.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.adserving.AdRequest;
import dev.primetech.ads.common.adserving.AdResponse;

import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.util.List;
import java.util.concurrent.Executors;
import java.util.concurrent.Semaphore;
import java.util.concurrent.atomic.LongAdder;
import java.util.function.Consumer;

/**
 * Asks the ad server for each break, asynchronously, like a player starting an ad pod. A cap on
 * requests in flight stops the simulator from piling up work when the ad server falls behind:
 * over the cap, the break is skipped (and counted) rather than queued.
 */
final class AdServerPodSource implements PodSource, AutoCloseable {
    private static final int MAX_AD_SECONDS = 30;

    final LongAdder requests = new LongAdder();
    final LongAdder adsReceived = new LongAdder();
    final LongAdder failures = new LongAdder();
    final LongAdder skipped = new LongAdder();

    private final URI endpoint;
    private final HttpClient http;
    private final Semaphore inFlight;

    AdServerPodSource(String adServerUrl, int maxInFlight) {
        this.endpoint = URI.create(adServerUrl.replaceAll("/+$", "") + "/v1/ad-requests");
        this.inFlight = new Semaphore(maxInFlight);
        this.http = HttpClient.newBuilder()
                .version(HttpClient.Version.HTTP_1_1)
                .connectTimeout(Duration.ofSeconds(2))
                .executor(Executors.newVirtualThreadPerTaskExecutor())
                .build();
    }

    @Override
    public void request(SessionPlanner.Viewer v, Consumer<List<SessionPlanner.PodAd>> onPod) {
        if (!inFlight.tryAcquire()) {
            skipped.increment();
            onPod.accept(List.of());
            return;
        }
        byte[] body;
        try {
            body = Json.MAPPER.writeValueAsBytes(new AdRequest(v.sessionId(), v.deviceId(), v.consent(),
                    v.deviceType(), v.country(), v.slots(), MAX_AD_SECONDS, v.slots() * MAX_AD_SECONDS));
        } catch (IOException e) {
            inFlight.release();
            throw new IllegalStateException(e);
        }
        requests.increment();
        var request = HttpRequest.newBuilder(endpoint)
                .timeout(Duration.ofSeconds(2))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofByteArray(body))
                .build();
        http.sendAsync(request, HttpResponse.BodyHandlers.ofByteArray()).whenComplete((response, error) -> {
            inFlight.release();
            List<SessionPlanner.PodAd> pod = List.of();
            if (error == null && response.statusCode() == 200) {
                try {
                    pod = Json.MAPPER.readValue(response.body(), AdResponse.class).ads().stream()
                            .map(a -> new SessionPlanner.PodAd(a.adId(), a.creativeId(), a.campaignId(), a.lineI
```

```
  ↪ temId(),
                                    a.durationSeconds(), a.priceMicros(), a.auctionId()))
                            .toList();
                } catch (IOException e) {
                    failures.increment();
                }
            } else {
                failures.increment();
            }
            adsReceived.add(pod.size());
            onPod.accept(pod);
        });
    }

    String summary() {
        return "adRequests=%d adsReceived=%d failures=%d skipped=%d".formatted(
                requests.sum(), adsReceived.sum(), failures.sum(), skipped.sum());
    }

    @Override
    public void close() {
        http.close();
    }
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/CatalogPublisher.java

49 lines · Stands in for the campaign management system until the ad server exists (Phase 3): writes the campaign catalog to the compacted metadata topic, keyed by campaign id.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.Topics;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.serialization.ByteArraySerializer;
import org.apache.kafka.common.serialization.StringSerializer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.Duration;
import java.util.List;
import java.util.Map;
import java.util.concurrent.TimeUnit;

/**
 * Stands in for the campaign management system until the ad server exists (Phase 3): writes the
 * campaign catalog to the compacted metadata topic, keyed by campaign id. Re-publishing is
 * harmless because compaction keeps the latest value per key.
 */
final class CatalogPublisher {
    private static final Logger log = LoggerFactory.getLogger(CatalogPublisher.class);

    private CatalogPublisher() {
    }

    static void publish(String bootstrapServers, List<CampaignMetadata> campaigns) throws Exception {
        Map<String, Object> config = Map.of(
                ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers,
                ProducerConfig.CLIENT_ID_CONFIG, "simulator-catalog",
                ProducerConfig.ACKS_CONFIG, "all",
                ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
        try (var producer = new KafkaProducer<>(config, new StringSerializer(), new ByteArraySerializer())) {
            for (CampaignMetadata c : campaigns) {
                producer.send(new ProducerRecord<>(Topics.CAMPAIGN_METADATA, c.campaignId(),
                        Json.MAPPER.writeValueAsBytes(c)));
            }
            producer.flush();
            // Surface any failed send rather than starting traffic against missing metadata.
            producer.send(new ProducerRecord<>(Topics.CAMPAIGN_METADATA, campaigns.get(0).campaignId(),
                    Json.MAPPER.writeValueAsBytes(campaigns.get(0)))).get(30, TimeUnit.SECONDS);
            producer.close(Duration.ofSeconds(10));
        }
        log.info("published {} campaigns to {}", campaigns.size(), Topics.CAMPAIGN_METADATA);
    }
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/FaultInjector.java

58 lines · Makes the traffic as messy as real device traffic, so the pipeline's handling of it can be tested: retried sends (duplicates), events buffered on a device that come in late, and broken payloads.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.ClientEvent;

```

```
import java.util.random.RandomGenerator;

/**
 * Makes the traffic as messy as real device traffic, so the pipeline's handling of it can be
 * tested: retried sends (duplicates), events buffered on a device that come in late, and broken
 * payloads.
 */
final class FaultInjector {
    enum Fault { NONE, DUPLICATE, LATE, INVALID }

    private final double duplicateRate;
    private final double lateRate;
    private final double invalidRate;

    FaultInjector(double duplicateRate, double lateRate, double invalidRate) {
        if (duplicateRate + lateRate + invalidRate > 1.0) {
            throw new IllegalArgumentException("fault rates must sum to at most 1");
        }
        this.duplicateRate = duplicateRate;
        this.lateRate = lateRate;
        this.invalidRate = invalidRate;
    }

    Fault choose(RandomGenerator random) {
        double r = random.nextDouble();
        if (r < duplicateRate) {
            return Fault.DUPLICATE;
        }
        r -= duplicateRate;
        if (r < lateRate) {
            return Fault.LATE;
        }
        r -= lateRate;
        if (r < invalidRate) {
            return Fault.INVALID;
        }
        return Fault.NONE;
    }

    /** Breaks the event in one of the ways the gateway is expected to catch. */
    static ClientEvent corrupt(ClientEvent e, long nowMs, RandomGenerator random) {
        return switch (random.nextInt(4)) {
            case 0 -> copy(e, e.eventTimeMs(), "usa", e.priceMicros(), e.campaignId());
            case 1 -> copy(e, nowMs + 3_600_000L, e.country(), e.priceMicros(), e.campaignId());
            case 2 -> copy(e, e.eventTimeMs(), e.country(), e.type().isBillable() ? -5L : 7L, e.campaignId());
            default -> copy(e, e.eventTimeMs(), e.country(), e.priceMicros(), null);
        };
    }

    private static ClientEvent copy(ClientEvent e, long time, String country, Long price, String campaignId) {
        return new ClientEvent(e.eventId(), e.type(), time, e.sessionId(), e.adId(), e.creativeId(), campaignId,
                e.lineItemId(), e.deviceId(), e.deviceType(), country, e.consent(), price, e.auctionId());
    }
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/GatewayClient.java

155 lines · Sends batches the way a well-behaved player SDK would: bounded concurrency, and the whole batch retried with exponential backoff on 5xx, 429 or network errors.

```
package dev.primetech.ads.simulator;

import com.fasterxml.jackson.databind.JsonNode;
import dev.primetech.ads.common.ClientEvent;
import dev.primetech.ads.common.IngestRequest;
import dev.primetech.ads.common.Json;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.util.List;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Semaphore;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.LongAdder;

/**
 * Sends batches the way a well-behaved player SDK would: bounded concurrency, and the whole batch
 * retried with exponential backoff on 5xx, 429 or network errors. Those retries are another real
 * source of duplicates.
 */
final class GatewayClient implements AutoCloseable {
```

```
    private static final Logger log = LoggerFactory.getLogger(GatewayClient.class);
    private static final int MAX_ATTEMPTS = 5;

    final LongAdder sentEvents = new LongAdder();
    final LongAdder accepted = new LongAdder();
    final LongAdder rejected = new LongAdder();
    final LongAdder retries = new LongAdder();
    final LongAdder failedEvents = new LongAdder();
    private final ConcurrentLinkedQueue<Long> latenciesMicros = new ConcurrentLinkedQueue<>();

    private final URI endpoint;
    private final HttpClient http;
    private final Semaphore inFlight;
    private final int concurrency;
    private final ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();

    GatewayClient(String gatewayUrl, int concurrency) {
        this.endpoint = URI.create(gatewayUrl.replaceAll("/+$", "") + "/v1/events");
        this.concurrency = concurrency;
        this.inFlight = new Semaphore(concurrency);
        this.http = HttpClient.newBuilder()
                .version(HttpClient.Version.HTTP_1_1)
                .connectTimeout(Duration.ofSeconds(5))
                .executor(executor)
                .build();
    }

    /** Blocks while {@code concurrency} batches are already in flight; that is the backpressure. */
    void submit(List<ClientEvent> batch) throws InterruptedException {
        inFlight.acquire();
        executor.execute(() -> {
            try {
                sendWithRetry(batch);
            } finally {
                inFlight.release();
            }
        });
    }

    int inFlight() {
        return concurrency - inFlight.availablePermits();
    }

    /** Waits for in-flight batches to finish. */
    void drain(Duration timeout) throws InterruptedException {
        if (inFlight.tryAcquire(concurrency, timeout.toMillis(), TimeUnit.MILLISECONDS)) {
            inFlight.release(concurrency);
        }
    }

    /** p99 of request latency since the last call, in milliseconds. */
    double drainP99Millis() {
        var samples = new java.util.ArrayList<Long>();
        Long v;
        while ((v = latenciesMicros.poll()) != null) {
            samples.add(v);
        }
        if (samples.isEmpty()) {
            return 0;
        }
        samples.sort(null);
        return samples.get((int) Math.min(samples.size() - 1, Math.ceil(samples.size() * 0.99) - 1)) / 1000.0;
    }

    private void sendWithRetry(List<ClientEvent> batch) {
        byte[] body;
        try {
            body = Json.MAPPER.writeValueAsBytes(new IngestRequest(batch));
        } catch (IOException e) {
            throw new IllegalStateException(e);
        }
        var request = HttpRequest.newBuilder(endpoint)
                .timeout(Duration.ofSeconds(15))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofByteArray(body))
                .build();

        for (int attempt = 1; attempt <= MAX_ATTEMPTS; attempt++) {
            long start = System.nanoTime();
            try {
                HttpResponse<byte[]> response = http.send(request, HttpResponse.BodyHandlers.ofByteArray());
                latenciesMicros.add((System.nanoTime() - start) / 1000);
                int status = response.statusCode();
                if (status == 200) {
                    JsonNode result = Json.MAPPER.readTree(response.body());
                    sentEvents.add(batch.size());
                    accepted.add(result.get("accepted").asLong());
                    rejected.add(result.get("rejected").asLong());
```

```
                    return;
                }
                if (status != 429 && status < 500) {
                    log.warn("batch dropped, gateway answered {}: {}", status, new String(response.body()));
                    failedEvents.add(batch.size());
                    return;
                }
            } catch (IOException e) {
                log.debug("send attempt {} failed: {}", attempt, e.toString());
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                failedEvents.add(batch.size());
                return;
            }
            if (attempt < MAX_ATTEMPTS) {
                retries.increment();
                sleepBackoff(attempt);
            }
        }
        log.warn("batch of {} events dropped after {} attempts", batch.size(), MAX_ATTEMPTS);
        failedEvents.add(batch.size());
    }

    private static void sleepBackoff(int attempt) {
        try {
            // 100ms, 200ms, 400ms, 800ms, with jitter so retries from many clients spread out.
            long base = 100L << (attempt - 1);
            Thread.sleep(base + (long) (Math.random() * base));
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    @Override
    public void close() {
        executor.shutdownNow();
    }
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/GroundTruth.java

50 lines · What the simulator actually produced.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.EventType;

import java.util.EnumMap;
import java.util.Map;
import java.util.concurrent.atomic.LongAdder;

/**
 * What the simulator actually produced. After de-duplication, the pipeline's counts should equal
 * {@link #uniqueValid} (bot events included: they are valid payloads, flagged downstream as IVT).
 */
final class GroundTruth {
    final Map<EventType, LongAdder> uniqueValid = new EnumMap<>(EventType.class);
    final LongAdder billedMicros = new LongAdder();
    final LongAdder sessions = new LongAdder();
    final LongAdder duplicates = new LongAdder();
    final LongAdder late = new LongAdder();
    final LongAdder invalid = new LongAdder();
    final LongAdder botSessions = new LongAdder();
    final LongAdder botEvents = new LongAdder();
    /** Breaks for which no ad was served: nothing to play, so no events. */
    final LongAdder noFill = new LongAdder();

    GroundTruth() {
        for (EventType t : EventType.values()) {
            uniqueValid.put(t, new LongAdder());
        }
    }

    long uniqueValidTotal() {
        return uniqueValid.values().stream().mapToLong(LongAdder::sum).sum();
    }

    String summary() {
        var sb = new StringBuilder();
        sb.append("sessions=").append(sessions.sum())
                .append(" uniqueValid=").append(uniqueValidTotal())
                .append(" duplicates=").append(duplicates.sum())
                .append(" late=").append(late.sum())
                .append(" invalid=").append(invalid.sum())
                .append(" botSessions=").append(botSessions.sum())
                .append(" botEvents=").append(botEvents.sum())
                .append(" noFill=").append(noFill.sum())
                .append(" billedMicros=").append(billedMicros.sum())
                .append(" byType={");
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
        uniqueValid.forEach((t, n) -> sb.append(t).append('=').append(n.sum()).append(' '));
        return sb.append('}').toString();
    }
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/PodSource.java

14 lines · Where a viewer's ad break comes from: the local catalog, or the ad server.

```
package dev.primetech.ads.simulator;

import java.util.List;
import java.util.function.Consumer;

/** Where a viewer's ad break comes from: the local catalog, or the ad server. */
interface PodSource {

    /**
     * Asks for the ads of one break. {@code onPod} receives them (possibly none) exactly once, on
     * any thread; it must not block.
     */
    void request(SessionPlanner.Viewer viewer, Consumer<List<SessionPlanner.PodAd>> onPod);
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/Session.java

73 lines · One viewer's ad break, planned up front but emitted lazily.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.ClientEvent;
import dev.primetech.ads.common.DeviceType;
import dev.primetech.ads.common.EventType;

import java.util.UUID;
import java.util.random.RandomGenerator;

/**
 * One viewer's ad break, planned up front but emitted lazily. Only the plan (a few arrays) is
 * held in memory, so hundreds of thousands of concurrent sessions stay cheap.
 */
final class Session {
    final String sessionId;
    final String deviceId;
    final DeviceType deviceType;
    final String country;
    final boolean consent;
    final boolean bot;

    private final SessionPlanner.PodAd[] ads;
    private final EventType[] types;
    private final long[] dueMs;
    private int next;

    Session(String sessionId, String deviceId, DeviceType deviceType, String country, boolean consent,
            boolean bot, SessionPlanner.PodAd[] ads, EventType[] types, long[] dueMs) {
        this.sessionId = sessionId;
        this.deviceId = deviceId;
        this.deviceType = deviceType;
        this.country = country;
        this.consent = consent;
        this.bot = bot;
        this.ads = ads;
        this.types = types;
        this.dueMs = dueMs;
    }

    boolean hasNext() {
        return next < types.length;
    }

    long nextDueMs() {
        return dueMs[next];
    }

    int size() {
        return types.length;
    }

    /** Emits the next planned event. {@code random} only supplies the event id. */
    ClientEvent emitNext(RandomGenerator random) {
        int i = next++;
        SessionPlanner.PodAd ad = ads[i];
        EventType type = types[i];
        return new ClientEvent(
                new UUID(random.nextLong(), random.nextLong()).toString(),
                type,
                dueMs[i],
                sessionId,
                ad.adId(),
```

```
                ad.creativeId(),
                ad.campaignId(),
                ad.lineItemId(),
                deviceId,
                deviceType,
                country,
                consent,
                type.isBillable() ? ad.priceMicros() : null,
                ad.auctionId());
    }
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/SessionPlanner.java

178 lines · Plans realistic ad breaks: a pod of ads played back to back, each firing an impression then quartiles until the viewer drops off, plus an occasional click.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.DeviceType;
import dev.primetech.ads.common.EventType;
import dev.primetech.ads.common.catalog.AdCatalog;

import java.util.ArrayList;
import java.util.List;
import java.util.UUID;
import java.util.random.RandomGenerator;

/**
 * Plans realistic ad breaks: a pod of ads played back to back, each firing an impression then
 * quartiles until the viewer drops off, plus an occasional click. The pod comes either from the
 * local catalog or from an ad server response.
 */
final class SessionPlanner {
    /** Chance the viewer reaches each next quartile given they reached the previous one. */
    static final double QUARTILE_CONTINUE = 0.97;
    static final double CLICK_RATE = 0.004;

    private static final EventType[] QUARTILES = {
            EventType.FIRST_QUARTILE, EventType.MIDPOINT, EventType.THIRD_QUARTILE, EventType.COMPLETE};
    private static final DeviceType[] DEVICE_TYPES = DeviceType.values();
    private static final double[] DEVICE_WEIGHTS = {0.70, 0.15, 0.10, 0.05};
    private static final String[] COUNTRIES = {"US", "CA", "GB", "DE", "FR", "BR", "MX", "JP", "KR", "AU"};
    private static final double[] COUNTRY_WEIGHTS = {0.45, 0.06, 0.08, 0.06, 0.05, 0.08, 0.06, 0.06, 0.05, 0.05}
    ;
    private static final double CONSENT_RATE = 0.85;
    static final int BOT_POD_SIZE = 3;

    /** Who is watching: decided before the ad request, because the ad server targets on it. */
    record Viewer(String sessionId, String deviceId, DeviceType deviceType, String country, boolean consent,
                  boolean bot, int slots) {
    }

    /** One ad to play, with what the player reports on its tracking events. */
    record PodAd(String adId, String creativeId, String campaignId, String lineItemId, int durationSeconds,
                 long priceMicros, String auctionId) {

        static PodAd of(AdCatalog.AdSpec spec) {
            return new PodAd(spec.adId(), spec.creativeId(), spec.campaignId(), spec.lineItemId(),
                    spec.durationSeconds(), spec.impressionPriceMicros(), null);
        }
    }

    private final AdCatalog catalog;

    SessionPlanner(AdCatalog catalog) {
        this.catalog = catalog;
    }

    Viewer viewer(RandomGenerator random) {
        return new Viewer(
                new UUID(random.nextLong(), random.nextLong()).toString(),
                "dev-" + Long.toHexString(random.nextLong()),
                DEVICE_TYPES[weighted(DEVICE_WEIGHTS, random)],
                COUNTRIES[weighted(COUNTRY_WEIGHTS, random)],
                random.nextDouble() < CONSENT_RATE,
                false,
                1 + random.nextInt(3));
    }

    /** Bots claim consent, so their hashed device id is visible to invalid-traffic detection. */
    Viewer botViewer(String botDeviceId, RandomGenerator random) {
        return new Viewer(new UUID(random.nextLong(), random.nextLong()).toString(), botDeviceId, DeviceType.CTV
    , "US",
                true, true, BOT_POD_SIZE);
    }

    /** A pod picked from the local catalog, for running without an ad server. */
    List<PodAd> localPod(Viewer viewer, RandomGenerator random) {
```

```
        var pod = new ArrayList<PodAd>(viewer.slots());
        for (int i = 0; i < viewer.slots(); i++) {
            pod.add(PodAd.of(catalog.pick(random)));
        }
        return pod;
    }

    Session plan(long startMs, RandomGenerator random) {
        Viewer viewer = viewer(random);
        return play(viewer, localPod(viewer, random), startMs, random);
    }

    Session planBot(long startMs, String botDeviceId, RandomGenerator random) {
        Viewer viewer = botViewer(botDeviceId, random);
        return play(viewer, localPod(viewer, random), startMs, random);
    }

    /** Turns a pod into the tracking events this viewer will fire, with their due times. */
    Session play(Viewer viewer, List<PodAd> pod, long startMs, RandomGenerator random) {
        return viewer.bot() ? playBot(viewer, pod, startMs) : playHuman(viewer, pod, startMs, random);
    }

    private Session playHuman(Viewer viewer, List<PodAd> pod, long startMs, RandomGenerator random) {
        var ads = new ArrayList<PodAd>();
        var types = new ArrayList<EventType>();
        var due = new ArrayList<Long>();

        long adStart = startMs;
        for (PodAd ad : pod) {
            long durationMs = ad.durationSeconds() * 1000L;
            ads.add(ad);
            types.add(EventType.IMPRESSION);
            due.add(adStart);

            int reached = 0;
            while (reached < QUARTILES.length && random.nextDouble() < QUARTILE_CONTINUE) {
                reached++;
                ads.add(ad);
                types.add(QUARTILES[reached - 1]);
                due.add(adStart + durationMs * reached / 4);
            }
            if (random.nextDouble() < CLICK_RATE) {
                long clickAt = adStart + (long) (random.nextDouble() * durationMs * reached / 4);
                insertSorted(ads, types, due, ad, EventType.CLICK, clickAt);
            }
            // A viewer who abandons an ad leaves the break; no more ads in this pod.
            if (reached < QUARTILES.length) {
                break;
            }
            adStart += durationMs;
        }
        return new Session(viewer.sessionId(), viewer.deviceId(), viewer.deviceType(), viewer.country(),
                viewer.consent(), false, ads.toArray(PodAd[]::new), types.toArray(EventType[]::new),
                due.stream().mapToLong(Long::longValue).toArray());
    }

    /**
     * Each ad: an impression, then a "complete" half a second later (no real player finishes a
     * 15-30s ad that fast), one ad per second, always from the same device.
     */
    private static Session playBot(Viewer viewer, List<PodAd> pod, long startMs) {
        int n = pod.size();
        var ads = new PodAd[n * 2];
        var types = new EventType[n * 2];
        var due = new long[n * 2];
        for (int a = 0; a < n; a++) {
            ads[2 * a] = pod.get(a);
            ads[2 * a + 1] = pod.get(a);
            types[2 * a] = EventType.IMPRESSION;
            types[2 * a + 1] = EventType.COMPLETE;
            due[2 * a] = startMs + a * 1_000L;
            due[2 * a + 1] = startMs + a * 1_000L + 500;
        }
        return new Session(viewer.sessionId(), viewer.deviceId(), viewer.deviceType(), viewer.country(),
                viewer.consent(), true, ads, types, due);
    }

    /** Average events per session, measured by sampling, so the engine can hit a target event rate. */
    double estimateEventsPerSession(RandomGenerator random, int samples) {
        long total = 0;
        for (int i = 0; i < samples; i++) {
            total += plan(0, random).size();
        }
        return (double) total / samples;
    }

    private static void insertSorted(ArrayList<PodAd> ads, ArrayList<EventType> types,
                                     ArrayList<Long> due, PodAd ad, EventType type, long at) {
```

```
        int i = due.size();
        while (i > 0 && due.get(i - 1) > at) {
            i--;
        }
        ads.add(i, ad);
        types.add(i, type);
        due.add(i, at);
    }

    private static int weighted(double[] weights, RandomGenerator random) {
        double r = random.nextDouble();
        for (int i = 0; i < weights.length; i++) {
            r -= weights[i];
            if (r < 0) {
                return i;
            }
        }
        return weights.length - 1;
    }
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/SimulatorApp.java

85 lines · Entry point: runs the traffic simulator against the gateway and ad server.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.ClientEvent;
import dev.primetech.ads.common.catalog.AdCatalog;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.time.Duration;
import java.util.List;
import java.util.SplittableRandom;

public final class SimulatorApp {
    private static final Logger log = LoggerFactory.getLogger(SimulatorApp.class);
    private static final long TICK_MS = 20;
    private static final long REPORT_EVERY_MS = 5_000;

    public static void main(String[] args) throws Exception {
        SimulatorConfig config = SimulatorConfig.fromEnv(System.getenv());
        log.info("starting: {}", config);

        var random = new SplittableRandom(config.seed());
        var catalog = AdCatalog.generate(config.campaigns(), random);
        boolean useAdServer = config.adServerUrl() != null && !config.adServerUrl().isBlank();
        // With an ad server, it owns and announces the campaign catalog.
        if (!useAdServer && config.kafkaBootstrap() != null && !config.kafkaBootstrap().isBlank()) {
            CatalogPublisher.publish(config.kafkaBootstrap(), catalog.campaigns());
        }
        var adServer = useAdServer ? new AdServerPodSource(config.adServerUrl(), config.adRequestConcurrency())
                : null;
        var truth = new GroundTruth();
        var engine = new TrafficEngine(new SessionPlanner(catalog),
                new FaultInjector(config.duplicateRate(), config.lateRate(), config.invalidRate()),
                random, truth, config.targetEventsPerSecond(), config.lateDelayMs(),
                new TrafficEngine.Bots(config.botSessionShare(), config.botDevices()), adServer);

        try (var client = new GatewayClient(config.gatewayUrl(), config.concurrency())) {
            Runtime.getRuntime().addShutdownHook(new Thread(() -> log.info("ground truth: {}", truth.summary()))
);

            long start = System.currentTimeMillis();
            long end = config.durationSeconds() > 0 ? start + config.durationSeconds() * 1000 : Long.MAX_VALUE;
            long nextReport = start + REPORT_EVERY_MS;
            long sentAtLastReport = 0;

            while (System.currentTimeMillis() < end) {
                long now = System.currentTimeMillis();
                List<ClientEvent> due = engine.tick(now);
                for (int i = 0; i < due.size(); i += config.batchSize()) {
                    client.submit(List.copyOf(due.subList(i, Math.min(due.size(), i + config.batchSize()))));
                }
                if (now >= nextReport) {
                    long sent = client.sentEvents.sum();
                    log.info("rate={}/s accepted={} rejected={} retries={} failed={} inFlight={} activeSessions=
{} p99={}ms",
                            (sent - sentAtLastReport) * 1000 / REPORT_EVERY_MS, client.accepted.sum(),
                            client.rejected.sum(), client.retries.sum(), client.failedEvents.sum(),
                            client.inFlight(), engine.activeSessions(), client.drainP99Millis());
                    if (adServer != null) {
                        log.info("ad server: {}", adServer.summary());
                    }
                    sentAtLastReport = sent;
                    nextReport += REPORT_EVERY_MS;
                }
                long sleep = TICK_MS - (System.currentTimeMillis() - now);
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
                if (sleep > 0) {
                    Thread.sleep(sleep);
                }
            }
            // Release held-back late and duplicate copies so the ground truth matches what was sent.
            List<ClientEvent> pending = engine.drainDelayed();
            for (int i = 0; i < pending.size(); i += config.batchSize()) {
                client.submit(List.copyOf(pending.subList(i, Math.min(pending.size(), i + config.batchSize()))));
            }
            client.drain(Duration.ofSeconds(30));
            long elapsed = Math.max(1, (System.currentTimeMillis() - start) / 1000);
            log.info("done: sent={} ({}/s avg) accepted={} rejected={} retries={} failed={}",
                    client.sentEvents.sum(), client.sentEvents.sum() / elapsed, client.accepted.sum(),
                    client.rejected.sum(), client.retries.sum(), client.failedEvents.sum());
            if (adServer != null) {
                log.info("ad server: {}", adServer.summary());
                adServer.close();
            }
        }
    }

    private SimulatorApp() {
    }
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/SimulatorConfig.java

44 lines · Simulator settings: rate, faults, campaigns, seed, ad server URL.

```
package dev.primetech.ads.simulator;

import java.util.Map;

public record SimulatorConfig(
        String gatewayUrl,
        double targetEventsPerSecond,
        int batchSize,
        int concurrency,
        long durationSeconds,
        int campaigns,
        double duplicateRate,
        double lateRate,
        long lateDelayMs,
        double invalidRate,
        double botSessionShare,
        int botDevices,
        String kafkaBootstrap,
        String adServerUrl,
        int adRequestConcurrency,
        long seed) {

    public static SimulatorConfig fromEnv(Map<String, String> env) {
        return new SimulatorConfig(
                env.getOrDefault("GATEWAY_URL", "http://localhost:8080"),
                Double.parseDouble(env.getOrDefault("TARGET_EPS", "2000")),
                Integer.parseInt(env.getOrDefault("BATCH_SIZE", "200")),
                Integer.parseInt(env.getOrDefault("CONCURRENCY", "16")),
                Long.parseLong(env.getOrDefault("DURATION_SECONDS", "0")),
                Integer.parseInt(env.getOrDefault("CAMPAIGNS", "50")),
                Double.parseDouble(env.getOrDefault("DUPLICATE_RATE", "0.01")),
                Double.parseDouble(env.getOrDefault("LATE_RATE", "0.02")),
                Long.parseLong(env.getOrDefault("LATE_DELAY_MS", "30000")),
                Double.parseDouble(env.getOrDefault("INVALID_RATE", "0.005")),
                Double.parseDouble(env.getOrDefault("BOT_SESSION_SHARE", "0.01")),
                Integer.parseInt(env.getOrDefault("BOT_DEVICES", "5")),
                // Optional: where to publish the campaign catalog. Unset skips publishing.
                env.get("KAFKA_BOOTSTRAP"),
                // Optional: get each break's ads from the ad server instead of the local catalog.
                env.get("AD_SERVER_URL"),
                Integer.parseInt(env.getOrDefault("AD_REQUEST_CONCURRENCY", "256")),
                Long.parseLong(env.getOrDefault("SEED", "42")));
    }
}
```

event-simulator/src/main/java/dev/primetech/ads/simulator/TrafficEngine.java

163 lines · Drives the simulation by time.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.ClientEvent;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.random.RandomGenerator;

```

```
12  /**
13   * Drives the simulation by time. Each {@link #tick(long)} starts new sessions at the rate needed to
14   * reach the target event rate and returns every event that has come due, in event-time order
15   * (apart from injected faults). It does no I/O and takes time as an argument, so tests can run
16   * minutes of traffic in milliseconds.
17   */
18  final class TrafficEngine {
19      private record Delayed(long sendAtMs, ClientEvent event) {
20      }
21
22      private record Ready(SessionPlanner.Viewer viewer, List<SessionPlanner.PodAd> pod) {
23      }
24
25      /** Bot traffic settings: share of new sessions run by bots, and how many bot devices exist. */
26      record Bots(double sessionShare, int devices) {
27          static final Bots NONE = new Bots(0, 1);
28      }
29
30      private final SessionPlanner planner;
31      private final FaultInjector faults;
32      private final RandomGenerator random;
33      private final GroundTruth truth;
34      private final double sessionsPerMs;
35      private final long lateDelayMs;
36      private final Bots bots;
37      private final String[] botDeviceIds;
38      private final PodSource pods;
39      // Pods arrive from ad server callbacks on other threads; the engine itself stays single-threaded.
40      private final ConcurrentLinkedQueue<Ready> ready = new ConcurrentLinkedQueue<>();
41
42      private final PriorityQueue<Session> active = new PriorityQueue<>(Comparator.comparingLong(Session::nextDueM
  ↪ s));
43      private final PriorityQueue<Delayed> delayed = new PriorityQueue<>(Comparator.comparingLong(Delayed::sendAtM
  ↪ s));
44      private long lastTickMs = -1;
45      private double sessionCarry;
46
47      /** Engine that picks ads from the local catalog. */
48      TrafficEngine(SessionPlanner planner, FaultInjector faults, RandomGenerator random, GroundTruth truth,
49                    double targetEventsPerSecond, long lateDelayMs, Bots bots) {
50          this(planner, faults, random, truth, targetEventsPerSecond, lateDelayMs, bots, null);
51      }
52
53      /** @param pods where breaks get their ads; null means the local catalog */
54      TrafficEngine(SessionPlanner planner, FaultInjector faults, RandomGenerator random, GroundTruth truth,
55                    double targetEventsPerSecond, long lateDelayMs, Bots bots, PodSource pods) {
56          this.planner = planner;
57          this.pods = pods != null ? pods : (viewer, onPod) -> onPod.accept(planner.localPod(viewer, random));
58          this.bots = bots;
59          this.botDeviceIds = new String[bots.devices()];
60          for (int i = 0; i < botDeviceIds.length; i++) {
61              botDeviceIds[i] = "bot-" + Long.toHexString(random.nextLong());
62          }
63          this.faults = faults;
64          this.random = random;
65          this.truth = truth;
66          this.lateDelayMs = lateDelayMs;
67          double eventsPerSession = planner.estimateEventsPerSession(random, 2_000);
68          this.sessionsPerMs = targetEventsPerSecond / eventsPerSession / 1000.0;
69      }
70
71      List<ClientEvent> tick(long nowMs) {
72          startSessions(nowMs);
73          Ready r;
74          while ((r = ready.poll()) != null) {
75              if (r.pod().isEmpty()) {
76                  truth.noFill.increment();
77              } else {
78                  active.add(planner.play(r.viewer(), r.pod(), nowMs, random));
79              }
80          }
81          var out = new ArrayList<ClientEvent>();
82
83          while (!active.isEmpty() && active.peek().nextDueMs() <= nowMs) {
84              Session s = active.poll();
85              ClientEvent e = s.emitNext(random);
86              route(e, s.bot, nowMs, out);
87              if (s.hasNext()) {
88                  active.add(s);
89              }
90          }
91          while (!delayed.isEmpty() && delayed.peek().sendAtMs() <= nowMs) {
92              out.add(delayed.poll().event());
93          }
94          return out;
95      }
96
97      /**
```

```
     * Ends the run: late and duplicate copies still waiting are released now, so every event counted
     * in the ground truth is actually sent. Sessions still in progress simply stop, which is fine
     * because truth only counts events already emitted.
     */
    List<ClientEvent> drainDelayed() {
        var out = new ArrayList<ClientEvent>(delayed.size());
        while (!delayed.isEmpty()) {
            out.add(delayed.poll().event());
        }
        return out;
    }

    int activeSessions() {
        return active.size();
    }

    private void startSessions(long nowMs) {
        if (lastTickMs < 0) {
            lastTickMs = nowMs;
            return;
        }
        sessionCarry += (nowMs - lastTickMs) * sessionsPerMs;
        lastTickMs = nowMs;
        int toStart = (int) sessionCarry;
        sessionCarry -= toStart;
        for (int i = 0; i < toStart; i++) {
            SessionPlanner.Viewer viewer;
            if (bots.sessionShare() > 0 && random.nextDouble() < bots.sessionShare()) {
                viewer = planner.botViewer(botDeviceIds[random.nextInt(botDeviceIds.length)], random);
                truth.botSessions.increment();
            } else {
                viewer = planner.viewer(random);
            }
            truth.sessions.increment();
            pods.request(viewer, pod -> ready.add(new Ready(viewer, pod)));
        }
    }

    private void route(ClientEvent e, boolean bot, long nowMs, List<ClientEvent> out) {
        switch (faults.choose(random)) {
            case INVALID -> {
                truth.invalid.increment();
                out.add(FaultInjector.corrupt(e, nowMs, random));
                return;
            }
            case LATE -> {
                truth.late.increment();
                delayed.add(new Delayed(nowMs + lateDelayMs, e));
            }
            case DUPLICATE -> {
                // A retry after a lost response: same event id, sent again a few seconds later.
                truth.duplicates.increment();
                out.add(e);
                delayed.add(new Delayed(nowMs + 1_000 + random.nextInt(9_000), e));
            }
            case NONE -> out.add(e);
        }
        truth.uniqueValid.get(e.type()).increment();
        if (bot) {
            truth.botEvents.increment();
        }
        if (e.priceMicros() != null) {
            truth.billedMicros.add(e.priceMicros());
        }
    }
}
```

event-simulator/src/main/resources/simplelogger.properties

7 lines · The Kafka client logs its full config at INFO on startup; keep only warnings.

```
org.slf4j.simpleLogger.defaultLogLevel=info
org.slf4j.simpleLogger.showDateTime=true
org.slf4j.simpleLogger.dateTimeFormat=yyyy-MM-dd'T'HH:mm:ss.SSSZ
org.slf4j.simpleLogger.showThreadName=false
org.slf4j.simpleLogger.showShortLogName=true
# The Kafka client logs its full config at INFO on startup; keep only warnings.
org.slf4j.simpleLogger.log.org.apache.kafka=warn
```

event-simulator/src/test/java/dev/primetech/ads/simulator/SessionPlannerTest.java

97 lines · Test.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.ClientEvent;
import dev.primetech.ads.common.EventType;
import dev.primetech.ads.common.EventValidator;
import dev.primetech.ads.common.catalog.AdCatalog;
import org.junit.jupiter.api.Test;
```

```

import java.util.ArrayList;
import java.util.List;
import java.util.SplittableRandom;

import static org.assertj.core.api.Assertions.assertThat;

class SessionPlannerTest {
    private static final long START = 1_760_000_000_000L;

    private static List<ClientEvent> drain(Session s, SplittableRandom random) {
        var out = new ArrayList<ClientEvent>();
        while (s.hasNext()) {
            out.add(s.emitNext(random));
        }
        return out;
    }

    @Test
    void sessionsAreWellFormedAndValid() {
        var random = new SplittableRandom(7);
        var planner = new SessionPlanner(AdCatalog.generate(20, random));
        var validator = EventValidator.withDefaults();

        for (int i = 0; i < 500; i++) {
            List<ClientEvent> events = drain(planner.plan(START, random), random);

            assertThat(events).isNotEmpty();
            assertThat(events.get(0).type()).isEqualTo(EventType.IMPRESSION);
            assertThat(events).extracting(ClientEvent::eventTimeMs).isSorted();
            assertThat(events).extracting(ClientEvent::sessionId).containsOnly(events.get(0).sessionId());
            assertThat(events).extracting(ClientEvent::eventId).doesNotHaveDuplicates();
            assertThat(events).allSatisfy(e -> {
                assertThat(validator.validate(e, e.eventTimeMs())).as("event %s", e).isEmpty();
                assertThat(e.priceMicros() != null).isEqualTo(e.type() == EventType.IMPRESSION);
            });
        }
    }

    @Test
    void quartilesForOneAdNeverSkipAStep() {
        var random = new SplittableRandom(11);
        var planner = new SessionPlanner(AdCatalog.generate(5, random));

        for (int i = 0; i < 500; i++) {
            EventType last = null;
            for (ClientEvent e : drain(planner.plan(START, random), random)) {
                if (e.type() == EventType.CLICK) {
                    continue;
                }
                if (e.type() == EventType.IMPRESSION) {
                    assertThat(last).isIn(null, EventType.COMPLETE);
                } else {
                    assertThat(last).isNotNull();
                    assertThat(e.type().ordinal()).isEqualTo(last.ordinal() + 1);
                }
                last = e.type();
            }
        }
    }

    @Test
    void isDeterministicForASeed() {
        List<ClientEvent> a = run(99);
        List<ClientEvent> b = run(99);
        assertThat(a).isEqualTo(b);
    }

    private static List<ClientEvent> run(long seed) {
        var random = new SplittableRandom(seed);
        var planner = new SessionPlanner(AdCatalog.generate(10, random));
        return drain(planner.plan(START, random), random);
    }

    @Test
    void catalogTrafficIsSkewedTowardsTopCampaigns() {
        var random = new SplittableRandom(3);
        var catalog = AdCatalog.generate(50, random);
        int top = 0, bottom = 0;
        for (int i = 0; i < 20_000; i++) {
            String campaign = catalog.pick(random).campaignId();
            if (campaign.equals("camp-0000")) {
                top++;
            } else if (campaign.equals("camp-0049")) {
                bottom++;
            }
        }
        assertThat(top).isGreaterThan(bottom * 5);
```

```
    }
}
```

event-simulator/src/test/java/dev/primetech/ads/simulator/TrafficEngineTest.java

145 lines · Test.

```
package dev.primetech.ads.simulator;

import dev.primetech.ads.common.ClientEvent;
import dev.primetech.ads.common.EventType;
import dev.primetech.ads.common.EventValidator;
import dev.primetech.ads.common.catalog.AdCatalog;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.SplittableRandom;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.within;

class TrafficEngineTest {
    private static final long START = 1_760_000_000_000L;

    private record Run(List<ClientEvent> events, GroundTruth truth, long simulatedSeconds) {
    }

    private static Run simulate(double eps, double dup, double late, double invalid, long seconds) {
        return simulate(eps, dup, late, invalid, seconds, TrafficEngine.Bots.NONE, false);
    }

    private static Run simulate(double eps, double dup, double late, double invalid, long seconds,
                                TrafficEngine.Bots bots, boolean drainAtEnd) {
        var random = new SplittableRandom(5);
        var truth = new GroundTruth();
        var engine = new TrafficEngine(new SessionPlanner(AdCatalog.generate(20, random)),
                new FaultInjector(dup, late, invalid), random, truth, eps, 30_000, bots);
        var out = new ArrayList<ClientEvent>();
        for (long t = START; t <= START + seconds * 1000; t += 20) {
            out.addAll(engine.tick(t));
        }
        if (drainAtEnd) {
            out.addAll(engine.drainDelayed());
        }
        return new Run(out, truth, seconds);
    }

    @Test
    void drainingDelayedEventsMakesGroundTruthExact() {
        Run run = simulate(2_000, 0.02, 0.03, 0.01, 60, TrafficEngine.Bots.NONE, true);
        var validator = EventValidator.withDefaults();
        long runEnd = START + run.simulatedSeconds() * 1000;

        long uniqueValid = run.events().stream()
                .filter(e -> validator.validate(e, runEnd).isEmpty())
                .map(ClientEvent::eventId).distinct().count();
        assertThat(uniqueValid).isEqualTo(run.truth().uniqueValidTotal());
    }

    @Test
    void botsReuseAFewDevicesAtHighFrequency() {
        Run run = simulate(2_000, 0, 0, 0, 120, new TrafficEngine.Bots(0.05, 5), true);

        var botEvents = run.events().stream().filter(e -> e.deviceId().startsWith("bot-")).toList();
        assertThat(botEvents).hasSize((int) run.truth().botEvents.sum());
        assertThat(botEvents.stream().map(ClientEvent::deviceId).distinct().count()).isLessThanOrEqualTo(5);
        assertThat(botEvents).allSatisfy(e -> assertThat(e.consent()).isTrue());
        // Each bot device fires far more impressions per minute than any person could.
        long perDevicePerMinute = botEvents.stream().filter(e -> e.type() == EventType.IMPRESSION).count() / 5 /
  ↪  2;
        assertThat(perDevicePerMinute).isGreaterThan(30);
    }

    @Test
    void playsPodsFromAnAsynchronousAdSource() throws Exception {
        var random = new SplittableRandom(9);
        var truth = new GroundTruth();
        var executor = java.util.concurrent.Executors.newSingleThreadExecutor();
        var requests = new java.util.concurrent.atomic.AtomicInteger();
        // Every third break goes unfilled; the others get one ad with an auction id, answered on another thread
  ↪ .
        PodSource source = (viewer, onPod) -> executor.execute(() -> onPod.accept(requests.incrementAndGet() % 3
  ↪  == 0
                ? List.of()
                : List.of(new SessionPlanner.PodAd("ad-x", "cr-x", "camp-x", "li-x", 15, 12_345, "auction-" + vi
  ↪ ewer.sessionId())));
        var engine = new TrafficEngine(new SessionPlanner(AdCatalog.generate(5, random)), new FaultInjector(0, 0
```

```
  ↪ , 0),
                random, truth, 500, 30_000, TrafficEngine.Bots.NONE, source);

        var events = new ArrayList<ClientEvent>();
        for (long t = START; t <= START + 60_000; t += 20) {
            events.addAll(engine.tick(t));
            if (t % 1_000 == 0) {
                Thread.sleep(1); // let callbacks land
            }
        }
        executor.shutdown();

        assertThat(events).isNotEmpty().allSatisfy(e -> {
            assertThat(e.campaignId()).isEqualTo("camp-x");
            assertThat(e.auctionId()).isEqualTo("auction-" + e.sessionId());
        });
        assertThat(events).filteredOn(e -> e.type() == EventType.IMPRESSION)
                .allSatisfy(e -> assertThat(e.priceMicros()).isEqualTo(12_345L));
        assertThat(truth.noFill.sum()).isPositive();
    }

    @Test
    void hitsTargetRateInSteadyState() {
        // Sessions last up to ~90s, so measure after ramp-up.
        Run run = simulate(5_000, 0, 0, 0, 300);
        long steadyFrom = START + 120_000;
        long steady = run.events().stream().filter(e -> e.eventTimeMs() >= steadyFrom).count();

        assertThat(steady / 180.0).isCloseTo(5_000, within(500.0));
    }

    @Test
    void groundTruthMatchesUniqueValidEventsSent() {
        Run run = simulate(2_000, 0.02, 0.03, 0.01, 120);
        var validator = EventValidator.withDefaults();

        var uniqueValidIds = new HashSet<String>();
        long invalid = 0, duplicates = 0;
        long runEnd = START + run.simulatedSeconds() * 1000;
        for (ClientEvent e : run.events()) {
            // Injected future events are an hour ahead of emission, so still in the future at run end.
            if (validator.validate(e, runEnd).isPresent()) {
                invalid++;
            } else if (!uniqueValidIds.add(e.eventId())) {
                duplicates++;
            }
        }
        // Delayed copies scheduled after the run ended are not in the output yet.
        assertThat(uniqueValidIds.size()).isLessThanOrEqualTo((int) run.truth().uniqueValidTotal())
                .isGreaterThan((int) (run.truth().uniqueValidTotal() * 0.99));
        assertThat(invalid).isEqualTo(run.truth().invalid.sum());
        assertThat(duplicates).isLessThanOrEqualTo(run.truth().duplicates.sum()).isPositive();
    }

    @Test
    void lateEventsArriveAfterNewerEvents() {
        Run run = simulate(1_000, 0, 0.05, 0, 90);
        long maxSeen = 0;
        long outOfOrderBy30s = 0;
        for (ClientEvent e : run.events()) {
            if (e.eventTimeMs() < maxSeen - 25_000) {
                outOfOrderBy30s++;
            }
            maxSeen = Math.max(maxSeen, e.eventTimeMs());
        }
        assertThat(outOfOrderBy30s).isPositive();
    }
}
```

infra

infra/alertmanager/alertmanager.yml

37 lines · Alerts go to the dashboard, which shows them and pushes them to every device that installed

```
# Alerts go to the dashboard, which shows them and pushes them to every device that installed
# the app and turned notifications on. Add email, Slack or PagerDuty receivers alongside.
route:
  receiver: dashboard
  group_by: [alertname, check, job]
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 6h
  routes:
    - matchers: [severity="critical"]
      receiver: dashboard
      repeat_interval: 1h
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```

receivers:
  - name: dashboard
    webhook_configs:
      - url: http://dashboard:3000/api/alerts/webhook
        send_resolved: true
        http_config:
          authorization:
            type: Bearer
            credentials_file: /tmp/webhook-token

# One cause, one notification: when the cause is already firing, hold back its symptoms.
inhibit_rules:
  - source_matchers: [alertname="FlinkJobCrashLooping"]
    target_matchers: [alertname=~"FlinkJobRestarting|FlinkCheckpointsStalled|DeliveryDataStale|FlinkConsumerLagH
↪ igh"]
  - source_matchers: [alertname="KafkaDiskLow"]
    target_matchers: [alertname=~"FlinkCheckpointsFailing|FlinkJobCrashLooping|FlinkJobRestarting"]
  - source_matchers: [alertname="FlinkJobNotRunning"]
    target_matchers: [alertname=~"FlinkCheckpointsStalled|FlinkConsumerLagHigh|DeliveryDataStale|FlinkJobRestart
↪ ing"]
  - source_matchers: [alertname="ServiceDown", job="pacing-service"]
    target_matchers: [alertname=~"PacingStateStale|AdServerOnPacingFallback"]
  - source_matchers: [alertname="ServiceDown", job="flink"]
    target_matchers: [alertname=~"FlinkCheckpointsStalled|FlinkConsumerLagHigh|DeliveryDataStale"]
  - source_matchers: [alertname="KafkaPartitionsUnavailable"]
    target_matchers: [alertname="IngestPublishFailures"]
```

infra/create-topics.sh

44 lines · Creates the event topics. Safe to re-run: existing topics are left alone.

```
#!/usr/bin/env bash
# Creates the event topics. Safe to re-run: existing topics are left alone.
set -euo pipefail

BROKERS="${BROKERS:-redpanda:9092}"

# create <topic> <partitions> [topic-config ...]
create() {
  local topic="$1" partitions="$2"
  shift 2
  if rpk topic describe "$topic" -X brokers="$BROKERS" >/dev/null 2>&1; then
    echo "topic $topic already exists"
    return
  fi
  local args=()
  for c in "$@"; do args+=(--topic-config "$c"); done
  rpk topic create "$topic" -X brokers="$BROKERS" --partitions "$partitions" "${args[@]}"
}

WEEK=604800000
TWO_WEEKS=1209600000
# Retention is by time AND size, whichever comes first. Time alone filled the disk in the Phase 6
# load test (40k events/s writes several GB a minute across the event topics) and Redpanda and
# Flink's checkpoints both failed. Size per partition; the default suits a single development
# machine. Size the disks so that time, not size, is what normally applies.
PARTITION_BYTES="${TOPIC_PARTITION_RETENTION_BYTES:-268435456}"

# 12 partitions lets the stream processor scale to 12 parallel readers.
create ad-events.v1 12 retention.ms=$WEEK retention.bytes=$PARTITION_BYTES compression.type=producer
# Bad payloads, kept longer so they can be investigated.
create ad-events-dlq.v1 3 retention.ms=$TWO_WEEKS retention.bytes=$PARTITION_BYTES compression.type=producer
# Reference data: compaction keeps the latest record per campaign forever.
create campaign-metadata.v1 1 cleanup.policy=compact
# Output of the stream processor: what reporting and billing read.
create ad-events-clean.v1 12 retention.ms=$WEEK retention.bytes=$PARTITION_BYTES compression.type=producer
# Upsert stream of minute totals: compaction keeps the latest revision of each line item minute,
# deletion drops minutes older than a week.
create line-item-minutes.v1 6 cleanup.policy=compact,delete retention.ms=$WEEK
# Ad server audit log: one record per ad request, joined to events by auction id.
create ad-decisions.v1 12 retention.ms=$WEEK retention.bytes=$PARTITION_BYTES compression.type=producer
# Direct line item terms (tier, budget per period), read by the pacing service.
create line-items.v1 1 cleanup.policy=compact
# One record per audit check run (reconciliations between independent sources), kept 30 days.
create audit-results.v1 1 retention.ms=2592000000
```

infra/druid/ad-events-supervisor.json

54 lines

```
{
  "type": "kafka",
  "spec": {
    "dataSchema": {
      "dataSource": "ad_events",
      "timestampSpec": { "column": "eventTimeMs", "format": "millis" },
      "dimensionsSpec": {
        "dimensions": [
          "type",
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
            "advertiserId",
            "advertiserName",
            "campaignId",
            "campaignName",
            "lineItemId",
            "creativeId",
            "deviceType",
            "country",
            { "type": "long", "name": "ivt" },
            "ivtReason",
            { "type": "long", "name": "consent" }
          ]
        },
        "metricsSpec": [
          { "type": "count", "name": "events" },
          { "type": "longSum", "name": "priceMicros", "fieldName": "priceMicros" },
          { "type": "HLLSketchBuild", "name": "devices", "fieldName": "deviceIdHash", "lgK": 12 }
        ],
        "granularitySpec": {
          "segmentGranularity": "hour",
          "queryGranularity": "minute",
          "rollup": true
        }
      },
      "ioConfig": {
        "type": "kafka",
        "topic": "ad-events-clean.v1",
        "inputFormat": { "type": "json" },
        "consumerProperties": {
          "bootstrap.servers": "redpanda:9092",
          "isolation.level": "read_committed"
        },
        "useEarliestOffset": true,
        "taskCount": 1,
        "replicas": 1,
        "taskDuration": "PT15M",
        "completionTimeout": "PT20M"
      },
      "tuningConfig": {
        "type": "kafka",
        "maxRowsInMemory": 100000,
        "maxRowsPerSegment": 5000000
      }
    }
}
```

infra/druid/druid.env

21 lines · Shared settings for every Druid service (based on Druid's own docker-compose example),

```
# Shared settings for every Druid service (based on Druid's own docker-compose example),
# trimmed so the whole cluster fits in ~3 GB next to the rest of the stack.
druid_extensions_loadList=["druid-datasketches", "postgresql-metadata-storage", "druid-multi-stage-query", "drui
↪ d-kafka-indexing-service"]
druid_zk_service_host=zookeeper
druid_metadata_storage_type=postgresql
druid_metadata_storage_connector_connectURI=jdbc:postgresql://postgres:5432/druid
druid_metadata_storage_connector_user=druid
druid_metadata_storage_connector_password=druid-local-only
druid_storage_type=local
druid_storage_storageDirectory=/opt/shared/segments
druid_indexer_logs_type=file
druid_indexer_logs_directory=/opt/shared/indexing-logs
druid_processing_numThreads=2
druid_processing_numMergeBuffers=2
druid_processing_buffer_sizeBytes=64MiB
druid_indexer_runner_javaOptsArray=["-server", "-Xms512m", "-Xmx512m", "-XX:MaxDirectMemorySize=512m", "-Duser.t
↪ imezone=UTC", "-Dfile.encoding=UTF-8", "-Djava.util.logging.manager=org.apache.logging.log4j.jul.LogManager"]
druid_indexer_fork_property_druid_processing_buffer_sizeBytes=64MiB
druid_worker_capacity=2
druid_emitter=noop
druid_sql_enable=true
DRUID_LOG4J=<?xml version="1.0" encoding="UTF-8" ?><Configuration status="WARN"><Appenders><Console name="Consol
↪ e" target="SYSTEM_OUT"><PatternLayout pattern="%d{ISO8601} %p [%t] %c - %m%n"/></Console></Appenders><Loggers><R
↪ oot level="warn"><AppenderRef ref="Console"/></Root><Logger name="org.apache.druid.server.initialization.jetty.J
↪ ettyServerModule" level="info" additivity="false"><AppenderRef ref="Console"/></Logger></Loggers></Configuration
↪ >
```

infra/druid/submit-supervisor.sh

15 lines · Registers the Kafka ingestion supervisor once the Druid router answers. Re-submitting the same

```
#!/bin/sh
# Registers the Kafka ingestion supervisor once the Druid router answers. Re-submitting the same
# spec is safe: Druid updates the existing supervisor for the datasource.
set -eu
ROUTER="${ROUTER:-http://router:8888}"

echo "waiting for Druid at $ROUTER"
until wget -q -O /dev/null "$ROUTER/status/health"; do sleep 3; done
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
# The overlord can come up after the router; retry until the spec is accepted.
until wget -q -O - --header='Content-Type: application/json' \
    --post-file=/specs/ad-events-supervisor.json "$ROUTER/druid/indexer/v1/supervisor"; do
  sleep 5
done
echo
echo "supervisor for ad_events submitted"
```

infra/grafana/provisioning/dashboards/dashboards.yml

7 lines

```
apiVersion: 1
providers:
  - name: ads-eventing
    folder: Ads Eventing
    type: file
    allowUiUpdates: false
    options: { path: /var/lib/grafana/dashboards }
```

infra/grafana/provisioning/datasources/datasources.yml

15 lines

```
apiVersion: 1
datasources:
  - name: Prometheus
    uid: prometheus
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    jsonData: { timeInterval: 15s }
  - name: Alertmanager
    uid: alertmanager
    type: alertmanager
    access: proxy
    url: http://alertmanager:9093
    jsonData: { implementation: prometheus, handleGrafanaManagedAlerts: false }
```

infra/prometheus/prometheus.yml

31 lines

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/rules/*.yml

alerting:
  alertmanagers:
    - static_configs:
        - targets: ["alertmanager:9093"]

scrape_configs:
  - job_name: gateway
    static_configs: [{ targets: ["gateway:8080"] }]
  - job_name: ad-server
    static_configs: [{ targets: ["ad-server:8090"] }]
  - job_name: pacing-service
    static_configs: [{ targets: ["pacing-service:8095"] }]
  - job_name: audit-service
    static_configs: [{ targets: ["audit-service:8096"] }]
  - job_name: flink
    # Flink's Prometheus reporter (metrics.reporter.prom in docker-compose.yml).
    static_configs: [{ targets: ["jobmanager:9249", "taskmanager:9249"] }]
  - job_name: redpanda
    metrics_path: /public_metrics
    static_configs: [{ targets: ["redpanda:9644"] }]
  - job_name: prometheus
    static_configs: [{ targets: ["localhost:9090"] }]
  - job_name: alertmanager
    static_configs: [{ targets: ["alertmanager:9093"] }]
```

infra/prometheus/rules/ads-eventing.yml

172 lines · Alerting rules. Each alert names what is wrong, what it costs, and where to look first.

```
# Alerting rules. Each alert names what is wrong, what it costs, and where to look first.
# severity: critical = money or data is at risk now (page); warning = degraded, act soon.
groups:
  - name: availability
    rules:
      - alert: ServiceDown
        expr: up{job=~"gateway|ad-server|pacing-service|audit-service|flink|redpanda"} == 0
        for: 1m
        labels: { severity: critical }
        annotations:
          summary: "{{ $labels.job }} is down"
          description: "Prometheus cannot scrape {{ $labels.instance }} for 1 minute."
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
13
14    - name: ingest
15      rules:
16        - alert: IngestPublishFailures
17          expr: increase(ads_ingest_publish_failures_total[5m]) > 0
18          labels: { severity: critical }
19          annotations:
20            summary: Gateway cannot write events to Kafka
21            description: "{{ $value | humanize }} batches were refused with 503 in 5 minutes. Players retry, but e
  ↪ vents may be delayed. Check Redpanda."
22        - alert: IngestRejectionRateHigh
23          expr: |
24            sum(rate(ads_ingest_events_rejected_total[5m]))
25              / (sum(rate(ads_ingest_events_accepted_total[5m])) + sum(rate(ads_ingest_events_rejected_total[5m]))
  ↪ ) > 0.05
26          for: 5m
27          labels: { severity: warning }
28          annotations:
29            summary: "{{ $value | humanizePercentage }} of incoming events are rejected"
30            description: Usually a client release sending malformed events. The dead-letter topic ad-events-dlq.v1
  ↪  has the details.
31
32    - name: ad-serving
33      rules:
34        - alert: AdDecisionLatencyHigh
35          expr: histogram_quantile(0.99, sum by (le) (rate(adserver_decision_latency_seconds_bucket[5m]))) > 0.3
36          for: 5m
37          labels: { severity: warning }
38          annotations:
39            summary: "Ad decision p99 is {{ $value | humanizeDuration }}"
40            description: Players time out waiting for ads above ~500 ms and show the break empty.
41        - alert: AdServerShedding
42          expr: sum(rate(adserver_requests_shed_total[5m])) / sum(rate(adserver_requests_total[5m])) > 0.02
43          for: 5m
44          labels: { severity: warning }
45          annotations:
46            summary: "Ad server rejects {{ $value | humanizePercentage }} of requests"
47            description: Admission control is shedding load (503). Add capacity or find what made decisions slower
  ↪ .
48        - alert: FillRateLow
49          expr: sum(rate(adserver_slots_filled_total[10m])) / sum(rate(adserver_slots_requested_total[10m])) < 0.7
50          for: 10m
51          labels: { severity: warning }
52          annotations:
53            summary: "Fill rate {{ $value | humanizePercentage }}"
54            description: Breaks go unfilled. Check DSP timeouts and errors, and direct line items' pacing.
55
56    - name: stream-processing
57      rules:
58        - alert: FlinkJobNotRunning
59          expr: max(flink_jobmanager_numRunningJobs) < 1 or absent(flink_jobmanager_numRunningJobs)
60          for: 1m
61          labels: { severity: critical }
62          annotations:
63            summary: The stream processing job is not running
64            description: No clean events, no minute totals, so pacing works from stale delivery and reporting stop
  ↪ s.
65        - alert: FlinkJobRestarting
66          expr: increase(flink_jobmanager_job_numRestarts[10m]) > 0
67          labels: { severity: warning }
68          annotations:
69            summary: "The Flink job restarted {{ $value | printf \"%.0f\" }} times in 10 minutes"
70            description: Each restart replays from the last checkpoint (exactly-once holds) but delays output. Che
  ↪ ck the TaskManager logs.
71        - alert: FlinkJobCrashLooping
72          expr: increase(flink_jobmanager_job_numRestarts[10m]) >= 3
73          labels: { severity: critical }
74          annotations:
75            summary: "The Flink job is crash-looping ({{ $value | printf \"%.0f\" }} restarts in 10 minutes)"
76            description: Restarts are not recovering it. The JobManager's exception history names the cause; in th
  ↪ e Phase 6 load test it was a full disk.
77        - alert: FlinkCheckpointsFailing
78          # Independent of job uptime: a crash-looping job never runs long enough for
79          # FlinkCheckpointsStalled, which is how a full disk went unnamed in the Phase 6 load test.
80          expr: increase(flink_jobmanager_job_numberOfFailedCheckpoints[5m]) >= 2
81          labels: { severity: critical }
82          annotations:
83            summary: "{{ $value | printf \"%.0f\" }} Flink checkpoints failed in 5 minutes"
84            description: Output is committed on checkpoints, and the job restarts after repeated failures. Usual c
  ↪ auses are local disk, the checkpoint store, or a checkpoint timing out under backpressure.
85        - alert: FlinkCheckpointsStalled
86          # Only for a job that has been running 2 minutes: one that just restarted is still restoring
87          # (the outage itself is ServiceDown or FlinkJobRestarting). Without this, drill 3 sent a
88          # stale-checkpoint page during recovery, after the cause had already resolved.
89          expr: |
90            time() - max(flink_jobmanager_job_lastCheckpointCompletedTimestamp) / 1000 > 120
91              and on () max(flink_jobmanager_job_uptime) > 120000
92          for: 1m
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
 93          labels: { severity: critical }
 94          annotations:
 95            summary: "No completed checkpoint for {{ $value | humanizeDuration }}"
 96            description: Transactional output is committed on checkpoints, so downstream consumers see nothing new
   ↪  until one completes.
 97        - alert: FlinkConsumerLagHigh
 98          expr: sum(flink_taskmanager_job_task_operator_pendingRecords{operator_name=~".*ad_events.*"}) > 200000
 99          for: 5m
100          labels: { severity: warning }
101          annotations:
102            summary: "{{ $value | humanize }} events waiting for the stream processor"
103            description: The job is falling behind its input; delivery data for pacing is getting older.
104
105    - name: pacing
106      rules:
107        - alert: PacingStateStale
108          expr: pacing_publish_age_seconds > 60
109          for: 30s
110          labels: { severity: critical }
111          annotations:
112            summary: "Pacing state not updated for {{ $value | humanizeDuration }}"
113            description: The ad server has fallen back to a fixed throttle and budget-only stop. It cannot overspe
   ↪ nd, but it under-delivers. Check Redis and the pacing service.
114        - alert: DeliveryDataStale
115          expr: pacing_delivery_lag_seconds > 300
116          for: 2m
117          labels: { severity: warning }
118          annotations:
119            summary: "Delivered spend is {{ $value | humanizeDuration }} behind"
120            description: Pacing steers on served spend alone while Flink output is late. Check the Flink job.
121        - alert: LineItemOverBudget
122          expr: pacing_overspent_line_items > 0
123          for: 1m
124          labels: { severity: critical }
125          annotations:
126            summary: "{{ $value }} line item(s) more than 1% over budget"
127            description: Overspend is billed to nobody. Check the pacing panel; the audit's budget-compliance chec
   ↪ k confirms against Druid.
128        - alert: AdServerOnPacingFallback
129          expr: sum(rate(adserver_pacing_fallback_total[5m])) > 0
130          for: 5m
131          labels: { severity: warning }
132          annotations:
133            summary: The ad server is pacing on its fallback
134            description: Some line items have no fresh pacing state. Check that the pacing service is running and
   ↪ writing to Redis.
135
136    - name: audit
137      rules:
138        - alert: AuditMismatch
139          expr: audit_check_ok == 0
140          labels: { severity: critical }
141          annotations:
142            summary: "Audit check {{ $labels.check }} found a mismatch"
143            description: Two independent records of the same money disagree. The audit panel and audit-results.v1
   ↪ list the line items.
144        - alert: AuditNotCompleting
145          expr: time() - audit_check_last_completed_seconds > 1200
146          for: 5m
147          labels: { severity: warning }
148          annotations:
149            summary: "Audit check {{ $labels.check }} has not completed for {{ $value | humanizeDuration }}"
150            description: A source it reads (Druid, Redis, Kafka) is unavailable, so nothing is being verified.
151
152    - name: kafka
153      rules:
154        - alert: KafkaPartitionsUnavailable
155          expr: max(redpanda_cluster_unavailable_partitions) > 0
156          for: 1m
157          labels: { severity: critical }
158          annotations:
159            summary: "{{ $value }} Kafka partitions unavailable"
160            description: Writes to those partitions fail; the gateway answers 503.
161        - alert: KafkaDiskLow
162          # Redpanda's own flag never tripped before it crashed on a full disk in the Phase 6 load
163          # test, so the free space is checked directly as well.
164          expr: |
165            min(redpanda_storage_disk_free_bytes) < 5368709120
166              or min(redpanda_storage_disk_free_bytes / redpanda_storage_disk_total_bytes) < 0.10
167              or max(redpanda_storage_disk_free_space_alert) > 0
168          for: 2m
169          labels: { severity: critical }
170          annotations:
171            summary: Kafka (Redpanda) is running out of disk
172            description: On a full disk Redpanda stops, and Flink's local state cannot checkpoint either. Check to
   ↪ pic retention (time and size) and add storage.
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

infra/prometheus/tests/ads-eventing-test.yml

193 lines · promtool test rules infra/prometheus/tests/ads-eventing-test.yml (run in CI)

```
# promtool test rules infra/prometheus/tests/ads-eventing-test.yml (run in CI)
rule_files:
  - ../rules/ads-eventing.yml
evaluation_interval: 15s

tests:
  - name: pacing state stops updating
    interval: 15s
    input_series:
      # Healthy for 2 minutes (age stays under 10 s), then Redis goes away and the age grows.
      - series: pacing_publish_age_seconds{job="pacing-service"}
        values: "5 5 5 5 5 5 5 5 20 35 50 65 80 95 110"
    alert_rule_test:
      - eval_time: 2m
        alertname: PacingStateStale
        exp_alerts: []
      - eval_time: 3m30s
        alertname: PacingStateStale
        exp_alerts:
          - exp_labels: { severity: critical, job: pacing-service }
            exp_annotations:
              summary: "Pacing state not updated for 1m 50s"
              description: The ad server has fallen back to a fixed throttle and budget-only stop. It cannot ove
↪ rspend, but it under-delivers. Check Redis and the pacing service.

  - name: an audit mismatch pages at once, and a healthy check does not
    interval: 1m
    input_series:
      - series: audit_check_ok{check="served-vs-decision-log"}
        values: "1 1 0 0"
      - series: audit_check_ok{check="flink-vs-druid"}
        values: "1 1 1 1"
    alert_rule_test:
      - eval_time: 1m
        alertname: AuditMismatch
        exp_alerts: []
      - eval_time: 2m
        alertname: AuditMismatch
        exp_alerts:
          - exp_labels: { severity: critical, check: served-vs-decision-log }
            exp_annotations:
              summary: "Audit check served-vs-decision-log found a mismatch"
              description: Two independent records of the same money disagree. The audit panel and audit-results
↪ .v1 list the line items.

  - name: a pending audit (NaN) is not a mismatch
    interval: 1m
    input_series:
      - series: audit_check_ok{check="budget-compliance"}
        values: "NaN NaN NaN"
    alert_rule_test:
      - eval_time: 2m
        alertname: AuditMismatch
        exp_alerts: []

  - name: checkpoints stall
    interval: 30s
    input_series:
      # The last completed checkpoint stays at t=0 (ms); time() moves on.
      - series: flink_jobmanager_job_lastCheckpointCompletedTimestamp{job="flink"}
        values: "0x20"
      # The job has been running for a long time (ms).
      - series: flink_jobmanager_job_uptime{job="flink"}
        values: "600000+30000x20"
    alert_rule_test:
      - eval_time: 2m
        alertname: FlinkCheckpointsStalled
        exp_alerts: []
      - eval_time: 3m30s
        alertname: FlinkCheckpointsStalled
        exp_alerts:
          - exp_labels: { severity: critical }
            exp_annotations:
              summary: "No completed checkpoint for 3m 30s"
              description: Transactional output is committed on checkpoints, so downstream consumers see nothing
↪  new until one completes.

  - name: the stream job disappearing entirely still alerts
    interval: 15s
    input_series:
      - series: up{job="gateway"}
        values: "1x10"
    alert_rule_test:
      - eval_time: 2m
        alertname: FlinkJobNotRunning
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
        exp_alerts:
          - exp_labels: { severity: critical }
            exp_annotations:
              summary: The stream processing job is not running
              description: No clean events, no minute totals, so pacing works from stale delivery and reporting
  ↪ stops.

  - name: rejection rate threshold
    interval: 1m
    input_series:
      # 1,000 events/min accepted; rejections at 2% then 10%.
      - series: ads_ingest_events_accepted_total
        values: "0+1000x20"
      - series: ads_ingest_events_rejected_total
        values: "0+20x10 200+100x10"
    alert_rule_test:
      - eval_time: 9m
        alertname: IngestRejectionRateHigh
        exp_alerts: []
      - eval_time: 20m
        alertname: IngestRejectionRateHigh
        exp_alerts:
          - exp_labels: { severity: warning }
            exp_annotations:
              summary: "9.091% of incoming events are rejected"
              description: Usually a client release sending malformed events. The dead-letter topic ad-events-dl
  ↪ q.v1 has the details.

  - name: a job that just restarted gets time to restore before checkpoints count as stalled
    interval: 30s
    input_series:
      - series: flink_jobmanager_job_lastCheckpointCompletedTimestamp{job="flink"}
        values: "0x20"
      # Restarted at t=0: uptime grows from zero and passes 2 minutes at 2m.
      - series: flink_jobmanager_job_uptime{job="flink"}
        values: "0+30000x20"
    alert_rule_test:
      - eval_time: 2m30s
        alertname: FlinkCheckpointsStalled
        exp_alerts: []
      - eval_time: 3m30s
        alertname: FlinkCheckpointsStalled
        exp_alerts:
          - exp_labels: { severity: critical }
            exp_annotations:
              summary: "No completed checkpoint for 3m 30s"
              description: Transactional output is committed on checkpoints, so downstream consumers see nothing
  ↪  new until one completes.

  - name: restart counts are whole numbers
    interval: 1m
    input_series:
      - series: flink_jobmanager_job_numRestarts{job="flink"}
        values: "0 0 1 2 3 3 3"
    alert_rule_test:
      - eval_time: 5m
        alertname: FlinkJobRestarting
        exp_alerts:
          - exp_labels: { severity: warning, job: flink }
            exp_annotations:
              summary: "The Flink job restarted 3 times in 10 minutes"
              description: Each restart replays from the last checkpoint (exactly-once holds) but delays output.
  ↪  Check the TaskManager logs.

  - name: a crash loop from a full disk is named, not just a restart warning
    interval: 1m
    input_series:
      # Restarting every minute, so uptime never reaches the 2 minute grace of FlinkCheckpointsStalled.
      - series: flink_jobmanager_job_numRestarts{job="flink"}
        values: "0 1 2 3 4 5 6"
      - series: flink_jobmanager_job_numberOfFailedCheckpoints{job="flink"}
        values: "0 1 2 3 4 5 6"
      - series: flink_jobmanager_job_uptime{job="flink"}
        values: "30000x6"
      - series: flink_jobmanager_job_lastCheckpointCompletedTimestamp{job="flink"}
        values: "0x6"
    alert_rule_test:
      - eval_time: 5m
        alertname: FlinkCheckpointsStalled
        exp_alerts: []
      - eval_time: 5m
        alertname: FlinkCheckpointsFailing
        exp_alerts:
          - exp_labels: { severity: critical, job: flink }
            exp_annotations:
              summary: "5 Flink checkpoints failed in 5 minutes"
              description: Output is committed on checkpoints, and the job restarts after repeated failures. Usu
  ↪ al causes are local disk, the checkpoint store, or a checkpoint timing out under backpressure.
```

```
      - eval_time: 5m
        alertname: FlinkJobCrashLooping
        exp_alerts:
          - exp_labels: { severity: critical, job: flink }
            exp_annotations:
              summary: "The Flink job is crash-looping (5 restarts in 10 minutes)"
              description: Restarts are not recovering it. The JobManager's exception history names the cause; i
  ↪ n the Phase 6 load test it was a full disk.

  - name: low Kafka disk by absolute free space, even when Redpanda's own flag is off
    interval: 1m
    input_series:
      - series: redpanda_storage_disk_free_bytes{job="redpanda"}
        values: "20000000000 20000000000 3000000000 3000000000 3000000000 3000000000"
      - series: redpanda_storage_disk_total_bytes{job="redpanda"}
        values: "250000000000x5"
      - series: redpanda_storage_disk_free_space_alert{job="redpanda"}
        values: "0x5"
    alert_rule_test:
      - eval_time: 1m
        alertname: KafkaDiskLow
        exp_alerts: []
      - eval_time: 5m
        alertname: KafkaDiskLow
        exp_alerts:
          - exp_labels: { severity: critical }
            exp_annotations:
              summary: Kafka (Redpanda) is running out of disk
              description: On a full disk Redpanda stops, and Flink's local state cannot checkpoint either. Chec
  ↪ k topic retention (time and size) and add storage.
```

ingest-gateway

ingest-gateway/pom.xml

47 lines

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.primetech.ads</groupId>
    <artifactId>ads-eventing</artifactId>
    <version>0.1.0-SNAPSHOT</version>
  </parent>

  <artifactId>ingest-gateway</artifactId>
  <name>Ingest Gateway</name>

  <properties>
    <main.class>dev.primetech.ads.gateway.GatewayApp</main.class>
  </properties>

  <dependencies>
    <dependency>
      <groupId>dev.primetech.ads</groupId>
      <artifactId>ads-common</artifactId>
    </dependency>
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
    </dependency>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
    </dependency>
  </dependencies>

  <build>
    <finalName>ingest-gateway</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-shade-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

ingest-gateway/src/main/java/dev/primetech/ads/gateway/EventSink.java

18 lines · Where the gateway puts events.

```java
package dev.primetech.ads.gateway;

import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.DeadLetter;

import java.util.concurrent.CompletableFuture;

/** Where the gateway puts events. Completes once the write is durable (Kafka acks=all). */
public interface EventSink extends AutoCloseable {

    CompletableFuture<Void> publish(AdEvent event);

    CompletableFuture<Void> publishDeadLetter(DeadLetter deadLetter);

    @Override
    default void close() {
    }
}
```

ingest-gateway/src/main/java/dev/primetech/ads/gateway/GatewayApp.java

43 lines · Entry point: starts the ingest gateway HTTP server and Kafka producer.

```java
package dev.primetech.ads.gateway;

import dev.primetech.ads.common.DeviceIdHasher;
import dev.primetech.ads.common.EventNormalizer;
import dev.primetech.ads.common.EventValidator;
import io.micrometer.core.instrument.binder.jvm.JvmGcMetrics;
import io.micrometer.core.instrument.binder.jvm.JvmMemoryMetrics;
import io.micrometer.core.instrument.binder.jvm.JvmThreadMetrics;
import io.micrometer.core.instrument.binder.system.ProcessorMetrics;
import io.micrometer.prometheusmetrics.PrometheusConfig;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;

public final class GatewayApp {

    public static void main(String[] args) throws Exception {
        GatewayConfig config = GatewayConfig.fromEnv(System.getenv());

        var registry = new PrometheusMeterRegistry(PrometheusConfig.DEFAULT);
        registry.config().commonTags("service", "ingest-gateway");
        new JvmMemoryMetrics().bindTo(registry);
        new JvmGcMetrics().bindTo(registry);
        new JvmThreadMetrics().bindTo(registry);
        new ProcessorMetrics().bindTo(registry);

        var metrics = new IngestMetrics(registry);
        var sink = new KafkaEventSink(config.kafkaBootstrap(), registry);
        var ingest = new IngestService(
                EventValidator.withDefaults(),
                new EventNormalizer(new DeviceIdHasher(config.deviceIdSecret())),
                sink, metrics, System::currentTimeMillis, config.publishTimeout());
        var server = new GatewayServer(config, ingest, metrics, registry);

        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            // Stop taking requests first, then flush whatever the producer still holds.
            server.close();
            sink.close();
        }, "shutdown"));
        server.start();
    }

    private GatewayApp() {
    }
}
```

ingest-gateway/src/main/java/dev/primetech/ads/gateway/GatewayConfig.java

33 lines · Gateway settings read from environment variables.

```java
package dev.primetech.ads.gateway;

import java.time.Duration;
import java.util.Map;

public record GatewayConfig(
        int port,
        String kafkaBootstrap,
        String deviceIdSecret,
        int maxEventsPerBatch,
        int maxBodyBytes,
        Duration publishTimeout,
        String corsOrigin) {

    public static GatewayConfig fromEnv(Map<String, String> env) {
        return new GatewayConfig(
                Integer.parseInt(env.getOrDefault("PORT", "8080")),
                env.getOrDefault("KAFKA_BOOTSTRAP", "localhost:19092"),
                required(env, "DEVICE_ID_SECRET"),
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
                Integer.parseInt(env.getOrDefault("MAX_EVENTS_PER_BATCH", "1000")),
                Integer.parseInt(env.getOrDefault("MAX_BODY_BYTES", String.valueOf(1024 * 1024))),
                Duration.ofMillis(Long.parseLong(env.getOrDefault("PUBLISH_TIMEOUT_MS", "12000"))),
                env.getOrDefault("CORS_ORIGIN", "*"));
    }

    private static String required(Map<String, String> env, String key) {
        String value = env.get(key);
        if (value == null || value.isBlank()) {
            throw new IllegalStateException(key + " must be set");
        }
        return value;
    }
}
```

ingest-gateway/src/main/java/dev/primetech/ads/gateway/GatewayServer.java

159 lines · HTTP front door.

```
package dev.primetech.ads.gateway;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import dev.primetech.ads.common.IngestRequest;
import dev.primetech.ads.common.Json;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.Map;
import java.util.concurrent.Executors;

/**
 * HTTP front door. Uses the JDK server on virtual threads: each request blocks while waiting for
 * Kafka acks, which is cheap on a virtual thread and keeps the code straightforward.
 */
public final class GatewayServer implements AutoCloseable {
    private static final Logger log = LoggerFactory.getLogger(GatewayServer.class);

    private final HttpServer server;
    private final IngestService ingest;
    private final IngestMetrics metrics;
    private final PrometheusMeterRegistry registry;
    private final GatewayConfig config;

    public GatewayServer(GatewayConfig config, IngestService ingest, IngestMetrics metrics,
                         PrometheusMeterRegistry registry) throws IOException {
        this.config = config;
        this.ingest = ingest;
        this.metrics = metrics;
        this.registry = registry;
        this.server = HttpServer.create(new InetSocketAddress(config.port()), 1024);
        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        server.createContext("/v1/events", this::handleEvents);
        server.createContext("/stats", ex -> handleGet(ex, () -> json(ex, 200, metrics.snapshot())));
        server.createContext("/metrics", ex -> handleGet(ex,
                () -> send(ex, 200, "text/plain; version=0.0.4", registry.scrape().getBytes(StandardCharsets.UTF
↪ _8))));
        server.createContext("/health", ex -> handleGet(ex, () -> json(ex, 200, Map.of("status", "UP"))));
    }

    public void start() {
        server.start();
        log.info("ingest gateway listening on :{}", port());
    }

    public int port() {
        return server.getAddress().getPort();
    }

    @Override
    public void close() {
        server.stop(2);
    }

    private void handleEvents(HttpExchange ex) throws IOException {
        long start = System.nanoTime();
        try (ex) {
            addCors(ex);
            if (ex.getRequestMethod().equals("OPTIONS")) {
                send(ex, 204, null, null);
                return;
            }
```

```
            if (!ex.getRequestMethod().equals("POST")) {
                error(ex, 405, "use POST");
                return;
            }
            byte[] body = readLimited(ex.getRequestBody(), config.maxBodyBytes());
            if (body == null) {
                error(ex, 413, "body larger than " + config.maxBodyBytes() + " bytes");
                return;
            }
            IngestRequest request;
            try {
                request = Json.MAPPER.readValue(body, IngestRequest.class);
            } catch (JsonProcessingException e) {
                error(ex, 400, "malformed JSON");
                return;
            }
            if (request.events() == null || request.events().isEmpty()) {
                error(ex, 400, "events must be a non-empty array");
                return;
            }
            if (request.events().size() > config.maxEventsPerBatch()) {
                error(ex, 413, "at most " + config.maxEventsPerBatch() + " events per batch");
                return;
            }
            try {
                json(ex, 200, ingest.ingest(request.events()));
            } catch (IngestService.PublishFailedException e) {
                log.warn("publish failed, asking client to retry: {}", e.getCause().toString());
                ex.getResponseHeaders().set("Retry-After", "1");
                error(ex, 503, "event store unavailable, retry the batch");
            }
        } catch (RuntimeException e) {
            log.error("unexpected error handling /v1/events", e);
            error(ex, 500, "internal error");
        } finally {
            metrics.recordLatency(System.nanoTime() - start);
        }
    }

    private void handleGet(HttpExchange ex, IoAction action) throws IOException {
        try (ex) {
            addCors(ex);
            if (!ex.getRequestMethod().equals("GET")) {
                error(ex, 405, "use GET");
                return;
            }
            action.run();
        }
    }

    private void addCors(HttpExchange ex) {
        var h = ex.getResponseHeaders();
        h.set("Access-Control-Allow-Origin", config.corsOrigin());
        h.set("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
        h.set("Access-Control-Allow-Headers", "Content-Type");
    }

    /** Reads at most {@code limit} bytes; returns null if the body is larger. */
    private static byte[] readLimited(InputStream in, int limit) throws IOException {
        byte[] bytes = in.readNBytes(limit + 1);
        return bytes.length > limit ? null : bytes;
    }

    private static void error(HttpExchange ex, int status, String message) throws IOException {
        json(ex, status, Map.of("error", message));
    }

    private static void json(HttpExchange ex, int status, Object body) throws IOException {
        send(ex, status, "application/json", Json.MAPPER.writeValueAsBytes(body));
    }

    private static void send(HttpExchange ex, int status, String contentType, byte[] body) throws IOException {
        if (contentType != null) {
            ex.getResponseHeaders().set("Content-Type", contentType);
        }
        if (body == null) {
            ex.sendResponseHeaders(status, -1);
            return;
        }
        ex.sendResponseHeaders(status, body.length);
        try (OutputStream out = ex.getResponseBody()) {
            out.write(body);
        }
    }

    @FunctionalInterface
    private interface IoAction {
        void run() throws IOException;
```

```
    }
}
```

ingest-gateway/src/main/java/dev/primetech/ads/gateway/IngestMetrics.java

112 lines · Gateway counters.

```
package dev.primetech.ads.gateway;

import dev.primetech.ads.common.EventType;
import dev.primetech.ads.common.RejectReason;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.Timer;
import io.micrometer.core.instrument.distribution.ValueAtPercentile;

import java.util.EnumMap;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.concurrent.TimeUnit;

/**
 * Gateway counters. Backed by Micrometer so Prometheus scrapes the same numbers the dashboard
 * reads from {@code /stats}. Accepted and rejected are separate metric names because Prometheus
 * requires every series of one metric to have the same label keys.
 */
public final class IngestMetrics {
    private final Map<EventType, Counter> accepted = new EnumMap<>(EventType.class);
    private final Map<RejectReason, Counter> rejected = new EnumMap<>(RejectReason.class);
    private final Counter publishFailures;
    private final Counter batches;
    private final Timer requestLatency;
    private final long startedAtMs = System.currentTimeMillis();

    public IngestMetrics(MeterRegistry registry) {
        for (EventType type : EventType.values()) {
            accepted.put(type, Counter.builder("ads.ingest.events.accepted")
                    .description("Events accepted and durably written to Kafka")
                    .tag("type", type.name())
                    .register(registry));
        }
        for (RejectReason reason : RejectReason.values()) {
            rejected.put(reason, Counter.builder("ads.ingest.events.rejected")
                    .description("Events rejected by validation and sent to the DLQ")
                    .tag("reason", reason.name())
                    .register(registry));
        }
        publishFailures = Counter.builder("ads.ingest.publish.failures")
                .description("Batches answered with 503 because Kafka did not acknowledge every event")
                .register(registry);
        batches = Counter.builder("ads.ingest.batches").register(registry);
        requestLatency = Timer.builder("ads.ingest.request.latency")
                .description("End-to-end latency of POST /v1/events, including the Kafka ack")
                .publishPercentiles(0.5, 0.99)
                .publishPercentileHistogram()
                .register(registry);
    }

    public void accepted(EventType type) {
        accepted.get(type).increment();
    }

    public void rejected(RejectReason reason) {
        rejected.get(reason).increment();
    }

    public void publishFailed() {
        publishFailures.increment();
    }

    public void batch() {
        batches.increment();
    }

    public void recordLatency(long nanos) {
        requestLatency.record(nanos, TimeUnit.NANOSECONDS);
    }

    public Snapshot snapshot() {
        Map<String, Long> byType = new LinkedHashMap<>();
        long acceptedTotal = 0;
        for (var e : accepted.entrySet()) {
            long n = (long) e.getValue().count();
            byType.put(e.getKey().name(), n);
            acceptedTotal += n;
        }
        Map<String, Long> byReason = new LinkedHashMap<>();
        long rejectedTotal = 0;
        for (var e : rejected.entrySet()) {
```

```
            long n = (long) e.getValue().count();
            byReason.put(e.getKey().name(), n);
            rejectedTotal += n;
        }
        double p50 = 0, p99 = 0;
        for (ValueAtPercentile v : requestLatency.takeSnapshot().percentileValues()) {
            if (v.percentile() == 0.5) {
                p50 = v.value(TimeUnit.MILLISECONDS);
            } else if (v.percentile() == 0.99) {
                p99 = v.value(TimeUnit.MILLISECONDS);
            }
        }
        return new Snapshot(System.currentTimeMillis(), (System.currentTimeMillis() - startedAtMs) / 1000,
                acceptedTotal, rejectedTotal, (long) batches.count(), (long) publishFailures.count(),
                byType, byReason, p50, p99);
    }

    public record Snapshot(
            long timestampMs,
            long uptimeSeconds,
            long accepted,
            long rejected,
            long batches,
            long publishFailures,
            Map<String, Long> acceptedByType,
            Map<String, Long> rejectedByReason,
            double latencyP50Ms,
            double latencyP99Ms) {
    }
}
```

ingest-gateway/src/main/java/dev/primetech/ads/gateway/IngestService.java

91 lines · Validates, scrubs and publishes one batch.

```
package dev.primetech.ads.gateway;

import dev.primetech.ads.common.ClientEvent;
import dev.primetech.ads.common.DeadLetter;
import dev.primetech.ads.common.EventNormalizer;
import dev.primetech.ads.common.EventValidator;
import dev.primetech.ads.common.IngestResponse;
import dev.primetech.ads.common.RejectReason;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.TimeUnit;
import java.util.function.LongSupplier;

/**
 * Validates, scrubs and publishes one batch. Delivery is at-least-once: the batch is acknowledged
 * only after Kafka confirms every event, and on any failure the client retries the whole batch.
 * Duplicates from those retries are removed downstream by event id.
 */
public final class IngestService {
    private final EventValidator validator;
    private final EventNormalizer normalizer;
    private final EventSink sink;
    private final IngestMetrics metrics;
    private final LongSupplier clock;
    private final Duration publishTimeout;

    public IngestService(EventValidator validator, EventNormalizer normalizer, EventSink sink,
                         IngestMetrics metrics, LongSupplier clock, Duration publishTimeout) {
        this.validator = validator;
        this.normalizer = normalizer;
        this.sink = sink;
        this.metrics = metrics;
        this.clock = clock;
        this.publishTimeout = publishTimeout;
    }

    /**
     * @throws PublishFailedException if any write did not complete; nothing in the batch should be
     *                                treated as delivered
     */
    public IngestResponse ingest(List<ClientEvent> events) throws PublishFailedException {
        long now = clock.getAsLong();
        var writes = new ArrayList<CompletableFuture<Void>>(events.size());
        var errors = new ArrayList<IngestResponse.Rejection>();
        var acceptedEvents = new ArrayList<ClientEvent>(events.size());

        for (int i = 0; i < events.size(); i++) {
            ClientEvent e = events.get(i);
            if (e == null) {
                errors.add(new IngestResponse.Rejection(i, null, RejectReason.MISSING_FIELD));
```

```
                continue;
            }
            Optional<RejectReason> problem = validator.validate(e, now);
            if (problem.isPresent()) {
                errors.add(new IngestResponse.Rejection(i, e.eventId(), problem.get()));
                writes.add(sink.publishDeadLetter(DeadLetter.of(problem.get(), now, e)));
            } else {
                acceptedEvents.add(e);
                writes.add(sink.publish(normalizer.normalize(e, now)));
            }
        }

        try {
            CompletableFuture.allOf(writes.toArray(CompletableFuture[]::new))
                    .get(publishTimeout.toMillis(), TimeUnit.MILLISECONDS);
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
            metrics.publishFailed();
            throw new PublishFailedException(ex);
        } catch (Exception ex) {
            metrics.publishFailed();
            throw new PublishFailedException(ex);
        }

        // Count only after the writes are durable, so the counters match what is in Kafka.
        metrics.batch();
        acceptedEvents.forEach(e -> metrics.accepted(e.type()));
        errors.forEach(r -> metrics.rejected(r.reason()));
        return new IngestResponse(acceptedEvents.size(), errors.size(), errors);
    }

    public static final class PublishFailedException extends Exception {
        PublishFailedException(Throwable cause) {
            super(cause);
        }
    }
}
```

ingest-gateway/src/main/java/dev/primetech/ads/gateway/KafkaEventSink.java

80 lines · Writes accepted events to Kafka (acks=all, idempotent) and rejected ones to the dead-letter topic.

```
package dev.primetech.ads.gateway;

import com.fasterxml.jackson.core.JsonProcessingException;
import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.DeadLetter;
import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.Topics;
import io.micrometer.core.instrument.MeterRegistry;
import io.micrometer.core.instrument.binder.kafka.KafkaClientMetrics;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.Producer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.serialization.ByteArraySerializer;
import org.apache.kafka.common.serialization.StringSerializer;

import java.time.Duration;
import java.util.Map;
import java.util.concurrent.CompletableFuture;

public final class KafkaEventSink implements EventSink {
    private final Producer<String, byte[]> producer;
    private final KafkaClientMetrics clientMetrics;

    public KafkaEventSink(String bootstrapServers, MeterRegistry registry) {
        Map<String, Object> config = Map.of(
                ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers,
                ProducerConfig.CLIENT_ID_CONFIG, "ingest-gateway",
                // Durability: an event is only acknowledged to the player once all in-sync replicas have it.
                ProducerConfig.ACKS_CONFIG, "all",
                // Idempotence stops producer-level retries from duplicating records within a partition.
                ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true,
                // Throughput: small linger plus large batches; lz4 is cheap on CPU and compresses JSON well.
                ProducerConfig.LINGER_MS_CONFIG, 5,
                ProducerConfig.BATCH_SIZE_CONFIG, 128 * 1024,
                ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4",
                // Fail fast so the HTTP request can return 503 and the client retries.
                ProducerConfig.MAX_BLOCK_MS_CONFIG, 2_000,
                ProducerConfig.REQUEST_TIMEOUT_MS_CONFIG, 5_000,
                ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, 10_000);
        this.producer = new KafkaProducer<>(config, new StringSerializer(), new ByteArraySerializer());
        this.clientMetrics = new KafkaClientMetrics(producer);
        clientMetrics.bindTo(registry);
    }

    @Override
    public CompletableFuture<Void> publish(AdEvent event) {
```

```
        // Keyed by session so every event of a playback lands on one partition, in order.
        return send(Topics.AD_EVENTS, event.sessionId(), event);
    }

    @Override
    public CompletableFuture<Void> publishDeadLetter(DeadLetter deadLetter) {
        return send(Topics.AD_EVENTS_DLQ, deadLetter.event().eventId(), deadLetter);
    }

    private CompletableFuture<Void> send(String topic, String key, Object value) {
        var result = new CompletableFuture<Void>();
        try {
            byte[] bytes = Json.MAPPER.writeValueAsBytes(value);
            producer.send(new ProducerRecord<>(topic, key, bytes), (metadata, error) -> {
                if (error != null) {
                    result.completeExceptionally(error);
                } else {
                    result.complete(null);
                }
            });
        } catch (JsonProcessingException | RuntimeException ex) {
            // send() itself throws when metadata is unavailable or the buffer stays full past max.block.ms.
            result.completeExceptionally(ex);
        }
        return result;
    }

    @Override
    public void close() {
        producer.close(Duration.ofSeconds(10));
        clientMetrics.close();
    }
}
```

ingest-gateway/src/main/resources/simplelogger.properties

7 lines · The Kafka client logs its full config at INFO on startup; keep only warnings.

```
org.slf4j.simpleLogger.defaultLogLevel=info
org.slf4j.simpleLogger.showDateTime=true
org.slf4j.simpleLogger.dateTimeFormat=yyyy-MM-dd'T'HH:mm:ss.SSSZ
org.slf4j.simpleLogger.showThreadName=false
org.slf4j.simpleLogger.showShortLogName=true
# The Kafka client logs its full config at INFO on startup; keep only warnings.
org.slf4j.simpleLogger.log.org.apache.kafka=warn
```

ingest-gateway/src/test/java/dev/primetech/ads/gateway/FakeSink.java

32 lines · Test.

```
package dev.primetech.ads.gateway;

import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.DeadLetter;

import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CopyOnWriteArrayList;

final class FakeSink implements EventSink {
    final List<AdEvent> events = new CopyOnWriteArrayList<>();
    final List<DeadLetter> deadLetters = new CopyOnWriteArrayList<>();
    volatile boolean failing;

    @Override
    public CompletableFuture<Void> publish(AdEvent event) {
        if (failing) {
            return CompletableFuture.failedFuture(new RuntimeException("broker down"));
        }
        events.add(event);
        return CompletableFuture.completedFuture(null);
    }

    @Override
    public CompletableFuture<Void> publishDeadLetter(DeadLetter deadLetter) {
        if (failing) {
            return CompletableFuture.failedFuture(new RuntimeException("broker down"));
        }
        deadLetters.add(deadLetter);
        return CompletableFuture.completedFuture(null);
    }
}
```

ingest-gateway/src/test/java/dev/primetech/ads/gateway/GatewayServerTest.java

111 lines · Test.

```
package dev.primetech.ads.gateway;

import com.fasterxml.jackson.databind.JsonNode;
import dev.primetech.ads.common.DeviceIdHasher;
```

```
import dev.primetech.ads.common.EventNormalizer;
import dev.primetech.ads.common.EventValidator;
import dev.primetech.ads.common.Json;
import io.micrometer.prometheusmetrics.PrometheusConfig;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

import static org.assertj.core.api.Assertions.assertThat;

class GatewayServerTest {
    private final FakeSink sink = new FakeSink();
    private final HttpClient http = HttpClient.newHttpClient();
    private GatewayServer server;

    @BeforeEach
    void start() throws Exception {
        var config = new GatewayConfig(0, "unused", "test-secret-0123456789", 2, 4096,
                Duration.ofSeconds(1), "*");
        var registry = new PrometheusMeterRegistry(PrometheusConfig.DEFAULT);
        var metrics = new IngestMetrics(registry);
        var ingest = new IngestService(EventValidator.withDefaults(),
                new EventNormalizer(new DeviceIdHasher(config.deviceIdSecret())),
                sink, metrics, System::currentTimeMillis, config.publishTimeout());
        server = new GatewayServer(config, ingest, metrics, registry);
        server.start();
    }

    @AfterEach
    void stop() {
        server.close();
    }

    private HttpResponse<String> post(String body) throws Exception {
        return http.send(HttpRequest.newBuilder(uri("/v1/events"))
                .POST(HttpRequest.BodyPublishers.ofString(body)).build(), HttpResponse.BodyHandlers.ofString());
    }

    private HttpResponse<String> get(String path) throws Exception {
        return http.send(HttpRequest.newBuilder(uri(path)).GET().build(), HttpResponse.BodyHandlers.ofString());
    }

    private URI uri(String path) {
        return URI.create("http://localhost:" + server.port() + path);
    }

    private static String validEvent(String id) {
        return """
                {"eventId":"%s","type":"COMPLETE","eventTimeMs":%d,"sessionId":"s1","adId":"a1",
                 "creativeId":"c1","campaignId":"camp1","lineItemId":"li1","deviceId":"d1",
                 "deviceType":"CTV","country":"US","consent":true}
                """.formatted(id, System.currentTimeMillis());
    }

    @Test
    void acceptsBatchAndReportsStats() throws Exception {
        var response = post("{\"events\":[" + validEvent("e1") + "," + validEvent("e2") + "]}");

        assertThat(response.statusCode()).isEqualTo(200);
        JsonNode body = Json.MAPPER.readTree(response.body());
        assertThat(body.get("accepted").asInt()).isEqualTo(2);
        assertThat(sink.events).hasSize(2);
        assertThat(response.headers().firstValue("Access-Control-Allow-Origin")).contains("*");

        JsonNode stats = Json.MAPPER.readTree(get("/stats").body());
        assertThat(stats.get("accepted").asLong()).isEqualTo(2);
        assertThat(stats.get("acceptedByType").get("COMPLETE").asLong()).isEqualTo(2);
    }

    @Test
    void rejectsMalformedOversizedAndEmptyRequests() throws Exception {
        assertThat(post("{not json").statusCode()).isEqualTo(400);
        assertThat(post("{\"events\":[]}").statusCode()).isEqualTo(400);
        assertThat(post("{\"events\":[" + validEvent("1") + "," + validEvent("2") + "," + validEvent("3") + "]}"
    )
                .statusCode()).as("more than maxEventsPerBatch").isEqualTo(413);
        assertThat(post("{\"events\":[\"" + "x".repeat(5000) + "\"]}").statusCode())
                .as("body over maxBodyBytes").isEqualTo(413);
        assertThat(get("/v1/events").statusCode()).isEqualTo(405);
    }

```

```
    @Test
    void returns503WithRetryAfterWhenKafkaIsDown() throws Exception {
        sink.failing = true;
        var response = post("{\"events\":[" + validEvent("e1") + "]}");

        assertThat(response.statusCode()).isEqualTo(503);
        assertThat(response.headers().firstValue("Retry-After")).contains("1");
    }

    @Test
    void exposesPrometheusMetricsAndHealth() throws Exception {
        post("{\"events\":[" + validEvent("e1") + "]}");

        assertThat(get("/health").statusCode()).isEqualTo(200);
        assertThat(get("/metrics").body())
                .contains("ads_ingest_events_accepted_total{type=\"COMPLETE\"} 1.0")
                .contains("ads_ingest_events_rejected_total{reason=\"MISSING_PRICE\"}")
                .contains("ads_ingest_request_latency_seconds");
    }
}
```

ingest-gateway/src/test/java/dev/primetech/ads/gateway/IngestServiceTest.java

81 lines · Test.

```
package dev.primetech.ads.gateway;

import dev.primetech.ads.common.ClientEvent;
import dev.primetech.ads.common.DeviceIdHasher;
import dev.primetech.ads.common.DeviceType;
import dev.primetech.ads.common.EventNormalizer;
import dev.primetech.ads.common.EventType;
import dev.primetech.ads.common.EventValidator;
import dev.primetech.ads.common.RejectReason;
import io.micrometer.core.instrument.simple.SimpleMeterRegistry;
import org.junit.jupiter.api.Test;

import java.time.Duration;
import java.util.Arrays;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class IngestServiceTest {
    static final long NOW = 1_760_000_000_000L;

    private final FakeSink sink = new FakeSink();
    private final IngestMetrics metrics = new IngestMetrics(new SimpleMeterRegistry());
    private final IngestService service = new IngestService(
            EventValidator.withDefaults(),
            new EventNormalizer(new DeviceIdHasher("test-secret-0123456789")),
            sink, metrics, () -> NOW, Duration.ofSeconds(1));

    static ClientEvent event(String id, EventType type, Long price) {
        return new ClientEvent(id, type, NOW, "sess-1", "ad-1", "cr-1", "camp-1", "li-1",
                "raw-device", DeviceType.CTV, "US", true, price);
    }

    @Test
    void publishesValidEventsAndDeadLettersInvalidOnes() throws Exception {
        var response = service.ingest(List.of(
                event("a", EventType.IMPRESSION, 1_000L),
                event("b", EventType.MIDPOINT, 5L),
                event("c", EventType.COMPLETE, null)));

        assertThat(response.accepted()).isEqualTo(2);
        assertThat(response.rejected()).isEqualTo(1);
        assertThat(response.errors()).singleElement().satisfies(r -> {
            assertThat(r.index()).isEqualTo(1);
            assertThat(r.eventId()).isEqualTo("b");
            assertThat(r.reason()).isEqualTo(RejectReason.PRICE_ON_NON_BILLABLE);
        });
        assertThat(sink.events).extracting(e -> e.eventId()).containsExactly("a", "c");
        assertThat(sink.events).allSatisfy(e -> assertThat(e.receivedTimeMs()).isEqualTo(NOW));
        assertThat(sink.deadLetters).singleElement().satisfies(d -> {
            assertThat(d.reason()).isEqualTo(RejectReason.PRICE_ON_NON_BILLABLE);
            assertThat(d.event().deviceId()).as("DLQ must not keep raw device ids").isNull();
        });

        var snapshot = metrics.snapshot();
        assertThat(snapshot.accepted()).isEqualTo(2);
        assertThat(snapshot.rejected()).isEqualTo(1);
        assertThat(snapshot.acceptedByType()).containsEntry("IMPRESSION", 1L).containsEntry("COMPLETE", 1L);
    }

    @Test
    void nullEntryIsRejectedNotFatal() throws Exception {
        var response = service.ingest(Arrays.asList(null, event("a", EventType.CLICK, null)));
```

```
65
66          assertThat(response.accepted()).isEqualTo(1);
67          assertThat(response.errors()).singleElement()
68                  .satisfies(r -> assertThat(r.reason()).isEqualTo(RejectReason.MISSING_FIELD));
69      }
70
71      @Test
72      void failsWholeBatchWhenSinkFailsAndCountsNothing() {
73          sink.failing = true;
74
75          assertThatThrownBy(() -> service.ingest(List.of(event("a", EventType.CLICK, null))))
76                  .isInstanceOf(IngestService.PublishFailedException.class);
77          var snapshot = metrics.snapshot();
78          assertThat(snapshot.accepted()).isZero();
79          assertThat(snapshot.publishFailures()).isEqualTo(1);
80      }
81  }
```

loadtest

loadtest/__pycache__/report.cpython-313.pyc

148 lines

```
  1  
  2  
  3  
  4  
  5  
  6  
  7  
  8  
  ↪ es.
  9  
 10  
 11  
 12  
 13  
 14  
 15  
 16  
 17  
 18  
 19  
 20  
 21  
 22  
 23  
  ↪ igits
 24  
 25  
  ↪ 
 26  
 27  
 28  
 29  
 30  
 31  
 32  
 33  
  ↪ 
 34  
 35  
 36  
 37  
  ↪ 
 38  
 39  
 40  
 41  
 42  
  ↪ 
 43  
 44  
 45  
 46  
 47  
 48  
 49  
  ↪ 
 50  
 51  
 52  
 53  
 54  
 55  
 56  
```

```
'@D@□□
>@D@□
@□:@□@Q@q@  @
□@E@J@J@t@□$@
$@t@
+@S@6@C@□@I@3E@-E@
F@E@
□@□@

@k@N@□@□□@1@\@?@□#@S@□@=@)9@$@%G@□@Q@O_@M`@in@Io@□□@1@^@□$@T@□2@C@□@:V@8W@`d@4e@□□@q@□)@□*@□/@a@4@□7@□@□;@□<@C@□
↪ @Q@GW@EX@E]@\]@ae@De@gh@@i@?j@jl@□m@□ @□@
@
@u@□@□□□3@□@□□
□@□□□@A@
.@
/@□@□=@s@1@EV@CW@`e@?f@□□@C@1@\@?@#7@a@
8@!@S@□A@□□□C□@C@□@!@H@□
@
@
@T@E@J@J@s@O@□+@d@□2@□3@□□□□@□□
□@9@9@U@
□@□□r□□□□c□□□□
```

```
140  ▯▯▯▯#▯m▯▯▯{▯▯▯\▯▯▯▯M▯
141  *▯
142  h▯▯Q▯$}▯▯▯
143  ▯▯▯▯▯▯▯&▯
144  ▯▯,▯
145  ▯▯
146  ▯(▯▯
147  ▯▯▯▯)▯▯▯▯
148  ▯z▯▯▯▯▯▯▯F▯▯▯▯▯r▯▯▯▯
```

loadtest/__pycache__/run.cpython-313.pyc

318 lines

```
  1  ▯
  2
  3  ▯▯▯▯▯▯2▯jx ▯▯▯▯▯▯▯
```

```
  ↪ □□n□0□n□U□R□□□
```

```
146  □□□□□□□□□□□a□□□ □□□g□f□=□□□f□)□NzBhttp://localhost:8888/druid/indexer/v1/supervisor/ad_events/status□
147  □□□r□□□□□□payload□
148  aggregateLag)□r□□□□r□□□□r□□□□r□□□□r□□□□r□□□□□get□    Exception)□r□□□□r□□□s□□□□  r□□□□□  druid_lagr□□□□g□□□sg□□□□□□□
     ↪ □□□□
149  □□^□^□
150  #□
151  #□$h□rt□
152  #□
153  u□yz□□□□i□i□□□l□9□□-□G□□□□v□□□□{□{□>□□*□□*□□□□v□□
154  u□□□□
155  □□□□□□□□□□□□□□□□□□s'□□□□
156  A$□□□A□□□□A$□□□
157  A!□□
158  □A$□□$
159  A1□□0□A1□c□□□□□□□□□□□□□□□□□
```

```
   ↪ lertstate="firing"}))□@
221  cpu_percentr@□□□@
222  firing_alerts)
223  r□□□□r@□□□@minr@□□□r>□□□rV□□□r
224  □□□@□msr@□□□r@□□□)
225  r@□□□rA□□□@window@□t0@□freshrR□□□@□w@
226  pending_now@□pending_beforer□□□□s
227  □□□          r□□□□@measurer@□□□y□□□s□□□□@□@
228  @□@□@□@B@
229  □@c@"@f@o@
230  &@E@□□@J@J@s@1@f@q@j@D@I@I@K@"@$4@□5@□6@□7@
231  □@
232
233  @C@□□@J@J@s@1@f@□@ @   @
234  @b@ 0@□1@□2@□3@
235  □@(@!@
236  @A@□□@□p@□q@K@□□@
237  |@}~@|□@□□@□A□@□□□B□@□□□C□@N@□□  □@□□@c@□□   □@□□@□@□□   □@□□    □@t@
238  H@□@□@3@
239  O@□P@□□   □@□□    □@□@'Q@RS@QT@TW@%X@ Y@  □   □@
240  □   □@"@T@$x@yz@x{@{□@□□#A□@□□
241  B□@□□□C□@
242  □  □@
243  □    □@"@T@$y@z{@y|@□□}□A□@□□#B□@□□
244  C□@□□□D□@
245  □   □@□□    □@D@#R@ST@RU@UX@!Y@
246  Z@□□    □@□□    □@□@□□@□□A□B□@□□@□C□@□□C□F□@□□
247  G□@□□□H□@□□  □@□□    □@□@□□  □@□□    %@k@.A@^@E[@d@bm@b~@□□C□I□@□□b□I□@□□    □@□□    $@T@,j@kl@jm@ms@*t@%u@□□
   ↪  □@□□    □@□@ _@`a@_b@bh@
248  i@□j@□□    □@
249  □
250  @Y@[@
251  □    □@
252  □    □@d@V\@n@$@!B@1@#@S@□I@□J@bf@□□ □@ □    □@□□H□L□@□□P□V□@□□H□V□@2@d@%y@z{@y|@□□}□A□@□□$B□@□□□C□@□□
253  D□@□□\□`□@!□  □@"□    □@y}@□□B□H□@□□z□H□@d@
254  C@A@3@Fm@no@mp@ps@□t@u@□□N□R□@#□ □@$□    □@U@%□  □@&□□□@□□@□@□
255  @
256  B@□C@□H@q@+□    □@A@.□
257
258  @Hr-□□□c□□□□□□□□□□□□□□□□□□□□□□□□□□□□□@*□□□@□U□c□□□S□$□[□□□□□□□□□□□□U□S□-□□□S□5□□□□□□□□$□)□Nr0□□□r
259  □□□)□rr□□□)□r@□□□s□□□□ r□□□□r@□□□r@□□□@□□□s
260  □□□@□@□□□?@4@□@□@g@□@n@a@(@@□@r-□□□c□□□□□□□□□□□□□
261  □□□□□□□□@□□□@□[□□□□□□□□□□□□R□□□□□□□□□□□□□□□□□□□
```

```
 295   A
 296  I I g
 297  +!
 298   C
 299  D% %  S
 300   
 301   s
 302  I I I I
 303  
 304
 305  I
 306     __main__r)
 307   __doc__r rs r& r r
 308  urllib.parser urllib.requestr r r r r r __file__r(r
 309  r!rD rV r r r r r r r r __name__ r- r <module>r s x'@ A B
 310    
 311  
 312     w w r w w r w w x'@ A B
 313  !
 314  I
 315  
 316  \ B &,
 317  F A &,
 318  z F r-
```

loadtest/report.py

74 lines

```python
#!/usr/bin/env python3
"""Renders load test results (from run.py) as Markdown tables.

    python3 loadtest/report.py docs/loadtest/events.json docs/loadtest/full.json > docs/loadtest/RESULTS.md
"""
import json
import sys

# Containers grouped by the stage of the pipeline they belong to.
STAGES = {
    "load generator": ["simulator"],
    "gateway": ["gateway"],
    "Kafka": ["redpanda"],
    "Flink": ["jobmanager", "taskmanager"],
    "Druid": ["coordinator", "broker", "historical", "middlemanager", "router", "zookeeper", "postgres"],
    "ad serving": ["ad-server", "dsp-premium", "dsp-budget", "dsp-slow", "redis"],
    "pacing, audit, observability": ["pacing-service", "audit-service", "prometheus", "alertmanager", "grafana",
 "dashboard"],
}


def num(v, digits=0, suffix=""):
    if v is None:
        return "—"
    return f"{v:,.{digits}f}{suffix}"


def keeps_up(r):
    growth = r.get("flink_pending_growth_per_s") or 0
    return growth < 0.01 * r["gateway_eps"]


def table(results):
    full = results[0]["mode"] == "full"
    head = ["Offered events/s", "Gateway accepted", "Gateway p99", "Flink processed", "Flink backlog growth",
            "Committed output visible p50 / p99", "Keeps up"]
    if full:
        head[4:4] = ["Auctions/s", "Decision p99", "Shed"]
    lines = ["| " + " | ".join(head) + " |", "|" + "---|" * len(head)]
    for r in results:
        f = r["freshness"]
        row = [num(r["target_eps"]), num(r["gateway_eps"], suffix="/s"), num(r["gateway_p99_ms"], suffix=" ms"),
               num(r["flink_in_eps"], suffix="/s"), num(r["flink_pending_growth_per_s"], suffix="/s"),
               f"{num((f['visible_p50_ms'] or 0) / 1000, 1)} / {num((f['visible_p99_ms'] or 0) / 1000, 1)} s",
               "yes" if keeps_up(r) else "**no**"]
        if full:
            row[4:4] = [num(r["auctions_per_s"], suffix="/s"), num(r["decision_p99_ms"], suffix=" ms"),
                        num(100 * (r["shed_ratio"] or 0), 1, "%")]
        lines.append("| " + " | ".join(row) + " |")
    return "\n".join(lines)


def cpu_table(results):
    head = ["Offered events/s"] + list(STAGES) + ["total (of 400%)"]
    lines = ["| " + " | ".join(head) + " |", "|" + "---|" * len(head)]
    for r in results:
        c = r["cpu_percent"]
        cells = [sum(c.get(n, 0) for n in names) for names in STAGES.values()]
        lines.append("| " + " | ".join([num(r["target_eps"])] + [num(v, suffix="%") for v in cells]
                                       + [num(sum(c.values()), suffix="%")]) + " |")
```

```
    return "\n".join(lines)


def main():
    for path in sys.argv[1:]:
        results = json.load(open(path))
        mode = results[0]["mode"]
        print(f"### {'Event path only' if mode == 'events' else 'Full system (every ad break auctioned)'}\n")
        print(table(results) + "\n")
        print("CPU by stage (one `docker stats` sample mid-window, so ±20%):\n")
        print(cpu_table(results) + "\n")


if __name__ == "__main__":
    main()
```

loadtest/run.py

183 lines

```
#!/usr/bin/env python3
"""Steps the full stack through rising load and records what each component does.

Each step restarts the simulator at a target rate, waits for things to settle, then measures a
window. Everything comes from places an operator would look: Prometheus (every service, Flink,
Redpanda), `docker stats` (CPU per container), the clean topic (how fresh committed output is),
Druid's supervisor (ingestion lag) and the audit service (is the money still right).

    python3 loadtest/run.py --mode events --steps 5000,10000,20000 --out docs/loadtest/events.json

Modes:
  events  players send tracking events only: the ingest -> Kafka -> Flink -> Druid path
  full    every ad break is also requested from the ad server (auction, caps, pacing)

Needs the stack running with its ports published (plain `docker compose up`, not the
production overlay) and nothing else loading the machine.
"""
import argparse
import json
import os
import statistics
import subprocess
import threading
import time
import urllib.parse
import urllib.request

PROM = "http://localhost:9090"
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))


def prom(query, at=None):
    """Instant query; returns the first sample's value as float, or None."""
    params = {"query": query}
    if at:
        params["time"] = str(at)
    with urllib.request.urlopen(f"{PROM}/api/v1/query?{urllib.parse.urlencode(params)}", timeout=20) as r:
        result = json.load(r)["data"]["result"]
    if not result:
        return None
    v = float(result[0]["value"][1])
    return None if v != v else v  # NaN -> None


def compose(*args, env=None):
    subprocess.run(["docker", "compose", *args], cwd=ROOT, check=True, env={**os.environ, **(env or {})},
                   stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)


def start_simulator(eps, mode, seed):
    # More in-flight batches at higher rates, so the generator is never what limits throughput.
    concurrency = max(16, int(eps / 1000) * 4)
    compose("rm", "-sf", "simulator")
    compose("up", "-d", "--no-deps", "simulator", env={
        "TARGET_EPS": str(eps), "CONCURRENCY": str(concurrency), "DURATION_SECONDS": "0", "SEED": str(seed),
        "SIM_AD_SERVER_URL": "http://ad-server:8090" if mode == "full" else "",
    })


def cpu_by_container():
    out = subprocess.run(["docker", "stats", "--no-stream", "--format", "{{.Name}}\t{{.CPUPerc}}"],
                         capture_output=True, text=True, check=True).stdout
    cpu = {}
    for line in out.strip().splitlines():
        name, pct = line.split("\t")
        cpu[name.removeprefix("ads-eventing-").removesuffix("-1")] = float(pct.rstrip("%"))
    return cpu


```

```
def freshness(seconds):
    """Reads the clean topic as it is committed and measures, per record, how long after the
    gateway received the event it became readable (read_committed: Flink's transactions commit on
    each checkpoint). Also returns the gateway -> Flink processing part of that delay."""
    proc = subprocess.Popen(["docker", "compose", "exec", "-T", "redpanda", "rpk", "topic", "consume",
                             "ad-events-clean.v1", "-o", "end", "--read-committed", "-f", "%v\\n"],
                            cwd=ROOT, stdout=subprocess.PIPE, stderr=subprocess.DEVNULL, text=True)
    visible, processing = [], []
    stop = time.time() + seconds

    def read():
        for line in proc.stdout:
            now_ms = time.time() * 1000
            try:
                e = json.loads(line)
            except ValueError:
                continue
            visible.append(now_ms - e["receivedTimeMs"])
            processing.append(e["processedTimeMs"] - e["receivedTimeMs"])

    t = threading.Thread(target=read, daemon=True)
    t.start()
    time.sleep(max(0, stop - time.time()))
    proc.kill()
    t.join(timeout=5)

    def q(xs, p):
        return round(statistics.quantiles(xs, n=100)[p - 1]) if len(xs) > 100 else None

    return {"samples": len(visible), "visible_p50_ms": q(visible, 50), "visible_p99_ms": q(visible, 99),
            "processing_p50_ms": q(processing, 50), "processing_p99_ms": q(processing, 99)}


def druid_lag():
    try:
        with urllib.request.urlopen("http://localhost:8888/druid/indexer/v1/supervisor/ad_events/status", timeou
↪ t=10) as r:
            payload = json.load(r)["payload"]
        return payload.get("aggregateLag")
    except Exception:  # noqa: BLE001 - lag is informative, not essential
        return None


def audit():
    try:
        with urllib.request.urlopen("http://localhost:8096/stats", timeout=10) as r:
            checks = json.load(r)["checks"]
        return {k: (v.get("latestCompleted") or v.get("latest") or {}).get("status") for k, v in checks.items()}
    except Exception:  # noqa: BLE001
        return None


def measure(eps, mode, window):
    t0 = time.time()
    fresh = freshness(min(60, window))
    time.sleep(max(0, window / 2 - (time.time() - t0)))
    cpu = cpu_by_container()
    time.sleep(max(0, window - (time.time() - t0)))
    w = f"{window}s"
    pending_now = prom('sum(flink_taskmanager_job_task_operator_pendingRecords{operator_name=~".*ad_events.*"})'
↪ )
    pending_before = prom(f'sum(flink_taskmanager_job_task_operator_pendingRecords{{operator_name=~".*ad_events.
↪ *"}} offset {w})')
    r = {
        "target_eps": eps,
        "mode": mode,
        "gateway_eps": prom(f"sum(rate(ads_ingest_events_accepted_total[{w}]))"),
        "gateway_rejected_eps": prom(f"sum(rate(ads_ingest_events_rejected_total[{w}]))"),
        "gateway_p50_ms": ms(prom(f"histogram_quantile(0.5, sum by (le) (rate(ads_ingest_request_latency_seconds
↪ _bucket[{w}])))")),
        "gateway_p99_ms": ms(prom(f"histogram_quantile(0.99, sum by (le) (rate(ads_ingest_request_latency_second
↪ s_bucket[{w}])))")),
        "publish_failures": prom(f"sum(increase(ads_ingest_publish_failures_total[{w}]))"),
        # Counter rate over the same window as the gateway's (Flink's per-second meters lag).
        "flink_in_eps": prom(f'sum(rate(flink_taskmanager_job_task_operator_numRecordsOut{{operator_name="Source
↪ :_ad_events"}}[{w}]))'),
        "flink_pending": pending_now,
        "flink_pending_growth_per_s": None if pending_now is None or pending_before is None else (pending_now -
↪ pending_before) / window,
        "flink_backpressured_tasks": prom(f"max_over_time(sum(flink_taskmanager_job_task_isBackPressured)[{w}:15
↪ s])"),
        "checkpoint_ms": prom(f"max_over_time(max(flink_jobmanager_job_lastCheckpointDuration)[{w}:15s])"),
        "druid_lag_records": druid_lag(),
        "auctions_per_s": prom(f"sum(rate(adserver_requests_total[{w}]))") if mode == "full" else None,
        "decision_p99_ms": ms(prom(f"histogram_quantile(0.99, sum by (le) (rate(adserver_decision_latency_second
↪ s_bucket[{w}])))")) if mode == "full" else None,
        "shed_ratio": prom(f"sum(rate(adserver_requests_shed_total[{w}])) / sum(rate(adserver_requests_total[{w}
↪ ]))") if mode == "full" else None,
```

```
        "freshness": fresh,
        "cpu_percent": cpu,
        "audit": audit(),
        "firing_alerts": prom('count(ALERTS{alertstate="firing"})') or 0,
    }
    return r


def ms(seconds):
    return None if seconds is None else round(seconds * 1000, 1)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--mode", choices=["events", "full"], default="events")
    ap.add_argument("--steps", default="2000,5000,10000,20000")
    ap.add_argument("--warmup", type=int, default=60)
    ap.add_argument("--window", type=int, default=120)
    ap.add_argument("--out", required=True)
    args = ap.parse_args()
    results = []
    for i, eps in enumerate(int(s) for s in args.steps.split(",")):
        print(f"[{time.strftime('%H:%M:%S')}] {args.mode} {eps} events/s: warming up {args.warmup}s", flush=True)
        start_simulator(eps, args.mode, seed=1000 + i)
        time.sleep(args.warmup)
        r = measure(eps, args.mode, args.window)
        results.append(r)
        print(json.dumps({k: v for k, v in r.items() if k not in ("cpu_percent",)}), flush=True)
        os.makedirs(os.path.dirname(os.path.abspath(args.out)), exist_ok=True)
        with open(args.out, "w") as f:
            json.dump(results, f, indent=1)
    compose("rm", "-sf", "simulator")


if __name__ == "__main__":
    main()
```

mock-dsp

mock-dsp/pom.xml

43 lines

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.primetech.ads</groupId>
    <artifactId>ads-eventing</artifactId>
    <version>0.1.0-SNAPSHOT</version>
  </parent>

  <artifactId>mock-dsp</artifactId>
  <name>Mock DSP (OpenRTB bidder)</name>

  <properties>
    <main.class>dev.primetech.ads.dsp.DspApp</main.class>
  </properties>

  <dependencies>
    <dependency>
      <groupId>dev.primetech.ads</groupId>
      <artifactId>openrtb</artifactId>
    </dependency>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
    </dependency>
  </dependencies>

  <build>
    <finalName>mock-dsp</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-shade-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

mock-dsp/src/main/java/dev/primetech/ads/dsp/Bidder.java

99 lines · The bidding logic of a mock DSP, separate from HTTP so it can be tested directly.

```
package dev.primetech.ads.dsp;

import dev.primetech.ads.openrtb.BidRequest;
import dev.primetech.ads.openrtb.BidResponse;
import dev.primetech.ads.openrtb.OpenRtb;

import java.util.ArrayList;
import java.util.List;
import java.util.UUID;
import java.util.random.RandomGenerator;

/**
 * The bidding logic of a mock DSP, separate from HTTP so it can be tested directly. For each
 * impression it decides whether to bid, what the impression is worth, and what to bid given the
 * auction type and floor.
 */
public final class Bidder {
    private static final int[] DURATIONS = {15, 30};

    private final DspProfile profile;
    private final String winUrlBase;

    public Bidder(DspProfile profile, String winUrlBase) {
        this.profile = profile;
        this.winUrlBase = winUrlBase;
    }

    /** Returns null for no bid (answered with HTTP 204). */
    public BidResponse bid(BidRequest request, RandomGenerator random) {
        boolean firstPrice = request.at() == null || request.at() == BidRequest.FIRST_PRICE;
        var bids = new ArrayList<BidResponse.Bid>();
        for (BidRequest.Imp imp : request.imp()) {
            if (random.nextDouble() >= profile.bidRate()) {
                continue;
            }
            int maxDuration = imp.video() != null && imp.video().maxduration() != null ? imp.video().maxduration
    () : 30;
            int duration = pickDuration(maxDuration, random);
            if (duration < 0) {
                continue;
            }
            double value = logNormal(profile.cpmMedian(), profile.cpmSigma(), random);
            // First price: shade below value. Second price: bidding true value is the dominant strategy.
            double price = firstPrice ? value * profile.firstPriceShade() : value;
            double floor = imp.bidfloor() == null ? 0 : imp.bidfloor();
            if (price < floor) {
                if (value < floor) {
                    continue;
                }
                price = floor;
            }
            price = Math.round(price * 100) / 100.0;

            String advertiser = profile.advertisers().get(random.nextInt(profile.advertisers().size()));
            String campaign = campaignId(advertiser, random.nextInt(2));
            String creative = campaign + "-" + duration + "s";
            bids.add(new BidResponse.Bid(
                    UUID.randomUUID().toString(), imp.id(), price,
                    winUrlBase + "/win?auction=" + OpenRtb.AUCTION_ID_MACRO + "&imp=" + imp.id()
                            + "&price=" + OpenRtb.AUCTION_PRICE_MACRO,
                    vast(creative, duration), List.of(advertiser), campaign, creative, duration));
        }
        if (bids.isEmpty()) {
            return null;
        }
        return new BidResponse(request.id(), List.of(new BidResponse.SeatBid(profile.name(), bids)), "USD", null
);
    }

    /** How long this response takes, drawn per request. */
    public long latencyMs(RandomGenerator random) {
        return Math.round(logNormal(profile.latencyMedianMs(), profile.latencySigma(), random));
    }

    static String campaignId(String advertiserDomain, int n) {
        return advertiserDomain.substring(0, advertiserDomain.indexOf('.')) + "-c" + n;
    }

    private static int pickDuration(int maxDuration, RandomGenerator random) {
        int fitting = 0;
        for (int d : DURATIONS) {
            if (d <= maxDuration) {
                fitting++;
            }
        }
```

```
        if (fitting == 0) {
            return -1;
        }
        return DURATIONS[random.nextInt(fitting)];
    }

    private static double logNormal(double median, double sigma, RandomGenerator random) {
        return median * Math.exp(sigma * random.nextGaussian());
    }

    private static String vast(String creativeId, int duration) {
        return "<VAST version=\"4.2\"><Ad id=\"" + creativeId + "\"><InLine><Creatives><Creative><Linear>"
                + "<Duration>00:00:" + String.format("%02d", duration) + "</Duration>"
                + "</Linear></Creative></Creatives></InLine></Ad></VAST>";
    }
}
```

mock-dsp/src/main/java/dev/primetech/ads/dsp/DspApp.java

33 lines · Entry point: runs one mock DSP with a named bidding preset.

```
package dev.primetech.ads.dsp;

import io.micrometer.prometheusmetrics.PrometheusConfig;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public final class DspApp {
    private static final Logger log = LoggerFactory.getLogger(DspApp.class);

    public static void main(String[] args) throws Exception {
        var env = System.getenv();
        String presetName = env.getOrDefault("DSP_PROFILE", "budget");
        DspProfile profile = DspProfile.PRESETS.get(presetName);
        if (profile == null) {
            throw new IllegalArgumentException("DSP_PROFILE must be one of " + DspProfile.PRESETS.keySet());
        }
        int port = Integer.parseInt(env.getOrDefault("PORT", "9000"));
        // Where the ad server should send win notices: this DSP's address as the ad server sees it.
        String publicUrl = env.getOrDefault("PUBLIC_URL", "http://localhost:" + port);

        var registry = new PrometheusMeterRegistry(PrometheusConfig.DEFAULT);
        registry.config().commonTags("service", "mock-dsp", "dsp", profile.name());
        var server = new DspServer(port, new Bidder(profile, publicUrl), registry,
                Long.parseLong(env.getOrDefault("SEED", String.valueOf(presetName.hashCode()))), true);
        Runtime.getRuntime().addShutdownHook(new Thread(server::close, "shutdown"));
        server.start();
        log.info("{} listening on :{} ({})", profile.name(), port, profile);
    }

    private DspApp() {
    }
}
```

mock-dsp/src/main/java/dev/primetech/ads/dsp/DspProfile.java

39 lines · How a mock DSP behaves.

```
package dev.primetech.ads.dsp;

import java.util.List;
import java.util.Map;

/**
 * How a mock DSP behaves. Values are drawn from log-normal distributions, which is what both bid
 * prices and network latencies look like in practice: most near the median, a long right tail.
 *
 * @param bidRate          share of impressions the DSP bids on at all
 * @param cpmMedian        median value (USD CPM) the DSP's advertisers place on an impression
 * @param cpmSigma         spread of that value (log-normal sigma)
 * @param latencyMedianMs  median time to answer
 * @param latencySigma     spread of latency; a high value means a fat tail of timeouts
 * @param firstPriceShade  in first-price auctions, bid this fraction of value (bid shading)
 * @param advertisers      advertiser domains this DSP buys for
 */
public record DspProfile(
        String name,
        double bidRate,
        double cpmMedian,
        double cpmSigma,
        double latencyMedianMs,
        double latencySigma,
        double firstPriceShade,
        List<String> advertisers) {

    /** Three DSPs with deliberately different trade-offs, to make the auction interesting. */
    public static final Map<String, DspProfile> PRESETS = Map.of(
            // Selective, high value, reliably fast.
            "premium", new DspProfile("premium-dsp", 0.55, 34.0, 0.25, 35, 0.30, 0.85,
```

```
                    List.of("auroraluxe.example", "velvetline.example", "summitwatch.example")),
            // Bids on almost everything at modest prices.
            "budget", new DspProfile("budget-dsp", 0.90, 17.0, 0.35, 22, 0.25, 0.90,
                    List.of("snackbox.example", "quickcart.example", "playful.example", "brightsoda.example")),
            // Values impressions highly but is slow: a share of its bids misses the deadline.
            "slow", new DspProfile("slow-dsp", 0.75, 40.0, 0.30, 85, 0.45, 0.85,
                    List.of("roadster.example", "voltcar.example")));
    }
```

mock-dsp/src/main/java/dev/primetech/ads/dsp/DspServer.java

153 lines · HTTP front of a mock DSP: the OpenRTB bid endpoint, win notices and its own counters.

```
package dev.primetech.ads.dsp;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpServer;
import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.http.JsonHttp;
import dev.primetech.ads.openrtb.BidRequest;
import dev.primetech.ads.openrtb.BidResponse;
import io.micrometer.core.instrument.Counter;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;

import java.io.IOException;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.SplittableRandom;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicReference;

/** HTTP front of a mock DSP: the OpenRTB bid endpoint, win notices and its own counters. */
public final class DspServer implements AutoCloseable {
    private final HttpServer server;
    private final Bidder bidder;
    private final PrometheusMeterRegistry registry;
    private final boolean simulateLatency;
    // SplittableRandom is not thread-safe; each request splits its own generator off this one.
    private final AtomicReference<SplittableRandom> seed;

    private final Counter requests;
    private final Counter bidResponses;
    private final Counter bids;
    private final Counter noBids;
    private final Counter wins;
    private final Counter winSpendMicros;

    public DspServer(int port, Bidder bidder, PrometheusMeterRegistry registry, long seed, boolean simulateLaten
↪ cy)
            throws IOException {
        this.bidder = bidder;
        this.registry = registry;
        this.simulateLatency = simulateLatency;
        this.seed = new AtomicReference<>(new SplittableRandom(seed));
        requests = registry.counter("dsp.bid.requests");
        bidResponses = registry.counter("dsp.bid.responses");
        bids = registry.counter("dsp.bids");
        noBids = registry.counter("dsp.nobids");
        wins = registry.counter("dsp.wins");
        winSpendMicros = registry.counter("dsp.win.spend.micros");

        server = HttpServer.create(new InetSocketAddress(port), 1024);
        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        server.createContext("/openrtb2/bid", this::handleBid);
        server.createContext("/win", this::handleWin);
        server.createContext("/stats", ex -> {
            try (ex) {
                JsonHttp.json(ex, 200, stats());
            }
        });
        server.createContext("/metrics", ex -> {
            try (ex) {
                JsonHttp.send(ex, 200, "text/plain; version=0.0.4", registry.scrape().getBytes(StandardCharsets.
↪ UTF_8));
            }
        });
        server.createContext("/health", ex -> {
            try (ex) {
                JsonHttp.json(ex, 200, Map.of("status", "UP"));
            }
        });
    }

    public void start() {
        server.start();
    }
```

```

    public int port() {
        return server.getAddress().getPort();
    }

    @Override
    public void close() {
        server.stop(1);
    }

    private SplittableRandom random() {
        return seed.updateAndGet(SplittableRandom::split);
    }

    private void handleBid(HttpExchange ex) throws IOException {
        try (ex) {
            if (!ex.getRequestMethod().equals("POST")) {
                JsonHttp.error(ex, 405, "use POST");
                return;
            }
            byte[] body = JsonHttp.readLimited(ex.getRequestBody(), 256 * 1024);
            if (body == null) {
                JsonHttp.error(ex, 413, "request too large");
                return;
            }
            BidRequest request;
            try {
                request = Json.MAPPER.readValue(body, BidRequest.class);
            } catch (JsonProcessingException e) {
                JsonHttp.error(ex, 400, "malformed bid request");
                return;
            }
            requests.increment();
            var random = random();
            if (simulateLatency) {
                sleep(bidder.latencyMs(random));
            }
            BidResponse response = bidder.bid(request, random);
            if (response == null) {
                noBids.increment();
                JsonHttp.noContent(ex);
                return;
            }
            bidResponses.increment();
            bids.increment(response.seatbid().get(0).bid().size());
            JsonHttp.json(ex, 200, response);
        }
    }

    private void handleWin(HttpExchange ex) throws IOException {
        try (ex) {
            String price = JsonHttp.query(ex, "price");
            if (price != null) {
                wins.increment();
                winSpendMicros.increment(Math.round(Double.parseDouble(price) * 1_000_000 / 1000));
            }
            JsonHttp.noContent(ex);
        }
    }

    private Map<String, Object> stats() {
        var m = new LinkedHashMap<String, Object>();
        m.put("requests", (long) requests.count());
        m.put("bidResponses", (long) bidResponses.count());
        m.put("bids", (long) bids.count());
        m.put("noBids", (long) noBids.count());
        m.put("wins", (long) wins.count());
        m.put("winSpendMicros", (long) winSpendMicros.count());
        return m;
    }

    private static void sleep(long ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

mock-dsp/src/main/resources/simplelogger.properties

7 lines · The Kafka client logs its full config at INFO on startup; keep only warnings.

```
org.slf4j.simpleLogger.defaultLogLevel=info
org.slf4j.simpleLogger.showDateTime=true
org.slf4j.simpleLogger.dateTimeFormat=yyyy-MM-dd'T'HH:mm:ss.SSSZ
org.slf4j.simpleLogger.showThreadName=false
org.slf4j.simpleLogger.showShortLogName=true
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
# The Kafka client logs its full config at INFO on startup; keep only warnings.
org.slf4j.simpleLogger.log.org.apache.kafka=warn
```

mock-dsp/src/test/java/dev/primetech/ads/dsp/BidderTest.java

87 lines · Test.

```
package dev.primetech.ads.dsp;

import dev.primetech.ads.openrtb.BidRequest;
import dev.primetech.ads.openrtb.BidResponse;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.List;
import java.util.SplittableRandom;
import java.util.stream.IntStream;

import static org.assertj.core.api.Assertions.assertThat;

class BidderTest {
    private static final DspProfile ALWAYS = new DspProfile("test-dsp", 1.0, 20.0, 0.3, 30, 0.3, 0.8,
            List.of("brand.example", "other.example"));

    static BidRequest request(int slots, double floor, int maxDuration, int auctionType) {
        var imps = IntStream.rangeClosed(1, slots).mapToObj(i -> new BidRequest.Imp(String.valueOf(i),
                new BidRequest.Video(List.of("video/mp4"), 5, maxDuration, List.of(7), 1920, 1080, 1, "pod", i == 1 ? 1 : 0, 0, null),
                floor, "USD", 1)).toList();
        return new BidRequest("auc", imps, null, new BidRequest.Device(3, new BidRequest.Geo("USA"), "h", 0),
                auctionType, 120, List.of("USD"), null);
    }

    private static List<BidResponse.Bid> bids(DspProfile profile, BidRequest request, int rounds) {
        var bidder = new Bidder(profile, "http://dsp");
        var random = new SplittableRandom(1);
        var out = new ArrayList<BidResponse.Bid>();
        for (int i = 0; i < rounds; i++) {
            BidResponse r = bidder.bid(request, random);
            if (r != null) {
                out.addAll(r.seatbid().get(0).bid());
            }
        }
        return out;
    }

    @Test
    void bidsOncePerImpressionWithWellFormedBids() {
        var bidder = new Bidder(ALWAYS, "http://dsp");
        BidResponse r = bidder.bid(request(3, 0, 30, 1), new SplittableRandom(2));

        assertThat(r.id()).isEqualTo("auc");
        assertThat(r.seatbid().get(0).bid()).extracting(BidResponse.Bid::impid).containsExactly("1", "2", "3");
        assertThat(r.seatbid().get(0).bid()).allSatisfy(b -> {
            assertThat(b.adomain()).hasSize(1);
            assertThat(b.nurl()).contains("${AUCTION_PRICE}").contains("${AUCTION_ID}");
            assertThat(b.adm()).startsWith("<VAST");
            assertThat(b.crid()).startsWith(b.cid());
        });
    }

    @Test
    void neverBidsBelowTheFloor() {
        var all = bids(ALWAYS, request(1, 25.0, 30, 1), 2_000);
        assertThat(all).isNotEmpty().allSatisfy(b -> assertThat(b.price()).isGreaterThanOrEqualTo(25.0));
    }

    @Test
    void shadesBidsInFirstPriceAuctionsOnly() {
        double firstPrice = bids(ALWAYS, request(1, 0, 30, BidRequest.FIRST_PRICE), 4_000).stream()
                .mapToDouble(BidResponse.Bid::price).average().orElseThrow();
        double secondPrice = bids(ALWAYS, request(1, 0, 30, BidRequest.SECOND_PRICE), 4_000).stream()
                .mapToDouble(BidResponse.Bid::price).average().orElseThrow();
        assertThat(firstPrice / secondPrice).isBetween(0.75, 0.85);
    }

    @Test
    void respectsMaxDurationAndBidRate() {
        var shortSlots = bids(ALWAYS, request(1, 0, 15, 1), 500);
        assertThat(shortSlots).allSatisfy(b -> assertThat(b.dur()).isEqualTo(15));
        assertThat(bids(ALWAYS, request(1, 0, 10, 1), 100)).as("nothing fits 10s").isEmpty();

        var half = new DspProfile("half", 0.5, 20, 0.3, 30, 0.3, 0.8, List.of("a.example"));
        assertThat(bids(half, request(1, 0, 30, 1), 4_000)).hasSizeBetween(1_800, 2_200);
    }

    @Test
    void latencyFollowsTheProfile() {
        var bidder = new Bidder(DspProfile.PRESETS.get("slow"), "http://dsp");
```

```
        var random = new SplittableRandom(3);
        long overDeadline = IntStream.range(0, 10_000).filter(i -> bidder.latencyMs(random) > 110).count();
        assertThat(overDeadline).as("the slow preset misses a 110 ms budget a meaningful share of the time")
                .isBetween(1_500L, 4_000L);
    }
}
```

openrtb

openrtb/pom.xml

21 lines

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.primetech.ads</groupId>
    <artifactId>ads-eventing</artifactId>
    <version>0.1.0-SNAPSHOT</version>
  </parent>

  <artifactId>openrtb</artifactId>
  <name>OpenRTB 2.6 model (CTV subset)</name>

  <dependencies>
    <dependency>
      <groupId>dev.primetech.ads</groupId>
      <artifactId>ads-common</artifactId>
    </dependency>
  </dependencies>
</project>
```

openrtb/src/main/java/dev/primetech/ads/openrtb/BidRequest.java

75 lines · OpenRTB 2.6 bid request, limited to the fields a CTV video ad pod needs.

```
package dev.primetech.ads.openrtb;

import java.util.List;

/**
 * OpenRTB 2.6 bid request, limited to the fields a CTV video ad pod needs. Field names follow the
 * specification exactly so the JSON is wire-compatible with real exchanges and DSPs.
 *
 * @param id     unique auction id
 * @param imp    one impression per slot in the pod
 * @param at     auction type: 1 first price, 2 second price plus
 * @param tmax   time the bidder has to respond, in milliseconds, including network latency
 * @param cur    allowed bid currencies (ISO-4217)
 * @param source transaction id shared by every party in the supply chain
 */
public record BidRequest(
        String id,
        List<Imp> imp,
        App app,
        Device device,
        Integer at,
        Integer tmax,
        List<String> cur,
        Source source) {

    public static final int FIRST_PRICE = 1;
    public static final int SECOND_PRICE = 2;

    /** @param bidfloor minimum CPM for this impression, in {@code bidfloorcur} */
    public record Imp(String id, Video video, Double bidfloor, String bidfloorcur, Integer secure) {
    }

    /**
     * Video placement. The pod fields are new in 2.6: every slot of one ad break shares a
     * {@code podid}, {@code slotinpod} says where in the break this one sits.
     *
     * @param plcmt     placement type: 1 = instream
     * @param slotinpod 1 first, -1 last, 2 first or last, 0 any
     * @param podseq    position of the pod in the content: 1 pre-roll, 0 mid-roll, -1 post-roll
     */
    public record Video(
            List<String> mimes,
            Integer minduration,
            Integer maxduration,
            List<Integer> protocols,
            Integer w,
            Integer h,
            Integer plcmt,
            String podid,
```

```
            Integer slotinpod,
            Integer podseq,
            Integer poddur) {
    }

    public record App(String id, String name, String bundle, Publisher publisher) {
    }

    public record Publisher(String id, String name) {
    }

    /**
     * @param devicetype OpenRTB device type list, see {@link OpenRtb#deviceType}
     * @param ifa        advertising id: here the pseudonymised device id, only with consent
     * @param lmt        limit ad tracking: 1 when the viewer did not consent
     */
    public record Device(Integer devicetype, Geo geo, String ifa, Integer lmt) {
    }

    /** @param country ISO-3166-1 alpha-3, as OpenRTB requires */
    public record Geo(String country) {
    }

    public record Source(String tid) {
    }
}
```

openrtb/src/main/java/dev/primetech/ads/openrtb/BidResponse.java

37 lines · OpenRTB 2.6 bid response.

```
package dev.primetech.ads.openrtb;

import java.util.List;

/**
 * OpenRTB 2.6 bid response. A bidder that does not want to bid answers HTTP 204 with no body,
 * or a response with an empty {@code seatbid} and a no-bid reason.
 *
 * @param nbr no-bid reason code
 */
public record BidResponse(String id, List<SeatBid> seatbid, String cur, Integer nbr) {

    public record SeatBid(String seat, List<Bid> bid) {
    }

    /**
     * @param impid   the {@link BidRequest.Imp#id()} this bid is for
     * @param price   bid CPM in the response currency
     * @param nurl    win notice URL; may contain {@code ${AUCTION_PRICE}} and {@code ${AUCTION_ID}}
     * @param adm     ad markup (a VAST document for video)
     * @param adomain advertiser domains, used for competitive separation and blocking
     * @param cid     campaign id on the bidder's side
     * @param crid    creative id on the bidder's side
     * @param dur     creative duration in seconds (2.6)
     */
    public record Bid(
            String id,
            String impid,
            double price,
            String nurl,
            String adm,
            List<String> adomain,
            String cid,
            String crid,
            Integer dur) {
    }
}
```

openrtb/src/main/java/dev/primetech/ads/openrtb/OpenRtb.java

52 lines · Conversions between our model and OpenRTB conventions.

```
package dev.primetech.ads.openrtb;

import dev.primetech.ads.common.DeviceType;

import java.util.Locale;
import java.util.MissingResourceException;

/** Conversions between our model and OpenRTB conventions. */
public final class OpenRtb {
    public static final String AUCTION_PRICE_MACRO = "${AUCTION_PRICE}";
    public static final String AUCTION_ID_MACRO = "${AUCTION_ID}";
    /** VAST 4.x wrapper and inline, the protocols a CTV player supports. */
    public static final java.util.List<Integer> VAST_PROTOCOLS = java.util.List.of(7, 8);
    public static final int PLACEMENT_INSTREAM = 1;

    private OpenRtb() {
    }
```

```

    /** OpenRTB device type list (AdCOM). */
    public static int deviceType(DeviceType type) {
        return switch (type) {
            case CTV -> 3;          // Connected TV
            case MOBILE -> 4;       // Phone
            case WEB -> 2;          // Personal computer
            case GAME_CONSOLE -> 6; // Connected device
        };
    }

    /** OpenRTB geo uses ISO-3166-1 alpha-3; our events use alpha-2. */
    public static String alpha3(String alpha2) {
        try {
            return Locale.of("", alpha2).getISO3Country();
        } catch (MissingResourceException e) {
            throw new IllegalArgumentException("unknown country " + alpha2, e);
        }
    }

    /** CPM in currency units (OpenRTB {@code price}) to micro-units, as we store money. */
    public static long cpmToMicros(double cpm) {
        return Math.round(cpm * 1_000_000);
    }

    public static double microsToCpm(long cpmMicros) {
        return cpmMicros / 1_000_000.0;
    }

    /** Substitutes the standard macros in a win or billing notice URL. */
    public static String expandMacros(String url, String auctionId, long clearingCpmMicros) {
        return url.replace(AUCTION_PRICE_MACRO, String.format(Locale.ROOT, "%.6f", microsToCpm(clearingCpmMicros
↪ )))
                .replace(AUCTION_ID_MACRO, auctionId);
    }
}
```

openrtb/src/test/java/dev/primetech/ads/openrtb/OpenRtbTest.java

69 lines · Test.

```
package dev.primetech.ads.openrtb;

import com.fasterxml.jackson.databind.JsonNode;
import dev.primetech.ads.common.DeviceType;
import dev.primetech.ads.common.Json;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;

class OpenRtbTest {

    @Test
    void requestSerializesWithSpecFieldNamesAndNoNulls() throws Exception {
        var video = new BidRequest.Video(List.of("video/mp4"), 5, 30, OpenRtb.VAST_PROTOCOLS, 1920, 1080,
                OpenRtb.PLACEMENT_INSTREAM, "pod-1", 1, 0, 90);
        var request = new BidRequest("auc-1", List.of(new BidRequest.Imp("1", video, 5.0, "USD", 1)),
                new BidRequest.App("app-1", "Streaming", "com.example.tv", null),
                new BidRequest.Device(3, new BidRequest.Geo("USA"), "hash", 0),
                BidRequest.FIRST_PRICE, 120, List.of("USD"), new BidRequest.Source("tid-1"));

        JsonNode json = Json.MAPPER.readTree(Json.MAPPER.writeValueAsBytes(request));

        assertThat(json.get("tmax").asInt()).isEqualTo(120);
        assertThat(json.at("/imp/0/video/podid").asText()).isEqualTo("pod-1");
        assertThat(json.at("/imp/0/bidfloorcur").asText()).isEqualTo("USD");
        assertThat(json.at("/device/geo/country").asText()).isEqualTo("USA");
        assertThat(json.at("/app/publisher").isMissingNode()).as("nulls are omitted").isTrue();
        assertThat(Json.MAPPER.treeToValue(json, BidRequest.class)).isEqualTo(request);
    }

    @Test
    void parsesAResponseWithUnknownExtensions() throws Exception {
        String body = """
                {"id":"auc-1","cur":"USD","seatbid":[{"seat":"dsp","bid":[
                  {"id":"b1","impid":"1","price":12.5,"adomain":["brand.example"],"cid":"c1","crid":"cr1","dur":
↪ 15,
                   "nurl":"https://dsp/win?p=${AUCTION_PRICE}","ext":{"anything":true}}]}]}
                """;
        BidResponse response = Json.MAPPER.readValue(body, BidResponse.class);

        var bid = response.seatbid().get(0).bid().get(0);
        assertThat(bid.price()).isEqualTo(12.5);
        assertThat(bid.adomain()).containsExactly("brand.example");
        assertThat(bid.dur()).isEqualTo(15);
    }
```

```

    @Test
    void mapsCountriesToAlpha3() {
        assertThat(OpenRtb.alpha3("US")).isEqualTo("USA");
        assertThat(OpenRtb.alpha3("GB")).isEqualTo("GBR");
        assertThat(OpenRtb.alpha3("KR")).isEqualTo("KOR");
        assertThatThrownBy(() -> OpenRtb.alpha3("ZZ")).isInstanceOf(IllegalArgumentException.class);
    }

    @Test
    void mapsDeviceTypes() {
        assertThat(OpenRtb.deviceType(DeviceType.CTV)).isEqualTo(3);
        assertThat(OpenRtb.deviceType(DeviceType.WEB)).isEqualTo(2);
    }

    @Test
    void convertsMoneyAndExpandsMacros() {
        assertThat(OpenRtb.cpmToMicros(12.345678)).isEqualTo(12_345_678L);
        assertThat(OpenRtb.expandMacros("https://dsp/win?p=${AUCTION_PRICE}&a=${AUCTION_ID}", "auc-9", 12_500_00
  ↳ 0L))
                .isEqualTo("https://dsp/win?p=12.500000&a=auc-9");
    }
}
```

pacing-service

pacing-service/pom.xml

51 lines

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.primetech.ads</groupId>
    <artifactId>ads-eventing</artifactId>
    <version>0.1.0-SNAPSHOT</version>
  </parent>

  <artifactId>pacing-service</artifactId>
  <name>Pacing Service (budget control loop)</name>

  <properties>
    <main.class>dev.primetech.ads.pacing.PacingApp</main.class>
  </properties>

  <dependencies>
    <dependency>
      <groupId>dev.primetech.ads</groupId>
      <artifactId>ads-common</artifactId>
    </dependency>
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
    </dependency>
    <dependency>
      <groupId>redis.clients</groupId>
      <artifactId>jedis</artifactId>
    </dependency>
    <dependency>
      <groupId>io.micrometer</groupId>
      <artifactId>micrometer-registry-prometheus</artifactId>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
    </dependency>
  </dependencies>

  <build>
    <finalName>pacing-service</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-shade-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

pacing-service/src/main/java/dev/primetech/ads/pacing/LineItemPacer.java

227 lines · Paces one line item: spends its period budget evenly by setting the probability that it enters an auction (the throttle).

```
package dev.primetech.ads.pacing;
```

```
 2
 3  import dev.primetech.ads.common.catalog.LineItemConfig;
 4  import dev.primetech.ads.common.pacing.PacingPeriod;
 5  import dev.primetech.ads.common.pacing.PacingState;
 6
 7  import java.util.ArrayList;
 8  import java.util.List;
 9  import java.util.Map;
10  import java.util.TreeMap;
11
12  /**
13   * Paces one line item: spends its period budget evenly by setting the probability that it enters
14   * an auction (the throttle). Pure logic, no I/O, so it can be tested in a closed loop.
15   *
16   * <h2>Inputs</h2>
17   * <ul>
18   *   <li><b>Delivered spend</b> per event-time minute from the stream processor: authoritative
19   *       (de-duplicated, IVT excluded) but only known for minutes the watermark has closed, so it
20   *       lags real time by a minute or so. {@code horizonMs} is how far it is known.</li>
21   *   <li><b>Served spend</b>: the ad server's own counter of what it has sold. Immediate, but it
22   *       overstates delivery because viewers abandon breaks before every served ad plays.</li>
23   * </ul>
24   *
25   * <h2>Control law</h2>
26   * The pacer learns a <em>yield</em>: delivered spend per minute the line item would get at
27   * throttle 1. It is estimated every tick from the growth of served spend (immediate) times the
28   * delivered/served ratio, divided by the throttle that was in force. Learning from delivered
29   * spend instead would lag by over a minute and overspend early; in simulation that let delivery
30   * stray 15% from the even curve when traffic was plentiful, against 3% with served spend.
31   *
32   * <p>Each tick sets {@code throttle = required rate / yield}, where the required rate spreads the
33   * remaining budget over the remaining time. Using remaining budget makes past shortfall or
34   * overshoot self-correcting (integral action). Each tick can cut the throttle by up to 4× but only
35   * double it: overspend cannot be undone, underspend can be caught up. Catching up is capped at
36   * {@link #MAX_CATCH_UP} times the even rate, so a line item that starts late in a period (or comes
37   * back from an outage) spends what it reasonably can instead of its whole budget in one burst.
38   *
39   * <h2>Hard stop</h2>
40   * Delivered spend lags, so stopping on it alone overspends. The pacer converts the remaining
41   * budget into served spend using the observed delivered/served ratio and publishes
42   * {@code stopAtServedMicros}; the ad server stops the moment its live served counter reaches it.
43   */
44  public final class LineItemPacer {
45      static final double MIN_THROTTLE = 0.0005;
46      static final double INITIAL_THROTTLE = 0.02;
47      static final double YIELD_SMOOTHING = 0.2;
48      /** Delivered/served ratio assumed until one is observed: 1 never lets served spend pass the budget. */
49      static final double DEFAULT_DELIVERY_RATIO = 1.0;
50      /** Tolerance around the even-delivery curve before a line item counts as ahead or behind. */
51      static final double PACE_TOLERANCE = 0.05;
52      /** Fastest catch-up, as a multiple of the even spending rate. */
53      static final double MAX_CATCH_UP = 2.0;
54      /**
55       * Typical time from serving an ad to playing it: a pod's ads play one after another, so the
56       * price-weighted mean measured end to end is about 15 s. Delivery up to the horizon is compared
57       * with what was served this much earlier. Comparing with everything served up to the horizon
58       * counts ads that have not played yet, which understated the delivery ratio and the yield and
59       * overshot; allowing too much (30 s, the first guess) counted delivered spend as still in
60       * flight and under-delivered by 6%.
61       */
62      static final long SERVE_TO_PLAY_MS = 15_000;
63      private static final long MINUTE_MS = 60_000;
64
65      private final LineItemConfig config;
66      private final PacingPeriod period;
67
68      private long periodIndex = Long.MIN_VALUE;
69      private final TreeMap<Long, Long> spendByMinute = new TreeMap<>();
70      /** (time, served spend) at each tick, to know what had been served by the delivery horizon. */
71      private final List<long[]> servedHistory = new ArrayList<>();
72      private double throttle = INITIAL_THROTTLE;
73      private double yieldPerMinute = Double.NaN;
74      private long lastTickMs = Long.MIN_VALUE;
75      private long lastServedMicros;
76      private PacingState last;
77
78      public LineItemPacer(LineItemConfig config, PacingPeriod period) {
79          this.config = config;
80          this.period = period;
81      }
82
83      /** Records the latest total for one event-time minute (later revisions replace earlier ones). */
84      public synchronized void delivered(long minuteStartMs, long spendMicros) {
85          // Minutes of the next period can arrive before the tick that rolls over to it; keep them.
86          if (period.index(minuteStartMs) >= periodIndex) {
87              spendByMinute.put(minuteStartMs, spendMicros);
88          }
89      }
```

```
90
91      /**
92       * One control step.
93       *
94       * @param horizonMs    delivered spend is known for minutes ending at or before this time
95       * @param servedMicros the ad server's served-spend counter for this period, now
96       */
97      public synchronized PacingState tick(long nowMs, long horizonMs, long servedMicros) {
98          rollOver(nowMs);
99          long budget = config.periodBudgetMicros();
100         long start = period.start(periodIndex);
101         long end = period.end(periodIndex);
102         long horizon = Math.max(start, Math.min(horizonMs, nowMs));
103
104         long delivered = deliveredBefore(horizon);
105         long servedAtHorizon = Math.min(servedMicros, servedAt(Math.max(start, horizon - SERVE_TO_PLAY_MS)));
106         double ratio = servedAtHorizon > 0
107                 ? clamp((double) delivered / servedAtHorizon, 0.3, 1.0)
108                 : DEFAULT_DELIVERY_RATIO;
109         long stopAt = servedAtHorizon + (long) (Math.max(0, budget - delivered) / ratio);
110         // Spend already served but not yet reported, expected to turn into delivery.
111         long inFlight = (long) ((servedMicros - servedAtHorizon) * ratio);
112         long committed = delivered + inFlight;
113         long target = (long) (budget * ((double) (horizon - start) / (end - start)));
114
115         updateYield(nowMs, servedMicros, ratio);
116
117         PacingState.Status status;
118         if (delivered >= budget || servedMicros >= stopAt) {
119             throttle = 0;
120             status = PacingState.Status.EXHAUSTED;
121         } else {
122             double minutesLeft = Math.max(0.5, (end - nowMs) / (double) MINUTE_MS);
123             double evenPerMinute = budget / ((end - start) / (double) MINUTE_MS);
124             double requiredPerMinute = Math.min(Math.max(0, budget - committed) / minutesLeft, MAX_CATCH_UP * ev
    ↪ enPerMinute);
125             double wanted;
126             if (Double.isNaN(yieldPerMinute) || yieldPerMinute <= 0) {
127                 // Nothing learned yet: probe upwards while behind.
128                 wanted = committed < target ? Math.max(throttle, INITIAL_THROTTLE) * 2 : Math.max(throttle, INIT
    ↪ IAL_THROTTLE);
129             } else {
130                 wanted = requiredPerMinute / yieldPerMinute;
131             }
132             double previous = Math.max(throttle, MIN_THROTTLE);
133             // Cut fast, grow cautiously: overspend cannot be undone, underspend can be caught up.
134             throttle = clamp(clamp(wanted, previous / 4, previous * 2), MIN_THROTTLE, 1.0);
135
136             double gap = (double) (delivered - target) / budget;
137             if (gap > PACE_TOLERANCE) {
138                 status = PacingState.Status.AHEAD;
139             } else if (gap < -PACE_TOLERANCE) {
140                 status = throttle >= 1.0 ? PacingState.Status.UNDER_DELIVERING : PacingState.Status.BEHIND;
141             } else {
142                 status = PacingState.Status.ON_PACE;
143             }
144         }
145
146         servedHistory.add(new long[] {nowMs, servedMicros});
147         last = new PacingState(config.lineItemId(), periodIndex, budget, delivered, target, throttle, stopAt, st
    ↪ atus, nowMs);
148         return last;
149     }
150
151     public synchronized PacingState last() {
152         return last;
153     }
154
155     public LineItemConfig config() {
156         return config;
157     }
158
159     private void rollOver(long nowMs) {
160         long index = period.index(nowMs);
161         if (index == periodIndex) {
162             return;
163         }
164         periodIndex = index;
165         long start = period.start(index);
166         spendByMinute.headMap(start).clear();
167         servedHistory.clear();
168         // Keep the learned yield (traffic in the next period looks like this one) but not the
169         // throttle: a throttle raised to catch up, or cut to zero once spent, says nothing about
170         // what a fresh budget needs. Start the period at the even rate.
171         if (Double.isNaN(yieldPerMinute) || yieldPerMinute <= 0) {
172             throttle = INITIAL_THROTTLE;
173         } else {
174             double evenPerMinute = config.periodBudgetMicros() / (period.lengthMs() / (double) MINUTE_MS);
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
            throttle = clamp(evenPerMinute / yieldPerMinute, MIN_THROTTLE, 1.0);
        }
        // Served counters restart at zero each period.
        lastTickMs = Long.MIN_VALUE;
        lastServedMicros = 0;
    }

    private long deliveredBefore(long horizon) {
        long sum = 0;
        for (Map.Entry<Long, Long> e : spendByMinute.headMap(horizon).entrySet()) {
            if (e.getKey() + MINUTE_MS <= horizon) {
                sum += e.getValue();
            }
        }
        return sum;
    }

    /**
     * Folds the served spend since the last tick into the yield estimate: what the throttle in force
     * since then produced, scaled to throttle 1 and to delivered spend.
     */
    private void updateYield(long nowMs, long servedMicros, double deliveryRatio) {
        if (lastTickMs != Long.MIN_VALUE && nowMs > lastTickMs && throttle > MIN_THROTTLE * 2) {
            double perMinute = (servedMicros - lastServedMicros) * deliveryRatio * MINUTE_MS / (nowMs - lastTick
  ↪ Ms);
            double sample = perMinute / throttle;
            yieldPerMinute = Double.isNaN(yieldPerMinute) ? sample
                    : YIELD_SMOOTHING * sample + (1 - YIELD_SMOOTHING) * yieldPerMinute;
        }
        lastTickMs = nowMs;
        lastServedMicros = servedMicros;
    }

    /** Served spend at a past moment, interpolated between ticks. */
    private long servedAt(long timeMs) {
        long[] before = null;
        for (long[] s : servedHistory) {
            if (s[0] <= timeMs) {
                before = s;
            } else {
                if (before == null) {
                    return 0;
                }
                double f = (double) (timeMs - before[0]) / (s[0] - before[0]);
                return before[1] + (long) ((s[1] - before[1]) * f);
            }
        }
        return before == null ? 0 : before[1];
    }

    private static double clamp(double v, double lo, double hi) {
        return Math.max(lo, Math.min(hi, v));
    }
}
```

pacing-service/src/main/java/dev/primetech/ads/pacing/PacingApp.java

165 lines · Wires PacingService to Kafka (inputs), Redis (outputs) and HTTP (stats).

```
package dev.primetech.ads.pacing;

import com.sun.net.httpserver.HttpServer;
import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.LineItemMinute;
import dev.primetech.ads.common.Topics;
import dev.primetech.ads.common.catalog.LineItemConfig;
import dev.primetech.ads.common.http.JsonHttp;
import dev.primetech.ads.common.pacing.PacingPeriod;
import dev.primetech.ads.common.pacing.PacingState;
import io.micrometer.prometheusmetrics.PrometheusConfig;
import io.micrometer.prometheusmetrics.PrometheusMeterRegistry;
import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.common.TopicPartition;
import org.apache.kafka.common.serialization.ByteArrayDeserializer;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.io.IOException;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.Executors;
```

```
import java.util.concurrent.TimeUnit;

/**
 * Wires {@link PacingService} to Kafka (inputs), Redis (outputs) and HTTP (stats). State is rebuilt
 * from the topics on every start: both are compacted or short-lived, and the current period's
 * minutes are all it needs, so there is nothing to persist.
 */
public final class PacingApp {
    private static final Logger log = LoggerFactory.getLogger(PacingApp.class);

    public static void main(String[] args) throws Exception {
        var env = System.getenv();
        String bootstrap = env.getOrDefault("KAFKA_BOOTSTRAP", "localhost:19092");
        String redisUrl = env.getOrDefault("REDIS_URL", "redis://localhost:6379");
        var period = PacingPeriod.of(Duration.ofMinutes(Long.parseLong(env.getOrDefault("PACING_PERIOD_MINUTES",
 "30"))));
        long tickSeconds = Long.parseLong(env.getOrDefault("PACING_TICK_SECONDS", "10"));
        int port = Integer.parseInt(env.getOrDefault("PORT", "8095"));

        var registry = new PrometheusMeterRegistry(PrometheusConfig.DEFAULT);
        registry.config().commonTags("service", "pacing-service");
        var store = new RedisPacingStore(redisUrl);
        var service = new PacingService(period, store, System::currentTimeMillis, registry);

        Thread consumer = Thread.ofPlatform().name("pacing-consumer").daemon().start(() -> consume(bootstrap, se
rvice));
        var ticker = Executors.newSingleThreadScheduledExecutor();
        ticker.scheduleAtFixedRate(() -> {
            try {
                service.tick();
            } catch (RuntimeException e) {
                log.error("pacing tick failed", e);
            }
        }, tickSeconds, tickSeconds, TimeUnit.SECONDS);

        HttpServer http = HttpServer.create(new InetSocketAddress(port), 64);
        http.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        http.createContext("/stats", ex -> {
            try (ex) {
                JsonHttp.cors(ex, "*");
                JsonHttp.json(ex, 200, stats(service, period));
            }
        });
        http.createContext("/metrics", ex -> {
            try (ex) {
                JsonHttp.send(ex, 200, "text/plain; version=0.0.4", registry.scrape().getBytes(StandardCharsets.
UTF_8));
            }
        });
        http.createContext("/health", ex -> {
            try (ex) {
                JsonHttp.json(ex, 200, Map.of("status", consumer.isAlive() ? "UP" : "DOWN"));
            }
        });
        http.start();
        Runtime.getRuntime().addShutdownHook(new Thread(() -> {
            http.stop(1);
            ticker.shutdownNow();
            store.close();
        }, "shutdown"));
        log.info("pacing service on :{}, period {} min, tick {} s", port, period.lengthMs() / 60_000, tickSecond
s);
    }

    /** Reads both topics from the beginning, forever. No consumer group: every instance sees everything. */
    private static void consume(String bootstrap, PacingService service) {
        var config = Map.<String, Object>of(
                ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrap,
                // Minute totals are written transactionally: only count committed ones.
                ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed",
                ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false,
                ConsumerConfig.CLIENT_ID_CONFIG, "pacing-service");
        while (true) {
            try (var consumer = new KafkaConsumer<>(config, new StringDeserializer(), new ByteArrayDeserializer(
))) {
                List<TopicPartition> partitions = new ArrayList<>();
                for (String topic : List.of(Topics.LINE_ITEMS, Topics.LINE_ITEM_MINUTES)) {
                    consumer.partitionsFor(topic, Duration.ofSeconds(30))
                            .forEach(p -> partitions.add(new TopicPartition(topic, p.partition())));
                }
                consumer.assign(partitions);
                consumer.seekToBeginning(partitions);
                while (true) {
                    for (ConsumerRecord<String, byte[]> r : consumer.poll(Duration.ofMillis(500))) {
                        if (r.value() == null) {
                            continue;
                        }
                        try {
```

```
                            if (r.topic().equals(Topics.LINE_ITEMS)) {
                                service.lineItem(Json.MAPPER.readValue(r.value(), LineItemConfig.class));
                            } else {
                                service.minute(Json.MAPPER.readValue(r.value(), LineItemMinute.class));
                            }
                        } catch (IOException e) {
                            log.warn("skipping malformed record on {}: {}", r.topic(), e.toString());
                        }
                    }
                }
            } catch (RuntimeException e) {
                log.warn("consumer failed, restarting in 5 s: {}", e.toString());
                try {
                    Thread.sleep(5_000);
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }
    }

    private static Map<String, Object> stats(PacingService service, PacingPeriod period) {
        long now = System.currentTimeMillis();
        long index = period.index(now);
        var out = new LinkedHashMap<String, Object>();
        out.put("periodIndex", index);
        out.put("periodStartMs", period.start(index));
        out.put("periodEndMs", period.end(index));
        out.put("elapsed", period.elapsed(now));
        out.put("horizonMs", service.horizonMs());
        var items = new ArrayList<Map<String, Object>>();
        for (PacingState s : service.latest()) {
            LineItemConfig c = service.config(s.lineItemId());
            var m = new LinkedHashMap<String, Object>();
            m.put("lineItemId", s.lineItemId());
            m.put("campaignId", c == null ? null : c.campaignId());
            m.put("tier", c == null ? null : c.tier());
            m.put("budgetMicros", s.budgetMicros());
            m.put("deliveredMicros", s.deliveredMicros());
            m.put("targetMicros", s.targetMicros());
            m.put("throttle", s.throttle());
            m.put("status", s.status());
            items.add(m);
        }
        out.put("lineItems", items);
        return out;
    }

    private PacingApp() {
    }
}
```

pacing-service/src/main/java/dev/primetech/ads/pacing/PacingService.java

152 lines · Holds one pacer per line item, feeds them delivery data, and runs the control tick.

```
package dev.primetech.ads.pacing;

import dev.primetech.ads.common.LineItemMinute;
import dev.primetech.ads.common.catalog.LineItemConfig;
import dev.primetech.ads.common.pacing.PacingPeriod;
import dev.primetech.ads.common.pacing.PacingState;
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.Gauge;
import io.micrometer.core.instrument.MeterRegistry;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.LongSupplier;

/**
 * Holds one pacer per line item, feeds them delivery data, and runs the control tick. Inputs arrive
 * on the Kafka thread ({@link #lineItem}, {@link #minute}); {@link #tick} runs on a timer.
 */
public final class PacingService {
    private static final Logger log = LoggerFactory.getLogger(PacingService.class);

    private final PacingPeriod period;
    private final PacingStore store;
    private final LongSupplier clock;
    private final Map<String, LineItemPacer> pacers = new ConcurrentHashMap<>();
    /** Latest revision seen per line item minute, so older revisions arriving late are ignored. */
```

```
33      private final Map<String, Integer> revisions = new ConcurrentHashMap<>();
34      /**
35       * Delivery per line item and minute, kept here as well as in the pacers: the two input topics
36       * are read together, so a line item's minutes can arrive before its config, and a changed
37       * config replaces the pacer. Either way the new pacer is replayed from this.
38       */
39      private final Map<String, Map<Long, Long>> delivered = new ConcurrentHashMap<>();
40      /**
41       * How far delivery is known: the end of the latest minute reported by any line item. The stream
42       * processor's watermark is global, so all line items' minutes up to here have fired.
43       */
44      private final AtomicLong horizonMs = new AtomicLong(Long.MIN_VALUE);
45      private final Counter storeErrors;
46      private volatile List<PacingState> latest = List.of();
47      /** When pacing state last reached the store; the ad server falls back if this gets old. */
48      private final AtomicLong lastPublishMs;
49      /** Overspend threshold for the overspent-line-items gauge: 1% over budget. */
50      static final double OVERSPEND_TOLERANCE = 1.01;
51
52      public PacingService(PacingPeriod period, PacingStore store, LongSupplier clock, MeterRegistry registry) {
53          this.period = period;
54          this.store = store;
55          this.clock = clock;
56          this.storeErrors = registry.counter("pacing.store.errors");
57          this.lastPublishMs = new AtomicLong(clock.getAsLong());
58          registry.gauge("pacing.line.items", pacers, Map::size);
59          Gauge.builder("pacing.publish.age.seconds", () -> (clock.getAsLong() - lastPublishMs.get()) / 1000.0)
60                  .description("Seconds since pacing state was last written to Redis").register(registry);
61          Gauge.builder("pacing.delivery.lag.seconds", () -> horizonMs.get() == Long.MIN_VALUE
62                          ? Double.NaN : (clock.getAsLong() - horizonMs.get()) / 1000.0)
63                  .description("How far behind real time delivered spend from the stream processor is").register(r
  ↪ egistry);
64          Gauge.builder("pacing.period.elapsed.ratio", () -> period.elapsed(clock.getAsLong())).register(registry)
  ↪ ;
65          for (PacingState.Status status : PacingState.Status.values()) {
66              Gauge.builder("pacing.line.items.by.status", () -> latest.stream().filter(s -> s.status() == status)
  ↪ .count())
67                      .tag("status", status.name()).register(registry);
68          }
69          Gauge.builder("pacing.delivered.budget.ratio.max", () -> latest.stream()
70                          .filter(s -> s.budgetMicros() > 0)
71                          .mapToDouble(s -> (double) s.deliveredMicros() / s.budgetMicros()).max().orElse(0))
72                  .description("Highest delivered / budget among line items this period").register(registry);
73          Gauge.builder("pacing.overspent.line.items", () -> latest.stream()
74                          .filter(s -> s.deliveredMicros() > s.budgetMicros() * OVERSPEND_TOLERANCE).count())
75                  .description("Line items more than 1% over budget this period").register(registry);
76      }
77
78      public void lineItem(LineItemConfig config) {
79          pacers.compute(config.lineItemId(), (id, old) -> {
80              if (old != null && old.config().equals(config)) {
81                  return old;
82              }
83              var pacer = new LineItemPacer(config, period);
84              delivered.getOrDefault(id, Map.of()).forEach(pacer::delivered);
85              return pacer;
86          });
87      }
88
89      public void minute(LineItemMinute m) {
90          String key = m.key();
91          Integer seen = revisions.get(key);
92          if (seen != null && seen >= m.revision()) {
93              return;
94          }
95          revisions.put(key, m.revision());
96          horizonMs.accumulateAndGet(m.windowEndMs(), Math::max);
97          delivered.computeIfAbsent(m.lineItemId(), k -> new ConcurrentHashMap<>()).put(m.windowStartMs(), m.spend
  ↪ Micros());
98          LineItemPacer pacer = pacers.get(m.lineItemId());
99          if (pacer != null) {
100             pacer.delivered(m.windowStartMs(), m.spendMicros());
101         }
102     }
103
104     /** One control step for every line item. Store failures are logged and retried next tick. */
105     public List<PacingState> tick() {
106         long now = clock.getAsLong();
107         long periodIndex = period.index(now);
108         long horizon = horizonMs.get() == Long.MIN_VALUE ? period.start(periodIndex) : horizonMs.get();
109         Map<String, Long> served;
110         try {
111             served = store.served(periodIndex, pacers.keySet());
112         } catch (Exception e) {
113             storeErrors.increment();
114             log.warn("cannot read served spend, skipping tick: {}", e.toString());
115             return latest;
116         }
```

```
        var states = new ArrayList<PacingState>(pacers.size());
        for (LineItemPacer pacer : pacers.values()) {
            states.add(pacer.tick(now, horizon, served.getOrDefault(pacer.config().lineItemId(), 0L)));
        }
        states.sort(Comparator.comparing(PacingState::lineItemId));
        try {
            store.publish(states);
            lastPublishMs.set(now);
        } catch (Exception e) {
            storeErrors.increment();
            log.warn("cannot publish pacing state: {}", e.toString());
        }
        pruneRevisions(period.start(periodIndex));
        latest = List.copyOf(states);
        return latest;
    }

    public List<PacingState> latest() {
        return latest;
    }

    public long horizonMs() {
        return horizonMs.get();
    }

    public LineItemConfig config(String lineItemId) {
        LineItemPacer p = pacers.get(lineItemId);
        return p == null ? null : p.config();
    }

    /** Bookkeeping is only needed for the current period. */
    private void pruneRevisions(long periodStart) {
        revisions.keySet().removeIf(k -> Long.parseLong(k.substring(k.lastIndexOf('|') + 1)) < periodStart);
        delivered.values().forEach(byMinute -> byMinute.keySet().removeIf(m -> m < periodStart));
    }
}
```

pacing-service/src/main/java/dev/primetech/ads/pacing/PacingStore.java

16 lines · Where pacing reads served spend and publishes its decisions (Redis in production).

```
package dev.primetech.ads.pacing;

import dev.primetech.ads.common.pacing.PacingState;

import java.util.Collection;
import java.util.List;
import java.util.Map;

/** Where pacing reads served spend and publishes its decisions (Redis in production). */
public interface PacingStore {

    /** The ad server's served-spend counters for these line items in the given period. */
    Map<String, Long> served(long periodIndex, Collection<String> lineItemIds) throws Exception;

    void publish(List<PacingState> states) throws Exception;
}
```

pacing-service/src/main/java/dev/primetech/ads/pacing/RedisPacingStore.java

47 lines · Pacing state in one Redis hash, one field per line item, written in a single HSET per tick.

```
package dev.primetech.ads.pacing;

import dev.primetech.ads.common.Json;
import dev.primetech.ads.common.pacing.PacingState;
import dev.primetech.ads.common.pacing.ServedSpend;
import redis.clients.jedis.JedisPooled;

import java.io.IOException;
import java.net.URI;
import java.util.Collection;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/** Pacing state in one Redis hash, one field per line item, written in a single HSET per tick. */
public final class RedisPacingStore implements PacingStore, AutoCloseable {
    private final JedisPooled redis;

    public RedisPacingStore(String url) {
        this.redis = new JedisPooled(URI.create(url));
    }

    @Override
    public Map<String, Long> served(long periodIndex, Collection<String> lineItemIds) {
        var out = new HashMap<String, Long>();
        lineItemIds.forEach(id -> out.put(id, 0L));
        out.putAll(ServedSpend.read(redis, periodIndex, lineItemIds));
        return out;
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
    }

    @Override
    public void publish(List<PacingState> states) throws IOException {
        if (states.isEmpty()) {
            return;
        }
        var fields = new HashMap<String, String>();
        for (PacingState s : states) {
            fields.put(s.lineItemId(), Json.MAPPER.writeValueAsString(s));
        }
        redis.hset(PacingState.REDIS_HASH, fields);
    }

    @Override
    public void close() {
        redis.close();
    }
}
```

pacing-service/src/main/resources/simplelogger.properties

7 lines · The Kafka client logs its full config at INFO on startup; keep only warnings.

```
org.slf4j.simpleLogger.defaultLogLevel=info
org.slf4j.simpleLogger.showDateTime=true
org.slf4j.simpleLogger.dateTimeFormat=yyyy-MM-dd'T'HH:mm:ss.SSSZ
org.slf4j.simpleLogger.showThreadName=false
org.slf4j.simpleLogger.showShortLogName=true
# The Kafka client logs its full config at INFO on startup; keep only warnings.
org.slf4j.simpleLogger.log.org.apache.kafka=warn
```

pacing-service/src/test/java/dev/primetech/ads/pacing/LineItemPacerTest.java

194 lines · Closed-loop tests: the pacer drives a simulated market and must hit its budget.

```
package dev.primetech.ads.pacing;

import dev.primetech.ads.common.catalog.LineItemConfig;
import dev.primetech.ads.common.pacing.PacingPeriod;
import dev.primetech.ads.common.pacing.PacingState;
import org.junit.jupiter.api.Test;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.SplittableRandom;
import java.util.function.LongToDoubleFunction;

import static org.assertj.core.api.Assertions.assertThat;

/**
 * Closed-loop tests: the pacer drives a simulated market and must hit its budget. The market
 * mimics the real system's delays: spend is reported per event-time minute, about 70 s after the
 * minute starts, ads play up to 30 s after they are served, and only 80% of served spend
 * becomes delivered spend.
 */
class LineItemPacerTest {
    private static final long MINUTE = 60_000;
    private static final PacingPeriod PERIOD = PacingPeriod.of(Duration.ofMinutes(30));
    private static final long T0 = PERIOD.start(1_000_000);
    private static final double DELIVERY_RATIO = 0.8;
    private static final long REPORT_LAG_MS = 70_000;
    /** Served ads play between 0 and this long later (15 s on average, as measured end to end). */
    private static final long MAX_PLAY_DELAY_MS = 30_000;

    record Run(List<PacingState> states, long finalDelivered, long budget, double maxTrackingError) {
    }

    /**
     * @param yieldPerMinute delivered spend per minute the line item would get at throttle 1, over time
     */
    static Run simulate(long budgetMicros, LongToDoubleFunction yieldPerMinute, long seed) {
        Span span = simulate(budgetMicros, yieldPerMinute, seed, T0, PERIOD.end(1_000_000), MAX_PLAY_DELAY_MS);
        return new Run(span.states(), span.deliveredByPeriod().get(1_000_000L), budgetMicros,
                span.maxTrackingError().get(1_000_000L));
    }

    record Span(List<PacingState> states, java.util.Map<Long, Long> deliveredByPeriod,
                java.util.Map<Long, Double> maxTrackingError) {
    }

    /**
     * Runs the loop from {@code fromMs} to {@code toMs}, possibly across period boundaries. Served
     * ads play up to {@code maxPlayDelayMs} later, and are delivered (and attributed) at play time,
     * so ads served at the end of one period count towards the next, as they do in the real system.
     */
    static Span simulate(long budgetMicros, LongToDoubleFunction yieldPerMinute, long seed, long fromMs, long to
↪ Ms,
                         long maxPlayDelayMs) {
```

```
54          var config = new LineItemConfig("li", "camp", "adv", LineItemConfig.Tier.STANDARD, 40_000_000, budgetMic
  ↪ ros);
55          var pacer = new LineItemPacer(config, PERIOD);
56          var random = new SplittableRandom(seed);
57          long served = 0;
58          long servedPeriod = PERIOD.index(fromMs);
59          double throttle = LineItemPacer.INITIAL_THROTTLE;
60          long stopAt = Long.MAX_VALUE;
61          var deliveredByMinute = new java.util.TreeMap<Long, Long>();
62          var states = new ArrayList<PacingState>();
63          var maxTrackingError = new java.util.HashMap<Long, Double>();
64
65          // One-second steps: the ad server serves at the current throttle until its stop threshold.
66          for (long t = fromMs; t < toMs; t += 1_000) {
67              if (PERIOD.index(t) != servedPeriod) {
68                  // The ad server's served counter is per period.
69                  servedPeriod = PERIOD.index(t);
70                  served = 0;
71                  stopAt = Long.MAX_VALUE;
72              }
73              if (served < stopAt) {
74                  double perSecond = throttle * yieldPerMinute.applyAsDouble(t - fromMs) / 60 * (0.7 + 0.6 * rando
  ↪ m.nextDouble());
75                  long servedNow = (long) (perSecond / DELIVERY_RATIO);
76                  served += servedNow;
77                  long playedAt = t + (maxPlayDelayMs > 0 ? random.nextLong(maxPlayDelayMs) : 0);
78                  deliveredByMinute.merge(Math.floorDiv(playedAt, MINUTE) * MINUTE, (long) (servedNow * DELIVERY_R
  ↪ ATIO), Long::sum);
79              }
80              if ((t - fromMs) % 10_000 == 0) {
81                  // Report every minute whose end is at least REPORT_LAG_MS - 60 s in the past.
82                  long horizon = Math.floorDiv(t - (REPORT_LAG_MS - MINUTE), MINUTE) * MINUTE;
83                  for (var e : deliveredByMinute.headMap(horizon).entrySet()) {
84                      pacer.delivered(e.getKey(), e.getValue());
85                  }
86                  PacingState s = pacer.tick(t, horizon, served);
87                  throttle = s.throttle();
88                  stopAt = s.stopAtServedMicros();
89                  states.add(s);
90                  if (t - PERIOD.start(PERIOD.index(t)) > 5 * MINUTE) {
91                      maxTrackingError.merge(s.periodIndex(),
92                              Math.abs(s.deliveredMicros() - s.targetMicros()) / (double) budgetMicros, Math::max)
  ↪ ;
93                  }
94              }
95          }
96          var deliveredByPeriod = new java.util.HashMap<Long, Long>();
97          deliveredByMinute.forEach((minute, spend) -> deliveredByPeriod.merge(PERIOD.index(minute), spend, Long::
  ↪ sum));
98          return new Span(states, deliveredByPeriod, maxTrackingError);
99      }
100
101     @Test
102     void spendsTheBudgetEvenlyAndStopsOnTime() {
103         // $10/min available, $90 budget over 30 min: needs a throttle around 0.3.
104         Run run = simulate(90_000_000, t -> 10_000_000, 1);
105
106         assertThat((double) run.finalDelivered() / run.budget()).isBetween(0.97, 1.005);
107         assertThat(run.maxTrackingError()).as("delivered stays close to the even curve").isLessThan(0.04);
108         assertThat(run.states().get(run.states().size() - 1).status()).isIn(PacingState.Status.EXHAUSTED, Pacing
  ↪ State.Status.ON_PACE);
109     }
110
111     @Test
112     void absorbsATrafficSurgeWithoutOverspending() {
113         // Traffic triples halfway through.
114         Run run = simulate(90_000_000, t -> t < 15 * MINUTE ? 10_000_000 : 30_000_000, 2);
115
116         assertThat((double) run.finalDelivered() / run.budget()).isBetween(0.97, 1.005);
117         assertThat(run.maxTrackingError()).isLessThan(0.06);
118     }
119
120     @Test
121     void canSpendNearlyAllAvailableTraffic() {
122         // The budget needs 90% of everything the line item could win.
123         Run run = simulate(270_000_000, t -> 10_000_000, 5);
124
125         assertThat((double) run.finalDelivered() / run.budget()).isBetween(0.97, 1.005);
126         assertThat(run.maxTrackingError()).isLessThan(0.04);
127     }
128
129     @Test
130     void reportsUnderDeliveryWhenTrafficCannotCoverTheBudget() {
131         // $5/min available, $300 budget: impossible.
132         Run run = simulate(300_000_000, t -> 5_000_000, 3);
133
134         PacingState last = run.states().get(run.states().size() - 1);
135         assertThat(last.throttle()).isEqualTo(1.0);
```

```
136          assertThat(last.status()).isEqualTo(PacingState.Status.UNDER_DELIVERING);
137          assertThat(run.finalDelivered()).isLessThan(run.budget());
138      }
139
140      @Test
141      void holdsBackWhenTrafficIsPlentiful() {
142          // $100/min available for a $30 budget: throttle must go very low and stay smooth.
143          Run run = simulate(30_000_000, t -> 100_000_000, 4);
144
145          assertThat((double) run.finalDelivered() / run.budget()).isBetween(0.97, 1.005);
146          assertThat(run.maxTrackingError()).as("brief early overshoot while learning the yield").isLessThan(0.09)
    ↪ ;
147          assertThat(run.states().get(run.states().size() / 2).throttle()).isLessThan(0.05);
148      }
149
150      @Test
151      void startingLateInAPeriodDoesNotOverspendTheNext() {
152          // Found live: the stack started 85% into a period. Chasing 85% of the budget at once drove
153          // the throttle to 1, the next period inherited it, and ads served at throttle 1 just before
154          // the boundary played (and counted) in the next period, which overspent by up to 51%.
155          // Plentiful traffic ($100/min for a $30 budget) and a play delay of up to a minute.
156          long from = T0 + (long) (0.85 * 30 * MINUTE);
157          Span span = simulate(30_000_000, t -> 100_000_000, 6, from, PERIOD.end(1_000_001), MAX_PLAY_DELAY_MS);
158
159          assertThat(span.deliveredByPeriod().get(1_000_000L)).as("the partial period catches up at most 2x the ev
    ↪ en rate")
160                  .isLessThan(30_000_000L * 15 / 100 * 2 + 3_000_000);
161          double next = span.deliveredByPeriod().get(1_000_001L) / 30_000_000.0;
162          assertThat(next).as("the first full period").isBetween(0.97, 1.01);
163          assertThat(span.maxTrackingError().get(1_000_001L)).isLessThan(0.09);
164      }
165
166      @Test
167      void stopsOnServedSpendBeforeDeliveryIsReported() {
168          var config = new LineItemConfig("li", "c", "a", LineItemConfig.Tier.STANDARD, 1, 10_000_000);
169          var pacer = new LineItemPacer(config, PERIOD);
170          // Nothing reported yet (horizon at period start), but $11 already served. Until the
171          // delivery ratio is observed, the pacer assumes every served ad plays, so it stops at the
172          // budget in served spend: it cannot overspend however many served ads go unplayed.
173          PacingState s = pacer.tick(T0 + 5 * MINUTE, T0, 11_000_000);
174
175          assertThat(s.stopAtServedMicros()).isEqualTo(10_000_000);
176          assertThat(s.status()).isEqualTo(PacingState.Status.EXHAUSTED);
177          assertThat(s.throttle()).isZero();
178      }
179
180      @Test
181      void startsAFreshBudgetEachPeriod() {
182          var config = new LineItemConfig("li", "c", "a", LineItemConfig.Tier.STANDARD, 1, 10_000_000);
183          var pacer = new LineItemPacer(config, PERIOD);
184          pacer.delivered(T0, 10_000_000);
185          assertThat(pacer.tick(T0 + 2 * MINUTE, T0 + 2 * MINUTE, 12_500_000).status()).isEqualTo(PacingState.Stat
    ↪ us.EXHAUSTED);
186
187          long next = PERIOD.end(1_000_000);
188          PacingState s = pacer.tick(next + MINUTE, next, 0);
189          assertThat(s.periodIndex()).isEqualTo(1_000_001);
190          assertThat(s.deliveredMicros()).isZero();
191          assertThat(s.status()).isNotEqualTo(PacingState.Status.EXHAUSTED);
192          assertThat(s.throttle()).isPositive();
193      }
194  }
```

pacing-service/src/test/java/dev/primetech/ads/pacing/PacingServiceTest.java

120 lines · Test.

```
1   package dev.primetech.ads.pacing;
2
3   import dev.primetech.ads.common.LineItemMinute;
4   import dev.primetech.ads.common.catalog.LineItemConfig;
5   import dev.primetech.ads.common.pacing.PacingPeriod;
6   import dev.primetech.ads.common.pacing.PacingState;
7   import io.micrometer.core.instrument.simple.SimpleMeterRegistry;
8   import org.junit.jupiter.api.Test;
9
10  import java.time.Duration;
11  import java.util.Collection;
12  import java.util.HashMap;
13  import java.util.List;
14  import java.util.Map;
15  import java.util.concurrent.atomic.AtomicLong;
16
17  import static org.assertj.core.api.Assertions.assertThat;
18
19  class PacingServiceTest {
20      private static final PacingPeriod PERIOD = PacingPeriod.of(Duration.ofMinutes(30));
21      private static final long T0 = PERIOD.start(1_000_000);
22
```

```
    static final class FakeStore implements PacingStore {
        final Map<String, Long> served = new HashMap<>();
        List<PacingState> published = List.of();
        boolean failing;

        @Override
        public Map<String, Long> served(long periodIndex, Collection<String> ids) {
            if (failing) {
                throw new IllegalStateException("redis down");
            }
            return served;
        }

        @Override
        public void publish(List<PacingState> states) {
            published = states;
        }
    }

    private final FakeStore store = new FakeStore();
    private final AtomicLong now = new AtomicLong(T0 + 10 * 60_000);
    private final SimpleMeterRegistry registry = new SimpleMeterRegistry();
    private final PacingService service = new PacingService(PERIOD, store, now::get, registry);

    private double gauge(String name, String... tags) {
        return registry.get(name).tags(tags).gauge().value();
    }

    private static LineItemConfig config(String id, long budget) {
        return new LineItemConfig(id, "camp", "adv", LineItemConfig.Tier.STANDARD, 30_000_000, budget);
    }

    private static LineItemMinute minute(String li, long start, int revision, long spend) {
        return new LineItemMinute(li, "camp", "adv", start, start + 60_000, revision, 1, 0, 0, 0, 0, 0, spend, 0
  ↪ , start);
    }

    @Test
    void countsMinutesThatArriveBeforeTheirConfig() {
        service.minute(minute("li-1", T0, 1, 4_000_000));
        service.minute(minute("li-1", T0 + 60_000, 1, 6_000_000));
        service.lineItem(config("li-1", 90_000_000));

        PacingState s = service.tick().get(0);
        assertThat(s.deliveredMicros()).isEqualTo(10_000_000);
        assertThat(store.published).containsExactly(s);
    }

    @Test
    void keepsOnlyTheLatestRevisionOfEachMinute() {
        service.lineItem(config("li-1", 90_000_000));
        service.minute(minute("li-1", T0, 2, 7_000_000));
        service.minute(minute("li-1", T0, 1, 5_000_000)); // older revision arriving late

        assertThat(service.tick().get(0).deliveredMicros()).isEqualTo(7_000_000);
    }

    @Test
    void aChangedBudgetKeepsDeliverySoFar() {
        service.lineItem(config("li-1", 90_000_000));
        service.minute(minute("li-1", T0, 1, 9_000_000));
        service.lineItem(config("li-1", 10_000_000));

        PacingState s = service.tick().get(0);
        assertThat(s.budgetMicros()).isEqualTo(10_000_000);
        assertThat(s.deliveredMicros()).isEqualTo(9_000_000);
    }

    @Test
    void aStoreOutageKeepsTheLastDecisions() {
        service.lineItem(config("li-1", 90_000_000));
        List<PacingState> first = service.tick();
        store.failing = true;

        assertThat(service.tick()).isEqualTo(first);
    }

    @Test
    void exposesHealthAndOverspendAsMetrics() {
        service.lineItem(config("li-ok", 90_000_000));
        service.lineItem(config("li-over", 10_000_000));
        // Five minutes delivered by the horizon: li-over is at 120% of its budget.
        service.minute(minute("li-ok", T0, 1, 10_000_000));
        service.minute(minute("li-over", T0 + 4 * 60_000, 1, 12_000_000));
        service.tick();

        assertThat(gauge("pacing.overspent.line.items")).isEqualTo(1);
        assertThat(gauge("pacing.delivered.budget.ratio.max")).isEqualTo(1.2);
```

```
        assertThat(gauge("pacing.line.items.by.status", "status", "EXHAUSTED")).isEqualTo(1);
        assertThat(gauge("pacing.delivery.lag.seconds")).isEqualTo(5 * 60);
        assertThat(gauge("pacing.publish.age.seconds")).isZero();

        // Redis down for two minutes: state stops reaching the ad server, and the gauge shows it.
        store.failing = true;
        now.addAndGet(120_000);
        service.tick();
        assertThat(gauge("pacing.publish.age.seconds")).isEqualTo(120);
    }
}
```

stream-processor

stream-processor/pom.xml

91 lines

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>dev.primetech.ads</groupId>
    <artifactId>ads-eventing</artifactId>
    <version>0.1.0-SNAPSHOT</version>
  </parent>

  <artifactId>stream-processor</artifactId>
  <name>Stream Processor (Flink)</name>

  <properties>
    <main.class>dev.primetech.ads.stream.PipelineJob</main.class>
    <flink.version>2.2.1</flink.version>
    <flink.kafka.connector.version>5.0.0-2.2</flink.kafka.connector.version>
    <!-- The connector is built against this client; the other services stay on the parent's version. -->
    <flink.kafka.clients.version>4.2.0</flink.kafka.clients.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>dev.primetech.ads</groupId>
      <artifactId>ads-common</artifactId>
    </dependency>

    <!-- Provided by the Flink runtime; not packaged into the job jar. -->
    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-streaming-java</artifactId>
      <version>${flink.version}</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-clients</artifactId>
      <version>${flink.version}</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-connector-base</artifactId>
      <version>${flink.version}</version>
      <scope>provided</scope>
    </dependency>

    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-connector-kafka</artifactId>
      <version>${flink.kafka.connector.version}</version>
    </dependency>
    <dependency>
      <groupId>org.apache.kafka</groupId>
      <artifactId>kafka-clients</artifactId>
      <version>${flink.kafka.clients.version}</version>
    </dependency>

    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-test-utils</artifactId>
      <version>${flink.version}</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-streaming-java</artifactId>
      <version>${flink.version}</version>
```

© 2026 QUILA VISUALS. Prepared for the Ads Eventing project.

```
      <type>test-jar</type>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-runtime</artifactId>
      <version>${flink.version}</version>
      <type>test-jar</type>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <finalName>stream-processor</finalName>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-shade-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

stream-processor/src/main/java/dev/primetech/ads/stream/AdPipeline.java

82 lines · The processing topology, independent of Kafka so tests can drive it with in-memory sources.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.EnrichedAdEvent;
import dev.primetech.ads.common.LineItemMinute;
import org.apache.flink.api.common.typeinfo.TypeInformation;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.datastream.SingleOutputStreamOperator;
import org.apache.flink.streaming.api.windowing.assigners.TumblingEventTimeWindows;
import org.apache.flink.util.OutputTag;

import java.time.Duration;

/**
 * The processing topology, independent of Kafka so tests can drive it with in-memory sources.
 *
 * <pre>
 * events ─► dedup (by eventId) ─► enrich (by campaignId, + broadcast metadata) ─► IVT (by device) ─┬─► clean ev
↪ ents
 *                                                                                                  └─► 1-min wi
↪ ndows (by line item) ─► line-item minutes
 * </pre>
 *
 * The input stream must already carry event-time timestamps and watermarks.
 */
public final class AdPipeline {
    /** Events too late even for the allowed lateness. They are still in the clean output; only the real-time to
↪ tals miss them. */
    public static final OutputTag<EnrichedAdEvent> TOO_LATE =
            new OutputTag<>("too-late-for-minute-totals", TypeInformation.of(EnrichedAdEvent.class));

    public record Outputs(DataStream<EnrichedAdEvent> cleanEvents,
                          DataStream<LineItemMinute> lineItemMinutes,
                          DataStream<EnrichedAdEvent> tooLate) {
    }

    private AdPipeline() {
    }

    public static Outputs build(DataStream<AdEvent> events, DataStream<CampaignMetadata> metadata, PipelineConfi
↪ g config) {
        DataStream<AdEvent> unique = events
                .keyBy(AdEvent::eventId)
                .process(new Deduplicate(config.dedupTtl()))
                .name("dedup-by-event-id").uid("dedup");

        DataStream<EnrichedAdEvent> enriched = unique
                .keyBy(AdEvent::campaignId)
                .connect(metadata.broadcast(EnrichWithCampaign.METADATA))
                .process(new EnrichWithCampaign(config.metadataWait()))
                .name("enrich-with-campaign").uid("enrich");

        // Keying by a null id would funnel every non-consented event to one subtask, so only
        // events with a device id go through the per-device rule.
        DataStream<EnrichedAdEvent> withDevice = enriched.filter(e -> e.deviceIdHash() != null).name("has-device
↪ -id");
        DataStream<EnrichedAdEvent> withoutDevice = enriched.filter(e -> e.deviceIdHash() == null).name("no-devi
↪ ce-id");
        DataStream<EnrichedAdEvent> clean = withDevice
                .keyBy(EnrichedAdEvent::deviceIdHash)
                .process(new InvalidTrafficFilter(config.ivtMaxImpressionsPerDeviceMinute(), config.allowedLaten
```

```
  ↪ ess()))
57                  .name("flag-invalid-traffic").uid("ivt")
58                  .union(withoutDevice)
59                  .transform("restore-event-time", TypeInformation.of(EnrichedAdEvent.class), new RestoreEventTime
  ↪ ())
60                  .uid("restore-event-time");
61
62          SingleOutputStreamOperator<LineItemMinute> minutes = minuteTotals(clean, config);
63          return new Outputs(clean, minutes, minutes.getSideOutput(TOO_LATE));
64      }
65
66      /**
67       * One-minute event-time totals per line item. A window first fires when the watermark passes
68       * its end, then again (a new revision) for late events within the allowed lateness, at most
69       * once per late-firing delay.
70       */
71      static SingleOutputStreamOperator<LineItemMinute> minuteTotals(DataStream<EnrichedAdEvent> events,
72                                                                     PipelineConfig config) {
73          return events
74                  .keyBy(EnrichedAdEvent::lineItemId)
75                  .window(TumblingEventTimeWindows.of(Duration.ofMinutes(1)))
76                  .trigger(new CoalescingLateTrigger(config.lateFiringDelay()))
77                  .allowedLateness(config.allowedLateness())
78                  .sideOutputLateData(TOO_LATE)
79                  .aggregate(new LineItemTotals(), new LineItemTotals.Emit())
80                  .name("line-item-minute-totals").uid("line-item-minutes");
81      }
82  }
```

stream-processor/src/main/java/dev/primetech/ads/stream/CoalescingLateTrigger.java

74 lines · Event-time trigger with batched late firings.

```
 1  package dev.primetech.ads.stream;
 2
 3  import org.apache.flink.api.common.state.ValueState;
 4  import org.apache.flink.api.common.state.ValueStateDescriptor;
 5  import org.apache.flink.streaming.api.windowing.triggers.Trigger;
 6  import org.apache.flink.streaming.api.windowing.triggers.TriggerResult;
 7  import org.apache.flink.streaming.api.windowing.windows.TimeWindow;
 8
 9  import java.time.Duration;
10
11  /**
12   * Event-time trigger with batched late firings. The window fires once when the watermark passes
13   * its end, like the default trigger. Late events then schedule at most one extra firing per
14   * {@code lateDelay} instead of one per event, which would otherwise rewrite the same window
15   * hundreds of times when a burst of buffered device events arrives.
16   *
17   * <p>Caveat: a late firing still pending when the window's allowed lateness runs out is dropped
18   * with the window. That needs the watermark to advance by the whole allowed lateness within
19   * {@code lateDelay}, which only happens when catching up on a backlog.
20   */
21  public final class CoalescingLateTrigger extends Trigger<Object, TimeWindow> {
22      private static final ValueStateDescriptor<Long> PENDING = new ValueStateDescriptor<>("late-firing-at", Long.
  ↪ class);
23
24      private final long lateDelayMs;
25
26      public CoalescingLateTrigger(Duration lateDelay) {
27          this.lateDelayMs = lateDelay.toMillis();
28      }
29
30      @Override
31      public TriggerResult onElement(Object element, long timestamp, TimeWindow window, TriggerContext ctx) throws
  ↪  Exception {
32          if (window.maxTimestamp() > ctx.getCurrentWatermark()) {
33              ctx.registerEventTimeTimer(window.maxTimestamp());
34              return TriggerResult.CONTINUE;
35          }
36          // The window has already fired: this is a late event.
37          if (lateDelayMs == 0) {
38              return TriggerResult.FIRE;
39          }
40          ValueState<Long> pending = ctx.getPartitionedState(PENDING);
41          if (pending.value() == null) {
42              long fireAt = ctx.getCurrentProcessingTime() + lateDelayMs;
43              ctx.registerProcessingTimeTimer(fireAt);
44              pending.update(fireAt);
45          }
46          return TriggerResult.CONTINUE;
47      }
48
49      @Override
50      public TriggerResult onEventTime(long time, TimeWindow window, TriggerContext ctx) {
51          return time == window.maxTimestamp() ? TriggerResult.FIRE : TriggerResult.CONTINUE;
52      }
53
54      @Override
```

```
    public TriggerResult onProcessingTime(long time, TimeWindow window, TriggerContext ctx) throws Exception {
        ctx.getPartitionedState(PENDING).clear();
        return TriggerResult.FIRE;
    }

    @Override
    public void clear(TimeWindow window, TriggerContext ctx) throws Exception {
        ctx.deleteEventTimeTimer(window.maxTimestamp());
        ValueState<Long> pending = ctx.getPartitionedState(PENDING);
        if (pending.value() != null) {
            ctx.deleteProcessingTimeTimer(pending.value());
            pending.clear();
        }
    }

    @Override
    public String toString() {
        return "CoalescingLateTrigger(" + lateDelayMs + "ms)";
    }
}
```

stream-processor/src/main/java/dev/primetech/ads/stream/Deduplicate.java

51 lines · Drops repeats of an event id.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.AdEvent;
import org.apache.flink.api.common.functions.OpenContext;
import org.apache.flink.api.common.state.StateTtlConfig;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.metrics.Counter;
import org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import org.apache.flink.util.Collector;

import java.time.Duration;

/**
 * Drops repeats of an event id. Keyed by event id, so the state is one small flag per event seen
 * within the TTL; with checkpointed state and a transactional sink, each event reaches the output
 * exactly once even across failures.
 *
 * <p>The TTL bounds memory: a duplicate arriving later than the TTL gets through. Client retries
 * happen within seconds, so 30 minutes leaves a wide margin.
 */
public final class Deduplicate extends KeyedProcessFunction<String, AdEvent, AdEvent> {
    private final Duration ttl;
    private transient ValueState<Boolean> seen;
    private transient Counter duplicates;

    public Deduplicate(Duration ttl) {
        this.ttl = ttl;
    }

    @Override
    public void open(OpenContext openContext) {
        var descriptor = new ValueStateDescriptor<>("seen", Boolean.class);
        descriptor.enableTimeToLive(StateTtlConfig.newBuilder(ttl)
                .setUpdateType(StateTtlConfig.UpdateType.OnCreateAndWrite)
                .setStateVisibility(StateTtlConfig.StateVisibility.NeverReturnExpired)
                .build());
        seen = getRuntimeContext().getState(descriptor);
        duplicates = getRuntimeContext().getMetricGroup().counter("duplicatesDropped");
    }

    @Override
    public void processElement(AdEvent event, Context ctx, Collector<AdEvent> out) throws Exception {
        if (seen.value() != null) {
            duplicates.inc();
            return;
        }
        seen.update(true);
        out.collect(event);
    }
}
```

stream-processor/src/main/java/dev/primetech/ads/stream/EnrichWithCampaign.java

102 lines · Joins campaign metadata onto events.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.EnrichedAdEvent;
import org.apache.flink.api.common.functions.OpenContext;
import org.apache.flink.api.common.state.ListState;
import org.apache.flink.api.common.state.ListStateDescriptor;
import org.apache.flink.api.common.state.MapStateDescriptor;
```

```java
import org.apache.flink.api.common.state.ReadOnlyBroadcastState;
import org.apache.flink.metrics.Counter;
import org.apache.flink.streaming.api.functions.co.KeyedBroadcastProcessFunction;
import org.apache.flink.util.Collector;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;

/**
 * Joins campaign metadata onto events. Metadata is small and changes rarely, so it is broadcast to
 * every parallel instance instead of being looked up per event.
 *
 * <p>Events for a campaign whose metadata has not arrived yet (typical just after start-up) are
 * buffered per campaign and released as soon as it arrives. If it never does, they are released
 * unenriched after {@code maxWait} and counted, rather than held forever.
 *
 * <p>Records released from the broadcast side or a timer lose their event timestamp; the pipeline
 * restores it from the payload before windowing ({@link RestoreEventTime}).
 */
public final class EnrichWithCampaign
        extends KeyedBroadcastProcessFunction<String, AdEvent, CampaignMetadata, EnrichedAdEvent> {

    public static final MapStateDescriptor<String, CampaignMetadata> METADATA =
            new MapStateDescriptor<>("campaign-metadata", String.class, CampaignMetadata.class);
    static final ListStateDescriptor<AdEvent> BUFFER = new ListStateDescriptor<>("awaiting-metadata", AdEvent.cl
  ↪ ass);

    private final Duration maxWait;
    private transient ListState<AdEvent> buffer;
    private transient Counter released;
    private transient Counter missing;

    public EnrichWithCampaign(Duration maxWait) {
        this.maxWait = maxWait;
    }

    @Override
    public void open(OpenContext openContext) {
        buffer = getRuntimeContext().getListState(BUFFER);
        released = getRuntimeContext().getMetricGroup().counter("bufferedEventsReleased");
        missing = getRuntimeContext().getMetricGroup().counter("metadataMissing");
    }

    @Override
    public void processElement(AdEvent event, ReadOnlyContext ctx, Collector<EnrichedAdEvent> out) throws Except
  ↪ ion {
        ReadOnlyBroadcastState<String, CampaignMetadata> metadata = ctx.getBroadcastState(METADATA);
        CampaignMetadata meta = metadata.get(event.campaignId());
        if (meta != null) {
            out.collect(EnrichedAdEvent.of(event, meta, ctx.currentProcessingTime()));
            return;
        }
        if (!buffer.get().iterator().hasNext()) {
            ctx.timerService().registerProcessingTimeTimer(ctx.currentProcessingTime() + maxWait.toMillis());
        }
        buffer.add(event);
    }

    @Override
    public void processBroadcastElement(CampaignMetadata meta, Context ctx, Collector<EnrichedAdEvent> out)
            throws Exception {
        ctx.getBroadcastState(METADATA).put(meta.campaignId(), meta);
        long now = ctx.currentProcessingTime();
        ctx.applyToKeyedState(BUFFER, (campaignId, state) -> {
            if (!campaignId.equals(meta.campaignId())) {
                return;
            }
            for (AdEvent e : state.get()) {
                out.collect(EnrichedAdEvent.of(e, meta, now));
                released.inc();
            }
            state.clear();
        });
    }

    @Override
    public void onTimer(long timestamp, OnTimerContext ctx, Collector<EnrichedAdEvent> out) throws Exception {
        List<AdEvent> waiting = new ArrayList<>();
        buffer.get().forEach(waiting::add);
        if (waiting.isEmpty()) {
            return;
        }
        CampaignMetadata meta = ctx.getBroadcastState(METADATA).get(ctx.getCurrentKey());
        for (AdEvent e : waiting) {
            out.collect(EnrichedAdEvent.of(e, meta, ctx.currentProcessingTime()));
            if (meta == null) {
                missing.inc();
```

```
            } else {
                released.inc();
            }
        }
        buffer.clear();
    }
}
```

stream-processor/src/main/java/dev/primetech/ads/stream/InvalidTrafficFilter.java

71 lines · General invalid traffic (GIVT) rule: a device that fires more impressions in one event-time minute than a person could watch is a bot or a broken integration.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.EnrichedAdEvent;
import dev.primetech.ads.common.EventType;
import org.apache.flink.api.common.functions.OpenContext;
import org.apache.flink.api.common.state.MapState;
import org.apache.flink.api.common.state.MapStateDescriptor;
import org.apache.flink.metrics.Counter;
import org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import org.apache.flink.util.Collector;

import java.time.Duration;

/**
 * General invalid traffic (GIVT) rule: a device that fires more impressions in one event-time
 * minute than a person could watch is a bot or a broken integration. Once a device crosses the
 * threshold in a minute, its later events in that minute are flagged (kept, but not billable).
 *
 * <p>The first {@code maxImpressionsPerMinute} impressions of an offending minute stay unflagged:
 * the rule only knows a device is abusive once it has seen enough. "First" means first to arrive
 * here; upstream re-keying does not preserve a device's order, so which events get flagged can
 * vary between runs, but how many does not. Devices without consent have
 * no stable id, so this rule cannot see them; that gap is inherent to privacy-first tracking.
 */
public final class InvalidTrafficFilter extends KeyedProcessFunction<String, EnrichedAdEvent, EnrichedAdEvent> {
    public static final String DEVICE_IMPRESSION_RATE = "DEVICE_IMPRESSION_RATE";
    private static final long MINUTE_MS = 60_000;

    private final int maxImpressionsPerMinute;
    private final long retentionMs;
    private transient MapState<Long, Integer> impressionsByMinute;
    private transient Counter flagged;

    public InvalidTrafficFilter(int maxImpressionsPerMinute, Duration allowedLateness) {
        this.maxImpressionsPerMinute = maxImpressionsPerMinute;
        this.retentionMs = allowedLateness.toMillis();
    }

    @Override
    public void open(OpenContext openContext) {
        impressionsByMinute = getRuntimeContext().getMapState(
                new MapStateDescriptor<>("impressions-by-minute", Long.class, Integer.class));
        flagged = getRuntimeContext().getMetricGroup().counter("ivtFlagged");
    }

    @Override
    public void processElement(EnrichedAdEvent event, Context ctx, Collector<EnrichedAdEvent> out) throws Except
↪ ion {
        long minute = Math.floorDiv(event.eventTimeMs(), MINUTE_MS);
        Integer count = impressionsByMinute.get(minute);
        int impressions = count == null ? 0 : count;
        if (event.type() == EventType.IMPRESSION) {
            impressions++;
            impressionsByMinute.put(minute, impressions);
            if (count == null) {
                // Forget the minute once no late event can still land in it.
                ctx.timerService().registerEventTimeTimer((minute + 1) * MINUTE_MS + retentionMs);
            }
        }
        if (impressions > maxImpressionsPerMinute) {
            flagged.inc();
            out.collect(event.withIvt(DEVICE_IMPRESSION_RATE));
        } else {
            out.collect(event);
        }
    }

    @Override
    public void onTimer(long timestamp, OnTimerContext ctx, Collector<EnrichedAdEvent> out) throws Exception {
        impressionsByMinute.remove((timestamp - retentionMs) / MINUTE_MS - 1);
    }
}
```

stream-processor/src/main/java/dev/primetech/ads/stream/JsonKafka.java

67 lines · JSON (de)serialization for Kafka using the same mapper as the other services.

```java
package dev.primetech.ads.stream;

import dev.primetech.ads.common.Json;
import org.apache.flink.api.common.serialization.DeserializationSchema;
import org.apache.flink.api.common.typeinfo.TypeInformation;
import org.apache.flink.connector.kafka.sink.KafkaRecordSerializationSchema;
import org.apache.flink.connector.kafka.source.reader.deserializer.KafkaRecordDeserializationSchema;
import org.apache.flink.metrics.Counter;
import org.apache.flink.util.Collector;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.producer.ProducerRecord;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.function.Function;

/** JSON (de)serialization for Kafka using the same mapper as the other services. */
final class JsonKafka {
    private JsonKafka() {
    }

    /**
     * Skips records it cannot parse (and tombstones) instead of failing the job: one bad record
     * must not stop the pipeline. Skips are counted so they show up in monitoring.
     */
    static <T> KafkaRecordDeserializationSchema<T> deserializer(Class<T> type) {
        return new KafkaRecordDeserializationSchema<>() {
            private transient Counter malformed;

            @Override
            public void open(DeserializationSchema.InitializationContext context) {
                malformed = context.getMetricGroup().counter("malformedRecords");
            }

            @Override
            public void deserialize(ConsumerRecord<byte[], byte[]> record, Collector<T> out) {
                if (record.value() == null) {
                    return;
                }
                try {
                    out.collect(Json.MAPPER.readValue(record.value(), type));
                } catch (IOException e) {
                    malformed.inc();
                }
            }

            @Override
            public TypeInformation<T> getProducedType() {
                return TypeInformation.of(type);
            }
        };
    }

    static <T> KafkaRecordSerializationSchema<T> serializer(String topic, SerializableKey<T> key) {
        return (element, context, timestamp) -> {
            try {
                return new ProducerRecord<>(topic, null, timestamp,
                        key.apply(element).getBytes(StandardCharsets.UTF_8), Json.MAPPER.writeValueAsBytes(eleme
↪ nt));
            } catch (IOException e) {
                throw new IllegalStateException("cannot serialize " + element, e);
            }
        };
    }

    interface SerializableKey<T> extends Function<T, String>, java.io.Serializable {
    }
}
```

stream-processor/src/main/java/dev/primetech/ads/stream/LineItemTotals.java

110 lines · Incremental per-window totals, so a window holds one small accumulator instead of every event.

```java
package dev.primetech.ads.stream;

import dev.primetech.ads.common.EnrichedAdEvent;
import dev.primetech.ads.common.LineItemMinute;
import org.apache.flink.api.common.functions.AggregateFunction;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.streaming.api.functions.windowing.ProcessWindowFunction;
import org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

/**
 * Incremental per-window totals, so a window holds one small accumulator instead of every event.
 * Valid traffic feeds the delivery counts and spend; IVT is only counted.
 */
public final class LineItemTotals implements AggregateFunction<EnrichedAdEvent, LineItemTotals.Acc, LineItemTota
```

```
  ↪ ls.Acc> {
 17
 18      /** Flink POJO: public no-arg constructor and public fields. */
 19      public static final class Acc {
 20          public String lineItemId;
 21          public String campaignId;
 22          public String advertiserId;
 23          public long impressions;
 24          public long firstQuartiles;
 25          public long midpoints;
 26          public long thirdQuartiles;
 27          public long completes;
 28          public long clicks;
 29          public long spendMicros;
 30          public long ivtEvents;
 31
 32          public Acc() {
 33          }
 34      }
 35
 36      @Override
 37      public Acc createAccumulator() {
 38          return new Acc();
 39      }
 40
 41      @Override
 42      public Acc add(EnrichedAdEvent e, Acc acc) {
 43          acc.lineItemId = e.lineItemId();
 44          acc.campaignId = e.campaignId();
 45          if (e.advertiserId() != null) {
 46              acc.advertiserId = e.advertiserId();
 47          }
 48          if (e.isIvt()) {
 49              acc.ivtEvents++;
 50              return acc;
 51          }
 52          switch (e.type()) {
 53              case IMPRESSION -> acc.impressions++;
 54              case FIRST_QUARTILE -> acc.firstQuartiles++;
 55              case MIDPOINT -> acc.midpoints++;
 56              case THIRD_QUARTILE -> acc.thirdQuartiles++;
 57              case COMPLETE -> acc.completes++;
 58              case CLICK -> acc.clicks++;
 59          }
 60          acc.spendMicros += e.billableMicros();
 61          return acc;
 62      }
 63
 64      @Override
 65      public Acc getResult(Acc acc) {
 66          return acc;
 67      }
 68
 69      @Override
 70      public Acc merge(Acc a, Acc b) {
 71          a.lineItemId = a.lineItemId != null ? a.lineItemId : b.lineItemId;
 72          a.campaignId = a.campaignId != null ? a.campaignId : b.campaignId;
 73          a.advertiserId = a.advertiserId != null ? a.advertiserId : b.advertiserId;
 74          a.impressions += b.impressions;
 75          a.firstQuartiles += b.firstQuartiles;
 76          a.midpoints += b.midpoints;
 77          a.thirdQuartiles += b.thirdQuartiles;
 78          a.completes += b.completes;
 79          a.clicks += b.clicks;
 80          a.spendMicros += b.spendMicros;
 81          a.ivtEvents += b.ivtEvents;
 82          return a;
 83      }
 84
 85      /**
 86       * Stamps the window bounds and a revision. The first firing (at the watermark) is revision 1;
 87       * each late event inside the allowed lateness fires the window again with the corrected totals.
 88       */
 89      public static final class Emit extends ProcessWindowFunction<Acc, LineItemMinute, String, TimeWindow> {
 90          private static final ValueStateDescriptor<Integer> REVISION = new ValueStateDescriptor<>("revision", Int
  ↪ eger.class);
 91
 92          @Override
 93          public void process(String key, Context ctx, Iterable<Acc> totals, Collector<LineItemMinute> out)
 94                  throws Exception {
 95              ValueState<Integer> revisionState = ctx.windowState().getState(REVISION);
 96              int revision = revisionState.value() == null ? 1 : revisionState.value() + 1;
 97              revisionState.update(revision);
 98
 99              Acc t = totals.iterator().next();
100              out.collect(new LineItemMinute(key, t.campaignId, t.advertiserId, ctx.window().getStart(),
101                      ctx.window().getEnd(), revision, t.impressions, t.firstQuartiles, t.midpoints, t.thirdQuarti
  ↪ les,
```

```
                    t.completes, t.clicks, t.spendMicros, t.ivtEvents, ctx.currentProcessingTime()));
        }

        @Override
        public void clear(Context ctx) {
            ctx.windowState().getState(REVISION).clear();
        }
    }
}
```

stream-processor/src/main/java/dev/primetech/ads/stream/PipelineConfig.java

46 lines · Job settings, read from the environment so the same jar runs locally and in containers.

```
package dev.primetech.ads.stream;

import java.io.Serializable;
import java.time.Duration;
import java.util.Map;

/** Job settings, read from the environment so the same jar runs locally and in containers. */
public record PipelineConfig(
        String kafkaBootstrap,
        String consumerGroup,
        Duration checkpointInterval,
        Duration outOfOrderness,
        Duration sourceIdleness,
        Duration dedupTtl,
        Duration metadataWait,
        int ivtMaxImpressionsPerDeviceMinute,
        Duration allowedLateness,
        Duration lateFiringDelay) implements Serializable {

    public static PipelineConfig fromEnv(Map<String, String> env) {
        return new PipelineConfig(
                env.getOrDefault("KAFKA_BOOTSTRAP", "localhost:19092"),
                env.getOrDefault("CONSUMER_GROUP", "stream-processor"),
                seconds(env, "CHECKPOINT_INTERVAL_SECONDS", 10),
                seconds(env, "OUT_OF_ORDERNESS_SECONDS", 5),
                seconds(env, "SOURCE_IDLENESS_SECONDS", 30),
                // Long enough to cover client retries; every extra minute costs state (one entry per event).
                Duration.ofMinutes(Long.parseLong(env.getOrDefault("DEDUP_TTL_MINUTES", "30"))),
                seconds(env, "METADATA_WAIT_SECONDS", 60),
                Integer.parseInt(env.getOrDefault("IVT_MAX_IMPRESSIONS_PER_DEVICE_MINUTE", "20")),
                Duration.ofMinutes(Long.parseLong(env.getOrDefault("ALLOWED_LATENESS_MINUTES", "10"))),
                seconds(env, "LATE_FIRING_DELAY_SECONDS", 5));
    }

    /** Settings for tests: small, deterministic values. */
    public static PipelineConfig forTests() {
        return new PipelineConfig("unused", "test", Duration.ofSeconds(1), Duration.ofSeconds(5),
                Duration.ofSeconds(5), Duration.ofMinutes(30), Duration.ofSeconds(60), 20, Duration.ofMinutes(10
↪ ),
                // Fire on every late event: bounded test jobs end before processing-time timers would fire.
                Duration.ZERO);
    }

    private static Duration seconds(Map<String, String> env, String key, long def) {
        return Duration.ofSeconds(Long.parseLong(env.getOrDefault(key, String.valueOf(def))));
    }
}
```

stream-processor/src/main/java/dev/primetech/ads/stream/PipelineJob.java

102 lines · Wires AdPipeline to Kafka with exactly-once output.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.EnrichedAdEvent;
import dev.primetech.ads.common.LineItemMinute;
import dev.primetech.ads.common.Topics;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.connector.base.DeliveryGuarantee;
import org.apache.flink.connector.kafka.sink.KafkaSink;
import org.apache.flink.connector.kafka.source.KafkaSource;
import org.apache.flink.connector.kafka.source.enumerator.initializer.OffsetsInitializer;
import org.apache.flink.core.execution.CheckpointingMode;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.streaming.api.functions.sink.v2.DiscardingSink;
import org.apache.kafka.clients.consumer.OffsetResetStrategy;

import java.util.Properties;

/** Wires {@link AdPipeline} to Kafka with exactly-once output. */
public final class PipelineJob {

    public static void main(String[] args) throws Exception {
```

```
        PipelineConfig config = PipelineConfig.fromEnv(System.getenv());
        var env = StreamExecutionEnvironment.getExecutionEnvironment();
        env.enableCheckpointing(config.checkpointInterval().toMillis(), CheckpointingMode.EXACTLY_ONCE);
        env.getCheckpointConfig().setMinPauseBetweenCheckpoints(config.checkpointInterval().toMillis() / 2);

        KafkaSource<AdEvent> eventSource = KafkaSource.<AdEvent>builder()
                .setBootstrapServers(config.kafkaBootstrap())
                .setTopics(Topics.AD_EVENTS)
                .setGroupId(config.consumerGroup())
                // A checkpoint, when there is one, decides the offsets. Without one (e.g. the job manager
                // restarted without HA), continue from the offsets committed at the last checkpoint
                // rather than replaying the topic against empty dedup state. On a very first start,
                // process everything already in the topic.
                .setStartingOffsets(OffsetsInitializer.committedOffsets(OffsetResetStrategy.EARLIEST))
                .setDeserializer(JsonKafka.deserializer(AdEvent.class))
                .build();
        DataStream<AdEvent> events = env.fromSource(eventSource,
                WatermarkStrategy.<AdEvent>forBoundedOutOfOrderness(config.outOfOrderness())
                        .withTimestampAssigner((e, ts) -> e.eventTimeMs())
                        .withIdleness(config.sourceIdleness()),
                "ad-events").uid("ad-events-source");

        KafkaSource<CampaignMetadata> metadataSource = KafkaSource.<CampaignMetadata>builder()
                .setBootstrapServers(config.kafkaBootstrap())
                .setTopics(Topics.CAMPAIGN_METADATA)
                .setGroupId(config.consumerGroup() + "-metadata")
                .setStartingOffsets(OffsetsInitializer.earliest())
                .setDeserializer(JsonKafka.deserializer(CampaignMetadata.class))
                .build();
        // The metadata topic is quiet most of the time. Marking it idle keeps it from holding back
        // the watermark of the enrichment operator, which is the minimum of both inputs.
        DataStream<CampaignMetadata> metadata = env.fromSource(metadataSource,
                WatermarkStrategy.<CampaignMetadata>noWatermarks().withIdleness(config.sourceIdleness()),
                "campaign-metadata").uid("campaign-metadata-source");

        AdPipeline.Outputs out = AdPipeline.build(events, metadata, config);

        out.cleanEvents().sinkTo(transactionalSink(config, Topics.AD_EVENTS_CLEAN, "ads-clean",
                JsonKafka.<EnrichedAdEvent>serializer(Topics.AD_EVENTS_CLEAN, EnrichedAdEvent::sessionId)))
                .name("clean-events-sink").uid("clean-events-sink");
        out.lineItemMinutes().sinkTo(transactionalSink(config, Topics.LINE_ITEM_MINUTES, "ads-minutes",
                JsonKafka.<LineItemMinute>serializer(Topics.LINE_ITEM_MINUTES, LineItemMinute::key)))
                .name("line-item-minutes-sink").uid("line-item-minutes-sink");
        // Counted by the window operator's numLateRecordsDropped metric; nothing else to do with them.
        out.tooLate().sinkTo(new DiscardingSink<>()).name("too-late-for-totals").uid("too-late-sink");

        env.execute("ads-eventing-pipeline");
    }

    /**
     * Writes inside Kafka transactions committed on each checkpoint, so consumers reading with
     * {@code isolation.level=read_committed} see every record exactly once. The cost is latency:
     * output becomes visible only after the next checkpoint completes.
     */
    private static <T> KafkaSink<T> transactionalSink(PipelineConfig config, String topic, String txPrefix,
                                                      org.apache.flink.connector.kafka.sink.KafkaRecordSerializa
↪ tionSchema<T> schema) {
        var props = new Properties();
        // Must not exceed the broker's maximum (Redpanda: transaction_max_timeout_ms, 15 min by default).
        props.setProperty("transaction.timeout.ms", "600000");
        props.setProperty("compression.type", "lz4");
        // Bigger, fuller batches. Records only become visible when the transaction commits at the
        // next checkpoint, so waiting up to 50 ms to fill a batch adds no visible latency. With the
        // defaults (5 ms, 16 KB) a load-test profile spent 14% of the TaskManager's CPU in LZ4,
        // much of it writing the headers of small batches.
        props.setProperty("linger.ms", "50");
        props.setProperty("batch.size", "262144");
        return KafkaSink.<T>builder()
                .setBootstrapServers(config.kafkaBootstrap())
                .setRecordSerializer(schema)
                .setDeliveryGuarantee(DeliveryGuarantee.EXACTLY_ONCE)
                .setTransactionalIdPrefix(txPrefix)
                .setKafkaProducerConfig(props)
                .build();
    }

    private PipelineJob() {
    }
}
```

stream-processor/src/main/java/dev/primetech/ads/stream/RestoreEventTime.java

20 lines · Sets each record's Flink timestamp back to its event time.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.EnrichedAdEvent;
import org.apache.flink.streaming.api.operators.AbstractStreamOperator;
import org.apache.flink.streaming.api.operators.OneInputStreamOperator;
```

```
import org.apache.flink.streaming.runtime.streamrecord.StreamRecord;

/**
 * Sets each record's Flink timestamp back to its event time. Records re-emitted from a broadcast
 * input or a processing-time timer carry the wrong timestamp (or none), and event-time windows
 * assign by that timestamp. Watermarks pass through unchanged.
 */
public final class RestoreEventTime extends AbstractStreamOperator<EnrichedAdEvent>
        implements OneInputStreamOperator<EnrichedAdEvent, EnrichedAdEvent> {

    @Override
    public void processElement(StreamRecord<EnrichedAdEvent> element) {
        output.collect(element.replace(element.getValue(), element.getValue().eventTimeMs()));
    }
}
```

stream-processor/src/test/java/dev/primetech/ads/stream/AdPipelineTest.java

216 lines · Runs the whole topology on a local Flink cluster with in-memory sources.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.EnrichedAdEvent;
import dev.primetech.ads.common.EventType;
import dev.primetech.ads.common.LineItemMinute;
import org.apache.flink.api.common.eventtime.Watermark;
import org.apache.flink.api.common.eventtime.WatermarkGenerator;
import org.apache.flink.api.common.eventtime.WatermarkOutput;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.typeinfo.TypeInformation;
import org.apache.flink.runtime.testutils.MiniClusterResourceConfiguration;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.test.junit5.MiniClusterExtension;
import org.apache.flink.util.CloseableIterator;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.RegisterExtension;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;

import static dev.primetech.ads.stream.Events.CAMP_1;
import static dev.primetech.ads.stream.Events.CAMP_2;
import static dev.primetech.ads.stream.Events.T0;
import static dev.primetech.ads.stream.Events.event;
import static org.assertj.core.api.Assertions.assertThat;

/** Runs the whole topology on a local Flink cluster with in-memory sources. */
class AdPipelineTest {

    @RegisterExtension
    static final MiniClusterExtension FLINK = new MiniClusterExtension(
            new MiniClusterResourceConfiguration.Builder().setNumberTaskManagers(1).setNumberSlotsPerTaskManager
↪ (4).build());

    /** Emits a watermark after every event, so lateness is deterministic regardless of timing. */
    static final class PerEventWatermarks implements WatermarkGenerator<AdEvent> {
        private long max = Long.MIN_VALUE;

        @Override
        public void onEvent(AdEvent event, long timestamp, WatermarkOutput output) {
            max = Math.max(max, timestamp);
            output.emitWatermark(new Watermark(max - 5_000));
        }

        @Override
        public void onPeriodicEmit(WatermarkOutput output) {
        }
    }

    private record Result(List<EnrichedAdEvent> clean, List<LineItemMinute> minutes) {
    }

    private static Result run(List<AdEvent> input, List<CampaignMetadata> metadata) throws Exception {
        var env = StreamExecutionEnvironment.getExecutionEnvironment();
        env.setParallelism(2);
        DataStream<AdEvent> events = env.fromData(input, TypeInformation.of(AdEvent.class))
                .setParallelism(1)
                .assignTimestampsAndWatermarks(WatermarkStrategy.<AdEvent>forGenerator(ctx -> new PerEventWaterm
↪ arks())
                        .withTimestampAssigner((e, ts) -> e.eventTimeMs()))
                .setParallelism(1);
```

```
        DataStream<CampaignMetadata> meta = env.fromData(metadata, TypeInformation.of(CampaignMetadata.class));

        AdPipeline.Outputs out = AdPipeline.build(events, meta, PipelineConfig.forTests());
        CloseableIterator<EnrichedAdEvent> clean = out.cleanEvents().collectAsync();
        CloseableIterator<LineItemMinute> minutes = out.lineItemMinutes().collectAsync();
        env.execute("pipeline-test");

        var cleanList = new ArrayList<EnrichedAdEvent>();
        clean.forEachRemaining(cleanList::add);
        var minuteList = new ArrayList<LineItemMinute>();
        minutes.forEachRemaining(minuteList::add);
        return new Result(cleanList, minuteList);
    }

    /** Latest revision of each line item minute, as a reader of the upsert topic would see it. */
    private static Map<String, LineItemMinute> latest(List<LineItemMinute> minutes) {
        return minutes.stream().collect(Collectors.toMap(LineItemMinute::key, Function.identity(),
                (a, b) -> a.revision() >= b.revision() ? a : b));
    }

    @Test
    void endToEnd() throws Exception {
        var input = new ArrayList<AdEvent>();
        // Two people watching camp-1 in minute 0: impression and full quartiles.
        for (String viewer : List.of("v1", "v2")) {
            input.add(event(viewer + "-imp", EventType.IMPRESSION, T0 + 1_000, "camp-1", viewer));
            input.add(event(viewer + "-q1", EventType.FIRST_QUARTILE, T0 + 8_500, "camp-1", viewer));
            input.add(event(viewer + "-mid", EventType.MIDPOINT, T0 + 16_000, "camp-1", viewer));
            input.add(event(viewer + "-q3", EventType.THIRD_QUARTILE, T0 + 23_500, "camp-1", viewer));
            input.add(event(viewer + "-done", EventType.COMPLETE, T0 + 31_000, "camp-1", viewer));
        }
        // A retried batch: every event of v1 again.
        input.addAll(new ArrayList<>(input.subList(0, 5)));
        // A viewer without consent (no device id) on camp-2, with a click.
        input.add(event("anon-imp", EventType.IMPRESSION, T0 + 2_000, "camp-2", null));
        input.add(event("anon-click", EventType.CLICK, T0 + 3_000, "camp-2", null));
        // A bot on camp-2: 25 impressions within minute 0.
        for (int i = 0; i < 25; i++) {
            input.add(event("bot-" + i, EventType.IMPRESSION, T0 + 32_000 + i * 1_000, "camp-2", "bot-device"));
        }
        // Traffic in minute 1 pushes the watermark past minute 0, so minute 0 fires...
        input.add(event("v3-imp", EventType.IMPRESSION, T0 + 70_000, "camp-1", "v3"));
        // ...then a late impression for minute 0 arrives (buffered on the device) and fires it again.
        input.add(event("late-imp", EventType.IMPRESSION, T0 + 20_000, "camp-1", "v4"));
        // Advance the watermark past minute 1 so it fires too.
        input.add(event("v5-imp", EventType.IMPRESSION, T0 + 130_000, "camp-2", "v5"));

        Result result = run(input, List.of(CAMP_1, CAMP_2));

        // Dedup: each event id exactly once.
        assertThat(result.clean()).extracting(EnrichedAdEvent::eventId).doesNotHaveDuplicates();
        assertThat(result.clean()).hasSize(input.size() - 5);
        // Enrichment.
        assertThat(result.clean()).allSatisfy(e -> assertThat(e.advertiserName()).isNotNull());
        // IVT: the bot's impressions beyond 20 in the minute. Which five depends on arrival order
        // after the shuffles, so only the count and the device are fixed.
        assertThat(result.clean()).filteredOn(EnrichedAdEvent::isIvt).hasSize(5)
                .allSatisfy(e -> assertThat(e.deviceIdHash()).isEqualTo("bot-device"));

        Map<String, LineItemMinute> totals = latest(result.minutes());
        LineItemMinute camp1Minute0 = totals.get("camp-1-li-0|" + T0);
        assertThat(camp1Minute0.impressions()).as("v1, v2 and the late one").isEqualTo(3);
        assertThat(camp1Minute0.completes()).isEqualTo(2);
        assertThat(camp1Minute0.spendMicros()).isEqualTo(3 * 30_000L);
        // How many revisions minute 0 went through depends on how watermarks and records interleave
        // across parallel subtasks; minuteTotalsAreRevisedForLateEvents pins that down.
        assertThat(camp1Minute0.advertiserId()).isEqualTo("adv-1");

        LineItemMinute camp2Minute0 = totals.get("camp-2-li-0|" + T0);
        assertThat(camp2Minute0.impressions()).as("anon + 20 unflagged bot impressions").isEqualTo(21);
        assertThat(camp2Minute0.clicks()).isEqualTo(1);
        assertThat(camp2Minute0.ivtEvents()).isEqualTo(5);
        assertThat(camp2Minute0.spendMicros()).as("IVT is not billed").isEqualTo(21 * 30_000L);

        assertThat(totals.get("camp-1-li-0|" + (T0 + 60_000)).impressions()).isEqualTo(1);
    }

    /** Per-event watermarks, for streams of already enriched events. */
    static final class EnrichedWatermarks implements WatermarkGenerator<EnrichedAdEvent> {
        private long max = Long.MIN_VALUE;

        @Override
        public void onEvent(EnrichedAdEvent event, long timestamp, WatermarkOutput output) {
            max = Math.max(max, timestamp);
            output.emitWatermark(new Watermark(max - 5_000));
        }

        @Override
```

```
156          public void onPeriodicEmit(WatermarkOutput output) {
157          }
158      }
159
160      @Test
161      void minuteTotalsAreRevisedForLateEventsAndDropTooLateOnes() throws Exception {
162          // Parallelism 1: records and watermarks keep their order, so lateness is deterministic.
163          var env = StreamExecutionEnvironment.getExecutionEnvironment();
164          env.setParallelism(1);
165          var input = List.of(
166                  Events.enriched("a", EventType.IMPRESSION, T0 + 1_000, "d1"),
167                  Events.enriched("b", EventType.IMPRESSION, T0 + 2_000, "d2"),
168                  Events.enriched("next", EventType.IMPRESSION, T0 + 70_000, "d3"),    // watermark passes minute
    ↪ 0
169                  Events.enriched("late", EventType.IMPRESSION, T0 + 30_000, "d4"),    // within allowed lateness
170                  Events.enriched("far", EventType.IMPRESSION, T0 + 15 * 60_000, "d5"), // watermark passes latene
    ↪ ss
171                  Events.enriched("too-late", EventType.IMPRESSION, T0 + 40_000, "d6"));
172          var events = env.fromData(input, TypeInformation.of(EnrichedAdEvent.class))
173                  .assignTimestampsAndWatermarks(WatermarkStrategy.<EnrichedAdEvent>forGenerator(ctx -> new Enrich
    ↪ edWatermarks())
174                          .withTimestampAssigner((e, ts) -> e.eventTimeMs()));
175          var minutes = AdPipeline.minuteTotals(events, PipelineConfig.forTests());
176          CloseableIterator<LineItemMinute> out = minutes.collectAsync();
177          CloseableIterator<EnrichedAdEvent> tooLate = minutes.getSideOutput(AdPipeline.TOO_LATE).collectAsync();
178          env.execute("lateness-test");
179
180          var emitted = new ArrayList<LineItemMinute>();
181          out.forEachRemaining(emitted::add);
182          var dropped = new ArrayList<EnrichedAdEvent>();
183          tooLate.forEachRemaining(dropped::add);
184
185          var minute0 = emitted.stream().filter(m -> m.windowStartMs() == T0).toList();
186          assertThat(minute0).extracting(LineItemMinute::revision).containsExactly(1, 2);
187          assertThat(minute0).extracting(LineItemMinute::impressions).containsExactly(2L, 3L);
188          assertThat(dropped).extracting(EnrichedAdEvent::eventId).containsExactly("too-late");
189      }
190
191      @Test
192      void minuteTotalsMatchTheCleanEvents() throws Exception {
193          // Many interleaved sessions: the windowed totals must agree with a plain count of the output.
194          var input = new ArrayList<AdEvent>();
195          var types = EventType.values();
196          for (int i = 0; i < 2_000; i++) {
197              long t = T0 + (i * 97L) % 240_000;
198              input.add(event("e" + i, types[i % types.length], t, i % 3 == 0 ? "camp-2" : "camp-1", "dev-" + (i %
    ↪  400)));
199          }
200          input.sort(Comparator.comparingLong(AdEvent::eventTimeMs));
201          input.add(event("flush", EventType.IMPRESSION, T0 + 600_000, "camp-1", "dev-x"));
202
203          Result result = run(input, List.of(CAMP_1, CAMP_2));
204
205          Map<String, Long> expectedImpressions = new HashMap<>();
206          for (EnrichedAdEvent e : result.clean()) {
207              if (e.type() == EventType.IMPRESSION && !e.isIvt()) {
208                  expectedImpressions.merge(e.lineItemId() + "|" + (e.eventTimeMs() / 60_000 * 60_000), 1L, Long::
    ↪ sum);
209              }
210          }
211          Map<String, Long> actual = latest(result.minutes()).values().stream()
212                  .filter(m -> m.impressions() > 0)
213                  .collect(Collectors.toMap(LineItemMinute::key, LineItemMinute::impressions));
214          assertThat(actual).isEqualTo(expectedImpressions);
215      }
216  }
```

stream-processor/src/test/java/dev/primetech/ads/stream/CoalescingLateTriggerTest.java

74 lines · Test.

```
 1  package dev.primetech.ads.stream;
 2
 3  import org.apache.flink.streaming.api.windowing.triggers.TriggerResult;
 4  import org.apache.flink.streaming.api.windowing.windows.TimeWindow;
 5  import org.apache.flink.streaming.runtime.operators.windowing.TriggerTestHarness;
 6  import org.apache.flink.streaming.runtime.streamrecord.StreamRecord;
 7  import org.junit.jupiter.api.Test;
 8
 9  import java.time.Duration;
10
11  import static org.assertj.core.api.Assertions.assertThat;
12
13  class CoalescingLateTriggerTest {
14      private static final TimeWindow MINUTE = new TimeWindow(0, 60_000);
15
16      private static TriggerTestHarness<Object, TimeWindow> harness(Duration delay) throws Exception {
17          return new TriggerTestHarness<>(new CoalescingLateTrigger(delay), new TimeWindow.Serializer());
18      }
```

```

    @Test
    void firesOnceWhenTheWatermarkPassesTheWindow() throws Exception {
        var h = harness(Duration.ofSeconds(5));
        assertThat(h.processElement(new StreamRecord<>("a", 1_000), MINUTE)).isEqualTo(TriggerResult.CONTINUE);
        assertThat(h.processElement(new StreamRecord<>("b", 2_000), MINUTE)).isEqualTo(TriggerResult.CONTINUE);
        assertThat(h.numEventTimeTimers(MINUTE)).isEqualTo(1);

        assertThat(h.advanceWatermark(59_998)).isEmpty();
        assertThat(h.advanceWatermark(59_999, MINUTE)).isEqualTo(TriggerResult.FIRE);
    }

    @Test
    void coalescesABurstOfLateEventsIntoOneFiring() throws Exception {
        var h = harness(Duration.ofSeconds(5));
        h.processElement(new StreamRecord<>("a", 1_000), MINUTE);
        h.advanceWatermark(70_000, MINUTE);
        h.advanceProcessingTime(100_000);

        for (int i = 0; i < 50; i++) {
            assertThat(h.processElement(new StreamRecord<>("late" + i, 30_000), MINUTE)).isEqualTo(TriggerResult
 ↪ .CONTINUE);
        }
        assertThat(h.numProcessingTimeTimers(MINUTE)).isEqualTo(1);

        assertThat(h.advanceProcessingTime(104_999)).isEmpty();
        assertThat(h.advanceProcessingTime(105_000, MINUTE)).isEqualTo(TriggerResult.FIRE);

        // A later late event schedules a new firing.
        h.processElement(new StreamRecord<>("later", 30_000), MINUTE);
        assertThat(h.numProcessingTimeTimers(MINUTE)).isEqualTo(1);
    }

    @Test
    void zeroDelayFiresForEachLateEvent() throws Exception {
        var h = harness(Duration.ZERO);
        h.processElement(new StreamRecord<>("a", 1_000), MINUTE);
        h.advanceWatermark(70_000, MINUTE);

        assertThat(h.processElement(new StreamRecord<>("late", 30_000), MINUTE)).isEqualTo(TriggerResult.FIRE);
        assertThat(h.numProcessingTimeTimers()).isZero();
    }

    @Test
    void clearRemovesTimersAndState() throws Exception {
        var h = harness(Duration.ofSeconds(5));
        h.processElement(new StreamRecord<>("a", 1_000), MINUTE);
        h.advanceWatermark(70_000, MINUTE);
        h.processElement(new StreamRecord<>("late", 30_000), MINUTE);

        h.clearTriggerState(MINUTE);

        assertThat(h.numEventTimeTimers()).isZero();
        assertThat(h.numProcessingTimeTimers()).isZero();
        assertThat(h.numStateEntries()).isZero();
    }
}
```

`stream-processor/src/test/java/dev/primetech/ads/stream/DeduplicateTest.java`

59 lines · Test.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.EventType;
import org.apache.flink.api.common.typeinfo.Types;
import org.apache.flink.streaming.util.KeyedOneInputStreamOperatorTestHarness;
import org.apache.flink.streaming.util.ProcessFunctionTestHarnesses;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.time.Duration;

import static dev.primetech.ads.stream.Events.T0;
import static dev.primetech.ads.stream.Events.event;
import static org.assertj.core.api.Assertions.assertThat;

class DeduplicateTest {
    private KeyedOneInputStreamOperatorTestHarness<String, AdEvent, AdEvent> harness;

    @BeforeEach
    void setUp() throws Exception {
        harness = ProcessFunctionTestHarnesses.forKeyedProcessFunction(
                new Deduplicate(Duration.ofMinutes(30)), AdEvent::eventId, Types.STRING);
    }

    @AfterEach
```

```
    void tearDown() throws Exception {
        harness.close();
    }

    @Test
    void passesFirstOccurrenceAndDropsRepeats() throws Exception {
        var a = event("a", EventType.IMPRESSION, T0, "camp-1", "d1");
        var b = event("b", EventType.MIDPOINT, T0, "camp-1", "d1");

        harness.processElement(a, T0);
        harness.processElement(a, T0 + 1_000);
        harness.processElement(b, T0);
        harness.processElement(a, T0 + 9_000);

        assertThat(harness.extractOutputValues()).containsExactly(a, b);
    }

    @Test
    void forgetsIdsAfterTheTtl() throws Exception {
        var a = event("a", EventType.IMPRESSION, T0, "camp-1", "d1");
        harness.setStateTtlProcessingTime(0);
        harness.processElement(a, T0);

        harness.setStateTtlProcessingTime(Duration.ofMinutes(29).toMillis());
        harness.processElement(a, T0);
        assertThat(harness.extractOutputValues()).hasSize(1);

        harness.setStateTtlProcessingTime(Duration.ofMinutes(31).toMillis());
        harness.processElement(a, T0);
        assertThat(harness.extractOutputValues()).as("a repeat after the TTL is let through").hasSize(2);
    }
}
```

stream-processor/src/test/java/dev/primetech/ads/stream/EnrichWithCampaignTest.java

86 lines · Test.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.EnrichedAdEvent;
import dev.primetech.ads.common.EventType;
import org.apache.flink.api.common.typeinfo.Types;
import org.apache.flink.streaming.util.KeyedBroadcastOperatorTestHarness;
import org.apache.flink.streaming.util.ProcessFunctionTestHarnesses;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.time.Duration;

import static dev.primetech.ads.stream.Events.CAMP_1;
import static dev.primetech.ads.stream.Events.CAMP_2;
import static dev.primetech.ads.stream.Events.T0;
import static dev.primetech.ads.stream.Events.event;
import static org.assertj.core.api.Assertions.assertThat;

class EnrichWithCampaignTest {
    private KeyedBroadcastOperatorTestHarness<String, AdEvent, CampaignMetadata, EnrichedAdEvent> harness;

    @BeforeEach
    void setUp() throws Exception {
        harness = ProcessFunctionTestHarnesses.forKeyedBroadcastProcessFunction(
                new EnrichWithCampaign(Duration.ofSeconds(60)), AdEvent::campaignId, Types.STRING,
                EnrichWithCampaign.METADATA);
        harness.setProcessingTime(0);
    }

    @AfterEach
    void tearDown() throws Exception {
        harness.close();
    }

    @Test
    void enrichesWhenMetadataIsKnown() throws Exception {
        harness.processBroadcastElement(CAMP_1, T0);
        harness.processElement(event("a", EventType.IMPRESSION, T0, "camp-1", "d"), T0);

        assertThat(harness.extractOutputValues()).singleElement().satisfies(e -> {
            assertThat(e.campaignName()).isEqualTo("Spring Launch");
            assertThat(e.advertiserName()).isEqualTo("Harbor Bank");
        });
    }

    @Test
    void buffersEventsUntilTheirCampaignArrives() throws Exception {
        harness.processElement(event("a", EventType.IMPRESSION, T0, "camp-1", "d"), T0);
        harness.processElement(event("b", EventType.MIDPOINT, T0, "camp-1", "d"), T0);
```

```
        harness.processElement(event("c", EventType.IMPRESSION, T0, "camp-2", "d"), T0);
        assertThat(harness.extractOutputValues()).isEmpty();

        harness.processBroadcastElement(CAMP_1, T0);
        assertThat(harness.extractOutputValues()).extracting(EnrichedAdEvent::eventId).containsExactly("a", "b");

        harness.processBroadcastElement(CAMP_2, T0);
        assertThat(harness.extractOutputValues()).extracting(EnrichedAdEvent::eventId).containsExactly("a", "b", "c");
        assertThat(harness.extractOutputValues()).allSatisfy(e -> assertThat(e.campaignName()).isNotNull());
    }

    @Test
    void releasesUnenrichedAfterMaxWait() throws Exception {
        harness.processElement(event("a", EventType.IMPRESSION, T0, "camp-unknown", "d"), T0);

        harness.setProcessingTime(59_000);
        assertThat(harness.extractOutputValues()).isEmpty();

        harness.setProcessingTime(60_000);
        assertThat(harness.extractOutputValues()).singleElement().satisfies(e -> {
            assertThat(e.eventId()).isEqualTo("a");
            assertThat(e.campaignName()).isNull();
        });
    }

    @Test
    void releasedEventsAreNotEmittedTwiceWhenTheTimerFires() throws Exception {
        harness.processElement(event("a", EventType.IMPRESSION, T0, "camp-1", "d"), T0);
        harness.processBroadcastElement(CAMP_1, T0);
        harness.setProcessingTime(120_000);

        assertThat(harness.extractOutputValues()).hasSize(1);
    }
}
```

stream-processor/src/test/java/dev/primetech/ads/stream/Events.java

27 lines · Test fixtures.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.AdEvent;
import dev.primetech.ads.common.CampaignMetadata;
import dev.primetech.ads.common.DeviceType;
import dev.primetech.ads.common.EnrichedAdEvent;
import dev.primetech.ads.common.EventType;

/** Test fixtures. Times are relative to a minute boundary so window assignment is obvious. */
final class Events {
    static final long T0 = 1_760_000_040_000L; // a whole minute
    static final CampaignMetadata CAMP_1 = new CampaignMetadata("camp-1", "Spring Launch", "adv-1", "Harbor Bank");
    static final CampaignMetadata CAMP_2 = new CampaignMetadata("camp-2", "Summer Sale", "adv-2", "Juniper Foods");

    private Events() {
    }

    static AdEvent event(String id, EventType type, long timeMs, String campaign, String device) {
        return new AdEvent(1, id, type, timeMs, timeMs + 50, "sess-" + id, "ad-" + campaign, "cr-" + campaign,
                campaign, campaign + "-li-0", device, DeviceType.CTV, "US", device != null,
                type == EventType.IMPRESSION ? 30_000L : null);
    }

    static EnrichedAdEvent enriched(String id, EventType type, long timeMs, String device) {
        return EnrichedAdEvent.of(event(id, type, timeMs, "camp-1", device), CAMP_1, timeMs);
    }
}
```

stream-processor/src/test/java/dev/primetech/ads/stream/InvalidTrafficFilterTest.java

88 lines · Test.

```
package dev.primetech.ads.stream;

import dev.primetech.ads.common.EnrichedAdEvent;
import dev.primetech.ads.common.EventType;
import org.apache.flink.api.common.typeinfo.Types;
import org.apache.flink.streaming.util.KeyedOneInputStreamOperatorTestHarness;
import org.apache.flink.streaming.util.ProcessFunctionTestHarnesses;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import java.time.Duration;
import java.util.List;

import static dev.primetech.ads.stream.Events.T0;
```

```
import static dev.primetech.ads.stream.Events.enriched;
import static org.assertj.core.api.Assertions.assertThat;

class InvalidTrafficFilterTest {
    private static final int MAX = 3;
    private KeyedOneInputStreamOperatorTestHarness<String, EnrichedAdEvent, EnrichedAdEvent> harness;

    @BeforeEach
    void setUp() throws Exception {
        harness = ProcessFunctionTestHarnesses.forKeyedProcessFunction(
                new InvalidTrafficFilter(MAX, Duration.ofMinutes(10)), EnrichedAdEvent::deviceIdHash, Types.STRI
↪ NG);
    }

    @AfterEach
    void tearDown() throws Exception {
        harness.close();
    }

    private void send(String id, EventType type, long time, String device) throws Exception {
        harness.processElement(enriched(id, type, time, device), time);
    }

    private List<String> flaggedIds() {
        return harness.extractOutputValues().stream().filter(EnrichedAdEvent::isIvt).map(EnrichedAdEvent::eventI
↪ d).toList();
    }

    @Test
    void flagsImpressionsAboveTheRateAndLaterEventsInThatMinute() throws Exception {
        for (int i = 1; i <= 5; i++) {
            send("imp" + i, EventType.IMPRESSION, T0 + i * 1_000, "bot");
        }
        send("complete", EventType.COMPLETE, T0 + 6_000, "bot");

        assertThat(flaggedIds()).containsExactly("imp4", "imp5", "complete");
        assertThat(harness.extractOutputValues()).filteredOn(EnrichedAdEvent::isIvt)
                .allSatisfy(e -> assertThat(e.ivtReason()).isEqualTo(InvalidTrafficFilter.DEVICE_IMPRESSION_RATE
↪ ));
    }

    @Test
    void countsPerDeviceAndPerMinute() throws Exception {
        for (int i = 1; i <= 3; i++) {
            send("a" + i, EventType.IMPRESSION, T0 + i * 1_000, "dev-a");
            send("b" + i, EventType.IMPRESSION, T0 + i * 1_000, "dev-b");
        }
        // Next minute: the count starts again.
        for (int i = 1; i <= 3; i++) {
            send("a-next" + i, EventType.IMPRESSION, T0 + 60_000 + i * 1_000, "dev-a");
        }
        assertThat(flaggedIds()).isEmpty();
    }

    @Test
    void lateEventsStillCountTowardsTheirOwnMinute() throws Exception {
        for (int i = 1; i <= 3; i++) {
            send("imp" + i, EventType.IMPRESSION, T0 + i * 1_000, "bot");
        }
        send("next-minute", EventType.IMPRESSION, T0 + 61_000, "bot");
        send("late", EventType.IMPRESSION, T0 + 30_000, "bot");

        assertThat(flaggedIds()).containsExactly("late");
    }

    @Test
    void dropsMinuteStateOnceNothingCanArriveForIt() throws Exception {
        send("imp", EventType.IMPRESSION, T0, "dev");
        assertThat(harness.numEventTimeTimers()).isEqualTo(1);

        harness.processWatermark(T0 + 60_000 + Duration.ofMinutes(10).toMillis());

        assertThat(harness.numEventTimeTimers()).isZero();
        assertThat(harness.numKeyedStateEntries()).isZero();
    }
}
```